✨ نسخه جامع و کامل آموزشی — زبان فارسی

هوش مصنوعی مولد برای مبتدیان

دوره جامع و پروژه‌محور مایکروسافت در ۲۱ درس کامل
از مفاهیم مدل‌های زبان بزرگ (LLM) تا مهندسی پرامپت، سامانه‌های RAG، پایگاه‌های داده برداری و توسعه عامل‌های هوشمند (AI Agents)

مؤلف و منبع اصلی:
تیم مشاوران ابر مایکروسافت (Microsoft Cloud Advocates)

تنظیم و گردآوری:
مهدی حیدرپور — mahdi-hp.ir

محتوای دوره:
۲۲ فصل کامل، نمونه‌کدهای پایتون و تایپ‌اسکریپت

تاریخ تنظیم و گردآوری:
دوشنبه ۱۸ آبان ۱۴۰۵

تاریخ میلادی معادل:
9 نوامبر 2026 (9 November 2026)

نسخه نهایی مستندات رسمی‌تنظیم و گردآوری: مهدی حیدرپور (mahdi-hp.ir) — دوشنبه ۱۸ آبان ۱۴۰۵ (9 نوامبر 2026 (9 November 2026))
مایکروسافت (v3)

🌱 پیش‌گفتار و راهنمای دوره

به دوره آموزشی **«هوش مصنوعی مولد برای مبتدیان» (Generative AI for Beginners)** خوش آمدید! این دوره توسط تیم *مشاوران فناوری ابری مایکروسافت (Microsoft Cloud Advocates)* طراحی شده تا هر توسعه‌دهنده، محقق یا علاقه‌مندی بتواند از نقطه صفر تا سطوح پیشرفته، اصول ساخت برنامه‌های واقعی با هوش مصنوعی مولد را فرا بگیرد.

🚀 سناریوی محوری: استارتاپ آموزشی ما

برای اینکه یادگیری صرفاً تئوری نباشد، تمام دروس این دوره حول یک **استارتاپ فرضی در حوزه آموزش** شکل گرفته‌اند. مأموریت این استارتاپ چنین تعریف شده است:

> *«افزایش دسترسی به یادگیری در مقیاس جهانی، تضمین آموزش عادلانه و شخصی‌سازی تجربیات آموزشی برای هر دانش‌آموز بر اساس توانایی‌ها و نیازهای فردی.»*

در طول فصول این کتاب، خواهید دید که چگونه از مدل‌های زبانی برای حل مسائل واقعی مانند تدریس خصوصی ۲۴ ساعته، خلاصه‌سازی متون پیچیده، جستجوی معنایی در منابع درسی، تولید تصاویر کمک‌آموزشی و امنیت داده‌های کاربران استفاده می‌کنیم.

💡 ساختار دروس: «یادگیری» در برابر «ساخت»

دروس این کتاب به دو دسته اصلی تقسیم شده‌اند:

- **دروس یادگیری (Learn):** بر درک عمیق مفاهیم، زیرساخت‌ها، ریاضیات و معماری سیستم‌های هوش مصنوعی، توکنایزیشن، ایمنی و اخلاق تمرکز دارند.
- **دروس ساخت (Build):** همراه با کدهای عملی به دو زبان **Python** و **TypeScript**، به شما یاد می‌دهند چگونه با کتابخانه‌ها کار کنید، با APIها ارتباط برقرار نمایید و نرم‌افزارهای کامل بسازید.

راهنمای فصول

فهرست مطالب و پیش‌فصل‌ها

برای پرش مستقیم به هر فصل، روی ردیف مربوطه کلیک کنید:

۰۰ **راه‌اندازی محیط توسعه و شروع به کار** — محیط توسعه و مقدمات
راه‌اندازی GitHub Codespaces، اجرای محلی در ویندوز/مک با Python، و اتصال به ارائه‌دهندگان مدل (O penAI، Azure و Microsoft Foundry)

۰۱ **معرفی هوش مصنوعی مولد و مدل‌های زبان بزرگ (LLMs)** — مفاهیم پایه
آشنایی با چیستی هوش مصنوعی مولد، تاریخچه، شبکه‌های عصبی عمیق، توکنایزیشن و سناریوی استارتاپ آموزشی ما

۰۲ **کاوش و مقایسه مدل‌های بزرگ زبانی مختلف** — مفاهیم پایه
مقایسه معماری‌های ترنسفورمر، مدل‌های زبانی در برابر چندوجهی (Multimodal)، و تفاوت سرویس‌های متن‌باز با مد ل‌های اختصاصی تجاری

۰۳ **استفاده مسئولانه و اخلاقی از هوش مصنوعی مولد** — اخلاق و حاکمیت داده
اصول هوش مصنوعی مسئولانه مایکروسافت، کاهش سوگیری و تعصب الگوریتمی، شفافیت، و جلوگیری از ت وهم (Hallucination)

۰۴ **اصول و مبانی مهندسی پرامپت (Prompt Engineering)** — مهندسی پرامپت
ساختار یک پرامپت استاندارد، نقش دستورالعمل‌ها، تکنیک Zero-Shot و Few-Shot، و راهنمای تصویری معماری درخواست‌ها

۰۵ **طراحی و ساخت پرامپت‌های پیشرفته** — مهندسی پرامپت
تکنیک‌های پیشرفته مهندسی پرامپت، زنجیره تفکر (Chain of Thought - CoT)، جهت‌دهی خروجی ساختاریافته (JSON) و روش‌های ارزیابی پاسخ

۰۶ **ساخت برنامه‌های کاربردی تولید متن** — توسعه کاربردی
کدنویسی با SDK پایتون و تایپ‌اسکریپت برای فراخوانی API، کنترل پارامترهای Temperature و Top_P و تولید مت ن‌های خلاقانه و خلاصه

۰۷ ساخت برنامه‌های چت و ربات‌های گفتگو

معماری چت‌بات‌های مدرن، مدیریت پنجره زمینه (Context Window)، پیام‌های System/User/Assistant و حافظه مکالمه

توسعه کاربردی

۰۸ ساخت برنامه‌های جستجوی هوشمند با پایگاه‌های داده برداری

تبدیل متن به بردار (Embeddings)، محاسبه شباهت کسینوسی (Cosine Similarity)، و ساخت سامانه جستجوی معنایی متنی

جستجوی معنایی

۰۹ ساخت برنامه‌های تولید تصویر با DALL-E

آشنایی با مدل‌های انتشار (Diffusion Models)، تولید تصویر با متن، بازسازی و ویرایش تصویر با API و پیاده‌سازی گالری هوشمند

چندوجهی و بصری

۱۰ ساخت برنامه‌های هوش مصنوعی با ابزارهای کم‌کد (Low-Code)

یکپارچه‌سازی مدل‌های هوش مصنوعی با ابزارهای Power Apps، Microsoft Power Platform و ساخت گردش‌کارهای خودکار تجاری بدون کدنویسی پیچیده

توسعه سازمانی

۱۱ یکپارچه‌سازی سیستم‌ها با فراخوانی توابع (Function Calling)

اتصال مدل‌های هوش مصنوعی به پایگاه‌های داده و APIهای خارجی از طریق ابزارسازی و تعریف ساختار شمای توابع (JSON Schema)

توسعه پیشرفته

۱۲ طراحی تجربه کاربری (UX) برای برنامه‌های هوش مصنوعی

اصول طراحی رابط کاربری برای هوش مصنوعی، مدیریت زمان تاخیر (Streaming)، نحوه گزارش خطاها، ارجاع منابع و جلب اعتماد کاربر

طراحی و محصول

۱۳ امنیت و محافظت از برنامه‌های هوش مصنوعی مولد

تهدیدات امنیتی اختصاصی LLMها، حملات تزریق پرامپت (Prompt Injection)، نشت داده‌ها، نقض حریم خصوصی و راهکارهای دفاعی

امنیت و پدافند سایبری

۱۴ چرخه عمر برنامه‌ها و مدیریت LLMOps

فرآیند چرخه عمر محصولات هوش مصنوعی مولد از ایده تا استقرار، ارزیابی، نظارت مداوم و راهبردهای LLMOps

عملیات و استقرار

۱۵ تولید تقویت‌شده با بازیابی (RAG) و پایگاه‌های داده برداری

معماری جامع RAG، قطعه‌بندی مستندات (Chunking)، اندیس‌گذاری برداری، بازیابی اطلاعات و تولید پاسخ بر پایه دانش اختصاصی سازمان

معماری پیشرفته RAG

۱۶ مدل‌های متن‌باز و اکوسیستم Hugging Face

اکوسیستم متن‌باز

استفاده از مدل‌های متن‌باز روی پلتفرم Hugging Face، انتخاب مدل‌های سبک و استقرار بر روی زیرساخت‌های ا
بری و محلی

۱۷ عامل‌های هوش مصنوعی هوشمند (AI Agents)

سیستم‌های عامل هوشمند

تبدیل مدل‌های زبانی به کارگزاران مستقل خودکار، حافظه، برنامه‌ریزی و تصمیم‌گیری، و چارچوب‌های مح
بوب مانند AutoGen و Semantic Kernel

۱۸ تنظیم دقیق مدل‌های زبانی (Fine-Tuning)

یادگیری عمیق

چه زمانی به Fine-Tuning نیاز داریم؟ مقایسه با RAG، آماده‌سازی مجموعه داده آموزشی، و روش‌های بهینه‌سازی پار
امترها (PEFT / LoRA)

۱۹ مدل‌های زبانی کوچک (Small Language Models - SLM)

مدل‌های نوین

انقلاب مدل‌های کوچک با کارایی فوق‌العاده مانند سری Phi-3 مایکروسافت، اجرای محلی بدون نیاز به کارت گرافیک
سنگین و کاربرد در Edge

۲۰ ساخت و توسعه با مدل‌های خانواده Mistral

مدل‌های نوین

بررسی مدل‌های پیشرفته فرانسوی Mistral 7B و Mixtral 8x7B، معماری مخلوط متخصصین (MoE) و استفاده کاربر
دی از آن‌ها

۲۱ ساخت و توسعه با مدل‌های خانواده Meta و Llama

مدل‌های نوین

نقش بنیادین متا در هوش مصنوعی متن‌باز، معماری Llama 3، قابلیت‌های چندزبانه، پروانه‌های انتشار و کاربرد در پر
وژه‌های مقیاس‌پذیر

درس ۰۰ از ۲۱

راه‌اندازی محیط توسعه و شروع به کار

راه‌اندازی GitHub Codespaces، اجرای محلی در ویندوز/مک با Python، و اتصال به ارائه‌دهندگان مدل (OpenAI، Azure و Microsoft Foundry)

ما بسیار هیجان‌زده هستیم که شما این دوره را شروع کنید و ببینید که با هوش مصنوعی مولد چه چیزهایی می‌خواهید بسازید!

برای اطمینان از موفقیت شما، این صفحه مراحل راه‌اندازی، نیازهای فنی و محل دریافت کمک در صورت نیاز را بیان می‌کند.

مراحل راه‌اندازی

برای شروع این دوره، باید مراحل زیر را کامل کنید.

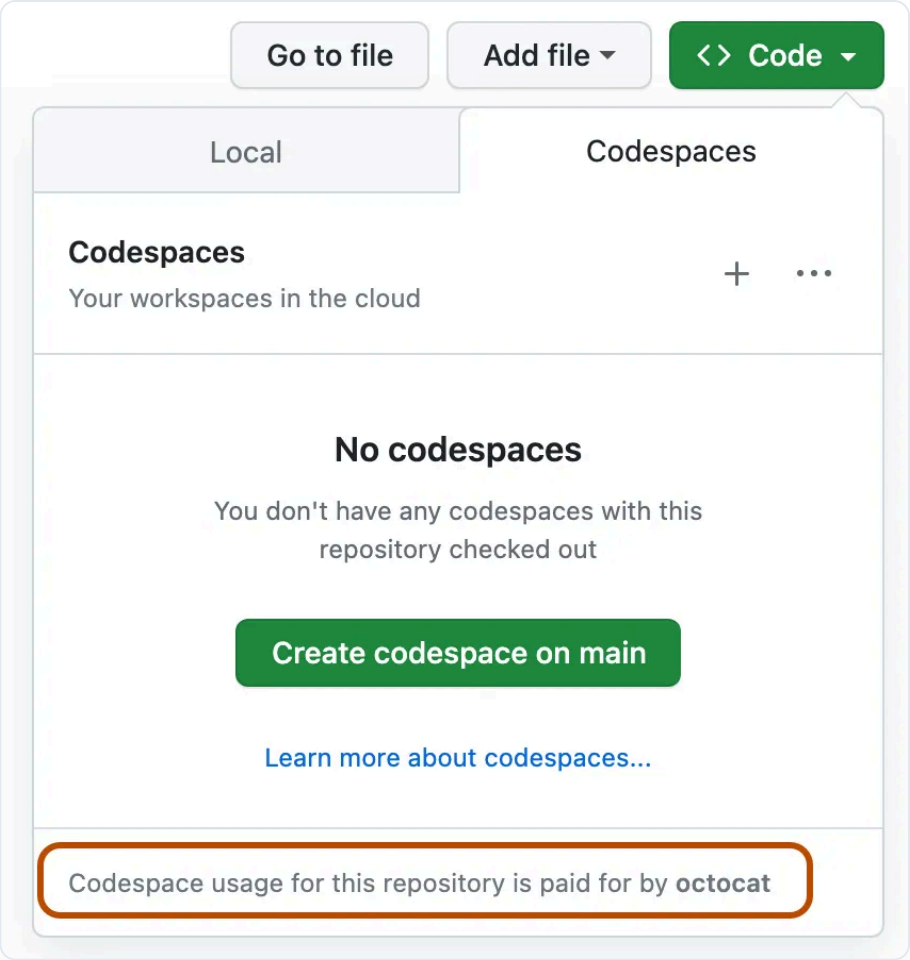
1. فورک کردن این مخزن

کل این مخزن را فورک کنید به حساب کاربری GitHub خودتان تا بتوانید هر کد را تغییر دهید و چالش‌ها را کامل کنید. همچنین می‌توانید این مخزن را ستاره (🌟) بدهید تا پیدا کردن آن و مخازن مرتبط راحت‌تر باشد.

2. ساخت یک کداسپیس

برای جلوگیری از هرگونه مشکل وابستگی هنگام اجرای کد، توصیه می‌کنیم این دوره را در GitHub Codespaces اجرا کنید.

در فورک شما: **Code -> Codespaces -> New on main**



Dialog showing buttons to create a codespace

> 📝 **راهنمای گزینه‌های پنجره GitHub Codespaces:**
>
> **گزینه Personal:** استفاده از سهمیه رایگان ماهیانه حساب کاربری شخصی گیتهاب (بدون هزینه).
>
> **گزینه Organization:** پرداخت هزینه‌های ماشین ابری از طریق سازمان متصل به گیتهاب.

2.1 اضافه کردن یک راز

1. ⚙️ آیکون چرخ‌دنده -> Add a new secret -> Codespaces : Manage user secret -> Command Palette.

2. نام را OPENAI_API_KEY بگذارید، کلید خود را الصاق کنید، ذخیره کنید.

3. بعدی چیست؟

من می‌خواهم...	بروم به...
شروع درس 1	`01-introduction-to-genai`
کار به صورت آفلاین	`setup-local.md`
راه‌اندازی ارائه‌دهنده LLM	`providers.md`
دیدار با سایر یادگیرندگان	عضویت در Discord ما

رفع اشکال

نشانه	راه‌حل
ساخت کانتینر بیش از ۱۰ دقیقه طول کشید	**Codespaces ← "Rebuild Container"**
`python: command not found`	ترمینال وصل نشده؛ روی + کلیک کنید ← *bash*
`401 Unauthorized` از OpenAI	کلید `OPENAI_API_KEY` اشتباه یا منقضی شده
VS Code نمایش می‌دهد "Dev container mounting"...	تب مرورگر را رفرش کنید— گاهی اتصال Codespaces قطع می‌شود
کرنل نوت‌بوک گم شده	منوی نوت‌بوک ← **Kernel ▸ Select Kernel ▸ Python 3**

سیستم‌های مبتنی بر یونیکس:

Terminal / Bash

```
touch .env
```

ویندوز:

Command Prompt

```
echo . > .env
```

3. **فایل `.env` را ویرایش کنید:** فایل `.env` را در ویرایشگر متن باز کنید (مثلاً Notepad، VS Code++ یا هر ویرایشگر دیگری). خطوط زیر را اضافه کنید و مقادیر واقعی نقطه انتهایی و کلید Microsoft Foundry Models خود را جایگزین کلیدهای نگهدارنده کنید (برای اطلاعات بیشتر به `providers.md` مراجعه کنید):

> **یادداشت تکمیلی**
>
> **توجه:** مدل‌های GitHub (و متغیر `GITHUB_TOKEN` مربوطه) تا پایان جولای ۲۰۲۶ بازنشسته می‌شوند. به جای آن از Microsoft Foundry Models استفاده کنید.

تنظیمات محیطی (.env)

```
AZURE_INFERENCE_ENDPOINT=your_foundry_endpoint_here
AZURE_INFERENCE_CREDENTIAL=your_foundry_api_key_here
```

4. **فایل را ذخیره کنید:** تغییرات را ذخیره کرده و ویرایشگر متن را ببندید.

5. **نصب `python-dotenv`** : اگر هنوز نصب نکرده‌اید، باید بسته `python-dotenv` را نصب کنید تا متغیرهای محیطی از فایل `.env` در برنامه پایتون شما بارگذاری شود. می‌توانید با `pip` آن را نصب کنید:

Terminal / Bash

```
pip install python-dotenv
```

6. **بارگذاری متغیرهای محیطی در اسکریپت پایتون شما:** در اسکریپت پایتون خود از بسته `python-dotenv` برای بارگذاری متغیرهای محیطی فایل `.env` استفاده کنید:

Python

```
from dotenv import load_dotenv
import os

# بارگذاری متغیرهای محیطی از فایل .env
load_dotenv()

# دسترسی به متغیرهای مدل‌های Microsoft Foundry
endpoint = os.getenv("AZURE_INFERENCE_ENDPOINT")
token = os.getenv("AZURE_INFERENCE_CREDENTIAL")

print(endpoint)
```

تمام شد! شما با موفقیت یک فایل `.env` ایجاد کرده، اطلاعات Microsoft Foundry Models خود را اضافه کرده و آنها را در برنامه پایتون خود بارگذاری کرده‌اید.

چگونه به صورت محلی روی کامپیوتر خود اجرا کنیم

برای اجرای کد به صورت محلی روی کامپیوترتان، باید نسخه‌ای از پایتون نصب شده داشته باشید.

سپس برای استفاده از مخزن، باید آن را کلون کنید:

SHELL

```
git clone https://github.com/microsoft/generative-ai-for-beginners
cd generative-ai-for-beginners
```

پس از اینکه همه چیز را دریافت کردید، می‌توانید شروع کنید!

مراحل اختیاری

نصب Miniconda

Miniconda یک نصب‌کننده سبک برای نصب Conda، پایتون و چند بسته است. خود Conda یک مدیر بسته است که تنظیم و جابجایی بین محیط‌های مجازی پایتون و بسته‌ها را آسان می‌کند. همچنین برای نصب بسته‌هایی که با `pip` در دسترس نیستند، مفید است.

می‌توانید با دنبال کردن راهنمای نصب MiniConda آن را راه‌اندازی کنید.

پس از نصب Miniconda، باید مخزن را کلون کنید (اگر قبلاً کلون نکردید)

سپس باید یک محیط مجازی بسازید. برای این کار با Conda، یک فایل محیط جدید (*environment.yml*) بسازید. اگر در Codespaces همراهی می‌کنید، این فایل را در پوشه `.devcontainer` بسازید، یعنی `.devcontainer/environment.yml`.

فایل محیط خود را با قطعه کد زیر پر کنید:

YML

```yml
name: <environment-name>
channels:
  - defaults
  - microsoft
dependencies:
  - python=<python-version>
  - openai
  - python-dotenv
  - pip
  - pip:
      - azure-ai-ml
```

اگر هنگام استفاده از conda خطا دریافت می‌کنید، می‌توانید به صورت دستی کتابخانه‌های Microsoft AI را با دستور زیر در ترمینال نصب کنید.

CODE

```
conda install -c microsoft azure-ai-ml
```

فایل محیط وابستگی‌هایی را که نیاز داریم مشخص می‌کند. `<environment-name>` نام محیط Conda شما و `<python-version>` نسخه پایتونی است که می‌خواهید استفاده کنید، مثلاً `3` جدیدترین نسخه اصلی پایتون است.

پس از آن، می‌توانید محیط Conda خود را با اجرای دستورات زیر در خط فرمان/ترمینال بسازید

Terminal / Bash

```
conda env create --name ai4beg --file .devcontainer/environment.yml # مسیر فرعی .devcontainer
اعمال می‌شود Codespace فقط برای تنظیمات
conda activate ai4beg
```

اگر مشکلی داشتید، به راهنمای محیط‌های Conda مراجعه کنید.

استفاده از Visual Studio Code با افزونه پشتیبانی پایتون

توصیه می‌کنیم از ویرایشگر Visual Studio Code (VS Code) همراه با افزونه پشتیبانی پایتون برای این دوره استفاده کنید. با این حال، این فقط یک توصیه است و الزام قطعی نیست.

یادداشت تکمیلی

توجه: با باز کردن مخزن دوره در VS Code، می‌توانید پروژه را داخل یک کانتینر راه‌اندازی کنید. این به خاطر پوشه خاص `.devcontainer` داخل مخزن دوره است. بعداً بیشتر در مورد آن توضیح می‌دهیم.

✨ نکته و پیشنهاد ویژه

توجه: پس از کلون و باز کردن دایرکتوری در VS Code، به صورت خودکار نصب افزونه پشتیبانی پایتون را پیشنهاد می‌دهد.

✨ نکته و پیشنهاد ویژه

توجه: اگر VS Code پیشنهاد داد مخزن را دوباره در یک کانتینر باز کنید، این درخواست را رد کنید تا از نسخه محلی پایتون استفاده کنید.

استفاده از Jupyter در مرورگر

همچنین می‌توانید پروژه را با استفاده از محیط Jupyter مستقیم در مرورگر اجرا کنید. هم Jupyter کلاسیک و هم Jupyter Hub محیط توسعه دلچسبی با ویژگی‌های تکمیل خودکار، برجسته‌سازی کد و ... ارائه می‌دهند.

برای شروع Jupyter به صورت محلی، به ترمینال/خط فرمان بروید، به دایرکتوری دوره بروید و فرمان زیر را اجرا کنید:

Terminal / Bash

```
jupyter notebook
```

یا

Terminal / Bash

```
jupyterhub
```

این یک نمونه Jupyter راه‌اندازی می‌کند و URL دسترسی را در پنجره خط فرمان نشان می‌دهد.

وقتی URL را باز کنید، باید طرح کلی دوره را ببینید و بتوانید به هر فایل `*.ipynb` ناوبری کنید. مثلاً `08-building-search-applications/python/oai-solution.ipynb`.

اجرای داخل کانتینر

راه دیگر راه‌اندازی همه چیز روی کامپیوتر یا Codespace استفاده از کانتینر است. پوشه خاص `.devcontainer` در مخزن دوره امکان راه‌اندازی پروژه داخل کانتینر توسط VS Code را فراهم می‌کند. خارج از Codespaces، این نیاز به نصب Docker دارد و کمی کار می‌برد، پس فقط به کسانی که تجربه کار با کانتینر دارند توصیه می‌شود.

یکی از بهترین روش‌ها برای امن نگه داشتن کلیدهای API هنگام استفاده از GitHub Codespaces، استفاده از رازهای Codespace است. لطفاً راهنمای مدیریت رازهای Codespaces را دنبال کنید.

درس‌ها و نیازهای فنی

دوره دارای درس‌های "یادگیری" است که مفاهیم هوش مصنوعی مولد را توضیح می‌دهد و درس‌های "ساخت" با مثال‌های کدنویسی عملی در هر دو زبان **Python** و **TypeScript** در صورت امکان.

برای درس‌های برنامه‌نویسی از Azure OpenAI در Microsoft Foundry استفاده می‌کنیم. شما نیاز به اشتراک Azure و کلید API دارید. دسترسی آزاد است - نیازی به درخواست ندارد - بنابراین می‌توانید یک منبع Microsoft Foundry ایجاد کرده و مدل را مستقر کنید و نقطه انتهایی و کلید خود را دریافت کنید.

هر درس برنامه‌نویسی همچنین یک فایل `README.md` دارد که می‌توانید کد و خروجی‌ها را بدون اجرای چیزی مشاهده کنید.

اولین بار استفاده از سرویس Azure OpenAI

اگر این اولین بار است که با سرویس Azure OpenAI کار می‌کنید، لطفاً این راهنما را برای ایجاد و استقرار یک منبع Azure OpenAI Service دنبال کنید.

اولین بار استفاده از API OpenAI

اگر اولین بار است که با API OpenAI کار می‌کنید، لطفاً راهنمای ایجاد و استفاده از رابط کاربری را دنبال کنید.

دیدار با سایر یادگیرندگان

ما کانال‌هایی در سرور رسمی AI Community Discord ایجاد کرده‌ایم تا با سایر یادگیرندگان ملاقات کنید. این روش خوبی برای شبکه‌سازی با دیگر کارآفرینان، سازندگان، دانشجویان و هر کسی است که می‌خواهد در حوزه هوش مصنوعی مولد پیشرفت کند.

تیم پروژه نیز در این سرور Discord حضور خواهد داشت تا به هر یادگیرنده‌ای کمک کند.

مشارکت

این دوره یک پروژه متن‌باز است. اگر بهبود یا مشکلی دیدید، لطفاً یک Pull Request ایجاد یا یک issue در GitHub گزارش کنید.

تیم پروژه تمام مشارکت‌ها را دنبال خواهد کرد. مشارکت در متن‌باز راه شگفت‌انگیزی برای ساختن حرفه شما در هوش مصنوعی مولد است.

اکثر مشارکت‌ها مستلزم توافق با قرارداد مجوز مشارکت‌کننده (CLA) است که شما حق دارید و واقعاً به ما حقوق استفاده از مشارکتتان را می‌دهید. برای اطلاعات بیشتر به وب‌سایت CLA، قرارداد مجوز مشارکت‌کننده مراجعه کنید.

مهم: هنگام ترجمه متن در این مخزن، لطفاً مطمئن شوید که از ترجمه ماشینی استفاده نمی‌کنید. ما ترجمه‌ها را توسط جامعه بررسی می‌کنیم، بنابراین فقط برای زبان‌هایی که مسلط هستید داوطلب ترجمه شوید.

وقتی یک درخواست کشش ارسال می‌کنید، ربات CLA به‌طور خودکار تعیین می‌کند که آیا نیاز به ارائه CLA دارید و PR را به‌طور مناسب تزیین می‌کند (مثلاً برچسب، نظر). به سادگی دستورالعمل‌های ارائه شده توسط ربات را دنبال کنید. شما فقط یکبار در تمام مخازنی که از CLA ما استفاده می‌کنند، باید این کار را انجام دهید.

این پروژه کد رفتار منبع باز مایکروسافت را پذیرفته است. برای اطلاعات بیشتر، به پرسش‌های متداول کد رفتار مراجعه کنید یا با ایمیل opencode برای سوالات یا نظرات اضافی تماس بگیرید.

بیایید شروع کنیم

اکنون که مراحل لازم برای تکمیل این دوره را طی کرده‌اید، بیایید با معرفی به هوش مصنوعی مولد و LLMها شروع کنیم.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

راه‌اندازی محلی 🖥️

اگر ترجیح می‌دهید همه چیز را روی لپ‌تاپ خود اجرا کنید، از این راهنما استفاده کنید.

شما دو مسیر دارید: **(A) پایتون بومی + virtual-env** یا **(B) کانتینر توسعه در VS Code با داکر**.

هر کدام که برایتان آسان‌تر است انتخاب کنید—هر دو به درس‌های یکسانی منجر می‌شوند.

1. پیش‌نیازها

ابزار	نسخه / یادداشت‌ها
پایتون	3.10 + (از https://python.org دریافت کنید)
گیت	آخرین نسخه (همراه با Xcode / گیت برای ویندوز / مدیر بسته لینوکس)
VS Code	اختیاری اما توصیه شده https://code.visualstudio.com
Docker Desktop	*فقط* برای گزینه B. نصب رایگان: https://docs.docker.com/desktop/

✨ **نکته و پیشنهاد ویژه**

💡 **نکته** – ابزارها را در ترمینال بررسی کنید:

`python --version` ، `git --version` ، `docker --version` ، `code --version`

2. گزینه A – پایتون بومی (سریع‌ترین)

گام 1 کلون کردن این مخزن

Terminal / Bash

```
git clone https://github.com/<your-github>/generative-ai-for-beginners
cd generative-ai-for-beginners
```

گام 2 ساخت و فعال‌سازی محیط مجازی

Terminal / Bash

```
python -m venv .venv          # بساز
source .venv/bin/activate     # مک‌اواس / لینوکس
.\.venv\Scripts\activate      # ویندوز پاورشل
```

✅ اکنون پرامپت باید با (.venv) شروع شود — یعنی داخل محیط مجازی هستید.

گام 3 نصب وابستگی‌ها

Terminal / Bash

```
pip install -r requirements.txt
```

به بخش 3 درباره کلیدهای API بروید

2. گزینه B – کانتینر توسعه VS Code (داکر)

این مخزن و دوره را با یک کانتینر توسعه راه‌اندازی کردیم که یک محیط اجرایی جهانی دارد که از پایتون ۳، .NET، Node.js و جاوا پشتیبانی می‌کند. تنظیمات مرتبط در فایل `devcontainer.json` در پوشه `.devcontainer/` در ریشه این مخزن تعریف شده است.

> 💡 **نکته کاربردی**
>
> **چرا این را انتخاب کنیم؟** محیطی مشابه Codespaces؛ بدون انحراف وابستگی.

گام 0 نصب موارد اضافی

Docker Desktop – تأیید کنید که `docker --version` کار می‌کند. افزونه VS Code Remote – Containers (شناسه: ms-vscode-remote.remote-containers).

گام 1 باز کردن مخزن در VS Code

فایل ▸ باز کردن پوشه… → generative-ai-for-beginners

پوشه .devcontainer/ را شناسایی کرده و یک اعلان باز می‌کند. VS Code

گام 2 دوباره باز کردن در کانتینر

روی «بازکردن دوباره در کانتینر» کلیک کنید. داکر تصویر را می‌سازد (≈ ۳ دقیقه در بار اول). وقتی پرامپت ترمینال ظاهر شد، داخل کانتینر هستید.

2. گزینه C – Miniconda

Miniconda یک نصب‌کننده سبک برای نصب Conda، پایتون و چند بسته دیگر است. Conda خود مدیر بسته‌ای است که کار راه‌اندازی و تعویض بین مختلف محیط‌های مجازی پایتون و بسته‌ها را آسان می‌کند. همچنین برای نصب بسته‌هایی که با `pip` در دسترس نیستند مفید است.

گام 0 نصب Miniconda

راهنمای نصب MiniConda را دنبال کنید تا آن را راه‌اندازی کنید.

Terminal / Bash

```
conda --version
```

گام 1 ساخت یک محیط مجازی

یک فایل محیط جدید (*environment.yml*) ایجاد کنید. اگر در Codespaces هستید، این فایل را در دایرکتوری `.devcontainer` بسازید، یعنی `.devcontainer/environment.yml`.

گام 2 پر کردن فایل محیط‌تان

قطعه زیر را به `environment.yml` اضافه کنید

YML

```yaml
name: <environment-name>
channels:
 - defaults
 - microsoft
dependencies:
- python=<python-version>
- openai
- python-dotenv
- pip
- pip:
    - azure-ai-ml
```

گام 3 ساخت محیط Conda

دستورات زیر را در خط فرمان/ترمینال خود اجرا کنید

Terminal / Bash

```
conda env create --name ai4beg --file .devcontainer/environment.yml # .devcontainer زیرمسیر
اعمال می‌شود Codespace فقط برای تنظیمات
conda activate ai4beg
```

اگر مشکلی پیش آمد به راهنمای محیط‌های Conda مراجعه کنید.

2 گزینه D – Jupyter کلاسیک / Jupyter Lab (در مرورگر شما)

> 💡 **نکته کاربردی**
>
> **این گزینه برای کیست؟**
>
> هر کسی که رابط کلاسیک Jupyter را دوست دارد یا می‌خواهد نوت‌بوک‌ها را بدون VS Code اجرا کند.

گام 1 اطمینان از نصب Jupyter

برای اجرای Jupyter به صورت محلی، به ترمینال/خط فرمان بروید، به دایرکتوری دوره مراجعه کنید و دستور زیر را اجرا کنید:

Terminal / Bash

```
jupyter notebook
```

یا

Terminal / Bash

```
jupyterhub
```

این باعث راه‌اندازی یک نمونه Jupyter می‌شود و آدرس URL دسترسی به آن در پنجره خط فرمان ظاهر خواهد شد.

وقتی به URL دسترسی پیدا کردید، باید طرح کلی دوره را ببینید و بتوانید به هر فایل `*.ipynb` وارد شوید. مثلاً `08-building-search-applications/python/oai-solution.ipynb`.

3. کلیدهای API خود را اضافه کنید

حفظ امنیت و محافظت کلیدهای API هنگام ساخت هر نوع برنامه‌ای مهم است. پیشنهاد می‌کنیم کلیدهای API را مستقیماً در کد خود ذخیره نکنید. قرار دادن این اطلاعات در مخزن عمومی ممکن است منجر به مشکلات امنیتی و حتی هزینه‌های ناخواسته در صورت استفاده توسط افراد سوءاستفاده‌گر شود. در اینجا راهنمای گام‌به‌گام برای ایجاد فایل `.env` در پایتون و افزودن اعتبارنامه‌های Microsoft Foundry Models ارائه شده است:

> **یادداشت تکمیلی**
>
> **توجه:** مدل‌های GitHub (و متغیر `GITHUB_TOKEN` آن) تا پایان جولای ۲۰۲۶ بازنشسته می‌شوند. این راهنما از Micro soft Foundry Models استفاده می‌کند. ترجیح می‌دهید کاملاً آفلاین کار کنید؟ به Foundry Local مراجعه کنید.

۱. **رفتن به دایرکتوری پروژه:** ترمینال یا خط فرمان خود را باز کنید و به ریشه دایرکتوری پروژه که می‌خواهید فایل `.env` را ایجاد کنید، بروید.

Terminal / Bash

```
cd path/to/your/project
```

۲. **ایجاد فایل `.env`** : از ویرایشگر متن مورد علاقه خود برای ایجاد یک فایل جدید به نام `.env` استفاده کنید. اگر از خط فرمان استفاده می‌کنید، می‌توانید از `touch` (در سیستم‌های مبتنی بر یونیکس) یا `echo` (در ویندوز) استفاده کنید:

سیستم‌های مبتنی بر یونیکس:

Terminal / Bash

```
touch .env
```

ویندوز:

Command Prompt

```
echo . > .env
```

۳. **ویرایش فایل `.env`** : فایل `.env` را در یک ویرایشگر متن (مثلاً Notepad، VS Code++ یا هر ویرایشگر دیگری) باز کنید. خطوط زیر را به فایل اضافه کنید و مقادیر جایگزین را با نقطه پایانی پروژه و کلید API واقعی Microsoft Foundry خود جایگزین کنید:

تنظیمات محیطی (.env)

```
AZURE_INFERENCE_ENDPOINT=your_foundry_endpoint_here
AZURE_INFERENCE_CREDENTIAL=your_foundry_api_key_here
```

۴. **ذخیره فایل**: تغییرات را ذخیره کرده و ویرایشگر متن را ببندید.

۵. **نصب `python-dotenv`** : اگر قبلاً نصب نکرده‌اید، باید بسته `python-dotenv` را نصب کنید تا متغیرهای محیطی از فایل `.env` در برنامه پایتون شما بارگذاری شوند. می‌توانید آن را با `pip` نصب کنید:

Terminal / Bash

```
pip install python-dotenv
```

۶. **بارگذاری متغیرهای محیطی در اسکریپت پایتون خود**: در اسکریپت پایتون خود، از بسته `python-dotenv` استفاده کنید تا متغیرهای محیطی را از فایل `.env` بارگذاری کنید:

Python

```
from dotenv import load_dotenv
import os

# بارگذاری متغیرهای محیطی از فایل .env
load_dotenv()

# دسترسی به متغیرهای مدل‌های Microsoft Foundry
endpoint = os.getenv("AZURE_INFERENCE_ENDPOINT")
token = os.getenv("AZURE_INFERENCE_CREDENTIAL")

print(endpoint)
```

همین بود! شما با موفقیت فایل `.env` ایجاد کرده‌اید، اعتبارنامه‌های Microsoft Foundry Models خود را اضافه کرده‌اید و آن‌ها را در برنامه پایتون خود بارگذاری کرده‌اید.

🔐 هرگز فایل .env را کامیت نکنید – این فایل در .gitignore قرار دارد. دستورالعمل‌های کامل ارائه‌دهنده در `providers.md` موجود است.

4. قدم بعدی چیست؟

می‌خواهم...	بروم به...
شروع درس ۱	`01-introduction-to-genai`
راه‌اندازی یک ارائه‌دهنده LLM	`providers.md`
ملاقات با دیگر یادگیرندگان	عضویت در Discord ما

5. رفع اشکال

نشانه	راه‌حل
`python not found` (پایتون پیدا نشد)	پایتون را به PATH اضافه کنید یا پس از نصب ترمینال را دوباره باز کنید.
`pip` نمی‌تواند wheel بسازد (ویندوز)	دستور `pip install --upgrade pip setuptools wheel` را اجرا کرده و دوباره تلاش کنید.
`ModuleNotFoundError: dotenv` (ماژول dotenv پیدا نشد)	دستور `pip install -r requirements.txt` را اجرا کنید (محیط نصب نشده بود).
ساخت داکر خطا می‌دهد *No space left*	در *Resources* ▸ *Settings* ▸ Docker Desktop → اندازه دیسک را افزایش دهید.
VS Code مدام پیشنهاد بازکردن مجدد می‌دهد	ممکن است هر دو گزینه فعال باشند؛ یکی را انتخاب کنید (venv) **یا** کانتینر).
خطاهای OpenAI ۴۰۱ / ۴۲۹	مقدار `OPENAI_API_KEY` و نرخ درخواست‌ها را بررسی کنید.
خطا در استفاده از Conda	کتابخانه‌های هوش مصنوعی مایکروسافت را با دستور `conda install -c microsoft azure-ai-ml` نصب کنید.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بخش تکمیلی / راهنمای همراه

انتخاب و پیکربندی ارائه‌دهنده LLM 🔑

تمرینات **ممکن است** برای کار با یک یا چند استقرار مدل زبان بزرگ (LLM) از طریق یک ارائه‌دهنده خدمات پشتیبانی شده مانند OpenAI، Azure یا Hugging Face تنظیم شوند. اینها یک *نقطه پایانی میزبانی شده* (API) فراهم می‌کنند که ما می‌توانیم به صورت برنامه‌نویسی با اعتبارنامه‌های مناسب (کلید API یا توکن) به آن دسترسی پیدا کنیم. در این دوره، این ارائه‌دهندگان را بررسی می‌کنیم:

- **OpenAI** با مدل‌های متنوع، از جمله سری اصلی GPT.
- **Azure OpenAI** برای مدل‌های OpenAI با تمرکز بر آمادگی سازمانی
- **Microsoft Foundry Models** برای یک نقطه پایانی و کلید API واحد برای دسترسی به صدها مدل از OpenAI، Meta، Microsoft، Cohere، Mistral و موارد دیگر (جایگزین GitHub Models می‌شود که در پایان ژوئیه ۲۰۲۶ بازنشسته می‌شود)
- **Hugging Face** برای مدل‌های متن‌باز و سرور استنتاج
- **Foundry Local** یا **Ollama** اگر ترجیح می‌دهید مدل‌ها را کاملاً آفلاین روی دستگاه خود اجرا کنید، بدون نیاز به اشتراک ابری

شما باید از حساب‌های خود برای این تمرینات استفاده کنید. تمرینات اختیاری هستند، بنابراین می‌توانید بر اساس علاقه خود یکی، همه یا هیچ‌کدام از ارائه‌دهندگان را راه‌اندازی کنید. راهنمایی‌هایی برای ثبت‌نام:

ثبت‌نام	هزینه	کلید API	محیط آزمایشی	توضیحات
OpenAI	قیمت‌گذاری	بر اساس پروژه	بدون کدنویسی، وب	مدل‌های متعدد موجود
Azure	قیمت‌گذاری	شروع سریع SDK	شروع سریع استودیو	باید از قبل درخواست دسترسی کرد
Microsoft Foundry	قیمت‌گذاری	صفحه نمای کلی پروژه	محیط آزمایشی Foundry	لایه رایگان موجود؛ یک نقطه پایانی + کلید برای ارائه‌دهندگان مدل متعدد
Hugging Face	قیمت‌گذاری	توکن‌های دسترسی	گفتگوی Hugging	مدل‌های محدودی دارد گفتگوی Hugging
Foundry Local	رایگان (روی دستگاه شما اجرا می‌شود)	نیاز نیست	CLI/SDK محلی	کاملاً آفلاین، نقطه پایانی سازگار با OpenAI

دستورالعمل‌های زیر را برای پیکربندی این مخزن برای استفاده با ارائه‌دهندگان مختلف دنبال کنید. تمریناتی که نیاز به ارائه‌دهنده خاصی دارند، یکی از این برچسب‌ها را در نام فایل خود خواهند داشت:

- `aoai` - نیاز به نقطه پایانی و کلید Azure OpenAI دارد
- `oai` - نیاز به نقطه پایانی و کلید OpenAI دارد
- `hf` - نیاز به توکن Hugging Face دارد
- `githubmodels` - نیاز به نقطه پایانی و کلید Microsoft Foundry Models دارد (GitHub Models در پایان ژوئیه ۲۰۲۶ بازنشسته می‌شود)

شما می‌توانید یکی، هیچ‌کدام یا همه ارائه‌دهندگان را پیکربندی کنید. تمرینات مرتبط به‌سادگی در صورت نبود اعتبارنامه‌ها به خطا برمی‌خورند.

ایجاد فایل `.env`

فرض می‌کنیم که شما قبلاً راهنمایی‌های بالا را خوانده‌اید و در ارائه‌دهنده مربوطه ثبت‌نام کرده و اعتبارنامه‌های احراز هویت مورد نیاز (کلید API یا توکن) را دریافت کرده‌اید. در مورد Azure OpenAI، فرض می‌کنیم که همچنین یک استقرار معتبر از سرویس Azure OpenAI (نقطه پایانی) دارید که حداقل یک مدل GPT برای تکمیل چت مستقر شده است.

گام بعدی این است که **متغیرهای محیطی محلی** خود را به شکل زیر پیکربندی کنید:

1. در پوشه اصلی به دنبال فایلی با نام `.env.copy` بگردید که باید محتویات مشابه زیر داشته باشد:

Terminal / Bash

```
# ارائه‌دهنده OpenAI
OPENAI_API_KEY='<add your OpenAI API key here>'

## آزور OpenAI در Microsoft Foundry
## (خدمات آزور OpenAI از بخشی اکنون Microsoft Foundry است: https://ai.azure.com)
AZURE_OPENAI_API_VERSION='2024-10-21' # پیش‌فرض تنظیم شده است! (نسخه پایدار فعلی GA API)
AZURE_OPENAI_API_KEY='<add your Foundry resource key here>'
AZURE_OPENAI_ENDPOINT='<add your Foundry resource endpoint here, e.g. https://<resource-name>.openai.azure.com>'
AZURE_OPENAI_DEPLOYMENT='<add your chat completion model deployment name here, e.g. gpt-5-mini>'
AZURE_OPENAI_EMBEDDINGS_DEPLOYMENT='<add your embeddings model deployment name here, e.g. text-embedding-3-small>'

## مدل‌های Microsoft Foundry (کاتالوگ مدل چند ارائه‌دهنده، جایگزین مدل‌های GitHub که تا پایان ژوئیه ۲۰۲۶ بازنشسته می‌شوند)
AZURE_INFERENCE_ENDPOINT='<add your Microsoft Foundry project endpoint here>'
AZURE_INFERENCE_CREDENTIAL='<add your Microsoft Foundry Models API key here>'

## Hugging Face
HUGGING_FACE_API_KEY='<add your HuggingFace API or token here>'
```

2. آن فایل را با دستور زیر به `.env` کپی کنید. این فایل در *gitignore* قرار دارد و اسرار را ایمن نگه می‌دارد.

Terminal / Bash

```
cp .env.copy .env
```

3. مقادیر را پر کنید (جایگزین‌های سمت راست علامت `=`) طبق توضیحات در بخش بعدی.

4. (اختیاری) اگر از GitHub Codespaces استفاده می‌کنید، می‌توانید متغیرهای محیطی را به عنوان *اسرار Codespaces* مرتبط با این مخزن ذخیره کنید. در این صورت نیازی به راه‌اندازی فایل محلی .env نخواهید داشت. **اما توجه داشته باشید که این گزینه فقط در صورت استفاده از GitHub Codespaces کار می‌کند.** اگر به جای آن از Docker Desktop استفاده می‌کنید، همچنان باید فایل .env را تنظیم کنید.

پر کردن فایل `.env`

بیایید نگاهی سریع به نام متغیرها بیندازیم تا بفهمیم هر کدام نماینده چه چیزی هستند:

متغیر	توضیح
HUGGING_FACE_API_KEY	این توکن دسترسی کاربری است که در پروفایل خود تنظیم کرده‌اید
OPENAI_API_KEY	این کلید مجوز استفاده از سرویس برای نقاط پایانی غیر Azure OpenAI است
AZURE_OPENAI_API_KEY	این کلید مجوز استفاده از سرویس Azure OpenAI است
AZURE_OPENAI_ENDPOINT	این نقطه پایانی مستقر شده برای منبع Azure OpenAI است
AZURE_OPENAI_DEPLOYMENT	این نقطه پایانی استقرار مدل *تولید متن* است
AZURE_OPENAI_EMBEDDINGS_DEPLOYMENT	این نقطه پایانی استقرار مدل *بردارهای متنی* است
AZURE_INFERENCE_ENDPOINT	این نقطه پایانی پروژه Microsoft Foundry شما است، برای استفاده از Microsoft Foundry Models
AZURE_INFERENCE_CREDENTIAL	این کلید API پروژه Microsoft Foundry شما است

توجه: دو متغیر آخر Azure OpenAI به ترتیب نمایانگر مدل پیش‌فرض برای تکمیل چت (تولید متن) و جستجوی برداری (بردارها) هستند. دستورالعمل تنظیم آن‌ها در تمرینات مربوطه تعریف خواهد شد.

پیکربندی Azure OpenAI: از طریق پرتال

> **یادداشت تکمیلی**
>
> **توجه:** سرویس Azure OpenAI اکنون بخشی از Microsoft Foundry است. منابع و استقرارها هنوز در پرتال Azure ظاهر می‌شوند، اما مدیریت روزمره مدل‌ها (استقرارها، محیط آزمایشی، نظارت) اکنون در پرتال Foundry انجام می‌شود به جای "استودیو Azure OpenAI" قدیمی.

مقادیر نقطه پایانی و کلید Azure OpenAI را در Azure Portal خواهید یافت، پس از آنجا شروع کنیم.

1. به Azure Portal بروید
2. در نوار کناری (منوی سمت چپ) گزینه **Keys and Endpoint** را کلیک کنید.
3. روی **Show Keys** کلیک کنید - باید موارد زیر را مشاهده کنید: KEY 1، KEY 2 و Endpoint.

4. مقدار KEY 1 را برای AZURE_OPENAI_API_KEY استفاده کنید
5. مقدار Endpoint را برای AZURE_OPENAI_ENDPOINT استفاده کنید

در مرحله بعد، به نقطه‌های پایانی مدل‌های خاصی که مستقر کرده‌ایم نیاز داریم.

1. گزینه **Model deployments** را در نوار کناری (منوی سمت چپ) برای منبع Azure OpenAI کلیک کنید.
2. در صفحه مقصد، روی **Go to Microsoft Foundry portal** (یا **Manage Deployments**، بسته به نوع منبع شما) کلیک کنید

این شما را به پرتال Microsoft Foundry منتقل می‌کند، جایی که سایر مقادیر را طبق توضیحات زیر پیدا خواهیم کرد.

پیکربندی Azure OpenAI: از طریق پرتال Microsoft Foundry

1. به Microsoft Foundry portal **از منبع خود** مطابق توضیح بالا بروید.
2. تب **Deployments** را کلیک کنید (نوار کناری، سمت چپ) تا مدل‌های مستقر شده را مشاهده کنید.
3. اگر مدل مورد نظر شما مستقر نشده است، با استفاده از **Deploy model** آن را از کتابخانه مدل‌ها مستقر کنید.
4. شما به یک مدل *تولید متن* نیاز خواهید داشت – پیشنهاد ما: **gpt-5-mini**
5. شما به یک مدل *بردار متنی* نیاز خواهید داشت – پیشنهاد ما: **text-embedding-3-small**

اکنون متغیرهای محیطی را برای بازتاب *نام استقرار* به‌روزرسانی کنید. این معمولاً همان نام مدل است مگر اینکه صریحاً آن را تغییر داده باشید. بنابراین، به عنوان مثال، ممکن است داشته باشید:

Terminal / Bash

```
AZURE_OPENAI_DEPLOYMENT='gpt-5-mini'
AZURE_OPENAI_EMBEDDINGS_DEPLOYMENT='text-embedding-3-small'
```

فراموش نکنید که پس از پایان فایل .env را ذخیره کنید. اکنون می‌توانید از فایل خارج شوید و به دستورالعمل‌های اجرای دفترچه بازگردید.

پیکربندی OpenAI: از پروفایل

کلید API OpenAI شما را می‌توان در حساب OpenAI شما یافت. اگر حساب ندارید، می‌توانید ثبت‌نام کنید و یک کلید API بسازید. پس از دریافت کلید، می‌توانید آن را در متغیر `OPENAI_API_KEY` در فایل `.env` قرار دهید.

پیکربندی Hugging Face: از پروفایل

توکن Hugging Face شما در پروفایل تحت بخش Access Tokens یافت می‌شود. این توکن‌ها را به صورت عمومی منتشر نکنید یا به اشتراک نگذارید. در عوض، برای این پروژه یک توکن جدید بسازید و آن را در فایل `.env` زیر متغیر `HUGGING_FACE_API_KEY` کپی کنید. *توجه:* این در واقع کلید API نیست اما برای احراز هویت استفاده می‌شود به همین دلیل نام‌گذاری را به این شکل حفظ کرده‌ایم.

پیکربندی Microsoft Foundry Models: از طریق پرتال

> **یادداشت تکمیلی**
>
> **توجه:** GitHub Models در پایان ژوئیه ۲۰۲۶ بازنشسته می‌شود. Microsoft Foundry Models جایگزین مستقیم آن است و همان تجربه کتابخانه مدل رایگان و SDK استنتاج Azure AI / OpenAI SDK را ارائه می‌دهد.

1. به Microsoft Foundry بروید و یک پروژه Foundry ایجاد یا باز کنید.
2. کتابخانه مدل را مرور کنید و یک مدل مستقر کنید، برای مثال `gpt-5-mini` .
3. در صفحه نمای کلی پروژه، **نقطه پایانی** و **کلید API** را کپی کنید.
4. مقدار نقطه پایانی را برای `AZURE_INFERENCE_ENDPOINT` و مقدار کلید را برای `AZURE_INFERENCE_CREDENTIAL` در فایل `.env` خود استفاده کنید.

ارائه‌دهندگان آفلاین / محلی

اگر ترجیح می‌دهید اصلاً از اشتراک ابری استفاده نکنید، می‌توانید مدل‌های سازگار و باز را مستقیماً روی دستگاه خود اجرا کنید:

- Foundry Local – اجرای دستگاه مایکروسافت. به طور خودکار بهترین ارائه‌دهنده اجرا (NPU، GPU یا CPU) را انتخاب می‌کند و یک نقطه پایانی سازگار با OpenAI ارائه می‌دهد، بنابراین می‌توانید اکثر کد نمونه در این دوره را با کمترین تغییرات مجدداً استفاده کنید. برای شروع، مستندات Foundry Local را ببینید یا با دستور `winget install Microsoft.FoundryLocal` (ویندوز) / `brew install microsoft/foundrylocal/foundrylocal` (macOS) نصب کنید.
- Ollama – جایگزین محبوب برای اجرای مدل‌های باز مانند Llama، Phi، Mistral و Gemma به صورت محلی.

برای مثال‌های عملی از هر دو گزینه، به درس ۱۹: ساخت با SLM ها مراجعه کنید.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۰۰ — هوش مصنوعی مولد مایکروسافت

درس ۰۱ از ۲۱

معرفی هوش مصنوعی مولد و مدل‌های زبان بزرگ (LLMs)

آشنایی با چیستی هوش مصنوعی مولد، تاریخچه، شبکه‌های عصبی عمیق، توکنایزیشن و سناریوی استارتاپ آموزشی ما

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/lFXQkBvEe0o?si=6ZBcQTwLJJDpnX0K

پخش در یوتیوب

اسکن کد QR

(برای دیدن ویدئوی این درس روی تصویر بالا کلیک کنید)

هوش مصنوعی مولد هوش مصنوعی‌ای است که قادر به تولید متن، تصویر و انواع دیگر محتوا است. چیزی که این فناوری را شگفت‌انگیز می‌کند این است که هوش مصنوعی را برای همه قابل دسترس می‌کند؛ هر کسی می‌تواند فقط با ارسال یک پیام متنی، یک جمله به زبان طبیعی، از آن استفاده کند. نیازی نیست زبان‌هایی مانند جاوا یا SQL را یاد بگیرید تا کاری ارزشمند انجام دهید، تنها کافی است زبان خود را به کار گیرید، بگویید چه می‌خواهید و یک پیشنهاد از مدل هوش مصنوعی دریافت کنید. کاربردها و تأثیرات این فناوری بسیار گسترده است، شما می‌توانید گزارش بنویسید یا بفهمید، برنامه‌های کاربردی بنویسید و خیلی بیشتر، همه فقط در چند ثانیه.

در این برنامه آموزشی، ما بررسی خواهیم کرد که چگونه استارتاپ ما از هوش مصنوعی مولد برای بازکردن سناریوهای جدید در دنیای آموزش بهره می‌برد و چگونه با چالش‌های اجتناب‌ناپذیر مرتبط با پیامدهای اجتماعی استفاده از آن و محدودیت‌های

فناوری مقابله می‌کنیم.

مقدمه

این درس شامل موارد زیر خواهد بود:

- معرفی سناریوی کسب‌وکار: ایده و مأموریت استارتاپ ما.
- هوش مصنوعی مولد و نحوه رسیدن ما به چشم‌انداز فناوری کنونی.
- عملکرد داخلی یک مدل زبان بزرگ.
- قابلیت‌های اصلی و موارد کاربرد عملی مدل‌های زبان بزرگ.

اهداف آموزشی

پس از اتمام این درس، شما درک خواهید داشت از:

- ماهیت هوش مصنوعی مولد و نحوه عملکرد مدل‌های زبان بزرگ.
- چگونگی بهره‌برداری از مدل‌های زبان بزرگ برای کاربردهای مختلف، با تمرکز بر سناریوهای آموزشی.

سناریو: استارتاپ آموزشی ما

هوش مصنوعی مولد نمایانگر اوج فناوری هوش مصنوعی است که مرزهای آنچه زمانی غیرممکن تصور می‌شد را جابه‌جا کرده است. مدل‌های هوش مصنوعی مولد دارای قابلیت‌ها و کاربردهای متعددی هستند، اما در این برنامه آموزشی ما بررسی خواهیم کرد که چگونه این فناوری آموزش را دگرگون می‌کند از طریق یک استارتاپ داستانی. ما این استارتاپ را *استارتاپ ما* نام خواهیم گذاشت. استارتاپ ما در حوزه آموزش فعالیت می‌کند با بیانیه مأموریتی بلندپروازانه:

> 💡 **نکته کاربردی**
>
> *افزایش دسترسی به یادگیری در مقیاسی جهانی، تضمین دسترسی عادلانه به آموزش و فراهم کردن تجربیات یادگیری شخصی‌سازی شده برای هر یادگیرنده، متناسب با نیازهایش.*

تیم استارتاپ ما می‌داند که بدون بهره گرفتن از یکی از قدرتمندترین ابزارهای زمانه – مدل‌های زبان بزرگ (LLM) – رسیدن به این هدف امکان‌پذیر نیست.

پیش‌بینی می‌شود هوش مصنوعی مولد نحوه یادگیری و آموزش امروزی را دگرگون کند، با در دسترس بودن معلمان مجازی برای دانش‌آموزان به صورت ۲۴ ساعته که حجم زیادی از اطلاعات و مثال‌ها را ارائه می‌دهند، و معلمان نیز قادر خواهند بود از ابزارهای نوآورانه برای ارزیابی دانش‌آموزانشان و ارائه بازخورد استفاده کنند.

پنج دانش‌آموز جوان که به یک مانیتور نگاه می‌کنند - تصویر توسط DALLE2

برای شروع، اجازه دهید برخی مفاهیم و اصطلاحات پایه را که در سرتاسر برنامه آموزشی استفاده خواهیم کرد تعریف کنیم.

چگونه به هوش مصنوعی مولد رسیدیم؟

با وجود هیجان فوق‌العاده‌ای که اخیراً با اعلام مدل‌های هوش مصنوعی مولد ایجاد شده است، این فناوری دهه‌ها سابقه دارد و اولین تلاش‌های پژوهشی به دهه ۶۰ میلادی بازمی‌گردد. امروزه ما به نقطه‌ای رسیده‌ایم که هوش مصنوعی دارای توانایی‌های شناختی انسانی مانند گفتگو است، همان‌طور که برای نمونه OpenAI ChatGPT یا Microsoft Copilot نشان داده‌اند؛ این‌ها همچنین از مدل GPT برای تجربه جستجوی مکالمه‌ای وب استفاده می‌کنند.

کمی برمی‌گردیم به عقب، نخستین نمونه‌های اولیه هوش مصنوعی شامل چت‌بات‌هایی بودند که به صورت تایپی ساخته شده بودند، مبتنی بر یک پایگاه دانش استخراج شده از گروهی از کارشناسان و نمایانده شده در کامپیوتر. پاسخ‌ها در پایگاه دانش با کلمات کلیدی موجود در متن ورودی تحریک می‌شدند. با این حال، به زودی مشخص شد که چنین رویکردی، استفاده از چت‌بات‌های تایپی، به خوبی مقیاس‌پذیر نیست.

رویکرد آماری به هوش مصنوعی: یادگیری ماشین

نقطه عطفی در دهه ۹۰ پیش آمد با به کارگیری رویکرد آماری در تحلیل متن. این منجر به توسعه الگوریتم‌های جدیدی به نام یادگیری ماشین شد که قادر به یادگیری الگوها از داده‌ها بدون برنامه‌نویسی صریح هستند. این رویکرد به ماشین‌ها اجازه می‌دهد تا درک زبان انسان را شبیه‌سازی کنند: مدل آماری بر اساس جفت‌های متن-برچسب آموزش می‌بیند و این امکان را می‌دهد که متن ورودی ناشناخته را با برچسبی از پیش تعیین شده، که نمایانگر نیت پیام است، طبقه‌بندی کند.

شبکه‌های عصبی و دستیاران مجازی مدرن

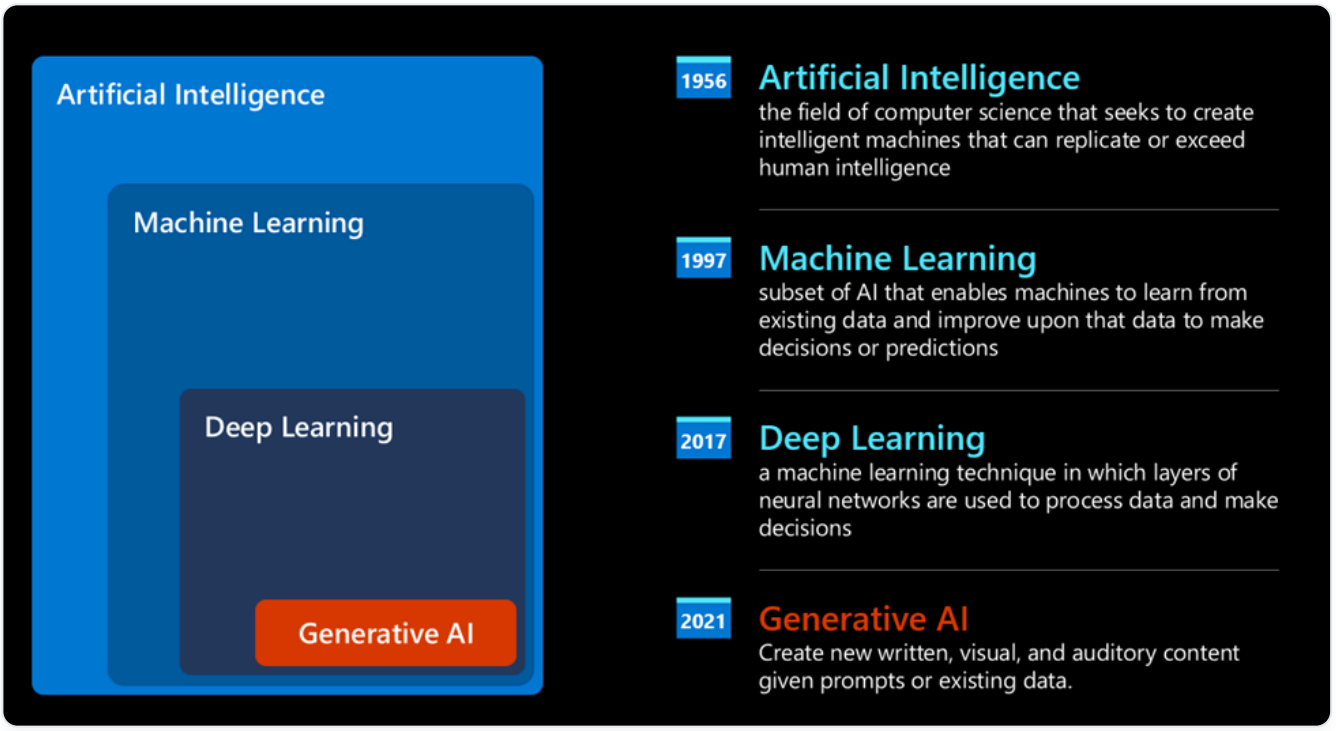
در سال‌های اخیر، پیشرفت فناوری سخت‌افزار که می‌تواند مقادیر بیشتری از داده‌ها و محاسبات پیچیده‌تر را پردازش کند، پژوهش در حوزه هوش مصنوعی را تشویق کرده و منجر به توسعه الگوریتم‌های پیشرفته یادگیری ماشین به‌نام شبکه‌های عصبی یا الگوریتم‌های یادگیری عمیق شده است.

شبکه‌های عصبی (به ویژه شبکه‌های عصبی بازگشتی – RNNها) به طور چشمگیری پردازش زبان طبیعی را بهبود بخشیده‌اند، به طوری که معنای متن را به طور معنادارتری بازنمایی می‌کنند و اهمیت زمینه یک کلمه در جمله را می‌سنجند.

این فناوری است که دستیاران مجازی متولد شده در دهه اول قرن جدید را توانمند کرده، که بسیار ماهر در تفسیر زبان انسانی، شناسایی نیاز و انجام عملی برای رفع آن هستند – مانند پاسخ دادن با یک اسکریپت از پیش تعریف‌شده یا استفاده از سرویس‌های شخص ثالث.

عصر حاضر، هوش مصنوعی مولد

بنابراین، این است نحوه رسیدن ما به هوش مصنوعی مولد امروزی، که می‌توان آن را زیرمجموعه‌ای از یادگیری عمیق دانست.



پس از دهه‌ها پژوهش در زمینه هوش مصنوعی، یک معماری مدل جدید به نام *ترنسفورمر* توانست محدودیت‌های RNNها را پشت سر بگذارد، به گونه‌ای که قادر به دریافت توالی‌های طولانی‌تری از متن به عنوان ورودی است. ترنسفورمرها بر اساس مکانیزم توجه ساخته شده‌اند، که امکان می‌دهد مدل به ورودی‌های دریافتی وزن‌های متفاوتی بدهد و «توجه بیشتری» به جایی بکند که اطلاعات مهم‌تر متمرکز شده‌اند، فارغ از ترتیب آن‌ها در توالی متن.

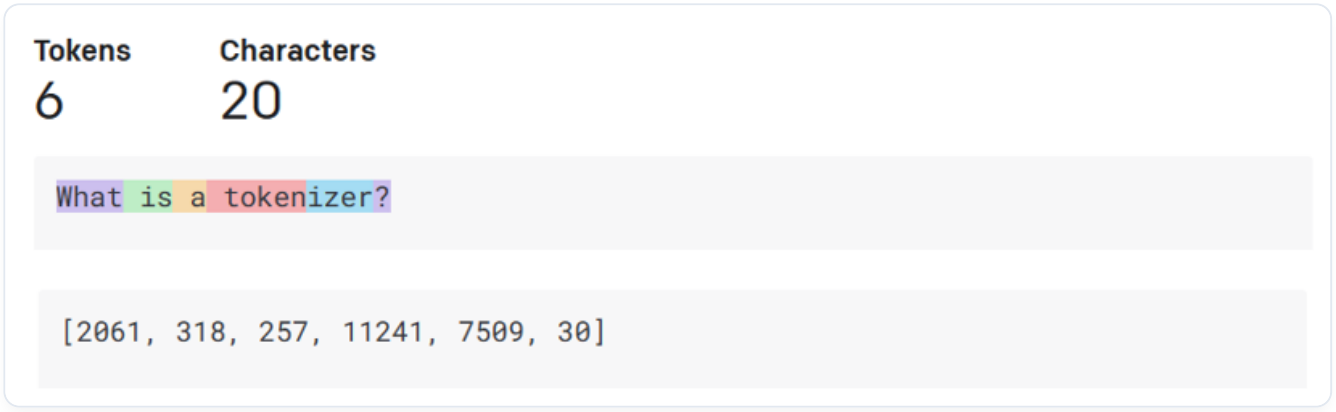
بیشتر مدل‌های هوش مصنوعی مولد اخیر – که به نام مدل‌های زبان بزرگ (LLM) نیز شناخته می‌شوند، چون با ورودی‌ها و خروجی‌های متنی کار می‌کنند – در واقع بر اساس این معماری ساخته شده‌اند. نکته جالب درباره این مدل‌ها که روی مقدار

زیادی داده بی‌برچسب از منابع مختلف مانند کتاب‌ها، مقالات و وب‌سایت‌ها آموزش دیده‌اند، این است که قادرند به طیف گسترده‌ای از وظایف تطبیق پیدا کنند و متنی گرامری صحیح و تا حدی خلاقانه تولید کنند. بنابراین، نه تنها ظرفیت ماشین برای «درک» متن ورودی به شکل شگفت‌انگیزی افزایش یافته، بلکه توانایی آن‌ها برای تولید پاسخ اصیل به زبان انسانی نیز ممکن شده است.

مدل‌های زبان بزرگ چطور کار می‌کنند؟

در فصل بعدی انواع مختلف مدل‌های هوش مصنوعی مولد را بررسی خواهیم کرد، اما فعلاً بیایید نگاه کنیم که مدل‌های زبان بزرگ چگونه کار می‌کنند، به ویژه با تمرکز بر مدل‌های OpenAI GPT (ترنسفورمر آموزش‌دیده مولد).

- **توکن‌ساز، تبدیل متن به عدد:** مدل‌های زبان بزرگ متنی را به عنوان ورودی دریافت و متنی را به عنوان خروجی تولید می‌کنند. اما چون مدل‌های آماری هستند، با اعداد بهتر از توالی‌های متنی کار می‌کنند. به همین دلیل هر ورودی توسط توکن‌ساز پردازش می‌شود، قبل از اینکه به مدل اصلی داده شود. توکن، بخش کوچکی از متن است – که شامل تعداد متغیری از کاراکترهاست، بنابراین وظیفه اصلی توکن‌ساز تقسیم ورودی به آرایه‌ای از توکن‌هاست. سپس هر توکن با یک شاخص توکن – که کدگذاری عددی آن بخش متن اصلی است – نگاشت می‌شود.



- **پیش‌بینی توکن‌های خروجی:** داده شده n توکن به عنوان ورودی (که حداکثر n از مدلی به مدل دیگر متفاوت است)، مدل قادر است یک توکن را به عنوان خروجی پیش‌بینی کند. سپس این توکن وارد ورودی مرحله بعدی می‌شود، در الگوی پنجره فزاینده، که تجربه بهتری از دریافت یک یا چند جمله به عنوان پاسخ ایجاد می‌کند. این توضیح می‌دهد چرا اگر با ChatGPT بازی کرده باشید، گاهی به نظر می‌رسد وسط جمله کارش را متوقف می‌کند.

- **فرایند انتخاب، توزیع احتمال:** توکن خروجی توسط مدل بر اساس احتمال وقوع آن پس از توالی متن فعلی انتخاب می‌شود. زیرا مدل توزیع احتمال را روی همه «توکن‌های بعدی» ممکن پیش‌بینی می‌کند، که بر اساس آموزش آن محاسبه شده است. اما همیشه توکنی که بالاترین احتمال را دارد، از توزیع نهایی انتخاب نمی‌شود. مقداری تصادفی بودن به این انتخاب اضافه می‌شود، به طوری که مدل به شکل غیرقطعی عمل می‌کند - ما برای همان ورودی همیشه خروجی یکسان نمی‌گیریم. این میزان تصادفی بودن به منظور شبیه‌سازی فرایند تفکر خلاق افزوده می‌شود و می‌توان آن را با پارامتری به نام دما (temperature) در مدل تنظیم کرد.

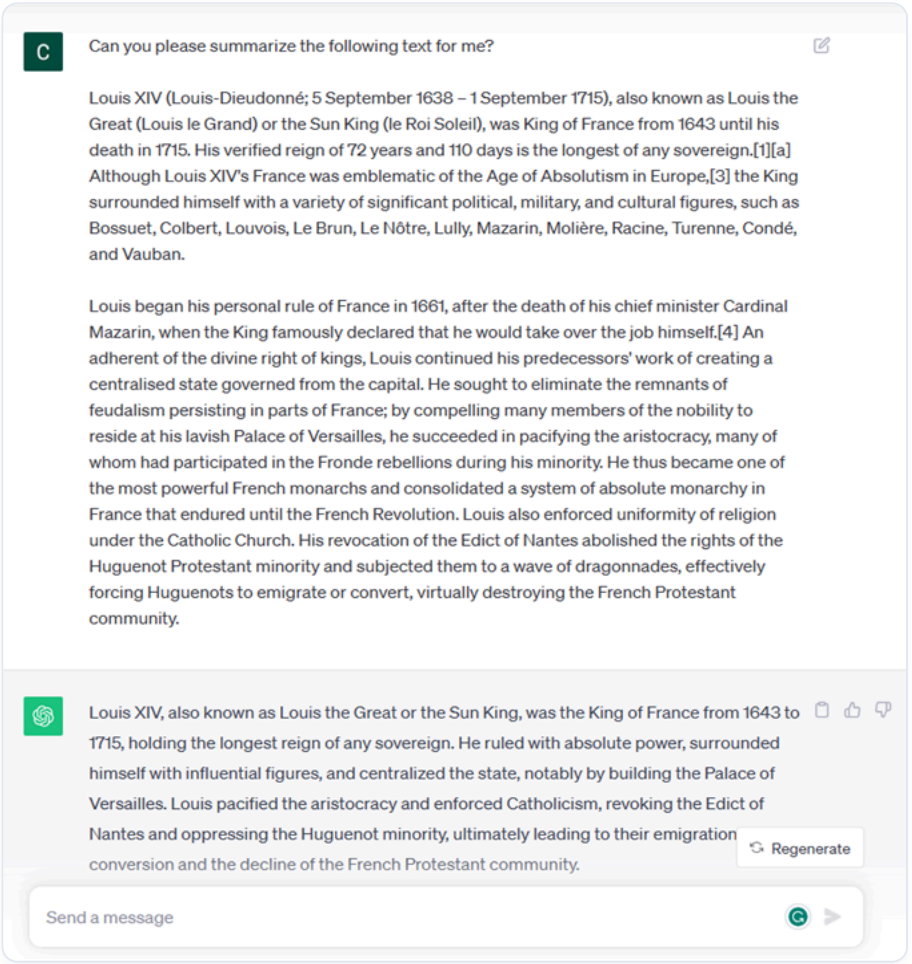
چگونه استارتاپ ما می‌تواند از مدل‌های زبان بزرگ بهره ببرد؟

حال که درک بهتری از عملکرد درونی مدل زبان بزرگ داریم، بیایید چند مثال عملی از کارهای رایجی که این مدل‌ها به خوبی انجام می‌دهند را با نگاه به سناریوی کسب‌وکارمان ببینیم. گفتیم که قابلیت اصلی یک مدل زبان بزرگ، *تولید متن از صفر، شروع از ورودی متنی به زبان طبیعی است*.

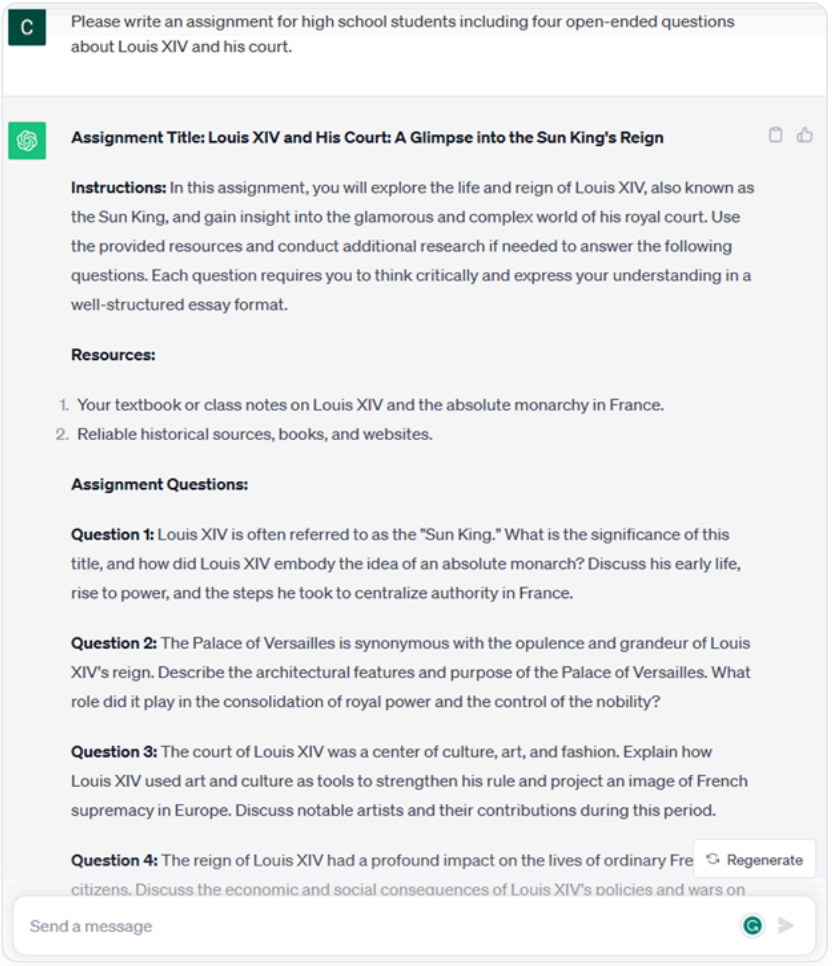
اما چه نوع ورودی و خروجی متنی؟ ورودی مدل زبان بزرگ به عنوان «پرامپت» شناخته می‌شود، در حالی که خروجی به عنوان «تکمیل» نامیده می‌شود، اصطلاحی که به مکانیزم مدل برای تولید توکن بعدی برای کامل کردن ورودی فعلی اشاره دارد. ما عمیق‌تر به معنای پرامپت و نحوه طراحی آن به گونه‌ای که بیشترین بهره را از مدل ببریم، خواهیم پرداخت. اما فعلاً بگوییم که پرامپت می‌تواند شامل موارد زیر باشد:

- یک **دستورالعمل** که نوع خروجی مورد انتظار از مدل را مشخص می‌کند. این دستور گاهی ممکن است شامل چند مثال یا داده اضافی باشد.

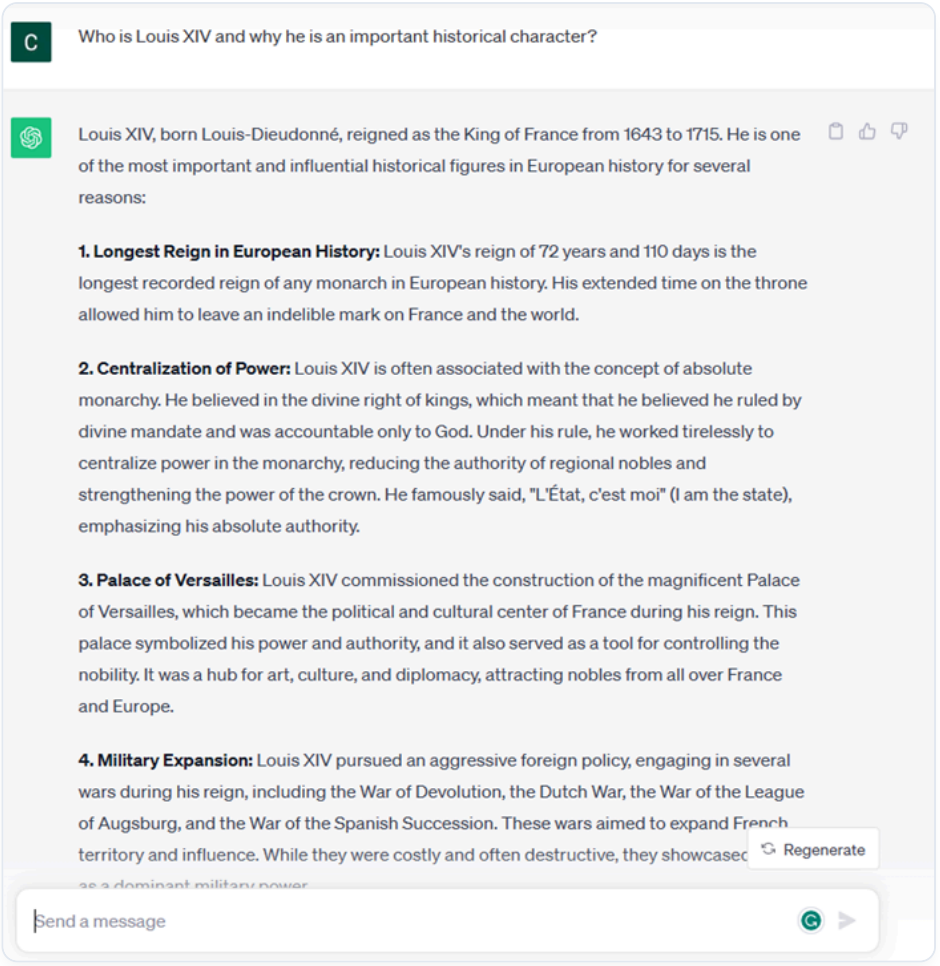
1. خلاصه‌سازی مقاله، کتاب، نقد و بررسی محصول و بیشتر، همراه با استخراج بینش‌ها از داده‌های بدون ساختار.



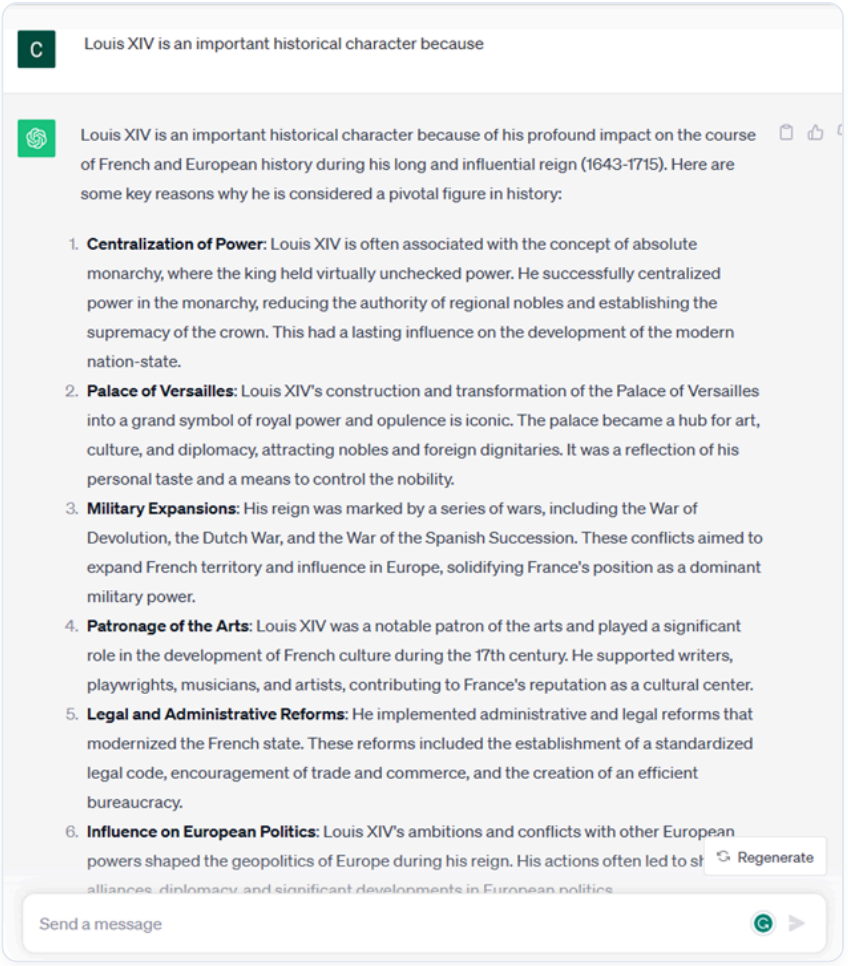
2. ایده‌پردازی خلاقانه و طراحی مقاله، انشاء، تکلیف یا بیشتر.



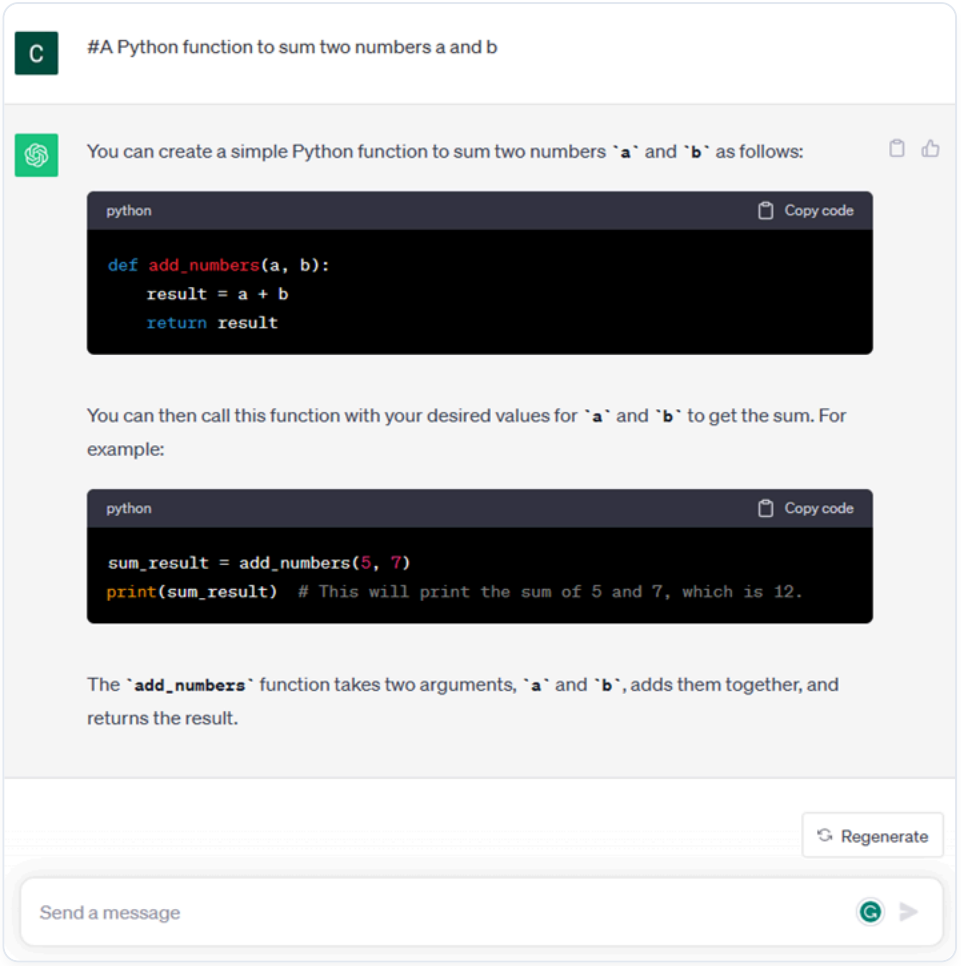
- یک **سوال** به صورت مکالمه با یک عامل (نماینده).



- بخشی از **متن برای تکمیل** که به طور ضمنی درخواست کمک برای نوشتن است.



- قسمتی از **کد** همراه با درخواست توضیح و مستندسازی آن، یا نظری که درخواست می‌کند قطعه‌ای کد بنویسد که وظیفه خاصی را انجام می‌دهد.



مثال‌های بالا ساده هستند و هدفشان نشان دادن کامل قابلیت‌های مدل‌های زبان بزرگ نیست. بلکه هدف نمایش پتانسیل استفاده از هوش مصنوعی مولد، به ویژه در زمینه‌های آموزشی، اما نه محدود به آن است.

همچنین، خروجی مدل هوش مصنوعی مولد کامل نیست و گاهی خلاقیت مدل ممکن است به ضد خودش تبدیل شود و خروجی‌ای داشته باشد که کاربر انسانی آن را به عنوان تحریف واقعیت تفسیر کند یا ممکن است توهین‌آمیز باشد. هوش مصنوعی مولد هوشمند نیست – حداقل در تعریف جامع‌تری از هوش که شامل تفکر نقادانه، خلاقانه یا هوش هیجانی است؛ غیرقطعی است و قابل اعتماد نیست، چون ممکن است ساختگی‌هایی مانند ارجاعات نادرست، محتوا و گزاره‌هایی همراه با اطلاعات صحیح ترکیب شده و به روشی قانع‌کننده و با اعتماد به نفس ارائه شوند. در درس‌های آینده به همه این محدودیت‌ها خواهیم پرداخت و می‌بینیم چه کارهایی می‌توانیم برای کاهش آن‌ها انجام دهیم.

تمرین

تمرین شما این است که بیشتر درباره هوش مصنوعی مولد مطالعه کنید و سعی کنید زمینه‌ای را شناسایی کنید که امروز فاقد هوش مصنوعی مولد است و شما می‌خواهید آن را اضافه کنید. تأثیر آن نسبت به روش «قدیمی» چگونه متفاوت خواهد بود؟ آیا می‌توانید کاری انجام دهید که قبلاً نمی‌توانستید یا سریع‌تر هستید؟ یک خلاصه ۳۰۰ کلمه‌ای بنویسید درباره اینکه استارتاپ هوش مصنوعی رویایی شما چگونه خواهد بود و شامل عناوینی مانند «مسئله»، «چگونه از هوش مصنوعی استفاده می‌کنم»، «تأثیر» و اختیاری یک برنامه کسب‌وکار باشد.

اگر این کار را انجام دادید، حتی ممکن است آماده باشید که برای شتاب‌دهنده مایکروسافت، Microsoft for Startups Found ers Hub درخواست دهید. ما اعتبار برای سرویس‌های OpenAI، Azure، راهنمایی و چیزهای بیشتر ارائه می‌کنیم، حتماً بررسی کنید!

بررسی دانش

درباره مدل‌های زبان بزرگ چه چیزی درست است؟

1. شما هر بار پاسخ دقیقا یکسانی دریافت می‌کنید.
2. این مدل‌ها کارها را بی‌نقص انجام می‌دهند، در جمع اعداد عالی هستند، کد کارا تولید می‌کنند و غیره.
3. پاسخ‌ها می‌توانند متفاوت باشند حتی اگر همان پرامپت استفاده شود. همچنین این مدل‌ها در ارائه پیش‌نویس اولیه چیزی، چه متن یا کد، عالی هستند. اما شما باید نتایج را بهبود بخشید.

پاسخ: ۳، یک مدل زبان بزرگ غیرقطعی است، پاسخ‌ها متفاوت‌اند، اما شما می‌توانید میزان آن را از طریق تنظیمات دما کنترل کنید. همچنین نباید انتظار داشته باشید که کارها را کاملاً بی‌نقص انجام دهد، بلکه برای انجام کارهای سنگین برای شما آمده است که معمولاً به این معنی است که شما یک تلاش اول خوب دریافت می‌کنید که باید به تدریج بهترش کنید.

کار عالی! ادامه مسیر

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش خود در زمینه هوش مصنوعی مولد را ارتقا دهید!

به درس ۲ بروید جایی که بررسی خواهیم کرد چگونه انواع مختلف مدل‌های زبانی بزرگ (LLM) را کاوش و مقایسه کنیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

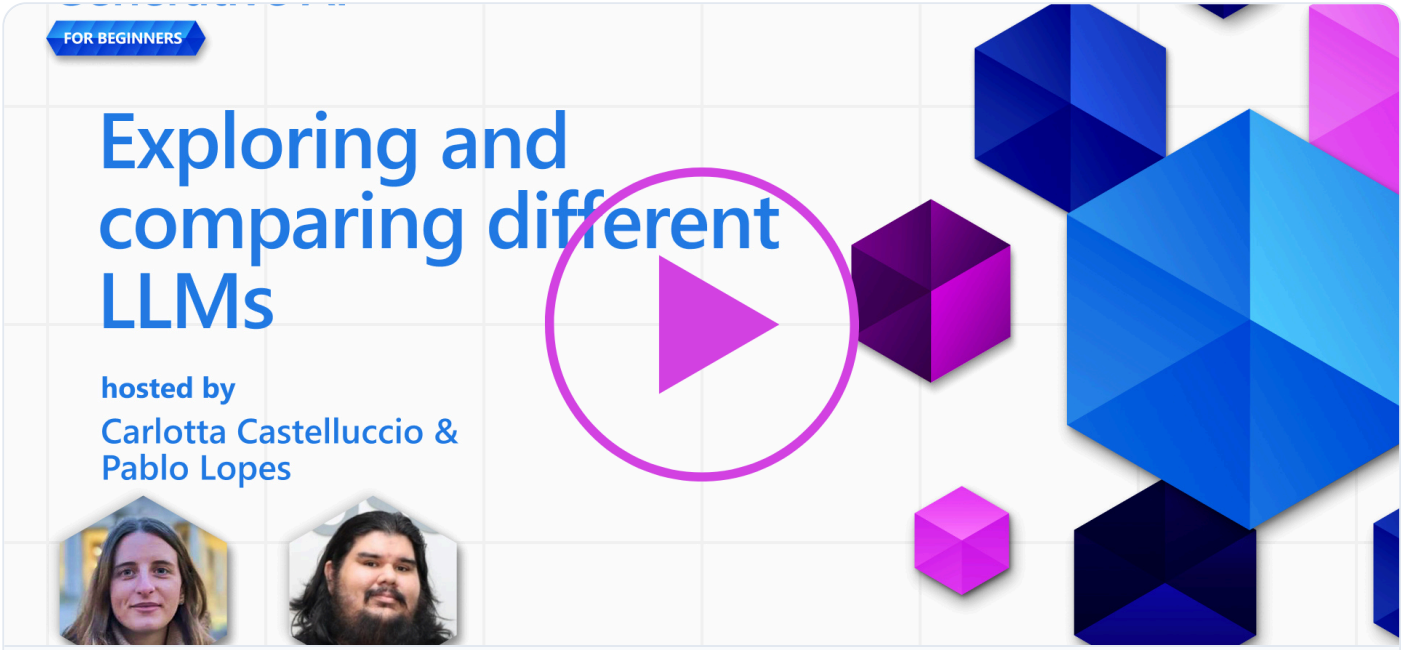
بازگشت به فهرست مطالب

درس ۰۱ — هوش مصنوعی مولد مایکروسافت

درس ۰۲ از ۲۱

کاوش و مقایسه مدل‌های بزرگ زبانی مختلف

مقایسه معماری‌های ترنسفورمر، مدل‌های زبانی در برابر چندوجهی (Multimodal)، و تفاوت سرویس‌های متن‌باز با مدل‌های اختصاصی تجاری



🎥 ویدیوی رسمی آموزش این درس

📱 برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/KIRUeDKscfI?si=8BHX1zvwzQBn-PlK

▶️ پخش در یوتیوب

اسکن کد QR

💡 نکته کاربردی

برای مشاهده ویدیو این درس، روی تصویر بالا کلیک کنید

با درس قبلی، دیدیم که چگونه هوش مصنوعی مولد در حال تغییر چشم‌انداز فناوری است، چگونه مدل‌های بزرگ زبانی (LLM) کار می‌کنند و چگونه یک کسب‌وکار – مثل استارتاپ ما – می‌تواند آن‌ها را در موارد استفاده خود به‌کار گیرد و رشد کند! در این فصل، قصد داریم انواع مختلف مدل‌های بزرگ زبانی (LLM) را مقایسه و تفاوت‌های آن‌ها را بررسی کنیم تا مزایا و معایب‌شان را بهتر درک کنیم.

مرحله بعدی در مسیر استارتاپ ما، بررسی چشم‌انداز فعلی مدل‌های بزرگ زبانی و درک این است که کدام‌یک برای استفاده ما مناسب‌تر هستند.

مقدمه

این درس شامل موارد زیر خواهد بود:

- انواع مختلف مدل‌های بزرگ زبانی در چشم‌انداز فعلی.
- آزمایش، تکرار و مقایسه مدل‌های مختلف برای مورد استفاده شما در Azure.
- چگونگی استقرار یک مدل بزرگ زبانی.

اهداف یادگیری

پس از اتمام این درس، شما قادر خواهید بود:

- مدل مناسب برای مورد استفاده خود را انتخاب کنید.
- یاد بگیرید چگونه مدل خود را آزمایش، تکرار و عملکرد آن را بهبود بخشید.
- بدانید کسب‌وکارها چگونه مدل‌ها را مستقر می‌کنند.

شناخت انواع مختلف مدل‌های بزرگ زبانی

مدل‌های بزرگ زبانی می‌توانند بر اساس معماری، داده‌های آموزشی و کاربردشان دسته‌بندی‌های مختلفی داشته باشند. درک این تفاوت‌ها کمک می‌کند استارتاپ ما مدل مناسب برای سناریو را انتخاب کند و بداند چگونه عملکرد را آزمایش، تکرار و بهبود دهد.

انواع مختلف زیادی از مدل‌های بزرگ زبانی وجود دارد، انتخاب شما به هدف استفاده، داده‌های شما، میزان بودجه و عوامل دیگر بستگی دارد.

بسته به این‌که مدل‌ها را برای متن، صدا، ویدئو، تولید تصویر و غیره می‌خواهید استفاده کنید، ممکن است نوع متفاوتی از مدل را انتخاب کنید.

- **تشخیص صدا و گفتار**. مدل‌های سبک Whisper همچنان مدل‌های عمومی useful برای تشخیص گفتار هستند، اما گزینه‌های تولیدی اکنون شامل مدل‌های جدیدتر گفتار به متن مانند `gpt-4o-transcribe`، `gpt-4o-mini-transcribe` ، و انواع دیاریزاسیون نیز می‌شوند. پوشش زبان، دیاریزاسیون، پشتیبانی لحظه‌ای، تأخیر و هزینه را برای سناریوی خود ارزیابی کنید. اطلاعات بیشتر در مستندات OpenAI گفتار به متن.
- **تولید تصویر**. مدل‌های شناخته شده مانند DALL-E و Midjourney گزینه‌های محبوب تولید تصویر هستند، اما API های تصویر OpenAI بر مدل‌های GPT Image مانند `gpt-image-2` متمرکز هستند، در حالی که Stable Diffusion،

Imagen، Flux و خانواده‌های مدل دیگر نیز گزینه‌های رایج‌اند. پایبندی به دستورات، پشتیبانی ویرایش، کنترل سبک، الزامات ایمنی و مجوزها را مقایسه کنید. اطلاعات بیشتر در راهنمای تولید تصویر OpenAI و فصل ۹ این دوره.

- **تولید متن**. مدل‌های متنی اکنون شامل مدل‌های frontier، مدل‌های استدلال، مدل‌های کوچکتر با تأخیر پایین، و مدل‌های وزن‌باز هستند. نمونه‌های فعلی شامل مدل‌های OpenAI GPT-5.x، مدل‌های Anthropic Claude 4.x، مدل‌های گوگل Gemini 3.x، مدل‌های Meta Llama 4 و مدل‌های Mistral هستند. فقط بر اساس تاریخ انتشار یا قیمت انتخاب نکنید؛ کیفیت انجام وظیفه، تأخیر، پنجره متن، استفاده از ابزار، رفتار ایمنی، دسترسی منطقه‌ای و هزینه کل را مقایسه کنید. فهرست مدل‌های Microsoft Foundry مکان خوبی برای مقایسه مدل‌های موجود در Azure است.

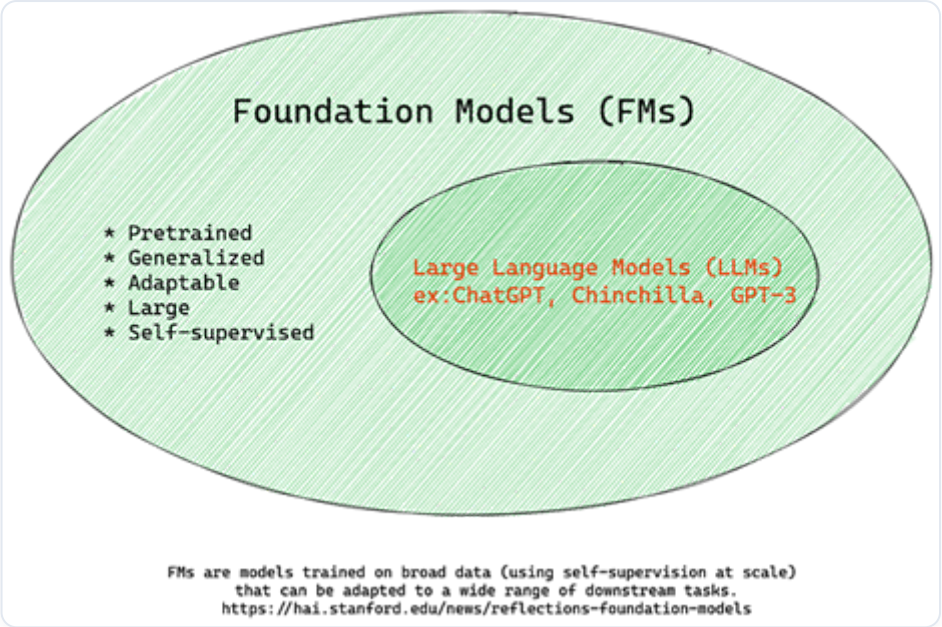
- **چند حالتی**. بسیاری از مدل‌های فعلی می‌توانند بیش از متن پردازش کنند. برخی ورودی تصویر، صدا یا ویدئو می‌پذیرند؛ برخی می‌توانند ابزارها را فراخوانی کنند؛ و مدل‌های تخصصی می‌توانند تصویر، صدا یا ویدئو تولید کنند. به عنوان مثال، مدل‌های فعلی OpenAI از ورودی متن و تصویر پشتیبانی می‌کنند، مدل‌های Gemini بسته به نوع، ورودی متن، کد، تصویر، صدا و ویدئو را پشتیبانی می‌کنند، و Llama 4 Scout و Maverick مدل‌های وزن‌باز چند حالتی بومی هستند. همیشه کارت مدل را برای حالت‌های ورودی و خروجی پشتیبانی شده قبل از ساخت گردش کار بررسی کنید.

انتخاب مدل به این معنی است که شما توانایی‌های پایه را دریافت می‌کنید، اما شاید کافی نباشد. اغلب داده‌های خاص شرکت وجود دارد که باید somehow به مدل بزرگ زبانی منتقل شود. چند راه برای انجام این کار وجود دارد که در بخش‌های بعدی به آن می‌پردازیم.

مدل‌های بنیادین در مقابل مدل‌های بزرگ زبانی

اصطلاح مدل بنیادین توسط پژوهشگران استنفورد ابداع شده است و به عنوان مدلی هوش مصنوعی تعریف شده که شرایطی از جمله موارد زیر دارد:

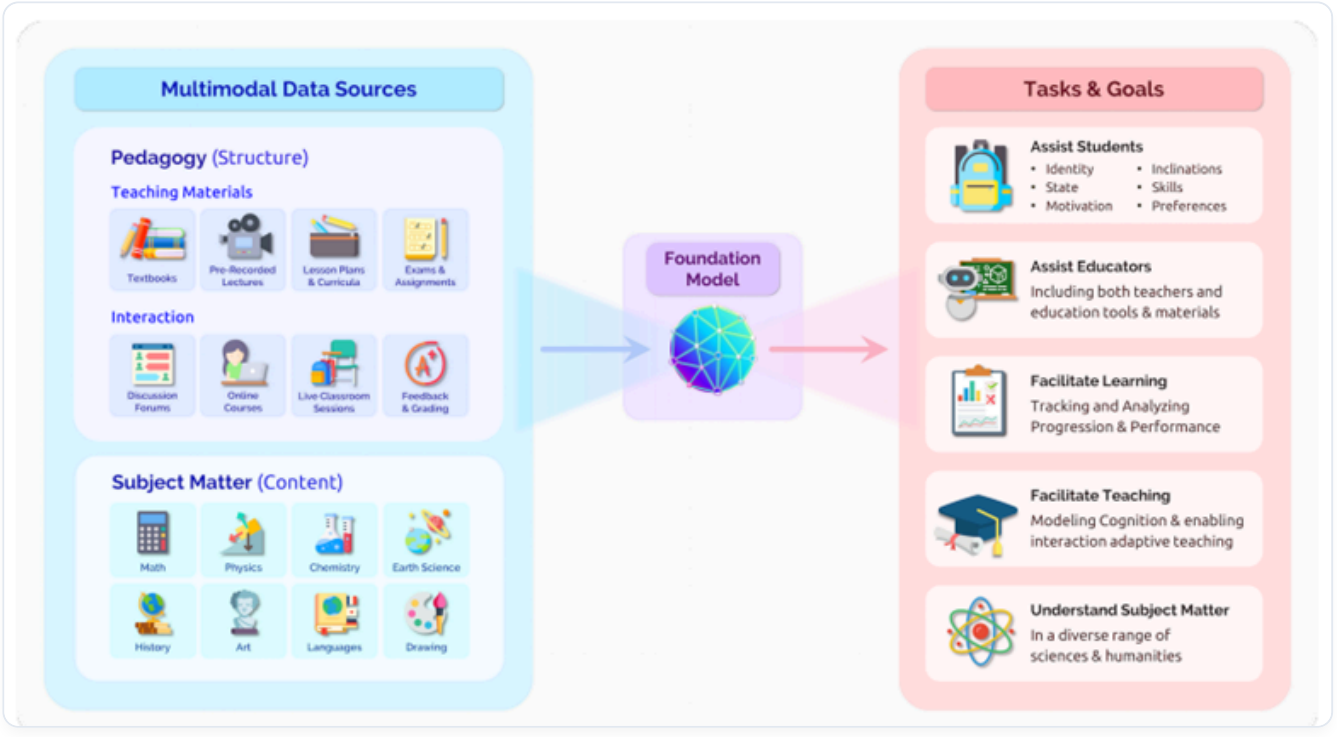
- **با یادگیری بدون نظارت یا خودنظارتی آموزش می‌بینند**، یعنی روی داده‌های چند حالتی بدون برچسب آموزش می‌بینند و نیازی به برچسب‌گذاری انسانی ندارند.
- **مدل‌های بسیار بزرگ هستند**، بر پایه شبکه‌های عصبی بسیار عمیق که روی میلیاردها پارامتر آموزش دیده‌اند.
- **معمولاً به عنوان «بنیاد» برای مدل‌های دیگر در نظر گرفته می‌شوند**، یعنی می‌توان به عنوان نقطه شروع برای ساخت مدل‌های دیگر استفاده کرد، که با تنظیم دقیق میسر می‌شود.



Foundation Models versus LLMs

منبع تصویر: راهنمای ضروری مدل‌های بنیادین و مدل‌های بزرگ زبانی | توسط Babar M Bhatti | Medium

برای روشن‌تر کردن این تفاوت، بیایید ChatGPT را به عنوان نمونه تاریخی بررسی کنیم. نسخه‌های اولیه ChatGPT از GPT-3.5 به عنوان مدل بنیادین استفاده کردند. سپس OpenAI با استفاده از داده‌های خاص چت و تکنیک‌های هم‌راستایی، نسخه‌ای تنظیم شده ایجاد کرد که در سناریوهای گفتگو، مانند چت‌بات‌ها، بهتر عمل می‌کرد. خدمات مدرن هوش مصنوعی اغلب بین چند نسخه مدل مختلف جابه‌جا می‌شوند، بنابراین نام خدمت و نام مدل زیرساختی همواره یکسان نیست.



Foundation Model

منبع تصویر: pdf (arxiv.org).2108.07258

مدل‌های وزن‌باز/متن‌باز در مقابل مدل‌های اختصاصی

روش دیگر برای دسته‌بندی مدل‌های بزرگ زبانی این است که آیا وزن‌باز، متن‌باز یا اختصاصی هستند.

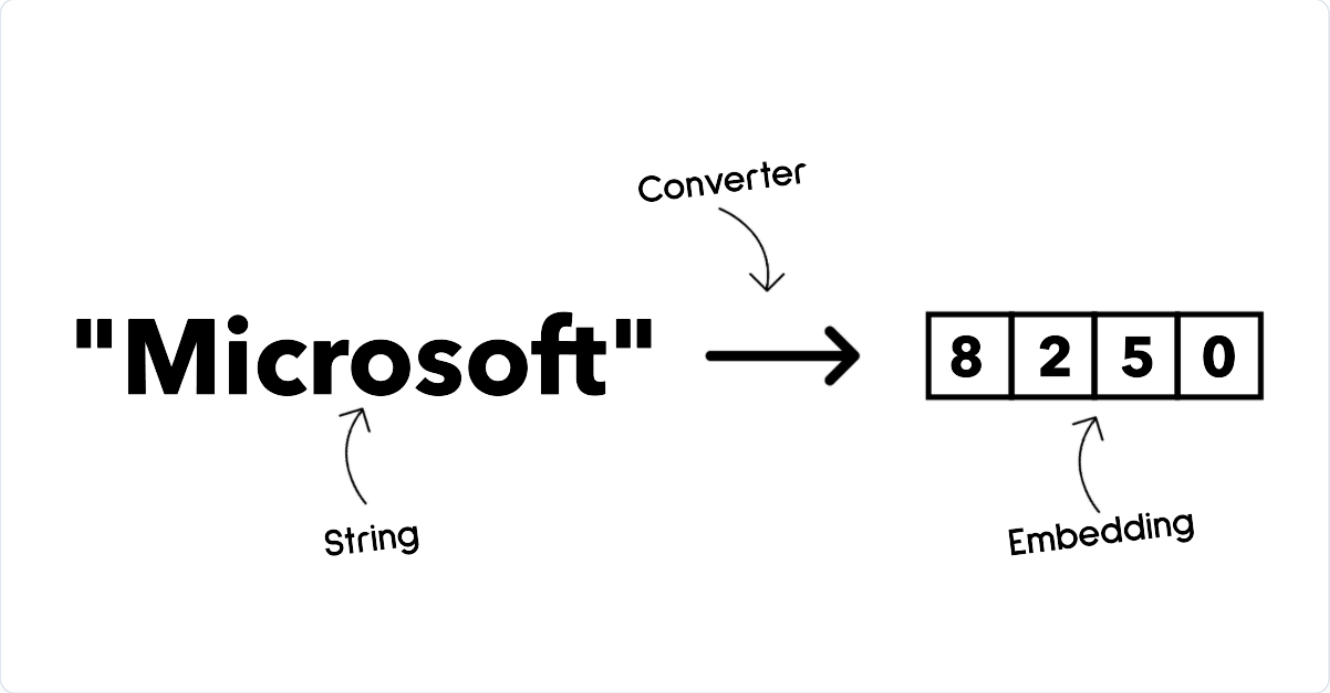
مدل‌های متن‌باز و وزن‌باز امکان دسترسی به آثار مدل برای بررسی، دانلود یا سفارشی‌سازی را فراهم می‌کنند، ولی مجوزهای آن‌ها متفاوت است. برخی کاملاً متن‌باز هستند، در حالی که برخی مدل‌های وزن‌باز با محدودیت‌های استفاده‌اند. این مدل‌ها مفیدند زمانی که کسب‌وکار نیاز به کنترل بیشتر روی استقرار، مکان داده، هزینه یا سفارشی‌سازی دارد. اما تیم‌ها باید قبل از استفاده عواقب مجوز، هزینه‌های سرویس‌دهی، نگهداری، به‌روزرسانی امنیتی و کیفیت ارزیابی را بررسی کنند. نمونه‌ها شامل Meta Lla ma 4، برخی مدل‌های Mistral و بسیاری مدل‌های میزبانی شده روی Hugging Face هستند.

مدل‌های اختصاصی متعلق به و میزبانی شده توسط یک ارائه‌دهنده هستند. این مدل‌ها اغلب برای استفاده مدیریت‌شده در تولید بهینه شده‌اند و می‌توانند پشتیبانی قوی، سیستم‌های ایمنی، ادغام ابزار و مقیاس‌پذیری ارائه دهند. اما معمولاً مشتریان نمی‌توانند وزن‌های مدل را بررسی یا اصلاح کنند و باید شرایط حریم خصوصی، نگهداری، انطباق و استفاده مجاز را بررسی کنند. نمونه‌ها شامل مدل‌های Google Gemini، OpenAI و Anthropic Claude هستند.

تعبیه در مقابل تولید تصویر در مقابل تولید متن و کد

مدل‌های بزرگ زبانی همچنین می‌توانند بر اساس خروجی تولید شده دسته‌بندی شوند.

تعبیه‌ها مجموعه‌ای از مدل‌ها هستند که قادرند متن را به صورت عددی تبدیل کنند، که به آن embedding گفته می‌شود، یعنی نمایشی عددی از متن ورودی. تعبیه‌ها فهم روابط بین کلمات یا جملات را برای ماشین‌ها آسان‌تر می‌کنند و می‌توانند به عنوان ورودی توسط مدل‌های دیگر مانند مدل‌های طبقه‌بندی یا خوشه‌بندی که عملکرد بهتری روی داده‌های عددی دارند، مصرف شوند. مدل‌های تعبیه معمولاً برای یادگیری انتقالی استفاده می‌شوند، جایی که مدل برای کاری موقت ساخته می‌شود که داده زیادی برای آن وجود دارد، سپس وزن‌های مدل (embeddingها) برای وظایف پایین‌دستی دیگر استفاده می‌شوند. نمونه‌ای از این دسته تعبیه‌های OpenAI است.



Embedding

مدل‌های تولید تصویر، مدل‌هایی هستند که تصاویر تولید می‌کنند. این مدل‌ها اغلب برای ویرایش تصویر، سنتز تصویر و ترجمه تصویر استفاده می‌شوند. مدل‌های تولید تصویر اغلب روی مجموعه داده‌های بزرگ تصاویر مانند LAION-5B آموزش دیده‌اند و می‌توانند تصاویر جدید تولید کنند یا با تکنیک‌هایی مثل super-resolution، inpainting و colorization تصاویر موجود را ویرایش کنند. نمونه‌ها شامل مدل‌های GPT Image، مدل‌های Stable Diffusion و مدل‌های Imagen هستند.

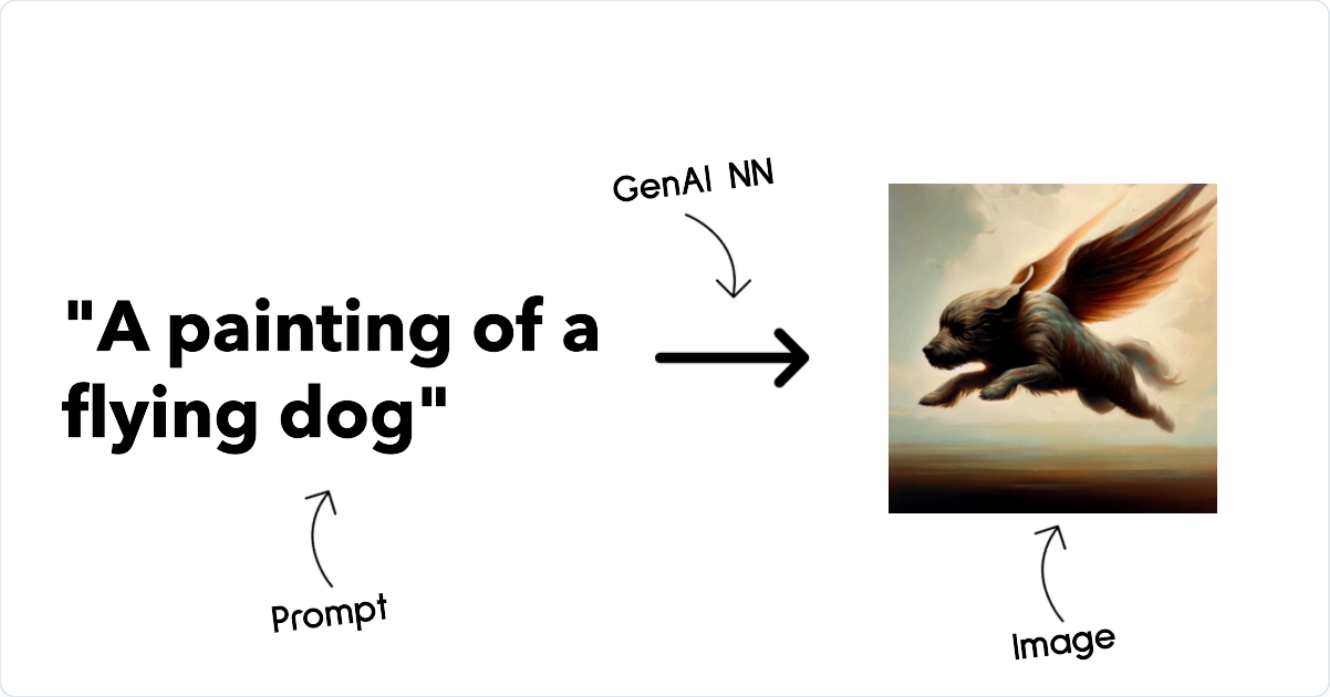
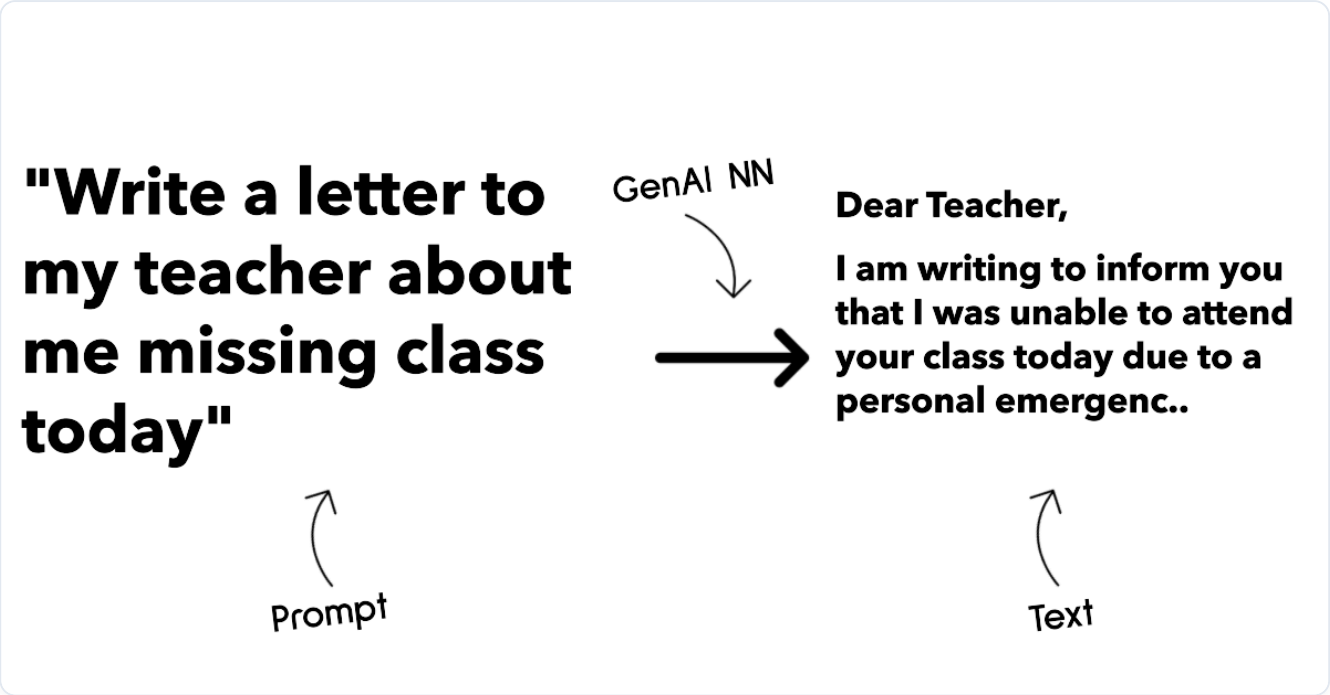


Image generation

مدل‌های تولید متن و کد، مدل‌هایی هستند که متن یا کد تولید می‌کنند. این مدل‌ها اغلب برای خلاصه‌سازی متن، ترجمه و پاسخ به سؤالات استفاده می‌شوند. مدل‌های تولید متن اغلب روی مجموعه داده‌های بزرگ متنی مانند BookCorpus آموزش دیده‌اند و می‌توانند متن جدید تولید کنند یا به سؤالات پاسخ دهند. مدل‌های تولید کد، مانند CodeParrot، معمولاً روی مجموعه داده‌های بزرگ کد مانند GitHub آموزش می‌بینند و می‌توانند کد جدید تولید کنند یا باگ‌های موجود را رفع کنند.



Text and code generation

مدل‌های کُدگذار-رمزگشا در مقابل فقط رمزگشا

برای صحبت درباره انواع معماری مدل‌های بزرگ زبانی، از یک قیاس استفاده کنیم.

تصور کنید مدیر شما وظیفه‌ای برای نوشتن یک آزمون برای دانش‌آموزان به شما داده است. دو همکار دارید؛ یکی مسئول ایجاد محتوا و دیگری مسئول بازبینی آن است.

سازنده محتوا مانند مدل فقط رمزگشا است: می‌تواند موضوع را ببیند، آنچه نوشته‌اید را مرور کند و سپس بر اساس آن متن تولید کند. آنها در نوشتن محتوای جذاب و آموزنده بسیار خوب هستند، اما بهترین انتخاب برای وظایفی مانند دسته‌بندی، بازیابی یا کدگذاری نیستند. خانواده مدل‌های فقط رمزگشا شامل مدل‌های GPT و Llama است.

بازبین مانند مدل فقط کُدگذار است؛ آنها به متن نوشته شده و پاسخ‌ها نگاه می‌کنند، رابطه بین آنها را مشاهده کرده و زمینه را درک می‌کنند، اما در تولید محتوا خوب نیستند. نمونه‌ای از مدل فقط کُدگذار، BERT است.

تصور کنید فردی باشد که هم بتواند آزمون را ایجاد و هم بازبینی کند، این یک مدل کُدگذار-رمزگشا است. نمونه‌هایی مانند BART و T5 وجود دارند.

سرویس در مقابل مدل

حالا بیایید تفاوت سرویس و مدل را توضیح دهیم. سرویس محصولی است که توسط یک ارائه‌دهنده خدمات ابری ارائه می‌شود و معمولاً ترکیبی از مدل‌ها، داده‌ها و اجزای دیگر است. مدل بخش اصلی یک سرویس است و معمولاً مدل بنیادین مانند یک مدل بزرگ زبانی است.

سرویس‌ها معمولاً برای استفاده تولیدی بهینه شده و استفاده از آن‌ها از طریق رابط کاربری گرافیکی آسان‌تر است. با این حال، خدمات همیشه رایگان نیستند و ممکن است به اشتراک یا پرداخت هزینه نیاز داشته باشند، که در عوض از تجهیزات و منابع مالک خدمت استفاده می‌کنند، هزینه‌ها را بهینه کرده و مقیاس‌پذیری آسان را فراهم می‌کنند. نمونه‌ای از سرویس، Azure Ope nAI Service است که برنامه نرخ پرداخت به اندازه مصرف را ارائه می‌دهد، یعنی کاربران بر اساس میزان استفاده هزینه پرداخت می‌کنند. این سرویس همچنین امنیت سازمانی و چارچوب هوش مصنوعی مسئولانه را در کنار قابلیت‌های مدل‌ها فراهم می‌کند.

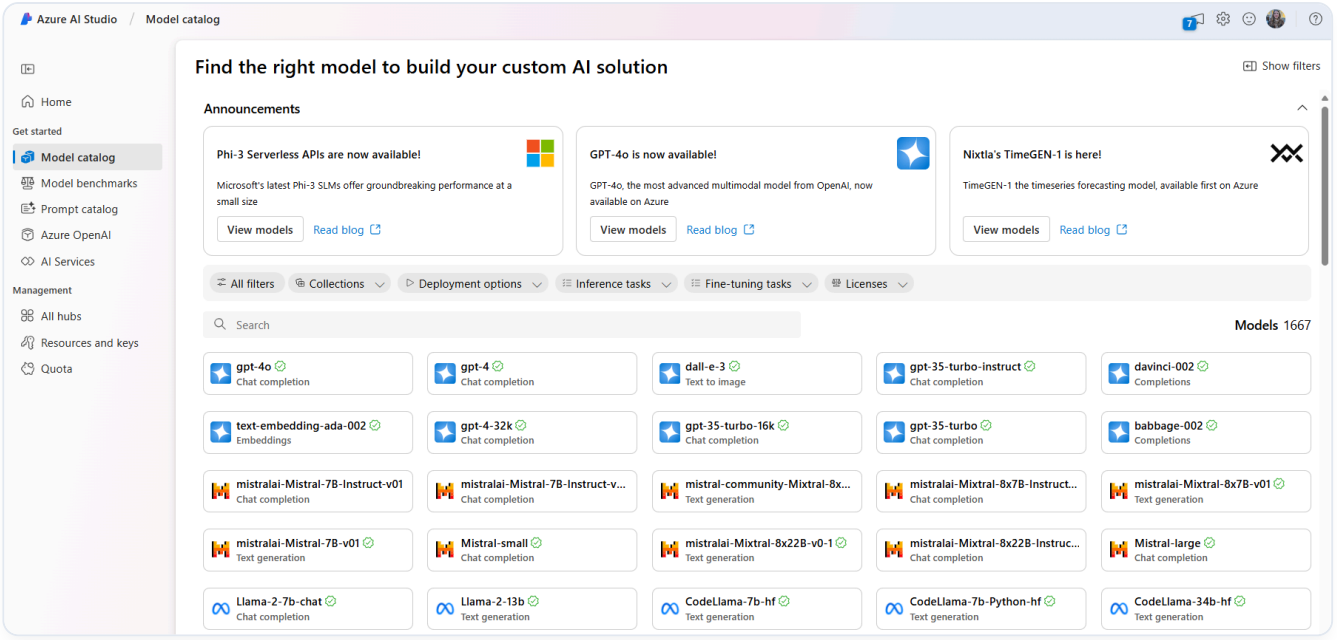
مدل‌ها مصنوعات شبکه عصبی هستند: پارامترها، وزن‌ها، معماری، توکنایزر و پیکربندی پشتیبان. اجرای مدل به صورت محلی یا در محیط خصوصی به سخت‌افزار مناسب، زیرساخت سرویس‌دهی، نظارت و مجوز متن‌باز/وزن‌باز سازگار یا مجوز تجاری نیاز دارد. مدل‌های وزن‌باز مانند Llama 4 یا مدل‌های Mistral می‌توانند خودمیزبانی شوند، اما همچنان به قدرت محاسباتی و تخصص عملیاتی نیاز دارند.

چگونه با مدل‌های مختلف آزمایش و تکرار کنیم تا عملکرد را در Azure درک کنیم

هنگامی که تیم ما چشم‌انداز فعلی مدل‌های زبان بزرگ (LLM) را بررسی کرده و برخی از گزینه‌های مناسب برای سناریوهایشان را شناسایی کرد، گام بعدی آزمایش آن‌ها روی داده‌ها و بار کاری خودشان است. این یک فرایند تکراری است که از طریق آزمایش‌ها و اندازه‌گیری‌ها انجام می‌شود. بیشتر مدل‌هایی که در بندهای قبلی ذکر کردیم (مدل‌های OpenAI، مدل‌های با وزن باز مانند Llama 4 و Mistral، و مدل‌های Hugging Face) در Microsoft Foundry Models در دسترس هستند.

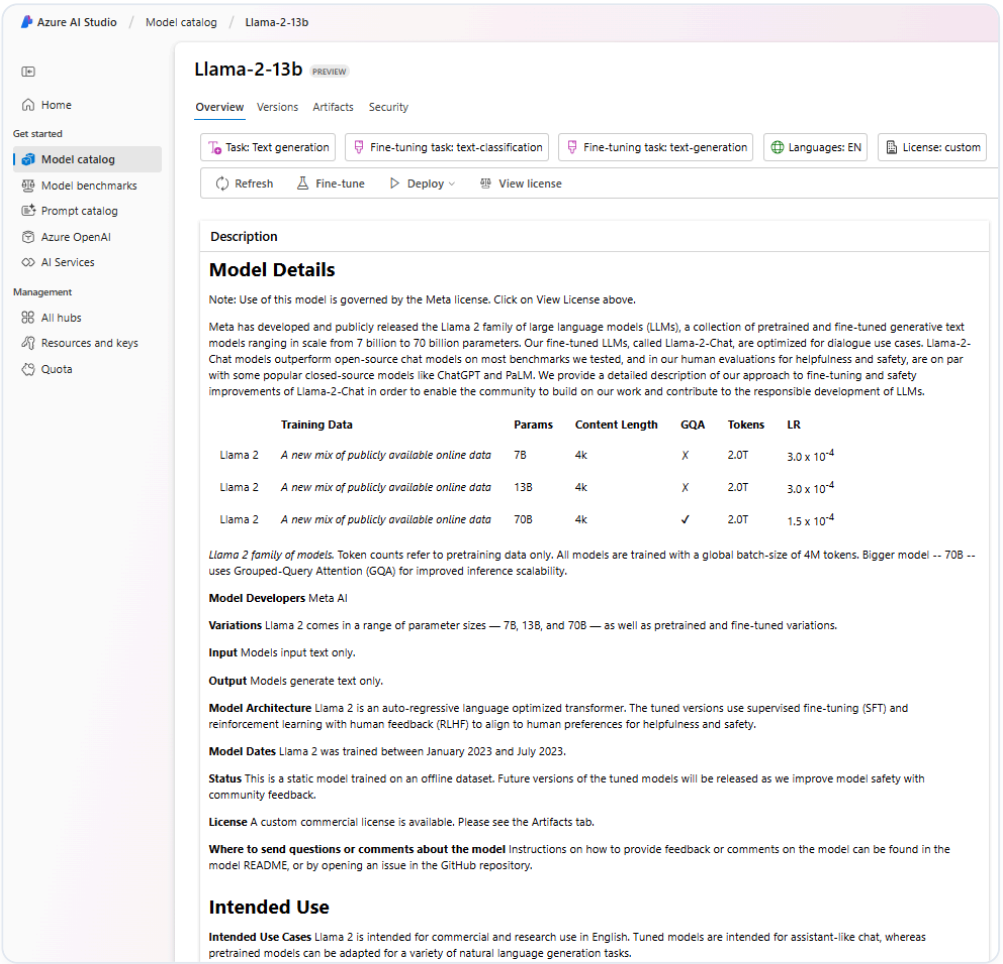
Microsoft Foundry، که قبلاً Azure AI Studio/Azure AI Foundry نام داشت، یک پلتفرم متحد Azure برای ساخت برنامه‌ها و عوامل هوش مصنوعی است. این پلتفرم به توسعه‌دهندگان کمک می‌کند چرخه عمر را از آزمایش و ارزیابی تا استقرار، مانیتورینگ و حاکمیت مدیریت کنند. فهرست مدل‌ها در Microsoft Foundry به کاربر امکان می‌دهد:

- مدل بنیادین مورد علاقه را در فهرست بیابد، شامل مدل‌هایی که توسط Azure فروخته می‌شوند و مدل‌هایی از شرکا و ارائه‌دهندگان جامعه. کاربران می‌توانند بر اساس کار، ارائه‌دهنده، مجوز، گزینه استقرار یا نام فیلتر کنند.



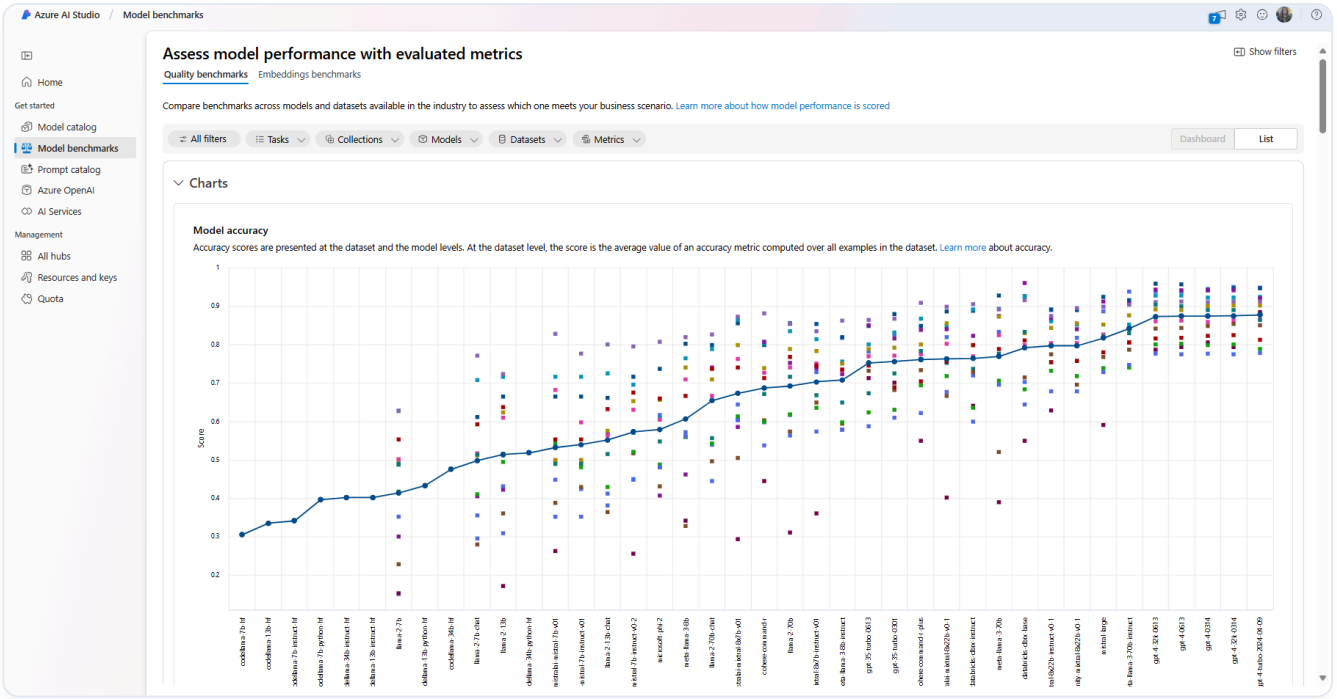
Model catalog

- کارت مدل را مرور کند، شامل توضیح مفصل در مورد استفاده مورد نظر و داده‌های آموزش، نمونه‌های کد و نتایج ارزیابی در کتابخانه ارزیابی داخلی.



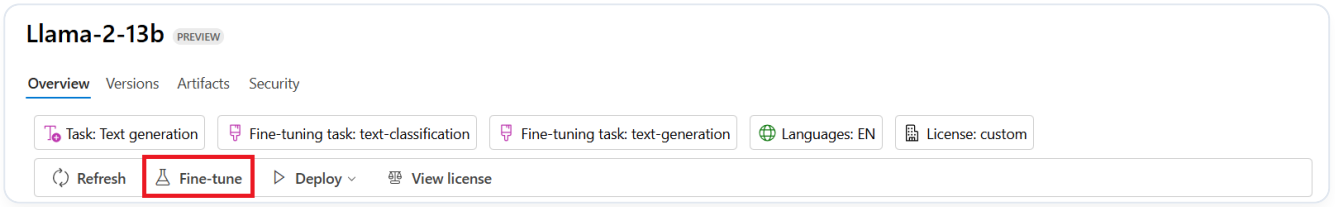
Model card

- معیارهای عملکرد را در بین مدل‌ها و مجموعه داده‌های موجود در صنعت برای ارزیابی اینکه کدام یک مناسب سناریوی کسب‌وکار است، از طریق پنل **Model Benchmarks** مقایسه کند.



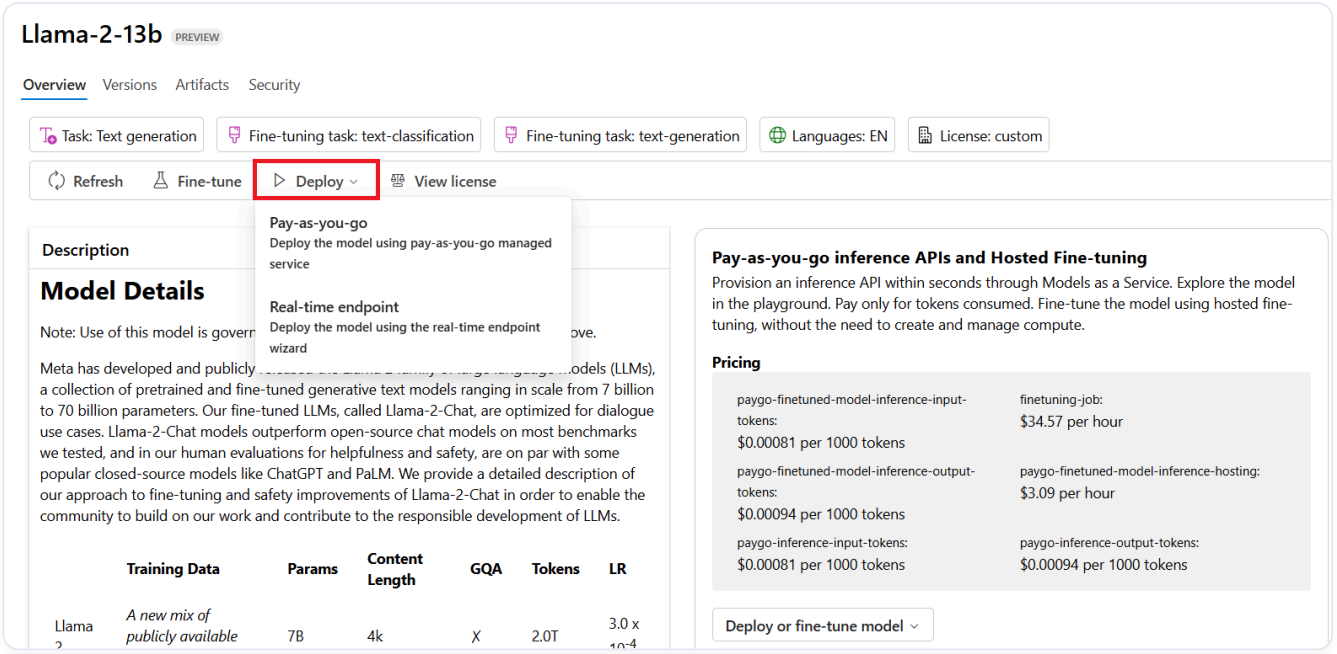
Model benchmarks

- مدل‌های پشتیبانی شده را روی داده‌های آموزشی سفارشی برای بهبود عملکرد مدل در بار کاری خاص ریزتنظیم کند، با استفاده از قابلیت‌های آزمایش و پیگیری Microsoft Foundry.



Model fine-tuning

- مدل پیش آموزش دیده اصلی یا نسخه ریزتنظیم شده را روی یک انتهای استنتاج بلادرنگ راه‌دور، با استفاده از گزینه‌های محاسبات مدیریت شده یا استقرار بدون سرور، مستقر کند تا برنامه‌ها بتوانند از آن استفاده کنند.



Model deployment

> **نکته کاربردی**
>
> همه مدل‌های موجود در فهرست در حال حاضر برای ریزتنظیم و/یا استقرار پرداخت به ازای استفاده در دسترس نیستند. برای جزئیات در مورد قابلیت‌ها و محدودیت‌های مدل، کارت مدل را بررسی کنید.

بهبود نتایج LLM

ما با تیم استارتاپ خود انواع مختلفی از LLMها و یک پلتفرم ابری (Microsoft Foundry) را بررسی کردیم که به ما اجازه می‌دهد مدل‌های مختلف را مقایسه کرده، آن‌ها را روی داده‌های آزمایشی ارزیابی کنیم، عملکرد را بهبود بخشیم و مدل‌ها را روی نقاط استنتاج مستقر کنیم.

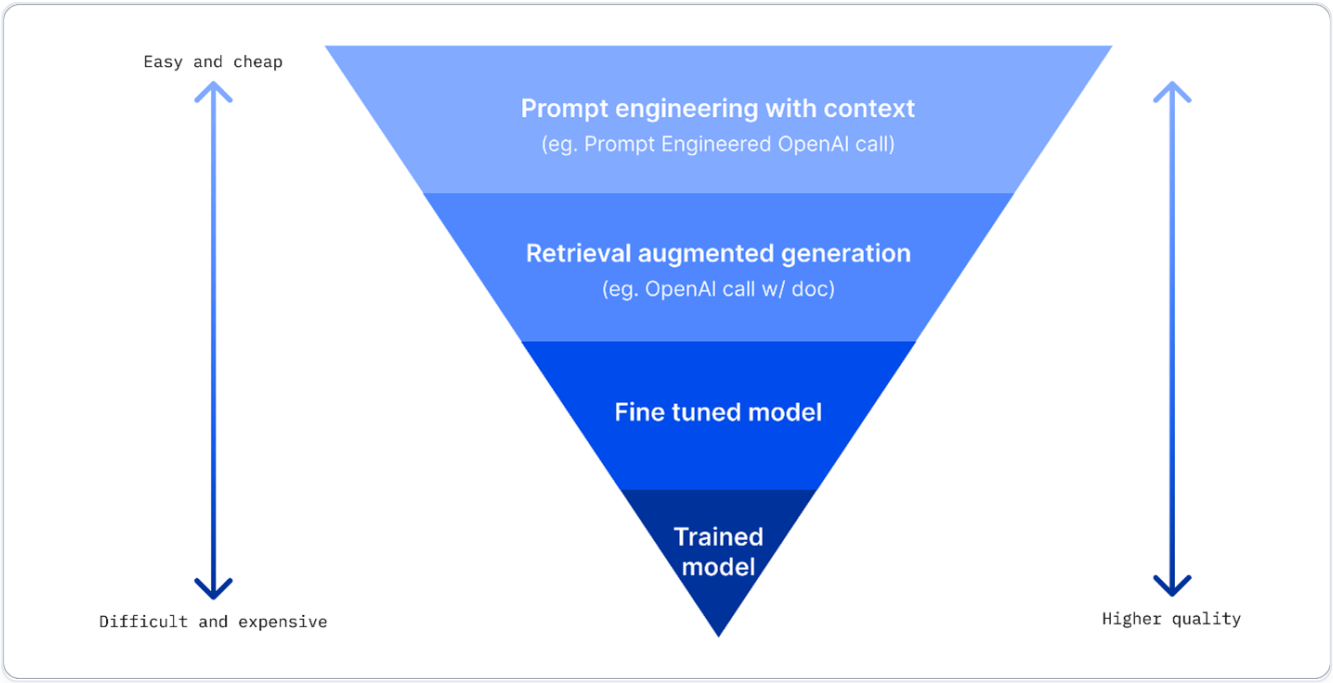
اما چه زمانی باید به جای استفاده از یک مدل پیش‌آموزش‌دیده، به ریزتنظیم مدل فکر کنند؟ آیا رویکردهای دیگری برای بهبود عملکرد مدل در بار کاری خاص وجود دارد؟

کسب‌وکارها می‌توانند از چندین رویکرد برای رسیدن به نتایج مورد نیاز خود از یک LLM استفاده کنند. می‌توانید انواع مختلفی از مدل‌ها را با درجات متفاوت آموزش هنگام استقرار LLM در تولید انتخاب کنید، با سطوح مختلفی از پیچیدگی، هزینه و کیفیت. در اینجا چند رویکرد متفاوت آمده است:

- **مهندسی پرامپت با زمینه**. ایده این است که هنگام دادن پرامپت زمینه کافی ارائه دهید تا پاسخ‌های مورد نیاز دریافت شود.
- **تولید تقویت شده با بازیابی، RAG**. داده‌های شما ممکن است در یک پایگاه داده یا نقطه انتهایی وب موجود باشد، برای اطمینان از اینکه این داده‌ها یا زیرمجموعه‌ای از آن‌ها در هنگام پرامپت وارد می‌شوند، می‌توانید داده‌های مرتبط را

بازیابی کرده و آن را بخشی از پرامپت کاربر کنید.

- **مدل ریزتنظیم شده**. در این حالت، مدل را بیشتر با داده‌های خودتان آموزش داده‌اید که باعث می‌شود مدل دقیق‌تر و پاسخگوتر به نیازهای شما باشد، اما شاید هزینه‌بر باشد.



LLMs deployment

منبع تصویر: چهار روش که شرکت‌ها LLMها را مستقر می‌کنند | وبلاگ Fiddler AI

مهندسی پرامپت با زمینه

LLMهای پیش‌آموزش‌دیده در کارهای زبان طبیعی عمومی بسیار خوب عمل می‌کنند، حتی وقتی فقط با یک پرامپت کوتاه، مانند جمله‌ای برای تکمیل یا سوالی — که به آن «یادگیری صفر-شات» گفته می‌شود — فرا خوانده شوند.

با این حال، هرچه کاربر بتواند پرسش خود را با درخواست دقیق و مثال‌ها — زمینه — بهتر قالب‌بندی کند، پاسخ دقیق‌تر و نزدیک‌تر به انتظار کاربر خواهد بود. در این حالت، اگر پرامپت تنها یک مثال داشته باشد «یادگیری یک-شات» و اگر چندین مثال داشته باشد «یادگیری چند-شات» نامیده می‌شود. مهندسی پرامپت با زمینه مقرون‌به‌صرفه‌ترین رویکرد برای شروع است.

تولید تقویت شده با بازیابی (RAG)

LLMها این محدودیت را دارند که فقط می‌توانند از داده‌هایی که در طول آموزششان استفاده شده برای تولید پاسخ استفاده کنند. این بدان معنی است که آن‌ها چیزی درباره واقعیت‌هایی که پس از فرایند آموزش رخ داده نمی‌دانند و نمی‌توانند به اطلاعات غیرعمومی (مانند داده‌های شرکت) دسترسی داشته باشند. این مسئله را می‌توان با RAG برطرف کرد، تکنیکی که پرامپت را با داده‌های خارجی به شکل بخش‌هایی از اسناد گسترش می‌دهد، با در نظر گرفتن محدودیت طول پرامپت. این توسط ابزارهای پایگاه داده برداری (مانند Azure Vector Search) پشتیبانی می‌شود که بخش‌های مفید را از منابع داده‌های پیش‌تعریف‌شده مختلف بازیابی کرده و به زمینه پرامپت اضافه می‌کنند.

این تکنیک برای کسب‌وکاری که داده، زمان یا منابع کافی برای ریزتنظیم LLM ندارد اما همچنان می‌خواهد عملکرد را در یک بار کاری خاص بهبود دهد و ریسک پاسخ‌های هذیانی، قدیمی یا بدون پشتیبانی را کاهش دهد، بسیار مفید است.

مدل ریزتنظیم شده

ریزتنظیم فرآیندی است که از یادگیری انتقالی برای «سازگار کردن» مدل با یک کار پایین‌دستی یا حل یک مسئله خاص استفاده می‌کند. بر خلاف یادگیری چند-شات و RAG، منجر به تولید یک مدل جدید با وزن‌ها و بایاس‌های به‌روز شده می‌شود. این فرایند نیازمند مجموعه‌ای از نمونه‌های آموزشی است که شامل ورودی واحد (پرامپت) و خروجی مرتبط آن (تکمیل) است. این رویکرد ترجیح داده می‌شود اگر:

- **استفاده از مدل‌های کوچکتر مخصوص کار**. کسب‌وکار می‌خواهد مدل کوچکتری را برای یک کار محدود ریزتنظیم کند تا به صورت مکرر از یک مدل بزرگتر پرامپت بگیرد که این منجر به راه‌حل مقرون‌به‌صرفه‌تر و سریع‌تری می‌شود.
- **در نظر گرفتن تأخیر**. تأخیر برای یک مورد استفاده خاص مهم است، بنابراین نمی‌توان از پرامپت‌های بسیار طولانی یا تعداد مثال‌هایی استفاده کرد که مدل باید از آنها یاد بگیرد و این تعداد با محدودیت طول پرامپت سازگار نیست.
- **سازگاری رفتار پایدار**. کسب‌وکار تعداد زیادی مثال با کیفیت بالا دارد و می‌خواهد مدل یک الگوی کاری، قالب خروجی، سبک لحن یا حوزه تخصصی را به طور مداوم دنبال کند. اگر مشکل اصلی، اطلاعات تازه یا دانش خصوصی است که اغلب تغییر می‌کند، به جای تنها تکیه بر ریزتنظیم، از RAG استفاده کنید.

مدل آموزش‌دیده

آموزش یک LLM از ابتدا بدون شک سخت‌ترین و پیچیده‌ترین رویکرد است که نیازمند حجم عظیمی از داده، منابع ماهر و قدرت محاسباتی مناسب است. این گزینه فقط در شرایطی باید در نظر گرفته شود که کسب‌وکار مورد استفاده‌ای خاص حوزه داشته باشد و مقدار زیادی داده متمرکز بر حوزه در دسترس باشد.

بررسی دانش

چه رویکردی می‌تواند روشی خوب برای بهبود نتایج تکمیل LLM باشد؟

1. مهندسی پرامپت با زمینه

2. RAG

3. مدل ریزتنظیم شده

پاسخ: هر سه می‌توانند کمک کنند. با مهندسی پرامپت و زمینه برای بهبود سریع شروع کنید و وقتی مدل به واقعیت‌های جاری یا داده خصوصی کسب‌وکار نیاز دارد از RAG استفاده کنید. ریزتنظیم را انتخاب کنید وقتی که نمونه‌های با کیفیت کافی دارید و می‌خواهید مدل به طور مداوم یک کار، قالب، لحن یا الگوی حوزه را دنبال کند.

🚀 چالش

بیشتر بخوانید درباره اینکه چگونه می‌توانید از RAG استفاده کنید برای کسب‌وکار خود.

کار عالی، یادگیری خود را ادامه دهید

پس از تکمیل این درس، مجموعه Generative AI Learning ما را بررسی کنید تا دانش خود در زمینه هوش مصنوعی مولد را ارتقا دهید!

به درس ۳ بروید جایی که به بررسی ساخت مسئولانه با هوش مصنوعی مولد می‌پردازیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

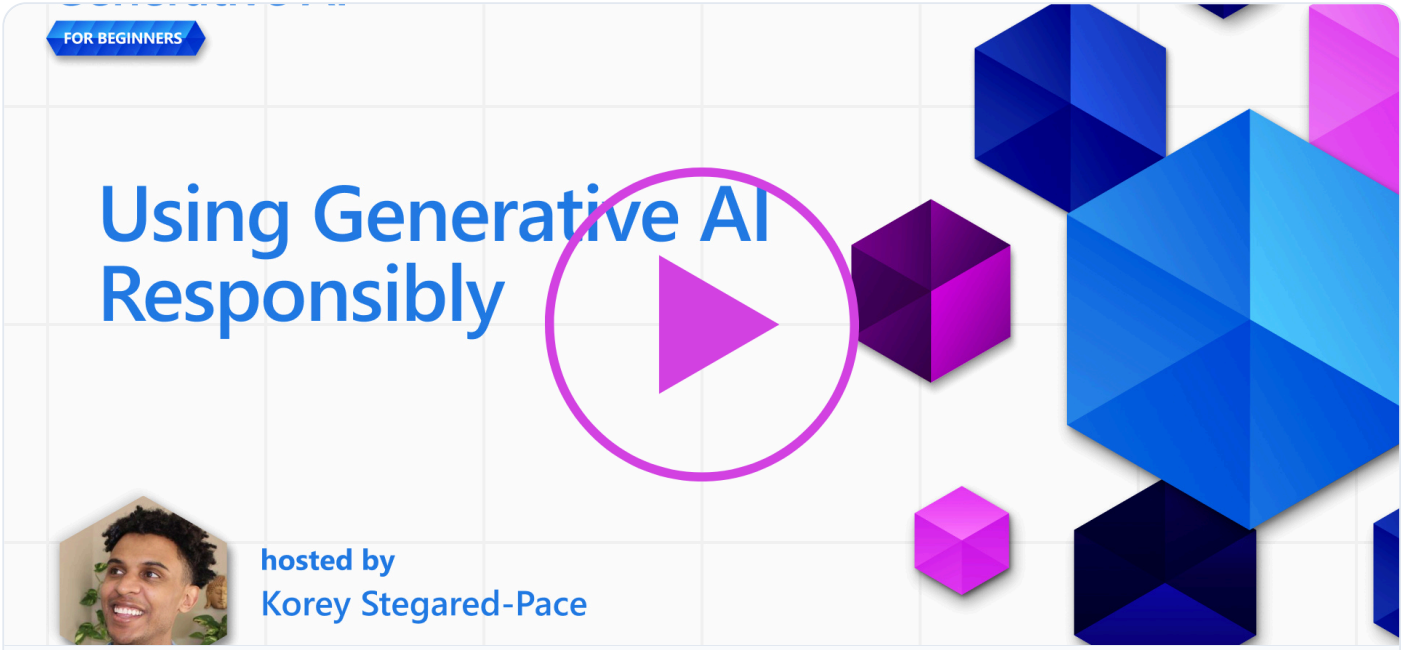
بازگشت به فهرست مطالب

درس ۰۲ — هوش مصنوعی مولد مایکروسافت

درس ۰۳ از ۲۱

استفاده مسئولانه و اخلاقی از هوش مصنوعی مولد

اصول هوش مصنوعی مسئولانه مایکروسافت، کاهش سوگیری و تعصب الگوریتمی، شفافیت، و جلوگیری از توهم (Hallucination)



🎥 ویدیوی رسمی آموزش این درس

📱 برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/YOp-e1GjZdA?si=7Wv4wu3x44L1DCVj

▶️ پخش در یوتیوب

اسکن کد QR

💡 نکته کاربردی

برای مشاهده ویدئوی این درس روی تصویر بالا کلیک کنید

هوش مصنوعی، به‌ویژه هوش مصنوعی تولیدی، می‌تواند بسیار جذاب باشد، اما باید در نظر داشته باشید که چگونه می‌توانید از آن به‌صورت مسئولانه استفاده کنید. باید به مواردی مانند اطمینان از خروجی منصفانه، غیرمضر و موارد دیگر توجه کنید. این فصل قصد دارد زمینه‌های ذکر شده، مواردی که باید در نظر گرفته شوند و نحوه برداشتن گام‌های فعال برای بهبود استفاده از هوش مصنوعی را به شما ارائه دهد.

مقدمه

این درس شامل موارد زیر خواهد بود:

- چرا باید هنگام ساخت برنامه‌های هوش مصنوعی تولیدی، اولویت را به هوش مصنوعی مسئولانه بدهید.
- اصول اصلی هوش مصنوعی مسئولانه و ارتباط آن‌ها با هوش مصنوعی تولیدی.
- نحوه به‌کارگیری این اصول هوش مصنوعی مسئولانه از طریق استراتژی و ابزارها.

اهداف یادگیری

پس از اتمام این درس، شما خواهید دانست:

- اهمیت هوش مصنوعی مسئولانه هنگام ساخت برنامه‌های هوش مصنوعی تولیدی.
- زمان فکر کردن و اعمال اصول اصلی هوش مصنوعی مسئولانه هنگام ساخت برنامه‌های هوش مصنوعی تولیدی.
- چه ابزارها و استراتژی‌هایی برای به‌کارگیری مفهوم هوش مصنوعی مسئولانه در دسترس شما هستند.

اصول هوش مصنوعی مسئولانه

هیجان پیرامون هوش مصنوعی تولیدی هیچ‌گاه به این اندازه بالا نبوده است. این هیجان باعث جذب تعداد زیادی توسعه‌دهنده جدید، توجه و سرمایه‌گذاری به این حوزه شده است. در حالی که این موضوع برای هر کسی که به دنبال ساخت محصولات و شرکت‌ها با استفاده از هوش مصنوعی تولیدی است بسیار مثبت است، اما مهم است که با مسئولیت پیش برویم.

در طول این دوره، ما بر ساخت استارتاپ و محصول آموزشی هوش مصنوعی خود تمرکز می‌کنیم. از اصول هوش مصنوعی مسئولانه استفاده خواهیم کرد: انصاف، شمول، قابلیت اطمینان/ایمنی، امنیت و حریم خصوصی، شفافیت و پاسخگویی. با این اصول، بررسی خواهیم کرد که چگونه این اصول با استفاده ما از هوش مصنوعی تولیدی در محصولاتمان مرتبط هستند.

چرا باید به هوش مصنوعی مسئولانه اولویت دهید

هنگام ساخت یک محصول، اتخاذ رویکردی انسان‌محور با در نظر گرفتن بهترین منافع کاربران، به بهترین نتایج منجر می‌شود.

ویژگی منحصر به فرد هوش مصنوعی تولیدی قدرت آن در ایجاد پاسخ‌های مفید، اطلاعات، راهنمایی و محتوا برای کاربران است. این کار می‌تواند بدون مراحل دستی زیاد انجام شود که منجر به نتایج بسیار چشمگیر می‌شود. بدون برنامه‌ریزی و استراتژی‌های مناسب، متأسفانه ممکن است به نتایج مضر برای کاربران، محصول شما و جامعه به‌طور کلی منجر شود.

بیایید به برخی از این نتایج بالقوه مضر نگاه کنیم:

توهمات

توهمات اصطلاحی است که برای توصیف زمانی استفاده می‌شود که یک مدل زبان بزرگ (LLM) محتوایی تولید می‌کند که یا کاملاً بی‌معنی است یا چیزی که می‌دانیم بر اساس منابع دیگر اطلاعاتی نادرست است.

به‌عنوان مثال، فرض کنید ما یک ویژگی برای استارتاپ خود ایجاد می‌کنیم که به دانش‌آموزان اجازه می‌دهد سوالات تاریخی از یک مدل بپرسند. یک دانش‌آموز سوالی مانند `چه کسی تنها بازمانده کشتی تایتانیک بود؟` می‌پرسد.

مدل پاسخی مانند زیر تولید می‌کند:

> Who was the sole survivor of Titanic?
>
> The sole survivor of the Titanic disaster was Charles Joughin. He was a British baker and the chief baker aboard the Titanic. Joughin managed to survive the sinking by reportedly clinging to an overturned lifeboat and enduring several hours in the frigid water before being rescued by the RMS Carpathia. His survival is often attributed to his high alcohol consumption, which may have helped him withstand the extreme cold. Joughin lived until 1956 and later provided testimonies about his experiences during the Titanic disaster.

> 💡 **نکته کاربردی**
>
> *(منبع: Flying bisons)*

این پاسخ بسیار مطمئن و کامل است. متأسفانه، این پاسخ نادرست است. حتی با حداقل تحقیق، می‌توان دریافت که بیش از یک بازمانده از فاجعه تایتانیک وجود داشته است. برای دانش‌آموزی که تازه شروع به تحقیق در این موضوع کرده است، این پاسخ می‌تواند به اندازه کافی قانع‌کننده باشد که مورد سوال قرار نگیرد و به‌عنوان حقیقت پذیرفته شود. پیامدهای این موضوع می‌تواند منجر به غیرقابل اعتماد بودن سیستم هوش مصنوعی و تأثیر منفی بر شهرت استارتاپ ما شود.

با هر نسخه جدید از یک مدل زبان بزرگ، شاهد بهبود عملکرد در کاهش توهمات بوده‌ایم. حتی با این بهبود، ما به‌عنوان سازندگان و کاربران برنامه‌ها همچنان باید از این محدودیت‌ها آگاه باشیم.

محتوای مضر

در بخش قبلی پوشش دادیم که زمانی که یک مدل زبان بزرگ پاسخ‌های نادرست یا بی‌معنی تولید می‌کند. خطر دیگری که باید از آن آگاه باشیم زمانی است که یک مدل با محتوای مضر پاسخ می‌دهد.

محتوای مضر می‌تواند به‌صورت زیر تعریف شود:

- ارائه دستورالعمل‌ها یا تشویق به خودآزاری یا آسیب به گروه‌های خاص.
- محتوای نفرت‌انگیز یا تحقیرآمیز.
- راهنمایی برای برنامه‌ریزی هر نوع حمله یا اعمال خشونت‌آمیز.
- ارائه دستورالعمل‌هایی برای یافتن محتوای غیرقانونی یا ارتکاب اعمال غیرقانونی.
- نمایش محتوای جنسی صریح.

برای استارتاپ ما، می‌خواهیم مطمئن شویم که ابزارها و استراتژی‌های مناسبی برای جلوگیری از دیده شدن این نوع محتوا توسط دانش‌آموزان داریم.

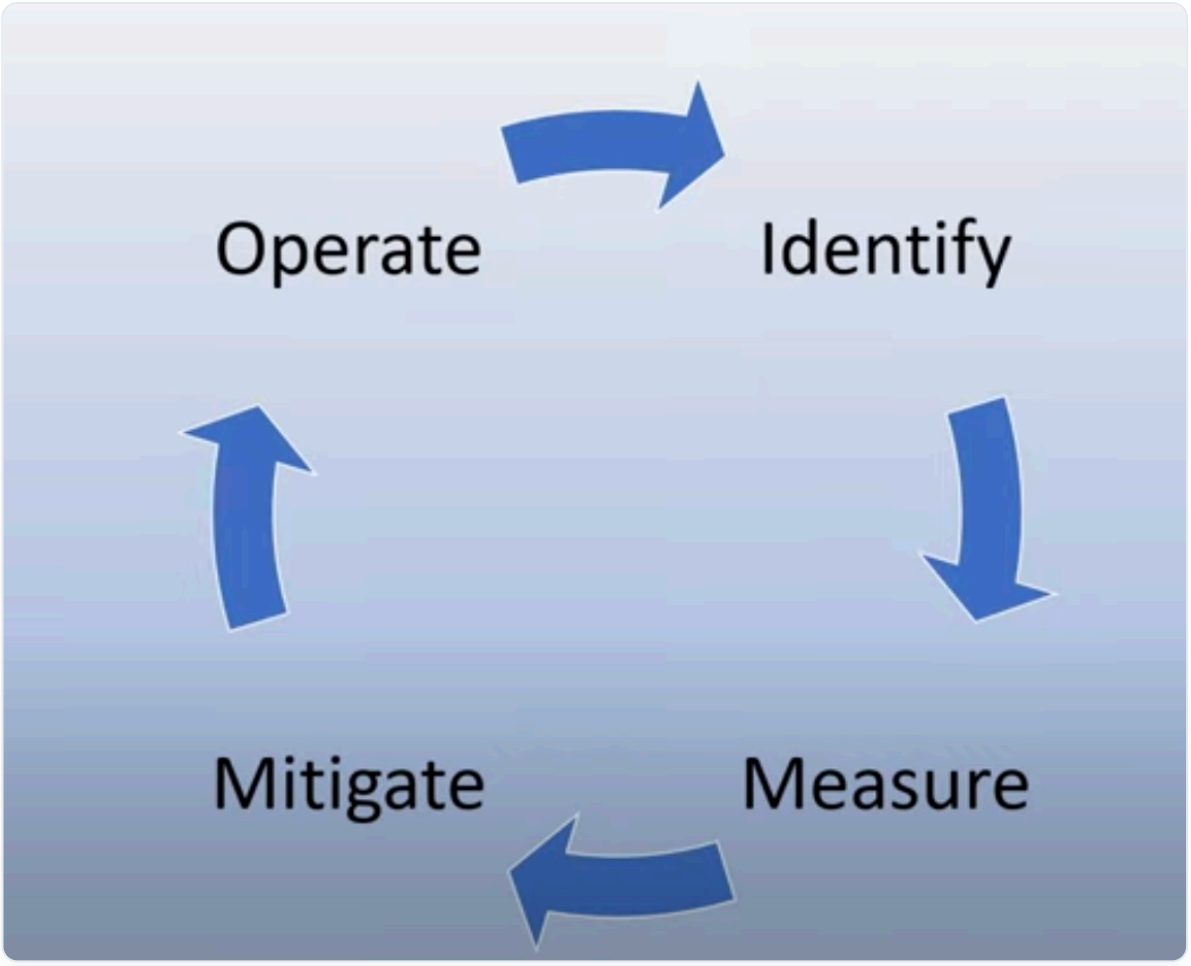
عدم انصاف

انصاف به‌عنوان "اطمینان از اینکه یک سیستم هوش مصنوعی عاری از تعصب و تبعیض است و همه را به‌طور منصفانه و برابر رفتار می‌کند" تعریف می‌شود. در دنیای هوش مصنوعی تولیدی، می‌خواهیم مطمئن شویم که دیدگاه‌های انحصاری گروه‌های به حاشیه رانده شده توسط خروجی مدل تقویت نمی‌شود.

این نوع خروجی‌ها نه تنها برای ایجاد تجربه‌های مثبت محصول برای کاربران ما مخرب هستند، بلکه باعث آسیب بیشتر به جامعه نیز می‌شوند. به‌عنوان سازندگان برنامه‌ها، باید همیشه یک پایگاه کاربری گسترده و متنوع را در نظر داشته باشیم هنگام ساخت راه‌حل‌ها با هوش مصنوعی تولیدی.

چگونه از هوش مصنوعی تولیدی به‌صورت مسئولانه استفاده کنیم

اکنون که اهمیت هوش مصنوعی تولیدی مسئولانه را شناسایی کرده‌ایم، بیایید به ۴ مرحله‌ای که می‌توانیم برای ساخت راه‌حل‌های هوش مصنوعی خود به‌صورت مسئولانه برداریم، نگاه کنیم:



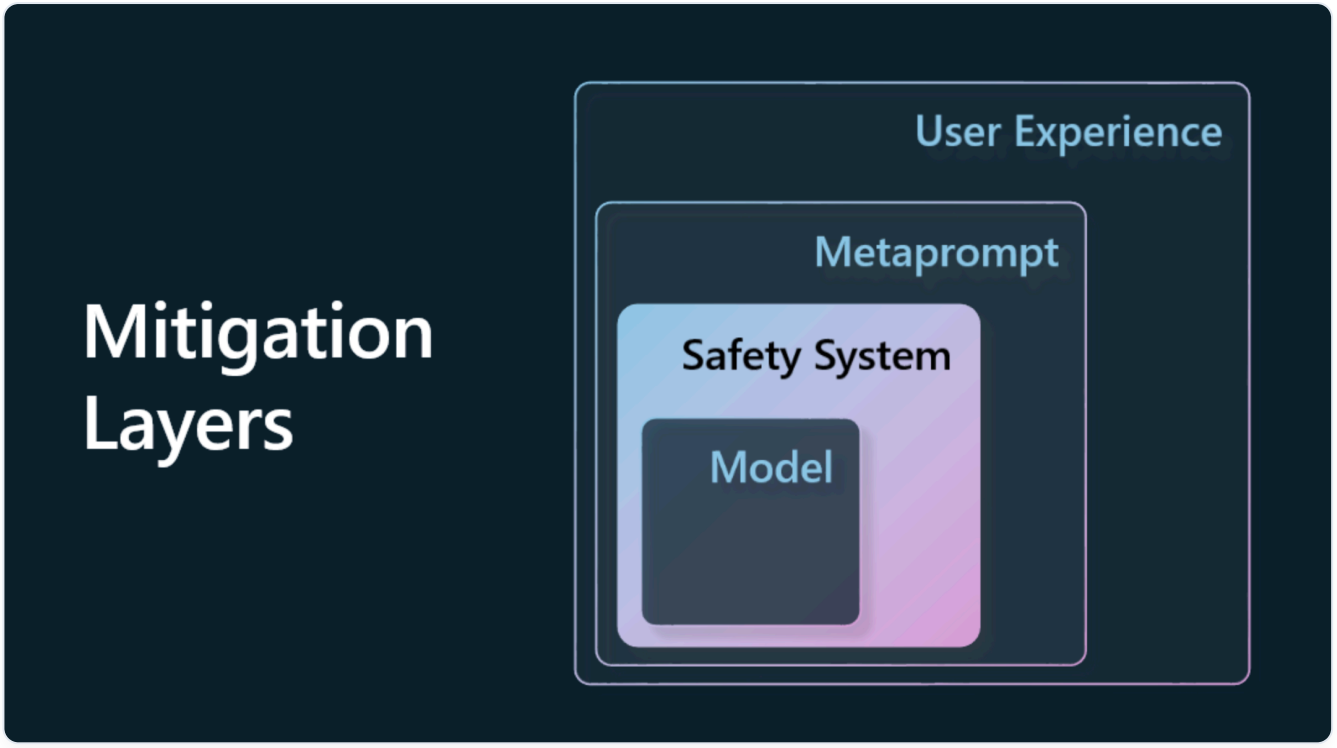
اندازه‌گیری آسیب‌های بالقوه

در تست نرم‌افزار، ما اقدامات مورد انتظار یک کاربر در یک برنامه را آزمایش می‌کنیم. به‌طور مشابه، آزمایش مجموعه‌ای متنوع از درخواست‌هایی که کاربران احتمالاً استفاده خواهند کرد، راه خوبی برای اندازه‌گیری آسیب‌های بالقوه است.

از آنجا که استارتاپ ما در حال ساخت یک محصول آموزشی است، خوب است که لیستی از درخواست‌های مرتبط با آموزش آماده کنیم. این می‌تواند شامل پوشش یک موضوع خاص، حقایق تاریخی و درخواست‌هایی درباره زندگی دانش‌آموزی باشد.

کاهش آسیب‌های بالقوه

اکنون زمان آن رسیده است که راه‌هایی پیدا کنیم که بتوانیم از آسیب‌های بالقوه ناشی از مدل و پاسخ‌های آن جلوگیری کنیم یا آن‌ها را محدود کنیم. می‌توانیم این موضوع را در ۴ لایه مختلف بررسی کنیم:



- **مدل**. انتخاب مدل مناسب برای مورد استفاده مناسب. مدل‌های بزرگ‌تر و پیچیده‌تر مانند GPT-4 می‌توانند خطر محتوای مضر بیشتری ایجاد کنند وقتی برای موارد استفاده کوچک‌تر و خاص‌تر اعمال شوند. استفاده از داده‌های آموزشی خود برای تنظیم دقیق نیز خطر محتوای مضر را کاهش می‌دهد.

- **سیستم ایمنی**. سیستم ایمنی مجموعه‌ای از ابزارها و تنظیمات در پلتفرم ارائه‌دهنده مدل است که به کاهش آسیب کمک می‌کند. نمونه‌ای از این سیستم، سیستم فیلتر محتوای سرویس Azure OpenAI است. سیستم‌ها باید حملات نفوذ و فعالیت‌های ناخواسته مانند درخواست‌های ربات‌ها را نیز شناسایی کنند.

- **متاپرومت**. متاپرومت‌ها و پایه‌گذاری روش‌هایی هستند که می‌توانیم مدل را بر اساس رفتارها و اطلاعات خاص هدایت یا محدود کنیم. این می‌تواند استفاده از ورودی‌های سیستمی برای تعریف محدودیت‌های خاص مدل باشد. علاوه بر این، ارائه خروجی‌هایی که بیشتر مرتبط با دامنه یا حوزه سیستم هستند.

همچنین می‌توان از تکنیک‌هایی مانند تولید مبتنی بر بازیابی (RAG) استفاده کرد تا مدل فقط اطلاعات را از مجموعه‌ای از منابع معتبر استخراج کند. درسی در ادامه این دوره برای **ساخت برنامه‌های جستجو** وجود دارد.

- **تجربه کاربری**. لایه نهایی جایی است که کاربر به‌طور مستقیم از طریق رابط برنامه ما به مدل تعامل می‌کند. به این ترتیب می‌توانیم طراحی UI/UX را برای محدود کردن کاربر در انواع ورودی‌هایی که می‌تواند به مدل ارسال کند و همچنین متن یا تصاویر نمایش داده شده به کاربر طراحی کنیم. هنگام استقرار برنامه هوش مصنوعی، باید شفاف باشیم که برنامه هوش مصنوعی تولیدی ما چه کاری می‌تواند انجام دهد و چه کاری نمی‌تواند انجام دهد.

ما یک درس کامل اختصاص داده‌ایم به **طراحی UX برای برنامه‌های هوش مصنوعی**

- **ارزیابی مدل**. کار با مدل‌های زبان بزرگ می‌تواند چالش‌برانگیز باشد زیرا همیشه کنترل کاملی بر داده‌هایی که مدل بر اساس آن‌ها آموزش دیده است نداریم. با این حال، باید همیشه عملکرد و خروجی‌های مدل را ارزیابی کنیم. همچنان مهم است که دقت، شباهت، پایه‌گذاری و ارتباط خروجی مدل را اندازه‌گیری کنیم. این به ارائه شفافیت و اعتماد به ذینفعان و کاربران کمک می‌کند.

راه‌اندازی یک راه‌حل مسئولانه هوش مصنوعی تولیدی

ساخت یک رویه عملیاتی پیرامون برنامه‌های هوش مصنوعی شما مرحله نهایی است. این شامل همکاری با بخش‌های دیگر استارتاپ ما مانند بخش حقوقی و امنیتی برای اطمینان از رعایت تمام سیاست‌های قانونی است. قبل از راه‌اندازی، همچنین می‌خواهیم برنامه‌هایی پیرامون تحویل، مدیریت حوادث و بازگشت به عقب ایجاد کنیم تا از هرگونه آسیب به کاربران خود جلوگیری کنیم.

ابزارها

در حالی که کار توسعه راه‌حل‌های هوش مصنوعی مسئولانه ممکن است زیاد به نظر برسد، این کار ارزش تلاش را دارد. با رشد حوزه هوش مصنوعی تولیدی، ابزارهای بیشتری برای کمک به توسعه‌دهندگان در ادغام مسئولیت به‌طور کارآمد در جریان کارشان توسعه خواهد یافت. به‌عنوان مثال، Azure AI Content Safety می‌تواند از طریق درخواست API به شناسایی محتوای مضر و تصاویر کمک کند.

بررسی دانش

چه مواردی را باید در نظر بگیرید تا از استفاده مسئولانه از هوش مصنوعی اطمینان حاصل کنید؟

1. اینکه پاسخ صحیح باشد.
2. استفاده مضر، اینکه هوش مصنوعی برای اهداف جنایی استفاده نشود.
3. اطمینان از اینکه هوش مصنوعی عاری از تعصب و تبعیض است.

پاسخ: گزینه‌های ۲ و ۳ صحیح هستند. هوش مصنوعی مسئولانه به شما کمک می‌کند تا نحوه کاهش اثرات مضر و تعصبات و موارد دیگر را در نظر بگیرید.

🚀 چالش

در مورد Azure AI Content Safety مطالعه کنید و ببینید چه چیزی را می‌توانید برای استفاده خود به کار ببرید.

کار عالی، ادامه یادگیری خود را ادامه دهید

پس از اتمام این درس، مجموعه آموزش هوش مصنوعی تولیدی ما را بررسی کنید تا دانش خود در زمینه هوش مصنوعی تولیدی را ارتقا دهید!

به درس چهارم بروید، جایی که اصول **مهندسی پرومت** را بررسی خواهیم کرد!

سلب مسئولیت:

این سند با استفاده از سرویس ترجمه هوش مصنوعی **Co-op Translator** ترجمه شده است. در حالی که ما تلاش می‌کنیم دقت را حفظ کنیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌ها باشند. سند اصلی به زبان اصلی آن باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما مسئولیتی در قبال سوء تفاهم‌ها یا تفسیرهای نادرست ناشی از استفاده از این ترجمه نداریم.

بازگشت به فهرست مطالب

درس ۰۳ — هوش مصنوعی مولد مایکروسافت

درس ۰۴ از ۲۱

اصول و مبانی مهندسی پرامپت (Prompt Engineering)

ساختار یک پرامپت استاندارد، نقش دستورالعمل‌ها، تکنیک Zero-Shot و Few-Shot، و راهنمای تصویری معماری درخواست‌ها

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/GElCu2kUlRs?si=qrXsBvXnCW12epb8

▶️ پخش در یوتیوب

اسکن کد QR

معرفی

این ماژول مفاهیم و تکنیک‌های ضروری برای ایجاد پرامپت‌های مؤثر در مدل‌های تولیدی هوش مصنوعی را پوشش می‌دهد. نحوه نوشتن پرامپت برای یک مدل زبان بزرگ نیز اهمیت دارد. یک پرامپت با دقت ساخته شده می‌تواند کیفیت پاسخ بهتری را به همراه داشته باشد. اما دقیقاً اصطلاحات *پرامپت* و *مهندسی پرامپت* به چه معنا هستند؟ و چگونه می‌توان ورودی پرامپت که به مدل ارسال می‌کنم را بهبود داد؟ این‌ها سوالاتی است که در این فصل و فصل بعدی سعی خواهیم کرد پاسخ دهیم.

هوش مصنوعی تولیدی توانایی خلق محتوای جدید (مثل متن، تصویر، صدا، کد و غیره) بر اساس درخواست‌های کاربر دارد. این موضوع با استفاده از *مدل‌های زبان بزرگ* مانند سری GPT شرکت OpenAI ("ترنسفورمر پیش‌آموزش داده شده تولیدی") محقق می‌شود که برای استفاده از زبان طبیعی و کد آموزش دیده‌اند.

کاربران اکنون می‌توانند با این مدل‌ها با استفاده از الگوهای آشنا مانند چت بدون نیاز به تخصص فنی یا آموزش تعامل کنند. مدل‌ها به صورت *پرامپت‌محور* هستند - کاربران یک ورودی متنی (پرامپت) ارسال می‌کنند و پاسخ هوش مصنوعی (تکمیل) را دریافت می‌کنند. سپس می‌توانند به صورت تعاملی با هوش مصنوعی "گفتگو" کنند و پرامپت خود را تا رسیدن پاسخ به انتظارات خود اصلاح کنند.

"پرامپت‌ها" اکنون به رابط اصلی *برنامه‌نویسی* برای اپلیکیشن‌های هوش مصنوعی تولیدی تبدیل شده‌اند که به مدل‌ها می‌گویند چه کاری انجام دهند و بر کیفیت پاسخ‌های بازگشتی تأثیر می‌گذارند. "مهندسی پرامپت" حوزه‌ای در حال رشد است که بر *طراحی و بهینه‌سازی* پرامپت‌ها برای ارائه پاسخ‌های پایدار و با کیفیت در مقیاس تمرکز دارد.

اهداف یادگیری

در این درس، یاد می‌گیریم مهندسی پرامپت چیست، چرا اهمیت دارد، و چگونه می‌توانیم پرامپت‌های مؤثرتری برای یک مدل و هدف کاربردی خاص ایجاد کنیم. مفاهیم بنیادی و بهترین شیوه‌های مهندسی پرامپت را درک خواهیم کرد - و درباره محیط تعاملی دفترچه یادداشت‌های Jupyter که این مفاهیم را در مثال‌های واقعی به کار می‌برد، یاد می‌گیریم.

در پایان این درس قادر خواهیم بود:

1. توضیح دهیم مهندسی پرامپت چیست و چرا اهمیت دارد.
2. اجزای یک پرامپت را شرح دهیم و نحوه استفاده آن‌ها را بیان کنیم.
3. بهترین شیوه‌ها و تکنیک‌های مهندسی پرامپت را یاد بگیریم.
4. تکنیک‌های آموخته شده را روی مثال‌های واقعی با استفاده از نقطه دسترسی OpenAI به کار ببریم.

اصطلاحات کلیدی

مهندسی پرامپت: عمل طراحی و بهینه‌سازی ورودی‌ها برای هدایت مدل‌های هوش مصنوعی به سمت تولید خروجی‌های مورد نظر. نشانه‌گذاری (توکنیزاسیون): فرایند تبدیل متن به واحدهای کوچک‌تر به نام توکن‌ها که مدل می‌تواند آن‌ها را درک و پردازش کند. مدل‌های زبان بزرگ تنظیم‌شده با دستورالعمل: مدل‌های زبان بزرگ (LLM) که به وسیله دستورالعمل‌های خاص برای بهبود دقت و مرتبط بودن پاسخ، بهینه شده‌اند.

محیط تمرین یادگیری

مهندسی پرامپت در حال حاضر بیشتر یک هنر است تا علم. بهترین راه برای بهبود شهود ما در آن، *تمرین بیشتر* و اتخاذ رویکرد آزمون و خطا است که تخصص حوزه کاربرد را با تکنیک‌های توصیه شده و بهینه‌سازی‌های مدل‌محور ترکیب می‌کند.

دفترچه یادداشت Jupyter همراه این درس محیط *تمرینی* را فراهم می‌کند که می‌توانید یادگیری خود را در آن امتحان کنید؛ چه هنگام درس و چه بخش چالش کدنویسی در پایان. برای اجرای تمرینات، شما به موارد زیر نیاز دارید:

1. **کلید API Azure OpenAI** - نقطه دسترسی سرویس برای یک مدل LLM مستقر شده.

2. **زمان اجرای پایتون** - که دفترچه یادداشت بتواند در آن اجرا شود.
3. **متغیرهای محیطی محلی** - *هم‌اکنون مراحل راه‌اندازی را کامل کنید تا آماده شوید.*

این دفترچه با تمرینات *شروع به کار* ارائه می‌شود - اما تشویق می‌شوید که بخش‌های *متن* (توضیح) و *کد* (درخواست‌های پرامپت) خود را اضافه کنید تا مثال‌ها یا ایده‌های بیشتری را امتحان کنید و شهود خود را برای طراحی پرامپت بسازید.

راهنمای تصویری

می‌خواهید قبل از شروع، تصویری کلی از موضوع این درس داشته باشید؟ این راهنمای تصویری را ببینید که به شما دید کلی درباره موضوعات اصلی پوشش داده شده و نکات کلیدی برای فکر کردن در هر بخش می‌دهد. نقشه راه درس شما را از درک مفاهیم و چالش‌های بنیادی تا پرداختن به آن‌ها با تکنیک‌ها و بهترین شیوه‌های مهندسی پرامپت مرتبط هدایت می‌کند. توجه داشته باشید بخش "تکنیک‌های پیشرفته" در این راهنما به محتوای فصل بعدی این دوره اشاره دارد.

استارتاپ ما

حالا بیایید صحبت کنیم که چگونه *این موضوع* به مأموریت استارتاپ ما برای آوردن نوآوری هوش مصنوعی به آموزش مرتبط است. ما می‌خواهیم برنامه‌های هوش مصنوعی مبتنی بر *یادگیری شخصی‌شده* بسازیم - پس بیایید فکر کنیم کاربران مختلف برنامه ما چگونه ممکن است پرامپت‌ها را "طراحی" کنند:

- **مدیران** ممکن است از هوش مصنوعی بخواهند *داده‌های برنامه درسی را برای شناسایی نقاط ضعف تحلیل کند*. هوش مصنوعی می‌تواند نتایج را خلاصه کند یا آن‌ها را با کد تجسم نماید.

- **معلمان** ممکن است از هوش مصنوعی بخواهند *یک طرح درس برای مخاطب و موضوع هدف بسازد*. هوش مصنوعی می‌تواند طرح شخصی‌سازی شده را در قالب مشخصی بسازد.
- **دانش‌آموزان** ممکن است از هوش مصنوعی بخواهند *آنها را در یک موضوع دشوار راهنمایی کند*. اکنون هوش مصنوعی می‌تواند با درس‌ها، نکات و مثال‌هایی مطابق سطح دانش‌آموزان آن‌ها را هدایت کند.

این فقط نوک کوه یخ است. به **پرامپت‌های آموزشی** نگاهی بیندازید - یک کتابخانه متن‌باز پرامپت که توسط کارشناسان آموزش گردآوری شده است - تا درک گسترده‌تری از امکانات به دست آورید! *تلاش کنید برخی از آن پرامپت‌ها را در محیط تمرینی یا در زمین بازی OpenAI اجرا کنید و ببینید چه رخ می‌دهد!*

مهندسی پرامپت چیست؟

ما این درس را با تعریف **مهندسی پرامپت** به عنوان فرایند *طراحی و بهینه‌سازی* ورودی‌های متنی (پرامپت‌ها) برای ارائه پاسخ‌هایی پایدار و با کیفیت (تکمیل‌ها) برای هدف کاربردی و مدل معین شروع کردیم. می‌توانیم این را به عنوان یک فرایند دو مرحله‌ای در نظر بگیریم:

- *طراحی* پرامپت اولیه برای مدل و هدف مشخص
- *اصلاح* پرامپت به صورت تکراری برای بهبود کیفیت پاسخ

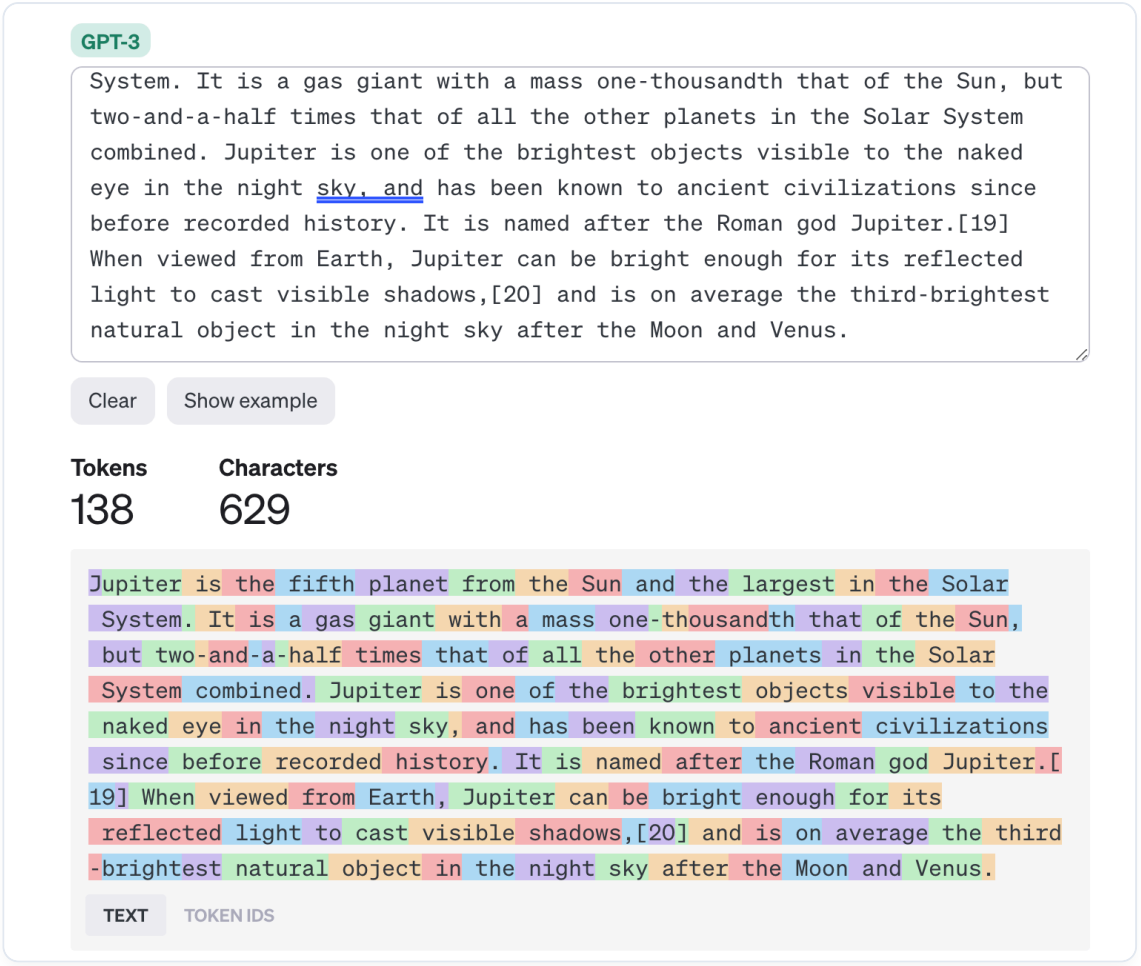
این لزوماً فرایندی آزمون و خطا است که نیاز به شهود و تلاش کاربر برای رسیدن به نتایج بهینه دارد. پس چرا این مهم است؟ برای پاسخ به این سوال ابتدا باید سه مفهوم را بفهمیم:

- *نشانه‌گذاری* = نحوه "دیدن" پرامپت توسط مدل
- *مدل‌های پایه زبان بزرگ* = نحوه "پردازش" پرامپت توسط مدل پایه
- *مدل‌های زبان بزرگ تنظیم‌شده با دستورالعمل* = نحوه "دیدن وظایف" توسط مدل

نشانه‌گذاری (Tokenization)

یک مدل زبان بزرگ پرامپت‌ها را به عنوان *دنباله‌ای از توکن‌ها* می‌بیند که مدل‌های مختلف (یا نسخه‌های مختلف یک مدل) ممکن است همان پرامپت را به شکل‌های متفاوتی نشانه‌گذاری کنند. از آن‌جا که مدل‌های زبان بزرگ روی توکن‌ها (نه متن خام) آموزش دیده‌اند، نحوه نشانه‌گذاری پرامپت‌ها به کیفیت پاسخ تولید شده تاثیر مستقیم دارد.

برای درک بهتر نحوه کار نشانه‌گذاری، ابزارهایی مثل **OpenAI Tokenizer** که در زیر نشان داده شده را امتحان کنید. پرامپت خود را کپی کنید و ببینید چگونه به توکن‌ها تبدیل می‌شود، با توجه به نحوه برخورد با کاراکترهای فاصله و نشانه‌گذاری. توجه کنید این مثال با یک مدل قدیمی‌تر (GPT-3) است - پس امتحان با مدل جدیدتر ممکن است نتیجه متفاوتی بدهد.

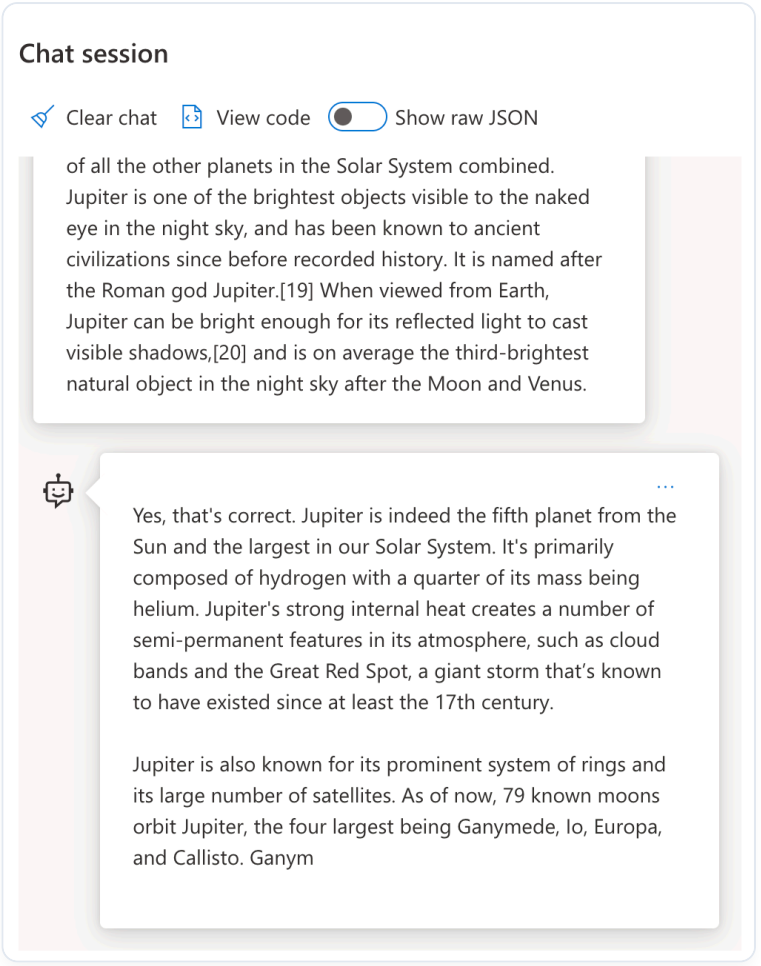


مفهوم: مدل‌های پایه

وقتی پرامپت نشانه‌گذاری شد، وظیفه اصلی "مدل پایه زبان بزرگ" (یا مدل بنیادین) پیش‌بینی توکن بعدی در دنباله است. چون مدل‌های زبان بزرگ روی دیتاست‌های متن حجیم آموزش دیده‌اند، آن‌ها حس خوبی از روابط آماری بین توکن‌ها دارند و می‌توانند با اطمینان نسبی این پیش‌بینی را انجام دهند. توجه داشته باشید که آن‌ها معنای *کلمات* در پرامپت یا توکن‌ها را نمی‌فهمند؛ فقط الگویی را می‌بینند که می‌توانند با پیش‌بینی بعدی آن را "تکمیل" کنند. پیش‌بینی دنباله را تا زمانی که کاربر متوقف کند یا شرایط از پیش تعیین شده صورت گیرد، ادامه می‌دهند.

می‌خواهید ببینید چطور تکمیل مبتنی بر پرامپت کار می‌کند؟ پرامپت بالا را با تنظیمات پیش‌فرض در زمین بازی Microsoft Foundry وارد کنید. سیستم به گونه‌ای پیکربندی شده که پرامپت‌ها را به عنوان درخواست اطلاعات در نظر بگیرد - بنابراین باید پاسخ تکمیل شده‌ای مشاهده کنید که با این زمینه سازگار است.

اما اگر کاربر بخواهد چیز خاصی را که شرط یا هدف وظیفه‌ای را برآورده می‌کند، ببیند چه؟ اینجاست که مدل‌های زبان بزرگ تنظیم‌شده با دستورالعمل وارد می‌شوند.



مفهوم: مدل‌های زبان بزرگ تنظیم‌شده با دستورالعمل

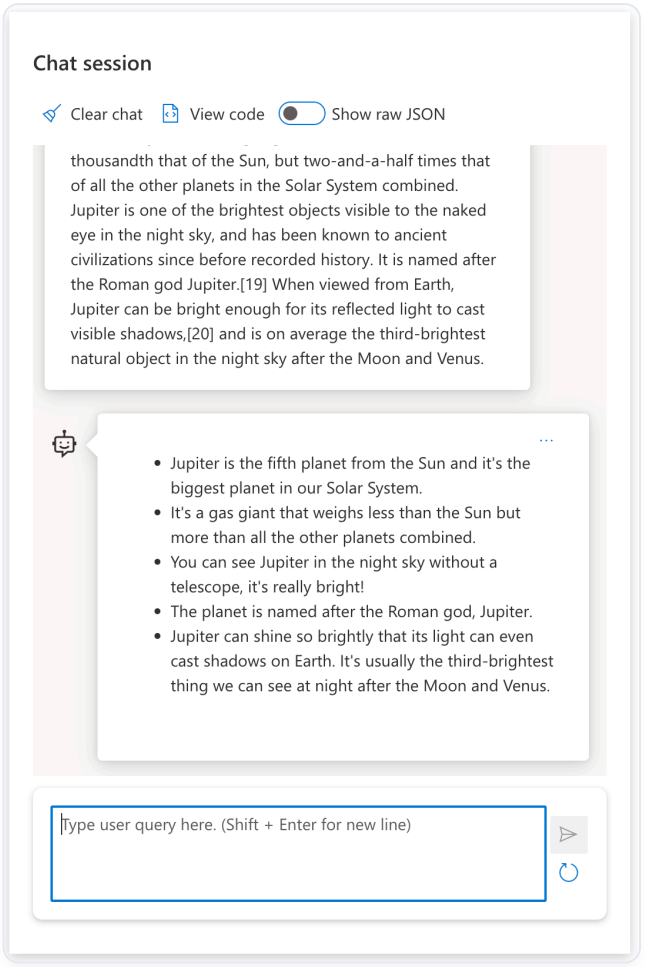
یک مدل زبان بزرگ تنظیم‌شده با دستورالعمل با شروع از مدل بنیادین، با مثال‌ها یا جفت‌های ورودی/خروجی (مثلاً "پیام‌های" چندمرحله‌ای) که شامل دستورالعمل‌های واضح هستند، بهینه‌سازی می‌شود - و پاسخ هوش مصنوعی تلاش می‌کند آن دستورالعمل را دنبال کند.

این فرایند از تکنیک‌هایی مثل یادگیری تقویتی با بازخورد انسانی (RLHF) استفاده می‌کند که می‌تواند مدل را آموزش دهد *دستورالعمل‌ها را دنبال کند* و *از بازخورد یاد بگیرد* تا پاسخ‌هایی تولید کند که بهتر به کاربردهای عملی می‌خورد و مرتبط‌تر با اهداف کاربران است.

بیایید امتحان کنیم - پرامپت بالا را بازبینی کنید، ولی اکنون پیام *سیستم* را به طوری تغییر دهید که دستورالعمل زیر را به عنوان زمینه ارائه دهد:

> **💡 نکته کاربردی**
>
> *خلاصه‌ای از محتوایی که دارید برای یک دانش‌آموز پایه دوم ارائه دهید. نتیجه را به یک پاراگراف با ۳-۵ نکته بولت‌شده محدود کنید.*

ببینید چطور نتیجه اکنون بر اساس هدف و قالب مطلوب تنظیم شده است؟ یک معلم می‌تواند این پاسخ را مستقیماً در اسلایدهای خود برای آن کلاس استفاده کند.



چرا به مهندسی پرامپت نیاز داریم؟

حال که می‌دانیم پرامپت‌ها چگونه توسط مدل‌های زبان بزرگ پردازش می‌شوند، بیایید درباره *چرا* به مهندسی پرامپت نیاز داریم صحبت کنیم. پاسخ در این واقعیت نهفته است که مدل‌های زبان بزرگ فعلی چندین چالش دارند که باعث می‌شود دستیابی به *پاسخ‌های قابل اعتماد و پایدار* بدون صرف تلاش در ساخت و بهینه‌سازی پرامپت دشوار باشد. برای مثال:

1. **پاسخ‌های مدل‌ها تصادفی هستند.** *همان پرامپت* احتمالاً با مدل‌ها یا نسخه‌های مختلف مدل پاسخ‌های متفاوتی تولید می‌کند. و حتی با *همان مدل* در زمان‌های مختلف ممکن است نتایج متفاوتی داشته باشد. *تکنیک‌های مهندسی پرامپت می‌توانند به ما کمک کنند این تغییرات را با ارائه راهنمایی‌های بهتر به حداقل برسانیم.*

2. **مدل‌ها ممکن است پاسخ‌های ساخته شده بسازند.** مدل‌ها با دیتاست‌های *بزرگ اما محدود* آموزش دیده‌اند، یعنی دانش آن‌ها در مورد مفاهیمی خارج از دامنه آموزش محدود است. در نتیجه ممکن است پاسخ‌های نادرست، خیالی یا متضاد با حقایق شناخته شده تولید کنند. *تکنیک‌های مهندسی پرامپت به کاربران کمک می‌کنند چنین پاسخ‌های ساختگی را شناسایی و کاهش دهند، مثلاً با درخواست ارجاعات یا استدلال از هوش مصنوعی.*

3. **توانمندی‌های مدل‌ها متفاوت خواهد بود.** مدل‌های جدیدتر یا نسل‌های مدل قابلیت‌های غنی‌تری دارند اما همچنین ویژگی‌ها و تعادل‌های خاصی در هزینه و پیچیدگی به همراه می‌آورند. *مهندسی پرامپت می‌تواند به ما کمک کند بهترین شیوه‌ها و جریان‌های کاری را توسعه دهیم که تفاوت‌ها را انتزاعی کنند و به نیازهای مدل‌محور مقیاس‌پذیر و بدون درز پاسخ دهند.*

بیایید این موضوع را در زمین بازی OpenAI یا Azure OpenAI ببینیم:

- از همان پرامپت با استقرارهای مختلف مدل زبان بزرگ (مثلاً OpenAI، Azure OpenAI، Hugging Face) استفاده کنید - آیا تفاوت‌ها را مشاهده کردید؟
- بارها همان پرامپت را با *همان* استقرار مدل زبان بزرگ (مثلاً زمین بازی Azure OpenAI) استفاده کنید - چگونه این تفاوت‌ها با هم فرق داشتند؟

مثال از پاسخ‌های ساختگی

در این دوره، ما از اصطلاح **"ساختگی"** برای اشاره به پدیده‌ای استفاده می‌کنیم که مدل‌های زبان بزرگ گاهی اطلاعات نادرست واقعی تولید می‌کنند، به دلیل محدودیت‌های آموزش یا دیگر موانع. ممکن است این موضوع در مقالات یا تحقیقات رایج با عنوان *"توهمات"* نیز بیان شده باشد. اما ما قویاً توصیه می‌کنیم از *"ساختگی"* استفاده کنید تا به طور تصادفی رفتار را انسان‌انگار نکنیم و ویژگی انسانی به نتیجه تولید شده توسط ماشین نسبت ندهیم. این همچنین از نظر واژگانی با **راهنمایی‌های هوش مصنوعی مسئولانه** مطابقت دارد، و اصطلاحاتی که ممکن است توهین‌آمیز یا غیرشامل باشند را حذف می‌کند.

می‌خواهید درک کنید ساختگی‌ها چگونه کار می‌کنند؟ فرض کنید پرامپتی دارید که از هوش مصنوعی می‌خواهد برای موضوعی غیرواقعی محتوا تولید کند (تا مطمئن شوید در دیتاست آموزش وجود ندارد). به عنوان مثال - من این پرامپت را امتحان کردم:

> 💡 **نکته کاربردی**
>
> **پرامپت:** یک طرح درس درباره جنگ مریخی سال ۲۰۷۶ تولید کن.

جستجوی وب نشان داد داستان‌های خیالی (مثل مجموعه تلویزیونی یا کتاب) درباره جنگ‌های مریخی وجود دارد - اما هیچ موردی در سال ۲۰۷۶ نیست. عقل سلیم نیز می‌گوید ۲۰۷۶ *در آینده است* و بنابراین نمی‌تواند مربوط به یک رویداد واقعی باشد.

پس چه اتفاقی می‌افتد وقتی این پرامپت را با ارائه‌دهندگان مختلف مدل‌های زبانی بزرگ اجرا می‌کنیم؟

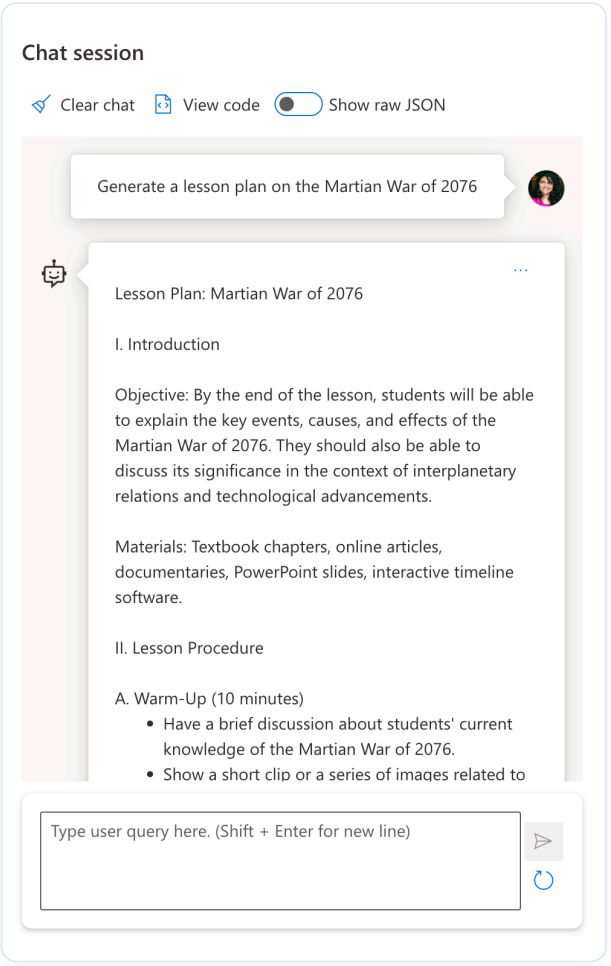
> 💡 **نکته کاربردی**
>
> **پاسخ ۱:** OpenAI Playground (GPT-35)

USER	Enter a user message here.
ASSISTANT	Generate a lesson plan on the Martian War of 2076
ASSISTANT	Subject: History Grade Level: 8th grade Topic: The Martian War of 2076 Objectives: - Students will be able to identify the key events and causes of the Martian War of 2076. - Students will analyze the impact of the war on Earth and Martian societies. - Students will examine the role of technology in the war. Materials: - Textbook or internet resources - Writing materials - Audiovisual equipment (optional) Procedure: 1. Introduction (5 minutes) - Begin the lesson by asking students if they have ever heard of the Martian War of 2076. - Provide a brief overview of the war and its significance in the history of Earth and Mars.

Response 1

💡 نکته کاربردی

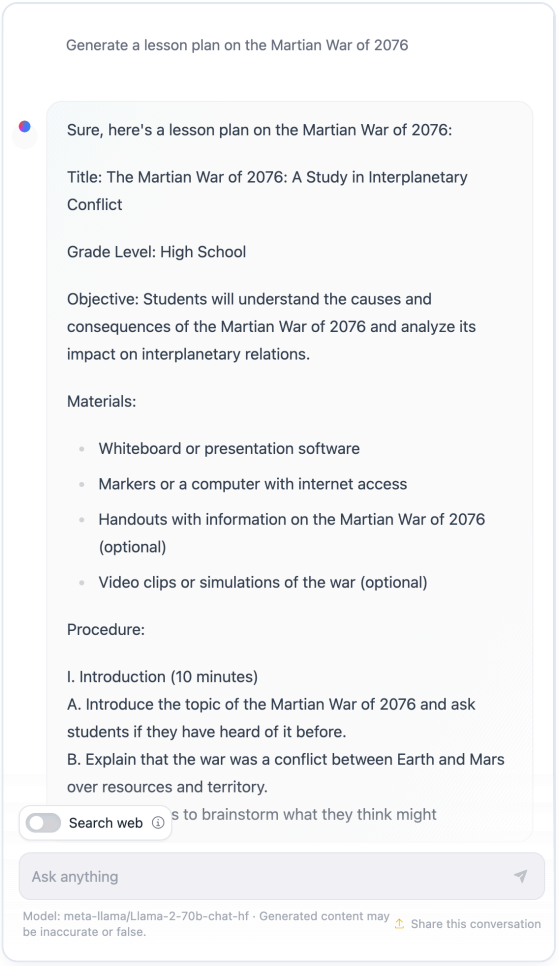
پاسخ ۲: Azure OpenAI Playground (GPT-35)



Response 2

💡 نکته کاربردی

پاسخ ۳: : Hugging Face Chat Playground (LLama-2)



Response 3

همان‌طور که انتظار می‌رود، هر مدل (یا نسخه مدل) پاسخ‌های کمی متفاوت تولید می‌کند که به خاطر رفتار تصادفی و تفاوت‌های قابلیت مدل است. برای مثال، یک مدل مخاطب پایه هشتم را هدف قرار می‌دهد در حالی که مدل دیگر دانش‌آموز دبیرستانی فرض می‌کند. اما هر سه مدل پاسخ‌هایی تولید کردند که می‌توانست یک کاربر ناآگاه را متقاعد کند که رویداد واقعی است.

تکنیک‌های مهندسی پرامپت مانند *متاپرامپتینگ* و *تنظیم دما* ممکن است تا حدی از ساخته‌پرداخته‌های مدل بکاهند. معماری‌های جدید مهندسی پرامپت نیز ابزارها و تکنیک‌های جدید را به‌طور یکپارچه در جریان پرامپت وارد می‌کنند تا برخی از این اثرات را کاهش داده یا تخفیف دهند.

مطالعه موردی: GitHub Copilot

بیایید این بخش را با درک اینکه مهندسی پرامپت چگونه در راه‌حل‌های واقعی به کار می‌رود جمع‌بندی کنیم، با نگاه به یک مطالعه موردی: GitHub Copilot.

GitHub Copilot "برنامه‌نویس هم‌کار هوش مصنوعی" شماست - که پرامپت‌های متنی را به تکمیل کد تبدیل می‌کند و در محیط توسعه شما (مثلاً Visual Studio Code) ادغام شده است تا تجربه کاربری روانی ایجاد کند. همان‌طور که در سری وبلاگ‌های زیر مستند شده، نسخه ابتدایی بر اساس مدل OpenAI Codex بود - با این حال مهندسان به سرعت نیاز به تنظیم

دقیق مدل و توسعه تکنیک‌های بهتر مهندسی پرامپت را برای بهبود کیفیت کد درک کردند. در جولای، آن‌ها **یک مدل هوش مصنوعی بهبود یافته را معرفی کردند که فراتر از Codex می‌رود** برای پیشنهادهای سریع‌تر.

پست‌ها را به ترتیب بخوانید تا مسیر یادگیری آن‌ها را دنبال کنید.

- **مه ۲۰۲۳** | **GitHub Copilot در درک کد شما بهتر می‌شود**
- **مه ۲۰۲۳** | **داخل GitHub: کار با مدل‌های زبانی بزرگ پشت GitHub Copilot**.
- **ژوئن ۲۰۲۳** | **چگونه پرامپت‌های بهتری برای GitHub Copilot بنویسیم**.
- **جولای ۲۰۲۳** | **GitHub Copilot .. با مدل هوش مصنوعی بهبود یافته فراتر از Codex می‌رود**
- **جولای ۲۰۲۳** | **راهنمای توسعه‌دهندگان برای مهندسی پرامپت و مدل‌های زبانی بزرگ**
- **سپتامبر ۲۰۲۳** | **چگونه یک اپلیکیشن مدل زبانی بزرگ شرکتی بسازیم: درس‌هایی از GitHub Copilot**

همچنین می‌توانید وبلاگ **مهندسی آن‌ها** را برای پست‌های بیشتری مانند **این پست** که نشان می‌دهد چگونه این مدل‌ها و تکنیک‌ها برای هدایت اپلیکیشن‌های دنیای واقعی *به کار می‌روند* مرور کنید.

ساختار پرامپت

دیدیم چرا مهندسی پرامپت مهم است - حالا بیایید بفهمیم پرامپت‌ها چگونه *ساخته* می‌شوند تا بتوانیم تکنیک‌های مختلف را برای طراحی مؤثرتر پرامپت ارزیابی کنیم.

پرامپت ساده

بیایید از پرامپت ساده شروع کنیم: یک ورودی متنی که بدون زمینه دیگر به مدل ارسال می‌شود. در اینجا یک مثال داریم - وقتی چند کلمه اول سرود ملی آمریکا را به OpenAI **Completion API** می‌فرستیم، پاسخ را بلافاصله با چند خط بعدی *تکمیل* می‌کند، که رفتار پیش‌بینی پایه‌ای را نشان می‌دهد.

پرامپت (ورودی)	تکمیل (خروجی)
Oh say can you see	به نظر می‌رسد شما در حال شروع متن "The Star-Spangled Banner" هستید، سرود ملی ایالات متحده. متن کامل این است ...

پرامپت پیچیده

حالا بیایید زمینه و دستورالعمل‌هایی به آن پرامپت ساده اضافه کنیم. **Chat Completion API** به ما اجازه می‌دهد که پرامپت پیچیده‌ای را به صورت مجموعه‌ای از *پیام‌ها* بسازیم با:

- جفت‌های ورودی/خروجی که ورودی *کاربر* و پاسخ *دستیار* را بازتاب می‌دهند.

- پیام سیستم که زمینه رفتار یا شخصیت دستیار را تعیین می‌کند.

درخواست اکنون به شکل زیر است که *توکن‌سازی* اطلاعات مرتبط از زمینه و گفتگو را به خوبی ضبط می‌کند. تغییر زمینه سیستم می‌تواند به همان اندازه کیفیت تکمیل‌ها تأثیرگذار باشد که ورودی‌های کاربر ارائه شده است.

Python

```
response = client.responses.create(
    model="gpt-5-mini",
    input=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Who won the world series in 2020?"},
        {"role": "assistant", "content": "The Los Angeles Dodgers won the World Series in
2020."},
        {"role": "user", "content": "Where was it played?"}
    ]
)
```

پرامپت دستوری

در مثال‌های بالا، پرامپت کاربر یک پرسش متنی ساده بود که می‌توان آن را به عنوان درخواستی برای اطلاعات تفسیر کرد. با پرامپت‌های *دستوری*، می‌توانیم از آن متن برای مشخص کردن یک وظیفه به شکل دقیق‌تر استفاده کنیم و راهنمایی بهتری به هوش مصنوعی بدهیم. در اینجا یک مثال داریم:

پرامپت (ورودی)	تکمیل (خروجی)	نوع دستور
یک توضیح درباره جنگ داخلی بنویسید	*یک پاراگراف ساده برگرداند*	ساده
یک توضیح درباره جنگ داخلی بنویسید. تاریخ‌ها و رویدادهای کلیدی را ارائه دهید و اهمیت آن‌ها را شرح دهید	*یک پاراگراف به همراه فهرستی از تاریخ‌های رویدادهای کلیدی با توضیحات برگرداند*	پیچیده
یک توضیح درباره جنگ داخلی در ۱ پاراگراف بنویسید. ۳ نکته کلیدی با تاریخ‌ها و اهمیتشان ارائه دهید. ۳ نکته دیگر درباره شخصیت‌های تاریخی مهم و مشارکتشان ارائه دهید. خروجی را به صورت فایل JSON بازگردانید	*جزئیات بیشتری در یک کادر متنی، قالب‌بندی شده به صورت JSON برمی‌گرداند که می‌توانید آن را کپی و در فایل ذخیره کنید و صحت آن را بررسی کنید*	پیچیده. قالب‌بندی شده

محتوای اصلی

در مثال‌های بالا، پرامپت هنوز تا حدی باز بود و به مدل اجازه می‌داد که خودش تصمیم بگیرد کدام بخش از داده‌های پیش‌آموزش داده شده برایش مرتبط است. با الگوی طراحی *محتوای اصلی*، متن ورودی به دو بخش تقسیم می‌شود:

- یک دستور (اقدام)

- محتوای مرتبط (که بر اقدام تأثیر می‌گذارد)

در اینجا مثالی داریم که در آن دستور "خلاصه در ۲ جمله" است.

پرامپت (ورودی)	تکمیل (خروجی)
مشتری پنجمین سیاره از خورشید و بزرگ‌ترین در منظومه شمسی است. این یک غول گازی است که جرم آن یک هزارم جرم خورشید است، ولی دو و نیم برابر تمام سیارات دیگر منظومه خورشیدی است. مشتری یکی از پرنورترین اجسام قابل دید با چشم غیرمسلح در آسمان شب است و از زمان‌های قدیم شناخته شده است. این سیاره به نام خدای رومی مشتری نامیده شده است.[۱۹] وقتی از زمین دیده می‌شود، مشتری می‌تواند به اندازه‌ای روشن باشد که سایه‌ها قابل مشاهده بیندازد[۲۰] و به طور متوسط سومین جسم طبیعی پرنور در آسمان شب پس از ماه و ناهید است. **این را در دو جمله کوتاه خلاصه کنید**	مشتری، پنجمین سیاره از خورشید، بزرگ‌ترین در منظومه شمسی است و به عنوان یکی از پرنورترین اجسام آسمان شب شناخته می‌شود. نام این سیاره برگرفته از خدای رومی مشتری است و غولی گازی است که جرم آن دو و نیم برابر سایر سیارات منظومه شمسی است.

بخش محتوای اصلی را می‌توان به روش‌های مختلفی برای تولید دستورالعمل‌های مؤثرتر به کار برد:

- **مثال‌ها** - به جای اینکه مدل را با دستور صریح بگویید چه کاری انجام دهد، برایش مثال‌هایی از آنچه باید انجام دهد بیاورید و اجازه دهید الگو را استنتاج کند.
- **سرنخ‌ها** - دستور را با یک "سرنخ" دنبال کنید که تکمیل را آماده می‌کند و مدل را به سمت پاسخ‌های مرتبط‌تر هدایت می‌کند.
- **قالب‌ها** - این‌ها دستورالعمل‌های قابل تکراری هستند با جای‌گذاری‌های (متغیرها) که می‌توان با داده‌های مربوط به موارد استفاده خاص، سفارشی‌شان کرد.

بیایید این‌ها را در عمل بررسی کنیم.

استفاده از مثال‌ها

این رویکردی است که در آن محتوای اصلی را برای "تغذیه مدل" با چند مثال از خروجی مطلوب برای یک دستور معین استفاده می‌کنید و اجازه می‌دهید الگو را برای خروجی مطلوب استنتاج کند. بر اساس تعداد مثال‌های داده شده، می‌توانیم پرامپت صفر شات، یک شات، چند شات و غیره داشته باشیم.

پرامپت اکنون از سه جزء تشکیل شده است:

- شرح وظیفه
- چند مثال از خروجی مطلوب
- شروع یک مثال جدید (که به طور ضمنی شرح وظیفه جدید است)

نوع یادگیری	پرامپت (ورودی)	تکمیل (خروجی)
صفر شات	"The Sun is Shining". ترجمه به اسپانیایی	"El Sol está brillando".
یک شات	"The Sun is Shining" => ""El Sol está brillando" <= "It's a Cold and Windy Day"	"Es un día frío y ventoso".
چند شات	بازیکن دورهای بازی را دوید => بیسبال بازیکن یک ای اس زد زد => تنیس بازیکن شش زد => کریکت بازیکن یک اسلم دانک زد =>	بسکتبال

توجه کنید که در پرامپت صفر شات مجبور بودیم دستور صریح ("ترجمه به اسپانیایی") را ارائه دهیم، اما در مثال یک شات این دستور ضمنی است. مثال چند شات نشان می‌دهد که افزودن مثال‌های بیشتر اجازه می‌دهد مدل‌ها بدون دستور صریح، استنتاج‌های دقیق‌تری انجام دهند.

سرنخ‌های پرامپت

یک تکنیک دیگر برای استفاده از محتوای اصلی، ارائه *سرنخ* به جای مثال است. در این حالت، با یک بخش کوتاه که قالب پاسخ مطلوب را بازتاب می‌دهد، مدل را *آغاز می‌کنیم*. سپس مدل "سرنخ را می‌گیرد" و در همان راستا ادامه می‌دهد.

تعداد سرنخ‌ها	پرامپت (ورودی)	تکمیل (خروجی)
۰	مشتری پنجمین سیاره از خورشید و بزرگ‌ترین در منظومه شمسی است. این یک غول گازی است که جرم آن یک هزارم جرم خورشید است، ولی دو و نیم برابر تمام سیارات دیگر منظومه خورشیدی است. مشتری یکی از پرنورترین اجسام قابل دید با چشم غیرمسلح در آسمان شب است و از زمان‌های قدیم شناخته شده است. **این را خلاصه کن**	مشتری بزرگ‌ترین سیاره منظومه شمسی و پنجمین سیاره از خورشید است. این یک غول گازی با جرمی ۱/۱۰۰۰ خورشید است، اما از تمام سیارات دیگر بیشتر است. تمدن‌های باستانی برای مدت‌ها مشتری را می‌شناخته‌اند و به آسانی در آسمان شب دیده می‌شود.
۱	مشتری پنجمین سیاره از خورشید و بزرگ‌ترین در منظومه شمسی است. این یک غول گازی است که جرم آن یک هزارم جرم خورشید است، ولی دو و نیم برابر تمام سیارات دیگر منظومه خورشیدی است. مشتری یکی از پرنورترین اجسام قابل دید با چشم غیرمسلح در آسمان شب است و از زمان‌های قدیم شناخته شده است. **این را خلاصه کن** آنچه یاد گرفتیم این است که مشتری	پنجمین سیاره از خورشید و بزرگ‌ترین در منظومه شمسی است. این یک غول گازی است که جرم آن یک هزارم جرم خورشید است، ولی دو و نیم برابر جرم تمام سیارات دیگر است. به راحتی با چشم غیرمسلح دیده می‌شود و از قدیم شناخته شده است.
2	مشتری پنجمین سیاره از خورشید و بزرگ‌ترین سیاره در منظومه شمسی است. این یک سیاره بزرگ گازی است که جرم آن یک هزارم جرم خورشید است، اما دو و نیم برابر جرم تمام سیارات دیگر منظومه شمسی به طور جمع شده است. مشتری یکی از درخشان‌ترین اجرامی است که با چشم غیرمسلح در آسمان شب قابل مشاهده است و از زمان تمدن‌های باستانی قبل از تاریخ ثبت شده شناخته شده است. **خلاصه این مطلب** سه حقیقت برتر که یاد گرفتیم:	1. مشتری پنجمین سیاره از خورشید و بزرگ‌ترین سیاره در منظومه شمسی است. 2. این یک سیاره بزرگ گازی است که جرم آن یک هزارم جرم خورشید است... 3. مشتری از زمان‌های قدیم با چشم غیرمسلح دیده شده است...

الگوهای درخواست

یک الگوی درخواست، *دستورالعمل از پیش تعریف شده برای یک درخواست* است که می‌توان آن را ذخیره و در صورت نیاز مجدداً استفاده کرد تا تجربه‌های کاربری سازگارتر و در مقیاس بزرگ ایجاد شود. به ساده‌ترین شکل، این مجموعه‌ای از نمونه‌های درخواست است مانند این نمونه از OpenAI که هم اجزای تعاملی درخواست (پیام‌های کاربر و سیستم) و هم قالب درخواست مبتنی بر API را ارائه می‌دهد - برای پشتیبانی از استفاده مجدد.

در شکل پیچیده‌تر آن مانند این مثال از LangChain شامل *جایگزین‌هایی* است که می‌توان آن‌ها را با داده‌هایی از منابع مختلف (ورودی کاربر، زمینه سیستمی، منابع داده خارجی و غیره) جایگزین کرد تا یک درخواست به صورت پویا تولید شود. این

امکان را به ما می‌دهد که کتابخانه‌ای از درخواست‌های قابل استفاده مجدد ایجاد کنیم که می‌توانند به صورت برنامه‌ای برای ایجاد تجربه‌های کاربری سازگار در مقیاس بزرگ استفاده شوند.

در نهایت، ارزش واقعی الگوها در توانایی ایجاد و انتشار *کتابخانه‌های درخواست* برای حوزه‌های کاربردی خاص است - جایی که الگوی درخواست در حال حاضر *بهینه* شده است تا زمینه یا مثال‌های مخصوص به کاربرد را منعکس کند که پاسخ‌ها را برای مخاطبان هدف مرتبط‌تر و دقیق‌تر می‌کند. مخزن Prompts For Edu نمونه‌ای عالی از این رویکرد است که کتابخانه‌ای از درخواست‌ها برای حوزه آموزش با تأکید بر اهداف اصلی مانند برنامه‌ریزی درس، طراحی برنامه درسی، آموزش دانش‌آموزان و غیره گردآوری می‌کند.

محتوای پشتیبانی‌کننده

اگر ساختار درخواست را شامل یک دستورالعمل (وظیفه) و یک هدف (محتوای اصلی) در نظر بگیریم، آنگاه *محتوای ثانویه* مانند زمینه اضافی است که ما برای **تأثیرگذاری بر خروجی به شیوه‌ای خاص** فراهم می‌کنیم. می‌تواند شامل پارامترهای تنظیم، دستورالعمل‌های قالب‌بندی، رده‌بندی موضوعات و غیره باشد که به مدل کمک می‌کند پاسخ خود را متناسب با اهداف یا انتظارات کاربر تنظیم کند.

به عنوان مثال: با فرض یک کاتالوگ دوره با متاداده‌های گسترده (نام، توضیح، سطح، برچسب‌های متاداده، مدرس و غیره) برای تمام دوره‌های موجود در برنامه درسی:

- می‌توانیم یک دستورالعمل تعریف کنیم برای "خلاصه کاتالوگ دوره برای پاییز ۲۰۲۳"
- می‌توانیم از محتوای اصلی برای ارائه چند نمونه از خروجی مطلوب استفاده کنیم
- می‌توانیم از محتوای ثانویه برای شناسایی ۵ برچسب برتر مورد علاقه استفاده کنیم.

حالا، مدل می‌تواند خلاصه‌ای در قالب نشان داده شده توسط چند نمونه ارائه دهد - اما اگر نتیجه چندین برچسب داشته باشد، می‌تواند ۵ برچسب شناسایی‌شده در محتوای ثانویه را اولویت‌بندی کند.

بهترین روش‌های درخواست

حال که می‌دانیم چگونه درخواست‌ها *ساخته* می‌شوند، می‌توانیم در مورد اینکه چگونه آن‌ها را به گونه‌ای *طراحی* کنیم که بهترین روش‌ها را منعکس کند، فکر کنیم. می‌توانیم این موضوع را در دو بخش بررسی کنیم - داشتن نگرش درست و به کار بردن تکنیک‌های مناسب.

نگرش مهندسی درخواست

مهندسی درخواست فرآیندی آزمایش و خطا است، پس سه عامل کلی راهنما را در ذهن داشته باشید:

1. **درک حوزه اهمیت دارد.** دقت و ارتباط پاسخ تابعی از *حوزه* است که در آن برنامه یا کاربر فعالیت می‌کند. شهود و تخصص حوزه خود را برای **شخصی‌سازی تکنیک‌ها** بیشتر به کار ببرید. برای مثال، در درخواست‌های سیستم شخصیت‌های *حوزه‌محور* تعریف کنید یا در درخواست‌های کاربر از *الگوهای حوزه‌محور* استفاده کنید. محتوای ثانویه‌ای فراهم کنید که زمینه‌های خاص حوزه را منعکس کند، یا از *نشانه‌ها و مثال‌های حوزه‌محور* برای هدایت مدل به الگوهای استفاده آشنا بهره ببرید.

2. **درک مدل اهمیت دارد.** می‌دانیم مدل‌ها ذاتاً تصادفی هستند. اما پیاده‌سازی مدل‌ها می‌تواند از نظر داده‌های آموزشی استفاده شده (دانش پیش‌آموزش)، قابلیت‌ها (مانند API یا SDK) و نوع محتوایی که برای آن بهینه شده‌اند (مثلاً کد، تصاویر، متن) متفاوت باشد. نقاط قوت و محدودیت‌های مدلی که استفاده می‌کنید را بشناسید و از آن دانش برای *اولویت‌بندی وظایف* یا ایجاد *الگوهای سفارشی* که برای قابلیت‌های مدل بهینه شده‌اند، استفاده کنید.

3. **تکرار و اعتبارسنجی اهمیت دارد.** مدل‌ها به سرعت در حال تکامل هستند، و تکنیک‌های مهندسی درخواست نیز همین‌طور. به عنوان یک کارشناس حوزه، ممکن است زمینه یا معیارهای خاص خود را برای *برنامه* خاصتان داشته باشید که برای جامعه وسیع‌تر کاربرد نداشته باشد. از ابزارها و تکنیک‌های مهندسی درخواست برای "شروع سریع" ساخت درخواست استفاده کنید، سپس نتایج را با شهود و تخصص حوزه خود بازبینی و اعتبارسنجی کنید. بینش‌های خود را ثبت کرده و یک **پایگاه دانش** (برای مثال کتابخانه درخواست‌ها) ایجاد کنید که دیگران بتوانند به عنوان پایه جدید در تکرارهای سریع‌تر آینده از آن بهره ببرند.

بهترین روش‌ها

حال به بررسی بهترین روش‌های رایجی که توسط OpenAI و Azure OpenAI توصیه شده‌اند، می‌پردازیم.

چه کاری انجام دهیم	چرا
مدل‌های جدید را ارزیابی کنید.	نسل‌های جدید مدل‌ها احتمالاً ویژگی‌ها و کیفیت بهبود یافته‌ای دارند - اما ممکن است هزینه‌های بیشتری نیز داشته باشند. آن‌ها را برای تأثیر بررسی کرده و سپس تصمیمات مهاجرت را اتخاذ کنید.
دستورالعمل‌ها و زمینه را جدا کنید	بررسی کنید آیا مدل یا ارائه‌دهنده شما *مرزگذار* تعریف می‌کند تا دستورالعمل‌ها، محتوای اصلی و ثانویه را واضح‌تر متمایز کند. این به مدل‌ها کمک می‌کند وزن‌ها را دقیق‌تر به توکن‌ها اختصاص دهند.
مشخص و واضح باشید	جزئیات بیشتری درباره زمینه، نتیجه، طول، قالب، سبک و غیره ارائه دهید. این کار کیفیت و سازگاری پاسخ‌ها را بهبود می‌بخشد. دستورالعمل‌ها را در قالب‌های قابل استفاده مجدد ثبت کنید.
توضیحی باشید، از مثال استفاده کنید	مدل‌ها ممکن است بهتر به رویکرد "نمایش و بیان" پاسخ دهند. با روش `zero-shot` شروع کنید که فقط دستورالعمل می‌دهید (بدون مثال)، سپس `few-shot` را به عنوان اصلاح امتحان کنید که چند نمونه از خروجی مطلوب را ارائه می‌دهد. از قیاس‌ها استفاده کنید.
از نشانه‌ها برای شروع تکمیل‌ها استفاده کنید	با دادن چند کلمه یا عبارت ابتدایی که مدل می‌تواند به عنوان نقطه شروع برای پاسخ استفاده کند، آن را به سمت نتیجه مطلوب سوق دهید.
دوباره تکرار کنید	گاهی اوقات ممکن است لازم باشد به مدل چند بار تکرار کنید. دستورالعمل‌ها را قبل و بعد از محتوای اصلی بدهید، از دستورالعمل و نشانه استفاده کنید و غیره. تکرار و اعتبارسنجی کنید تا ببینید چه چیزی بهتر جواب می‌دهد.
ترتیب اهمیت دارد	ترتیب ارائه اطلاعات به مدل ممکن است خروجی را تحت تأثیر قرار دهد، حتی در نمونه‌های یادگیری، به دلیل سوگیری تازگی. گزینه‌های مختلف را امتحان کنید تا ببینید چه چیزی بهترین نتیجه را می‌دهد.
به مدل یک «خروجی امن» بدهید	به مدل یک پاسخ *پشتیبان* بدهید که در صورت ناتوانی در تکمیل وظیفه برای هر دلیلی ارائه شود. این می‌تواند احتمال تولید پاسخ‌های نادرست یا ساختگی را کاهش دهد.

همانند هر بهترین روش دیگر، به یاد داشته باشید که *نتایج شما ممکن است بسته به مدل، وظیفه و حوزه متفاوت باشد.* این‌ها را به عنوان نقطه شروع استفاده کنید و به مرور زمان تکرار کنید تا به چیزی که برای شما بهتر است برسید. فرآیند مهندسی درخواست خود را به طور مداوم بازنگری کنید با عرضه مدل‌ها و ابزارهای جدید، با تمرکز بر مقیاس‌پذیری فرآیند و کیفیت پاسخ.

تمرین

تبریک! به پایان درس رسیدید! وقت آن است که برخی از این مفاهیم و تکنیک‌ها را با مثال‌های واقعی امتحان کنید!

برای تمرین، از دفترچه Jupyter استفاده می‌کنیم که می‌توانید به صورت تعاملی تمرین‌ها را کامل کنید. همچنین می‌توانید دفترچه را با سلول‌های Markdown و کد مخصوص خود گسترش دهید و ایده‌ها و تکنیک‌ها را به صورت مستقل بررسی کنید.

برای شروع، مخزن را فورک کنید، سپس

- (توصیه شده) GitHub Codespaces را راه‌اندازی کنید
- (گزینه دیگر) مخزن را روی دستگاه محلی خود کلون کرده و با Docker Desktop استفاده کنید
- (گزینه دیگر) دفترچه را با محیط اجرایی دفترچه مورد علاقه خود باز کنید.

سپس، متغیرهای محیطی را تنظیم کنید

- فایل `.env.copy` را از ریشه مخزن به `.env` کپی کنید و مقادیر `AZURE_OPENAI_API_KEY`، `AZURE_OPENAI_ENDPOINT` و `AZURE_OPENAI_DEPLOYMENT` را پر کنید. به بخش **محیط یادگیری** برگردید تا نحوه انجام آن را بیاموزید.

سپس، دفترچه Jupyter را باز کنید

- هسته اجرایی را انتخاب کنید. اگر از گزینه ۱ یا ۲ استفاده می‌کنید، فقط هسته پیش‌فرض Python 3.10.x که توسط کانتینر توسعه فراهم شده است را انتخاب کنید.

اکنون آماده اجرای تمرین‌ها هستید. توجه داشته باشید که اینجا پاسخ‌های *درست و غلط* وجود ندارد - فقط کاوش گزینه‌ها با روش آزمایش و خطا و ساخت شهود برای آنچه برای یک مدل و حوزه کاربرد مشخص خوب کار می‌کند است.

به همین دلیل در این درس بخش‌های راه‌حل کد وجود ندارد. در عوض، دفترچه شامل سلول‌های Markdown با عنوان "راه حل من:" است که یک مثال خروجی را به عنوان مرجع نشان می‌دهد.

ارزیابی دانش

کدام یک از موارد زیر یک درخواست خوب است که برخی از بهترین روش‌های منطقی را دنبال می‌کند؟

1. به من تصویری از یک ماشین قرمز نشان بده
2. به من تصویری از یک ماشین قرمز ساخت ولوو با مدل XC90 که کنار صخره پارک شده و خورشید در حال غروب است نشان بده
3. به من تصویری از یک ماشین قرمز ساخت ولوو با مدل XC90 نشان بده

پاسخ: گزینه ۲، بهترین درخواست است زیرا جزئیاتی درباره «چه چیزی» ارائه می‌دهد و به جزئیات می‌پردازد (نه فقط هر ماشین، بلکه ساخت و مدل خاص) و همچنین تنظیم کلی را توصیف می‌کند. گزینه ۳ دوم است زیرا هنوز توضیح نسبتا زیادی دارد.

🚀 چالش

ببینید آیا می‌توانید با استفاده از تکنیک "نشانه" درخواست: جمله را کامل کنید: "به من تصویری از ماشین قرمز ساخت ولوو و " را تکمیل کنید. مدل چگونه پاسخ می‌دهد و چگونه آن را بهبود می‌دهید؟

عالی بود! یادگیری خود را ادامه دهید

می‌خواهید درباره مفاهیم مختلف مهندسی درخواست بیشتر بدانید؟ به **صفحه یادگیری ادامه‌دار** مراجعه کنید تا منابع عالی دیگری در این موضوع پیدا کنید.

به درس ۵ بروید که در آن به بررسی **تکنیک‌های پیشرفته درخواست** می‌پردازیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

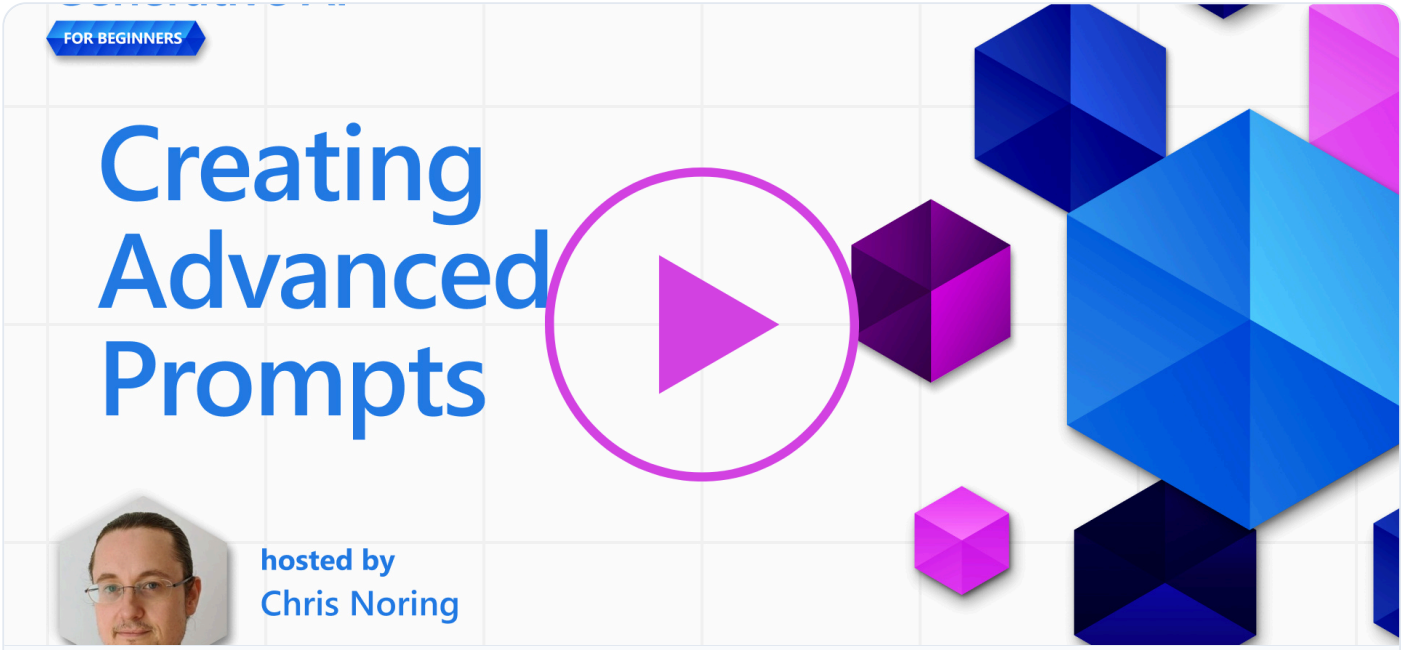
بازگشت به فهرست مطالب

درس ۰۴ — هوش مصنوعی مولد مایکروسافت

درس ۰۵ از ۲۱

طراحی و ساخت پرامپت‌های پیشرفته

تکنیک‌های پیشرفته مهندسی پرامپت، زنجیره تفکر (Chain of Thought - CoT)، جهت‌دهی خروجی ساختاریافته (JSON) و روش‌های ارزیابی پاسخ



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/BAjzkaCdRok?si=NmUIyRf7-cDgbjtt

پخش در یوتیوب

اسکن کد QR

بیایید برخی از مطالب فصل قبلی را مرور کنیم:

> **نکته کاربردی**
>
> مهندسی درخواست فرآیندی است که در آن ما **مدل را به سمت پاسخ‌های مرتبط‌تر هدایت می‌کنیم** با ارائه دستورالعمل‌ها یا زمینه‌های مفیدتر.

دو مرحله برای نوشتن درخواست‌ها وجود دارد: ساختن درخواست با ارائه زمینه مرتبط و **بهینه‌سازی**، یعنی بهبود تدریجی درخواست.

در این مرحله، ما درک اولیه‌ای از نحوه نوشتن درخواست‌ها داریم، اما باید عمیق‌تر شویم. در این فصل، شما از امتحان کردن درخواست‌های مختلف به درک اینکه چرا یک درخواست بهتر از دیگری است، خواهید رسید. شما یاد خواهید گرفت که چگونه درخواست‌ها را با استفاده از تکنیک‌های پایه‌ای که می‌توانند برای هر مدل زبانی بزرگ (LLM) اعمال شوند، بسازید.

مقدمه

در این فصل، موضوعات زیر را پوشش خواهیم داد:

- گسترش دانش خود در زمینه مهندسی درخواست با استفاده از تکنیک‌های مختلف در درخواست‌ها.
- تنظیم درخواست‌ها برای تغییر خروجی.

اهداف یادگیری

پس از اتمام این درس، شما قادر خواهید بود:

- تکنیک‌های مهندسی درخواست را که نتیجه درخواست‌های شما را بهبود می‌بخشند، اعمال کنید.
- درخواست‌هایی انجام دهید که یا متنوع باشند یا قطعی.

مهندسی درخواست

مهندسی درخواست فرآیند ایجاد درخواست‌هایی است که نتیجه مطلوب را تولید می‌کنند. مهندسی درخواست فقط نوشتن یک متن درخواست نیست. مهندسی درخواست یک رشته مهندسی نیست، بلکه مجموعه‌ای از تکنیک‌ها است که می‌توانید برای دستیابی به نتیجه مطلوب اعمال کنید.

نمونه‌ای از یک درخواست

بیایید یک درخواست ساده مانند این را بررسی کنیم:

> 💡 **نکته کاربردی**
>
> 10 سوال درباره جغرافیا تولید کنید.

در این درخواست، شما در واقع مجموعه‌ای از تکنیک‌های مختلف درخواست را اعمال می‌کنید.

بیایید این را تجزیه کنیم.

- **زمینه**، شما مشخص می‌کنید که باید درباره "جغرافیا" باشد.
- **محدود کردن خروجی**، شما می‌خواهید بیش از 10 سوال نباشد.

محدودیت‌های درخواست ساده

ممکن است نتیجه مطلوب را دریافت کنید یا نکنید. سوالات شما تولید خواهند شد، اما جغرافیا موضوع بزرگی است و ممکن است به دلیل دلایل زیر چیزی که می‌خواهید را دریافت نکنید:

- **موضوع بزرگ**، شما نمی‌دانید که آیا سوالات درباره کشورها، پایتخت‌ها، رودخانه‌ها و غیره خواهد بود.
- **فرمت**، اگر بخواهید سوالات به شکل خاصی فرمت شوند، چه؟

همانطور که می‌بینید، هنگام ایجاد درخواست‌ها باید موارد زیادی را در نظر گرفت.

تا کنون، ما یک نمونه درخواست ساده را دیده‌ایم، اما هوش مصنوعی مولد قادر به انجام کارهای بسیار بیشتری است تا به افراد در نقش‌ها و صنایع مختلف کمک کند. بیایید برخی تکنیک‌های پایه‌ای را بررسی کنیم.

تکنیک‌های درخواست‌دهی

ابتدا باید درک کنیم که درخواست‌دهی یک ویژگی *پدیدار* مدل زبانی بزرگ است، به این معنی که این ویژگی به طور خاص در مدل ساخته نشده است، بلکه چیزی است که در هنگام استفاده از مدل کشف می‌شود.

برخی تکنیک‌های پایه‌ای وجود دارند که می‌توانیم برای درخواست‌دهی از مدل زبانی بزرگ استفاده کنیم. بیایید آن‌ها را بررسی کنیم.

- **درخواست‌دهی بدون نمونه**، این ساده‌ترین شکل درخواست‌دهی است. این یک درخواست واحد است که پاسخ را فقط بر اساس داده‌های آموزشی مدل درخواست می‌کند.
- **درخواست‌دهی با چند نمونه**، این نوع درخواست‌دهی مدل را با ارائه یک یا چند مثال راهنمایی می‌کند تا بتواند پاسخ خود را تولید کند.
- **زنجیره تفکر**، این نوع درخواست‌دهی به مدل زبانی بزرگ می‌گوید که چگونه یک مسئله را به مراحل تقسیم کند.
- **دانش تولید شده**، برای بهبود پاسخ یک درخواست، می‌توانید حقایق یا دانش تولید شده را علاوه بر درخواست خود ارائه دهید.
- **کمترین به بیشترین**، مانند زنجیره تفکر، این تکنیک درباره تقسیم یک مسئله به مجموعه‌ای از مراحل و سپس درخواست انجام این مراحل به ترتیب است.
- **خود-بهبود**، این تکنیک درباره نقد خروجی مدل زبانی بزرگ و سپس درخواست بهبود آن است.
- **درخواست‌دهی مایوتیک**، در اینجا هدف این است که مطمئن شوید پاسخ مدل زبانی بزرگ صحیح است و از آن می‌خواهید بخش‌های مختلف پاسخ را توضیح دهد. این یک شکل از خود-بهبود است.

درخواست‌دهی بدون نمونه

این سبک درخواست‌دهی بسیار ساده است و شامل یک درخواست واحد می‌شود. این تکنیک احتمالاً همان چیزی است که شما هنگام شروع یادگیری درباره مدل‌های زبانی بزرگ استفاده می‌کنید. در اینجا یک مثال آورده شده است:

- درخواست: "جبر چیست؟"
- پاسخ: "جبر شاخه‌ای از ریاضیات است که به مطالعه نمادهای ریاضی و قوانین دستکاری این نمادها می‌پردازد."

درخواست‌دهی با چند نمونه

این سبک درخواست‌دهی به مدل کمک می‌کند با ارائه چند مثال همراه با درخواست. این شامل یک درخواست واحد با داده‌های خاص وظیفه اضافی است. در اینجا یک مثال آورده شده است:

- درخواست: "یک شعر به سبک شکسپیر بنویسید. در اینجا چند نمونه از سونات‌های شکسپیر آورده شده است: سونات 18: 'آیا تو را با روزی تابستانی مقایسه کنم؟ تو زیباتر و معتدل‌تر هستی...' سونات 116: 'اجازه ندهید به ازدواج ذهن‌های واقعی موانعی وارد شود. عشق عشق نیست که با تغییر تغییر کند...' سونات 132: 'چشمانت را دوست دارم، و آن‌ها، به عنوان ترحم به من، قلبت را که مرا با تحقیر شکنجه می‌کند، می‌شناسند...' حالا، یک سونات درباره زیبایی ماه بنویس."
- پاسخ: "بر فراز آسمان، ماه به آرامی می‌درخشد، در نوری نقره‌ای که لطف ملایم خود را می‌افکند..."

مثال‌ها به مدل زبانی بزرگ زمینه، فرمت یا سبک خروجی مطلوب را ارائه می‌دهند. آن‌ها به مدل کمک می‌کنند وظیفه خاص را درک کند و پاسخ‌های دقیق‌تر و مرتبط‌تری تولید کند.

زنجیره تفکر

زنجیره تفکر یک تکنیک بسیار جالب است زیرا درباره بردن مدل زبانی بزرگ از طریق یک سری مراحل است. ایده این است که مدل زبانی بزرگ را به گونه‌ای آموزش دهید که بفهمد چگونه کاری را انجام دهد. به مثال زیر توجه کنید، با و بدون زنجیره تفکر:

- درخواست: "آلیس 5 سیب دارد، 3 سیب پرت می‌کند، 2 سیب به باب می‌دهد و باب یکی را برمی‌گرداند، آلیس چند سیب دارد؟"
- پاسخ: 5

مدل زبانی بزرگ با پاسخ 5 پاسخ می‌دهد، که اشتباه است. پاسخ صحیح 1 سیب است، با توجه به محاسبه (5 - 3 - 2 + 1 = 1).

پس چگونه می‌توانیم به مدل زبانی بزرگ آموزش دهیم که این کار را به درستی انجام دهد؟

بیایید زنجیره تفکر را امتحان کنیم. اعمال زنجیره تفکر به این معناست که:

1. به مدل زبانی بزرگ یک مثال مشابه بدهید.
2. محاسبه را نشان دهید و نحوه محاسبه صحیح را توضیح دهید.
3. درخواست اصلی را ارائه دهید.

در اینجا نحوه انجام این کار آورده شده است:

- درخواست: "لیزا 7 سیب دارد، 1 سیب پرت می‌کند، 4 سیب به بارت می‌دهد و بارت یکی را برمی‌گرداند: 7 - 1 = 6 6 - 4 = 2 2 + 1 = 3
  آلیس 5 سیب دارد، 3 سیب پرت می‌کند، 2 سیب به باب می‌دهد و باب یکی را برمی‌گرداند، آلیس چند سیب دارد؟"
  پاسخ: 1

توجه کنید که چگونه درخواست‌ها را به طور قابل توجهی طولانی‌تر می‌نویسیم، با یک مثال دیگر، یک محاسبه و سپس درخواست اصلی، و به پاسخ صحیح 1 می‌رسیم.

همانطور که می‌بینید، زنجیره تفکر یک تکنیک بسیار قدرتمند است.

دانش تولید شده

بسیاری از مواقع که می‌خواهید یک درخواست بسازید، می‌خواهید این کار را با استفاده از داده‌های شرکت خود انجام دهید. شما می‌خواهید بخشی از درخواست از شرکت باشد و بخش دیگر درخواست اصلی باشد که به آن علاقه دارید.

به عنوان مثال، این چیزی است که درخواست شما می‌تواند به نظر برسد اگر شما در صنعت بیمه باشید:

متن / Text

```
{{company}}: {{company_name}}
{{products}}:
{{products_list}}
Please suggest an insurance given the following budget and requirements:
Budget: {{budget}}
Requirements: {{requirements}}
```

در بالا، می‌بینید که درخواست با استفاده از یک قالب ساخته شده است. در قالب تعدادی متغیر وجود دارد که با `{{variable}}` مشخص شده‌اند و با مقادیر واقعی از یک API شرکت جایگزین خواهند شد.

در اینجا یک مثال از نحوه ظاهر درخواست پس از جایگزینی متغیرها با محتوا از شرکت شما آورده شده است:

متن / Text

```
Insurance company: ACME Insurance
Insurance products (cost per month):
- Car, cheap, 500 USD
- Car, expensive, 1100 USD
- Home, cheap, 600 USD
- Home, expensive, 1200 USD
- Life, cheap, 100 USD

Please suggest an insurance given the following budget and requirements:
Budget: $1000
Requirements: Car, Home, and Life insurance
```

اجرای این درخواست از طریق مدل زبانی بزرگ یک پاسخ مانند این تولید خواهد کرد:

خروجی برنامه (Output) 📋

```
Given the budget and requirements, we suggest the following insurance package from ACME
Insurance:
- Car, cheap, 500 USD
- Home, cheap, 600 USD
- Life, cheap, 100 USD
Total cost: $1,200 USD
```

همانطور که می‌بینید، همچنین بیمه عمر را پیشنهاد می‌دهد، که نباید این کار را انجام دهد. این نتیجه نشان‌دهنده این است که ما باید درخواست را با تغییر آن به وضوح بیشتر بهینه کنیم. پس از چندین **آزمایش و خطا**، به درخواست زیر می‌رسیم:

متن / Text 📝

```
Insurance company: ACME Insurance
Insurance products (cost per month):
- type: Car, cheap, cost: 500 USD
- type: Car, expensive, cost: 1100 USD
- type: Home, cheap, cost: 600 USD
- type: Home, expensive, cost: 1200 USD
- type: Life, cheap, cost: 100 USD

Please suggest an insurance given the following budget and requirements:
Budget: $1000 restrict choice to types: Car, Home
```

توجه کنید که چگونه افزودن **نوع** و **هزینه** و همچنین استفاده از کلمه کلیدی **محدود کردن** به مدل زبانی بزرگ کمک می‌کند تا آنچه را که می‌خواهیم درک کند.

اکنون پاسخ زیر را دریافت می‌کنیم:

خروجی برنامه (Output) 📋

```
Given the budget and requirements, we suggest the Car, Cheap insurance product which costs 500
USD per month.
```

هدف این مثال این بود که نشان دهد حتی اگر از یک تکنیک پایه‌ای مانند **دانش تولید شده** استفاده کنیم، باز هم در اکثر موارد نیاز به بهینه‌سازی درخواست داریم تا به نتیجه مطلوب برسیم.

کمترین به بیشترین

ایده درخواست‌دهی کمترین به بیشترین این است که یک مسئله بزرگ‌تر را به زیرمسائل تقسیم کنید. به این ترتیب، شما به مدل زبانی بزرگ کمک می‌کنید تا مسئله بزرگ‌تر را "فتح" کند. یک مثال خوب می‌تواند برای علم داده باشد که می‌توانید از مدل

زبانی بزرگ بخواهید یک مسئله را به این صورت تقسیم کند:

> 💡 **نکته کاربردی**
>
> درخواست: چگونه علم داده را در 5 مرحله انجام دهیم؟

با پاسخ دستیار هوش مصنوعی:

1. جمع‌آوری داده‌ها
2. پاکسازی داده‌ها
3. تحلیل داده‌ها
4. رسم نمودار داده‌ها
5. ارائه داده‌ها

خود-بهبود، نقد نتایج

با هوش مصنوعی‌های مولد و مدل‌های زبانی بزرگ، نمی‌توانید به خروجی اعتماد کنید. باید آن را بررسی کنید. به هر حال، مدل زبانی بزرگ فقط چیزی را که احتمالاً بعدی گفته می‌شود ارائه می‌دهد، نه چیزی که صحیح است. بنابراین، ایده خوبی است که از مدل زبانی بزرگ بخواهید خودش را نقد کند، که ما را به تکنیک خود-بهبود می‌رساند.

نحوه کار این است که مراحل زیر را دنبال کنید:

1. درخواست اولیه برای حل یک مسئله از مدل زبانی بزرگ
2. مدل زبانی بزرگ پاسخ می‌دهد
3. شما پاسخ را نقد می‌کنید و از هوش مصنوعی می‌خواهید آن را بهبود دهد
4. مدل زبانی بزرگ دوباره پاسخ می‌دهد، این بار نقد را در نظر می‌گیرد و راه‌حل‌هایی را که ارائه کرده است پیشنهاد می‌دهد

شما می‌توانید این فرآیند را به تعداد دفعاتی که می‌خواهید تکرار کنید.

در اینجا یک مثال با استفاده از این تکنیک آورده شده است:

> 💡 **نکته کاربردی**
>
> درخواست: "یک API وب پایتون با مسیرهای محصولات و مشتریان ایجاد کنید"

پاسخ هوش مصنوعی:

Python

```
import flask
app = Flask(__name__)

@app.route('/products')
def products():
    return 'Products'

@app.route('/customers')
def customers():
    return 'Customers'
```

✨ نکته و پیشنهاد ویژه

درخواست: 3 بهبود برای کد بالا پیشنهاد دهید

پاسخ هوش مصنوعی:

متن / Text

```
Sure, here are three suggestions to improve the code:

Import the correct module: The first line should import Flask instead of flask. The correct
import statement should be from flask import Flask.

Add a main function: Add a if __name__ == '__main__': block to the end of the file to run the
application when the file is executed.

Use f-strings: Use f-strings to format the return statements in the route functions. For
example, return 'Products' can be changed to return f'This is the {request.path} page'.
```

Python

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/products')
def products():
    return f'This is the {request.path} page'

@app.route('/customers')
def customers():
    return f'This is the {request.path} page'

if __name__ == '__main__':
    app.run()
```

همانطور که می‌بینید، پاسخ هوش مصنوعی بالا کد پیشنهادی اول را به لطف نقد بهبود داده است.

درخواست‌دهی مایوتیک

درخواست‌دهی مایوتیک یک تکنیک است که مشابه خود-بهبود است اما بیشتر درباره درخواست از مدل زبانی بزرگ برای توضیح خودش است. هدف کاهش ناسازگاری‌ها در خروجی مدل زبانی بزرگ است تا مطمئن شویم به پاسخ صحیح می‌رسیم. روندی که باید دنبال کنید این است:

1. از مدل زبانی بزرگ بخواهید به یک سوال پاسخ دهد.
2. برای هر بخش از پاسخ، از مدل زبانی بزرگ بخواهید آن را با جزئیات بیشتری توضیح دهد.
3. اگر ناسازگاری‌هایی وجود دارد، بخش‌هایی که ناسازگار هستند را کنار بگذارید.

مراحل 2 و 3 را تکرار کنید تا زمانی که تمام بخش‌ها را بررسی کرده و از پاسخ راضی باشید.

در اینجا یک مثال درخواست آورده شده است:

> **💡 نکته کاربردی**
>
> درخواست: چگونه می‌توانم یک برنامه بحران برای کاهش یک پاندمی در 5 مرحله ایجاد کنم؟ پاسخ مدل زبانی بزرگ:

متن / Text

```
1. Identify the risks
1. Identify the stakeholders
1. Identify the resources
1. Identify the actions
1. Identify the communication channels
```

5 مرحله شناسایی شده است، اما آیا می‌توانیم تعیین کنیم که این درست است؟ بیایید از مدل زبانی بزرگ بخواهیم هر مرحله را با جزئیات بیشتری توضیح دهد:

> **💡 نکته کاربردی**
>
> درخواست: مرحله اول را با جزئیات بیشتری توضیح دهید، خطرات یک پاندمی به طور دقیق چیست؟

متن / Text

```
The risks are the following:

- Loss of life
- Loss of business
- Loss of reputation
- Loss of customers
- Loss of employees
```

در این مرحله، می‌خواهیم مطمئن شویم که مدل زبانی بزرگ درست است، بنابراین از آن می‌خواهیم اولین خطر را با جزئیات بیشتری توضیح دهد و امیدواریم که با پاسخ بالا سازگار باشد:

> 💡 **نکته کاربردی**
>
> درخواست: در یک پاندمی، بزرگ‌ترین خطر چیست و چرا؟

متن / Text

```
The biggest risk is loss of life because it's the most important thing to protect.
```

> 💡 **نکته کاربردی**
>
> بزرگ‌ترین دو خطر در یک پاندمی چیست؟

متن / Text

```
The two biggest risks are loss of life and loss of business.
```

در این مرحله، مدل زبانی بزرگ سازگار است و "زندگی" و "کسب‌وکار" را به عنوان دو خطر اصلی ذکر می‌کند. اکنون می‌توانیم به مرحله بعدی ادامه دهیم و احساس نسبتاً مطمئنی داشته باشیم. با این حال، نباید به مدل زبانی بزرگ به طور کورکورانه اعتماد کنیم، همیشه باید خروجی را بررسی کنیم.

تغییر خروجی خود

مدل‌های زبانی بزرگ به طور طبیعی غیرقطعی هستند، به این معنی که هر بار که یک درخواست مشابه را اجرا می‌کنید، نتایج متفاوتی دریافت خواهید کرد. به عنوان مثال، درخواست زیر را امتحان کنید:

💡 نکته کاربردی

"کدی برای یک API وب پایتون تولید کنید"

Python

```
# Import necessary modules
import flask
from flask import request, jsonify

# Create a Flask app
app = flask.Flask(__name__)

# Create a route for the API
@app.route('/api', methods=['GET'])
def api():
    # Get the data from the request
    data = request.args

    # Process the data
    result = process_data(data)

    # Return the result as JSON
    return jsonify(result)

# Function to process the data
def process_data(data):
    # Do something with the data
    result = data + 1

    # Return the result
    return result

# Run the app
if __name__ == '__main__':
    app.run()
```

اجرای دوباره همان درخواست، پاسخی کمی متفاوت تولید می‌کند:

Python

```
#import necessary packages
import flask
from flask import request, jsonify

#create the Flask application
app = flask.Flask(__name__)
app.config["DEBUG"] = True

#create a list of books
books = [
    {'id': 0,
     'title': 'A Fire Upon the Deep',
     'author': 'Vernor Vinge',
     'first_sentence': 'The coldsleep itself was dreamless.',
     'year_published': '1992'},
    {'id': 1,
     'title': 'The Ones Who Walk Away From Omelas',
     'author': 'Ursula K. Le Guin',
     'first_sentence': 'With a clamor of bells that set the swallows soaring, the Festival of
Summer came to the city Omelas, bright-towered by the sea.',
     'published': '1973'},
    {'id': 2,
     'title': 'Dhalgren',
     'author': 'Samuel R. Delany',
     'first_sentence': 'to wound the autumnal city.',
     'published': '1975'}
]

#create an endpoint for the API
@app.route('/', methods=['GET'])
def home():
    return '''<h1>Books API</h1>
<p>A prototype API for retrieving books.</p>'''

#create an endpoint to return all books
@app.route('/api/v1/resources/books/all', methods=['GET'])
def api_all():
    return jsonify(books)

#create an endpoint to return a single book
@app.route('/api/v1/resources/books', methods=['GET'])
def api_id():
    #check if an ID was provided as part of the URL
    #if ID is provided, assign it to a variable
    #if no ID is provided, display an error
    if 'id' in request.args:
        id = int(request.args['id'])
    else:
        return "Error: No id field provided. Please specify an id."

    #create an empty list for our results
    results = []

    #loop through the data and match results that fit the requested ID
    #IDs are unique, but other fields might return many results
    for book in books:
        if book['id'] == id:
            results.append(book)

    #use the jsonify function from Flask to convert our list of
    #Python dictionaries to the JSON format
    return jsonify(results)

app.run()
```

> 💡 **نکته کاربردی**
>
> آیا خروجی متنوع یک مشکل است؟

بستگی به این دارد که چه چیزی را می‌خواهید انجام دهید. اگر به یک پاسخ خاص نیاز دارید، این یک مشکل است. اگر با خروجی متنوع مانند "تولید هر 3 سوال درباره جغرافیا" مشکلی ندارید، پس مشکلی نیست.

استفاده از دما برای تغییر خروجی

خب، تصمیم گرفته‌ایم که می‌خواهیم خروجی را محدود کنیم تا قابل پیش‌بینی‌تر باشد، یعنی قطعی‌تر. چگونه این کار را انجام دهیم؟

دمای یک مقدار بین 0 و 1 است، که 0 قطعی‌ترین و 1 متنوع‌ترین است. مقدار پیش‌فرض 0.7 است. بیایید ببینیم چه اتفاقی می‌افتد با دو اجرای یک درخواست مشابه با دمای تنظیم شده به 0.1:

> 💡 **نکته کاربردی**
>
> "کدی برای یک API وب پایتون تولید کنید"

Python

```python
#import necessary libraries
import flask
from flask import request, jsonify

#create an instance of the Flask class
app = flask.Flask(__name__)

#create an endpoint for the API
@app.route('/api/v1/endpoint', methods=['GET'])
def api_endpoint():
    #get the request data
    data = request.get_json()

    #process the data
    result = process_data(data)

    #return the result
    return jsonify(result)

#function to process the data
def process_data(data):
    #process the data
    result = {'result': 'success'}

    #return the result
    return result

#run the Flask app
if __name__ == '__main__':
    app.run()
```

اجرای دوباره درخواست نتیجه زیر را می‌دهد:

Python

```
#import necessary libraries
import flask
from flask import request, jsonify

#create a Flask app
app = flask.Flask(__name__)

#create a route for the API
@app.route('/api', methods=['GET'])
def api_call():
    #get the data from the request
    data = request.get_json()

    #process the data
    result = process_data(data)

    #return the result
    return jsonify(result)

#define the process_data function
def process_data(data):
    #do something with the data
    result = data + 1

    #return the result
    return result

#run the app
if __name__ == '__main__':
    app.run()
```

فقط تفاوت کوچکی بین این دو خروجی وجود دارد. این بار برعکس عمل کنیم، دما را به 0.9 تنظیم کنیم:

Python

```python
# Import necessary libraries
import flask
from flask import request, jsonify

# Create a Flask app
app = flask.Flask(__name__)

# Create a route for the API
@app.route('/api', methods=['GET'])
def api_call():
    # Get the data from the request
    data = request.args

    # Process the data
    result = process_data(data)

    # Return the result
    return jsonify(result)

# Function to process the data
def process_data(data):
    # Do something with the data
    result = data + 1

    # Return the result
    return result

# Run the app
if __name__ == '__main__':
    app.run()
```

و تلاش دوم با مقدار دما 0.9:

Python

```
import flask
from flask import request, jsonify

# create the Flask app
app = flask.Flask(__name__)
app.config['DEBUG'] = True

# create some test data
books = [
    {'id': 0, 'title': 'A Fire Upon The Deep', 'author': 'Vernor Vinge', 'first_sentence':
'The coldsleep itself was dreamless.', 'year_published': '1992'},
    {'id': 1, 'title': 'The Ones Who Walk Away From Omelas', 'author': 'Ursula K. Le Guin',
'first_sentence': 'With a clamor of bells that set the swallows soaring, the Festival of
Summer came to the city Omelas, bright-towered by the sea.', 'published': '1973'},
    {'id': 2, 'title': 'Dhalgren', 'author': 'Samuel R. Delany', 'first_sentence': 'to wound
the autumnal city.', 'published': '1975'}
]

# create an endpoint
@app.route('/', methods=['GET'])
def home():
    return '''<h1>Welcome to our book API!</h1>'''

@app.route('/api/v1/resources/books
```

همانطور که می‌بینید، نتایج نمی‌توانستند متنوع‌تر باشند.

> 💡 **نکته کاربردی**
>
> توجه داشته باشید که پارامترهای بیشتری وجود دارند که می‌توانید برای تغییر خروجی تنظیم کنید، مانند top-k، top-p، جریمه تکرار، جریمه طول و جریمه تنوع، اما این موارد خارج از محدوده این دوره آموزشی هستند.

روش‌های خوب

روش‌های زیادی وجود دارند که می‌توانید برای دستیابی به نتیجه دلخواه خود به کار ببرید. با استفاده بیشتر از تکنیک‌های درخواست‌دهی، سبک خود را پیدا خواهید کرد.

علاوه بر تکنیک‌هایی که پوشش داده‌ایم، برخی روش‌های خوب وجود دارند که هنگام درخواست از یک مدل زبان بزرگ (LLM) باید در نظر بگیرید.

در اینجا چند روش خوب برای در نظر گرفتن آورده شده است:

- **مشخص کردن زمینه**. زمینه اهمیت دارد، هرچه بتوانید بیشتر مشخص کنید، مانند حوزه، موضوع و غیره، بهتر است.
- محدود کردن خروجی. اگر تعداد مشخصی از آیتم‌ها یا طول خاصی می‌خواهید، آن را مشخص کنید.

- **مشخص کردن چه چیزی و چگونه**. به یاد داشته باشید که هم آنچه می‌خواهید و هم نحوه انجام آن را ذکر کنید، برای مثال "یک API وب پایتون با مسیرهای محصولات و مشتریان ایجاد کنید، آن را به ۳ فایل تقسیم کنید".
- **استفاده از قالب‌ها**. اغلب، شما می‌خواهید درخواست‌های خود را با داده‌های شرکت خود غنی کنید. از قالب‌ها برای این کار استفاده کنید. قالب‌ها می‌توانند متغیرهایی داشته باشند که با داده‌های واقعی جایگزین شوند.
- **درست نوشتن**. مدل‌های زبان ممکن است پاسخ صحیحی به شما بدهند، اما اگر درست بنویسید، پاسخ بهتری دریافت خواهید کرد.

تکلیف

در اینجا کدی در پایتون آورده شده است که نشان می‌دهد چگونه می‌توان یک API ساده با استفاده از Flask ساخت:

Python

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/')
def hello():
    name = request.args.get('name', 'World')
    return f'Hello, {name}!'

if __name__ == '__main__':
    app.run()
```

از یک دستیار هوش مصنوعی مانند GitHub Copilot یا ChatGPT استفاده کنید و تکنیک "خود-بهبود" را برای بهبود کد اعمال کنید.

راه‌حل

لطفاً سعی کنید با افزودن درخواست‌های مناسب به کد، تکلیف را حل کنید.

> 💡 **نکته کاربردی**
>
> یک درخواست را برای بهبود فرموله کنید، ایده خوبی است که تعداد بهبودها را محدود کنید. همچنین می‌توانید درخواست کنید که بهبود در یک زمینه خاص مانند معماری، عملکرد، امنیت و غیره انجام شود.

راه‌حل

بررسی دانش

چرا باید از درخواست‌دهی زنجیره‌ای استفاده کنم؟ یک پاسخ صحیح و دو پاسخ نادرست را نشان دهید.

1. برای آموزش مدل زبان بزرگ (LLM) در مورد نحوه حل یک مسئله.
2. ب، برای آموزش مدل زبان بزرگ به یافتن خطاها در کد.
3. ج، برای دستور دادن به مدل زبان بزرگ برای ارائه راه‌حل‌های مختلف.

پاسخ: 1، زیرا درخواست‌دهی زنجیره‌ای به مدل زبان بزرگ نشان می‌دهد که چگونه یک مسئله را با ارائه یک سری مراحل و مسائل مشابه و نحوه حل آن‌ها حل کند.

🚀 چالش

شما به تازگی از تکنیک خود-بهبود در تکلیف استفاده کردید. هر برنامه‌ای که ساخته‌اید را بردارید و به این فکر کنید که چه بهبودهایی می‌خواهید در آن اعمال کنید. اکنون از تکنیک خود-بهبود برای اعمال تغییرات پیشنهادی استفاده کنید. به نظر شما نتیجه بهتر بود یا بدتر؟

کار عالی! یادگیری خود را ادامه دهید

پس از اتمام این درس، مجموعه آموزش هوش مصنوعی مولد ما را بررسی کنید تا دانش خود در زمینه هوش مصنوعی مولد را ارتقا دهید!

به درس ۶ بروید، جایی که دانش خود در زمینه مهندسی درخواست‌دهی را با ساخت اپلیکیشن‌های تولید متن به کار خواهیم گرفت.

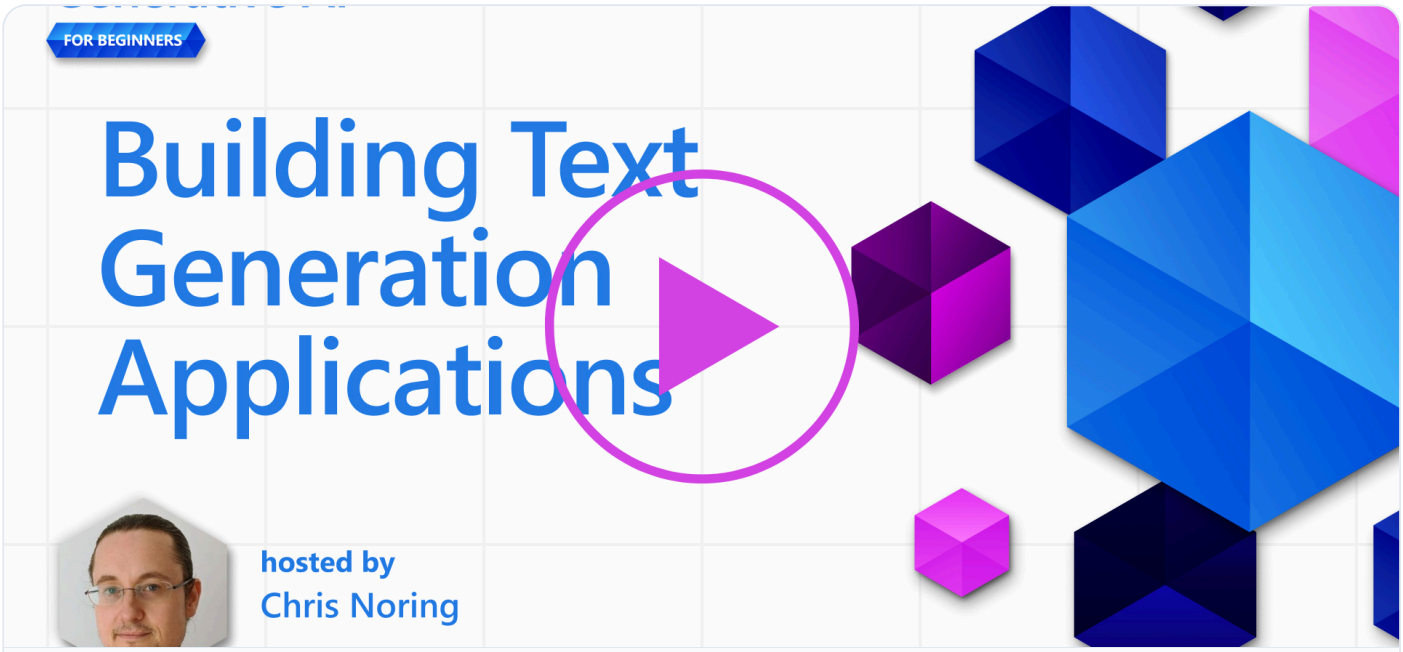
سلب مسئولیت:
این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما تلاش می‌کنیم دقت را حفظ کنیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌ها باشند. سند اصلی به زبان اصلی آن باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما مسئولیتی در قبال سوء تفاهم‌ها یا تفسیرهای نادرست ناشی از استفاده از این ترجمه نداریم.

بازگشت به فهرست مطالب
درس ۰۵ — هوش مصنوعی مولد مایکروسافت

درس ۰۶ از ۲۱

ساخت برنامه‌های کاربردی تولید متن

کدنویسی با SDK پایتون و تایپ‌اسکریپت برای فراخوانی API، کنترل پارامترهای Temperature و Top_P و تولید متن‌های خلاقانه و خلاصه



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/0Y5Luf5sRQA?si=t_xVg0clnAI4oUFZ

▶️ پخش در یوتیوب

اسکن کد QR

تا اینجا در این دوره دیده‌اید که مفاهیم اصلی‌ای مثل پرامپت‌ها و حتی حوزه‌ای به نام «مهندسی پرامپت» وجود دارد. ابزارهای زیادی مثل Microsoft Power Platform، Office 365، ChatGPT و غیره وجود دارند که با کمک پرامپت‌ها به شما در انجام کارها کمک می‌کنند.

برای اینکه چنین تجربه‌ای را به اپلیکیشن خود اضافه کنید، باید مفاهیمی مثل پرامپت‌ها، تکمیل‌ها را بفهمید و یک کتابخانه برای کار انتخاب کنید. دقیقا این موضوعات را در این فصل آموزش می‌بینید.

معرفی

در این فصل، شما خواهید:

- درباره کتابخانه openai و مفاهیم اصلی آن یاد بگیرید.
- یک اپلیکیشن تولید متن با استفاده از openai بسازید.
- درک کنید چگونه از مفاهیمی مانند پرامپت، دما و توکن‌ها برای ساخت اپلیکیشن تولید متن استفاده کنید.

اهداف یادگیری

در انتهای این درس، قادر خواهید بود:

- توضیح دهید اپ تولید متن چیست.
- با استفاده از openai یک اپ تولید متن بسازید.
- اپ خود را طوری تنظیم کنید که توکن‌های بیشتری یا کمتری استفاده کند و همچنین دما را برای خروجی متنوع تغییر دهید.

اپ تولید متن چیست؟

معمولاً وقتی اپلیکیشنی می‌سازید، دارای نوعی رابط کاربری مانند موارد زیر است:

- مبتنی بر فرمان (Command-based). اپ‌های کنسولی معمولی هستند که در آن‌ها فرمانی تایپ می‌کنید و کاری انجام می‌شود. برای مثال، `git` یک اپ مبتنی بر فرمان است.
- رابط کاربری (UI). برخی اپ‌ها دارای رابط گرافیکی هستند که در آنجا دکمه کلیک می‌کنید، متن وارد می‌کنید، گزینه‌ها را انتخاب می‌کنید و غیره.

اپ‌های کنسولی و رابط کاربری محدودیت دارند

آن را با اپ مبتنی بر فرمانی که شما فرمان می‌دهید مقایسه کنید:

- **محدود است**. نمی‌توانید هر فرمانی را تایپ کنید، فقط آن‌هایی که اپ پشتیبانی می‌کند.
- **مخصوص زبان خاص**. برخی اپ‌ها از زبان‌های زیادی پشتیبانی می‌کنند، اما به‌طور پیش‌فرض اپ برای یک زبان خاص ساخته شده است، حتی اگر بتوانید زبان‌های بیشتری اضافه کنید.

مزایای اپ‌های تولید متن

پس اپ تولید متن چه تفاوتی دارد؟

در یک اپ تولید متن، انعطاف بیشتری دارید، محدود به مجموعه فرمان یا زبان ورودی خاص نیستید. در عوض، می‌توانید با زبان طبیعی با اپ تعامل کنید. یک مزیت دیگر این است که شما در حال تعامل با منبع داده‌ای هستید که روی مجموعه وسیعی از اطلاعات آموزش دیده است، در حالی که یک اپ سنتی ممکن است به داده‌های یک پایگاه داده محدود باشد.

با اپ تولید متن چه می‌توان ساخت؟

می‌توانید چیزهای زیادی بسازید. برای مثال:

- **چت‌بات**. چت‌باتی که به پرسش‌ها درباره موضوعات مختلف مثل شرکت و محصولات شما پاسخ می‌دهد می‌تواند گزینه خوبی باشد.
- **دستیار**. مدل‌های زبان بزرگ (LLM) در کارهایی مانند خلاصه‌سازی متن، استخراج اطلاعات، تولید متنی مانند رزومه و غیره عالی هستند.
- **دستیار کد**. بسته به مدل زبانی که استفاده می‌کنید، می‌توانید دستیار کدی بسازید که در نوشتن کد به شما کمک کند. برای مثال، می‌توانید از محصولی مثل GitHub Copilot یا ChatGPT برای نوشتن کد کمک بگیرید.

چگونه شروع کنم؟

خوب، باید روشی برای اتصال به یک مدل زبان بزرگ پیدا کنید که معمولاً شامل دو رویکرد زیر است:

- استفاده از API. اینجا شما درخواست‌های وب را با پرامپت خود می‌سازید و متن تولید شده را دریافت می‌کنید.
- استفاده از کتابخانه. کتابخانه‌ها فراخوانی‌های API را بسته‌بندی می‌کنند و استفاده از آن‌ها را آسان می‌کنند.

کتابخانه‌ها / SDKها

چند کتابخانه شناخته شده برای کار با مدل‌های زبان بزرگ وجود دارد، مانند:

- **openai**، این کتابخانه اتصال به مدل شما و ارسال پرامپت‌ها را آسان می‌کند.

سپس کتابخانه‌هایی وجود دارند که در سطح بالاتر عمل می‌کنند، مانند:

- **Langchain**. لانگ‌چین معروف است و از زبان پایتون پشتیبانی می‌کند.
- **Semantic Kernel**. سماتیک کرنل کتابخانه‌ای از مایکروسافت است که از زبان‌های C#، پایتون و جاوا پشتیبانی می‌کند.

اولین اپ با استفاده از openai

بیایید ببینیم چگونه اولین اپ خود را می‌سازیم، به چه کتابخانه‌هایی نیاز داریم و چه مقدار لازم است و غیره.

نصب openai

کتابخانه‌های زیادی برای تعامل با OpenAI یا Azure OpenAI وجود دارد. ممکن است چندین زبان برنامه‌نویسی هم استفاده کنید مانند C#، پایتون، جاوااسکریپت، جاوا و غیره. ما انتخاب کردیم که از کتابخانه پایتون `openai` استفاده کنیم، پس از `pip` برای نصب آن استفاده می‌کنیم.

Terminal / Bash

```
pip install openai
```

ایجاد یک منبع

شما باید مراحل زیر را انجام دهید:

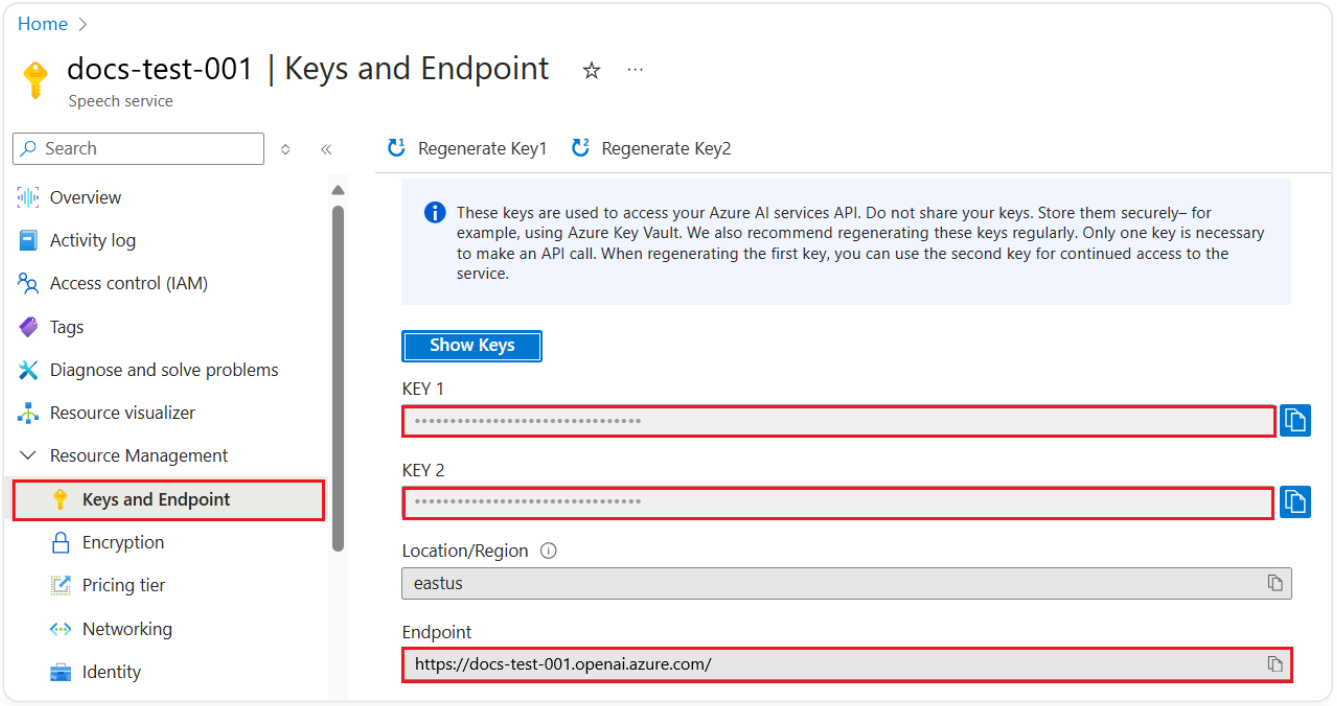
- یک حساب در Azure ایجاد کنید https://azure.microsoft.com/free/.
- به Azure OpenAI دسترسی پیدا کنید. به https://learn.microsoft.com/azure/ai-foundry/openai/overview#how-do-i-get-access-to-azure-openai بروید و درخواست دسترسی کنید.

> 💡 **نکته کاربردی**
>
> در زمان نگارش، باید برای دسترسی به Azure OpenAI درخواست دهید.

- نصب پایتون https://www.python.org/
- یک منبع سرویس Azure OpenAI ساخته باشید. این راهنما را ببینید برای اینکه چگونه یک منبع بسازید.

پیدا کردن کلید API و نقطه پایانی

اکنون باید به کتابخانه `openai` بگویید از چه کلید API استفاده کند. برای یافتن کلید API خود، به بخش "Keys and Endpoint" منبع Azure OpenAI خود بروید و مقدار "Key 1" را کپی کنید.



شکل: پنجره کلیدها و نقطه پایانی (Keys and Endpoint) در درگاه ابری Azure

📌 راهنمای کادرهای قرمز در تصویر بالا (Keys & Endpoint):

- **کادر شماره ۱ (KEY 1):** کلید محرمانه اصلی برای احراز هویت درخواست‌های کلاینت در کد برنامه.
- **کادر شماره ۲ (KEY 2):** کلید پشتیبان؛ در صورتی که بخواهید کلید اول را بازتولید (Regenerate) کنید، این کلید دسترسی مداوم را حفظ می‌کند.
- **کادر شماره ۳ (Endpoint):** آدرس پایانه اختصاصی وب‌سرویس شما برای اتصال کلاینت OpenAI.

💡 راهنمای طلایی: اجرای ۱۰۰٪ رایگان و محلی تمرین‌ها با Ollama (بدون نیاز به Azure یا ویزاکارت)

اگر حساب Azure یا کارت اعتباری بین‌المللی ندارید، اصلاً نگران نباشید! شما می‌توانید تمام تمرین‌های این درس و سراسر کتاب را بدون پرداخت حتی یک ریال، بدون تحریم و کاملاً آفلاین روی کامپیوتر شخصی خودتان اجرا کنید:

۱. **نصب Ollama:** نرم‌افزار رایگان Ollama را دانلود و نصب کنید.

۲. **اجرای یک مدل سریع و سبک:** در ترمینال سیستم خود تایپ کنید: `ollama run phi3` (یا `ollama run llama3.2:1b`).

۳. **اجرای کدهای پایتون این درس:** در کدهای پایتون این درس، کافیست دو خط زیر را تنظیم کنید:

```
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
```

کد شما بلافاصله به مدل محلی متصل شده و پاسخ‌ها را دقیقاً با همان کلاینت استاندارد پایتون تولید می‌کند!

اکنون که این اطلاعات را کپی کرده‌اید، بیایید به کتابخانه‌ها دستور دهیم که از آن استفاده کنند.

> **نکته کاربردی**
>
> ارزش دارد کلید API خود را از کد جدا کنید. می‌توانید این کار را با استفاده از متغیرهای محیطی انجام دهید.
>
> - متغیر محیطی `OPENAI_API_KEY` را به کلید API خود تنظیم کنید. `export OPENAI_API_KEY='sk-...'`

پیکربندی Azure

اگر از Azure OpenAI (که اکنون بخشی از Microsoft Foundry است) استفاده می‌کنید، این‌گونه پیکربندی می‌کنید. ما از کلاینت استاندارد `OpenAI` استفاده می‌کنیم که به نقطه پایانی Azure OpenAI `/openai/v1/` اشاره دارد، که با Responses API کار می‌کند و نیازی به `api_version` ندارد:

Python

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    base_url=f"{os.environ['AZURE_OPENAI_ENDPOINT'].rstrip('/')}/openai/v1/",
)
```

بالاتر موارد زیر را تنظیم می‌کنیم:

- `api_key`، این کلید API شما است که در پورتال Azure یا Microsoft Foundry پیدا می‌شود.
- `base_url`، این نقطه پایانی منبع Foundry شما است که با `/openai/v1/` اضافه شده است. نقطه پایانی پایدار v1 در OpenAI و Azure OpenAI بدون نیاز به مدیریت `api_version` کار می‌کند.

> **نکته کاربردی**
>
> `os.environ` > متغیرهای محیطی را می‌خواند. می‌توانید از آن برای خواندن متغیرهای محیطی مانند `AZURE_OPENAI_API_KEY` و `AZURE_OPENAI_ENDPOINT` استفاده کنید. این متغیرها را در ترمینال یا با استفاده از کتابخانه‌ای مانند `dotenv` تنظیم کنید.

تولید متن

روش تولید متن استفاده از Responses API از طریق متد `responses.create` است. مثال زیر را ببینید:

Python

```
prompt = "Complete the following: Once upon a time there was a"

response = client.responses.create(
    model="gpt-5-mini",  # این نام استقرار مدل شما است
    input=prompt,
    store=False,
)
print(response.output_text)
```

در کد بالا، پاسخ ایجاد می‌کنیم و مدل و پرامپت موردنظر را می‌دهیم. سپس متن تولید شده را با `response.output_text` چاپ می‌کنیم.

مکالمات چندمرحله‌ای

Responses API برای تولید متن تک مرحله‌ای و چت‌بات‌های چندمرحله‌ای مناسب است - شما لیستی از پیام‌ها را در `input` می‌دهید تا یک مکالمه بسازید:

Python

```
from openai import OpenAI

client = OpenAI(api_key="sk-...")

response = client.responses.create(model="gpt-5-mini", input="Hello world", store=False)
print(response.output_text)
```

درباره این قابلیت در فصل‌های بعد بیشتر توضیح داده خواهد شد.

تمرین - اولین اپ تولید متن شما

حالا که یاد گرفتیم چگونه openai را راه‌اندازی و پیکربندی کنیم، وقت ساخت اولین اپ تولید متن شما است. برای ساخت اپ خود، مراحل زیر را دنبال کنید:

1. یک محیط مجازی بسازید و openai را نصب کنید:

Terminal / Bash

```
python -m venv venv
source venv/bin/activate
pip install openai
```

> 💡 **نکته کاربردی**
>
> اگر از ویندوز استفاده می‌کنید، به جای `source venv/bin/activate` تایپ کنید `venv\Scripts\activate`.

> 💡 **نکته کاربردی**
>
> کلید Azure OpenAI خود را با مراجعه به https://portal.azure.com/ پیدا کنید؛ جستجو کنید `Open AI`، گزینه `Open AI resource` را انتخاب کنید، سپس `Keys and Endpoint` را انتخاب کنید و مقدار `Key 1` را انتخاب کنید و را کپی کنید.

2. یک فایل *app.py* بسازید و کد زیر را در آن قرار دهید:

Python

```
import os
from openai import OpenAI

client = OpenAI(
    api_key="<replace this value with your Azure OpenAI key>",
    base_url="<endpoint found in Azure Portal>/openai/v1/",
)
deployment_name = "<deployment name>"

# کد تکمیل خود را اضافه کنید
prompt = "Complete the following: Once upon a time there was a"

# پاسخ‌ها ایجاد کنید API درخواست با استفاده از
response = client.responses.create(model=deployment_name, input=prompt, store=False)

# پاسخ را چاپ کنید
print(response.output_text)
```

> 💡 **نکته کاربردی**
>
> اگر از OpenAI معمولی (نه Azure) استفاده می‌کنید، از `client = OpenAI(api_key="<replace this value with your OpenAI key>")` استفاده کنید (بدون `base_url`) و به جای نام استقرار، نام مدل مانند `gpt-5-mini` بدهید.

باید خروجی مشابه زیر ببینید:

خروجی برنامه (Output) 📋

```
very unhappy _____.

Once upon a time there was a very unhappy mermaid.
```

انواع مختلف پرامپت‌ها برای کارهای مختلف

اکنون دیدید چگونه با استفاده از یک پرامپت، متن تولید کنید. حتی یک برنامه دارید که اجرا می‌شود و می‌توانید آن را تغییر دهید تا انواع مختلف متن تولید کند.

پرامپت‌ها می‌توانند برای انواع کارها استفاده شوند. برای مثال:

- **تولید نوعی متن**. برای مثال می‌توانید شعر، سوالات یک آزمون و غیره تولید کنید.
- **جستجوی اطلاعات**. می‌توانید از پرامپت‌ها برای پرسش درباره اطلاعاتی مانند مثال زیر استفاده کنید: «CORS در توسعه وب چیست؟».
- **تولید کد**. می‌توانید از پرامپت‌ها برای تولید کد استفاده کنید، مثلا توسعه یک عبارت منظم برای اعتبارسنجی ایمیل‌ها یا حتی تولید یک برنامه کامل، مثل یک اپ وب.

یک مورد کاربردی‌تر: تولید کننده دستور پخت

تصور کنید مواد اولیه‌ای در خانه دارید و می‌خواهید چیزی بپزید. برای این کار به دستور پخت نیاز دارید. یکی از راه‌ها استفاده از موتور جستجو است یا می‌توانید از یک مدل زبان بزرگ استفاده کنید.

می‌توانید پرامپتی مانند زیر بنویسید:

> 💡 **نکته کاربردی**
>
> «۵ دستور پخت برای غذایی با مواد زیر نشان بده: مرغ، سیب‌زمینی و هویج. برای هر دستور، همه مواد استفاده شده را فهرست کن»

با توجه به پرامپت بالا، ممکن است پاسخی شبیه به این دریافت کنید:

خروجی برنامه (Output) 📋

```
1. Roasted Chicken and Vegetables:
Ingredients:
- 4 chicken thighs
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 2 tablespoons olive oil
- 2 cloves garlic, minced
- 1 teaspoon dried thyme
- 1 teaspoon dried oregano
- Salt and pepper, to taste

2. Chicken and Potato Stew:
Ingredients:
- 2 tablespoons olive oil
- 1 onion, diced
- 2 cloves garlic, minced
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 teaspoon dried oregano
- 1 teaspoon dried thyme
- 1 cup chicken broth
- Salt and pepper, to taste

3. Chicken and Potato Bake:
Ingredients:
- 2 tablespoons olive oil
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 onion, diced
- 2 cloves garlic, minced
- 1 teaspoon dried oregano
- 1 teaspoon dried thyme
- 1 cup chicken broth
- Salt and pepper, to taste

4. Chicken and Potato Soup:
Ingredients:
- 2 tablespoons olive oil
- 1 onion, diced
- 2 cloves garlic, minced
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 teaspoon dried oregano
- 1 teaspoon dried thyme
- 4 cups chicken broth
- Salt and pepper, to taste

5. Chicken and Potato Hash:
Ingredients:
- 2 tablespoons olive oil
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 onion, diced
- 2 cloves garlic, minced
- 1 teaspoon dried oregano
```

این نتیجه عالی است، من می‌دانم چه بپزم. در این نقطه، بهبودهای مفید می‌تواند اینها باشد:

- حذف موادی که دوست ندارم یا به آن‌ها حساسیت دارم.
- تولید لیست خرید، در صورتی که همه مواد را در خانه ندارم.

برای موارد بالا، بیایید یک پرامپت اضافی اضافه کنیم:

> **💡 نکته کاربردی**
>
> «لطفاً دستور پخت‌هایی که سیر دارند حذف کن چون به آن حساسیت دارم و آن را با چیز دیگری جایگزین کن. همچنین، لطفاً لیست خرید برای دستور پخت‌ها تولید کن، با توجه به اینکه من قبلاً مرغ، سیب‌زمینی و هویج در خانه دارم.»

اکنون یک نتیجه جدید دارید، به این صورت:

خروجی برنامه (Output) 📋

```
1. Roasted Chicken and Vegetables:
Ingredients:
- 4 chicken thighs
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 2 tablespoons olive oil
- 1 teaspoon dried thyme
- 1 teaspoon dried oregano
- Salt and pepper, to taste

2. Chicken and Potato Stew:
Ingredients:
- 2 tablespoons olive oil
- 1 onion, diced
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 teaspoon dried oregano
- 1 teaspoon dried thyme
- 1 cup chicken broth
- Salt and pepper, to taste

3. Chicken and Potato Bake:
Ingredients:
- 2 tablespoons olive oil
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 onion, diced
- 1 teaspoon dried oregano
- 1 teaspoon dried thyme
- 1 cup chicken broth
- Salt and pepper, to taste

4. Chicken and Potato Soup:
Ingredients:
- 2 tablespoons olive oil
- 1 onion, diced
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 teaspoon dried oregano
- 1 teaspoon dried thyme
- 4 cups chicken broth
- Salt and pepper, to taste

5. Chicken and Potato Hash:
Ingredients:
- 2 tablespoons olive oil
- 2 chicken breasts, cut into cubes
- 2 potatoes, cut into cubes
- 2 carrots, cut into cubes
- 1 onion, diced
- 1 teaspoon dried oregano

Shopping List:
- Olive oil
- Onion
- Thyme
- Oregano
- Salt
- Pepper
```

این پنج دستور پخت شما است، بدون هیچ اشاره‌ای به سیر و همچنین لیست خرید با توجه به مواد موجود در خانه‌تان دارید.

تمرین - ساخت تولید کننده دستور پخت

حال که این سناریو را اجرا کردیم، بیایید کد بنویسیم که با این سناریو مطابقت داشته باشد. برای این کار، مراحل زیر را دنبال کنید:

1. از فایل موجود *app.py* به عنوان نقطه شروع استفاده کنید
2. متغیر `prompt` را پیدا کنید و کد آن را به شکل زیر تغییر دهید:

Python

```
prompt = "Show me 5 recipes for a dish with the following ingredients: chicken, potatoes, and carrots. Per recipe, list all the ingredients used"
```

اگر کد را اکنون اجرا کنید، باید خروجی‌ای مشابه زیر ببینید:

خروجی برنامه (Output) 📋

```
-Chicken Stew with Potatoes and Carrots: 3 tablespoons oil, 1 onion, chopped, 2 cloves garlic, minced, 1 carrot, peeled and chopped, 1 potato, peeled and chopped, 1 bay leaf, 1 thyme sprig, 1/2 teaspoon salt, 1/4 teaspoon black pepper, 1 1/2 cups chicken broth, 1/2 cup dry white wine, 2 tablespoons chopped fresh parsley, 2 tablespoons unsalted butter, 1 1/2 pounds boneless, skinless chicken thighs, cut into 1-inch pieces
-Oven-Roasted Chicken with Potatoes and Carrots: 3 tablespoons extra-virgin olive oil, 1 tablespoon Dijon mustard, 1 tablespoon chopped fresh rosemary, 1 tablespoon chopped fresh thyme, 4 cloves garlic, minced, 1 1/2 pounds small red potatoes, quartered, 1 1/2 pounds carrots, quartered lengthwise, 1/2 teaspoon salt, 1/4 teaspoon black pepper, 1 (4-pound) whole chicken
-Chicken, Potato, and Carrot Casserole: cooking spray, 1 large onion, chopped, 2 cloves garlic, minced, 1 carrot, peeled and shredded, 1 potato, peeled and shredded, 1/2 teaspoon dried thyme leaves, 1/4 teaspoon salt, 1/4 teaspoon black pepper, 2 cups fat-free, low-sodium chicken broth, 1 cup frozen peas, 1/4 cup all-purpose flour, 1 cup 2% reduced-fat milk, 1/4 cup grated Parmesan cheese

-One Pot Chicken and Potato Dinner: 2 tablespoons olive oil, 1 pound boneless, skinless chicken thighs, cut into 1-inch pieces, 1 large onion, chopped, 3 cloves garlic, minced, 1 carrot, peeled and chopped, 1 potato, peeled and chopped, 1 bay leaf, 1 thyme sprig, 1/2 teaspoon salt, 1/4 teaspoon black pepper, 2 cups chicken broth, 1/2 cup dry white wine

-Chicken, Potato, and Carrot Curry: 1 tablespoon vegetable oil, 1 large onion, chopped, 2 cloves garlic, minced, 1 carrot, peeled and chopped, 1 potato, peeled and chopped, 1 teaspoon ground coriander, 1 teaspoon ground cumin, 1/2 teaspoon ground turmeric, 1/2 teaspoon ground ginger, 1/4 teaspoon cayenne pepper, 2 cups chicken broth, 1/2 cup dry white wine, 1 (15-ounce) can chickpeas, drained and rinsed, 1/2 cup raisins, 1/2 cup chopped fresh cilantro
```

> 💡 **نکته کاربردی**
>
> توجه، مدل زبان شما غیرقطعی است، بنابراین ممکن است هر بار که برنامه را اجرا می‌کنید نتایج متفاوتی بگیرید.

عالی، بیایید ببینیم چگونه می‌توانیم بهترش کنیم. برای بهبود می‌خواهیم مطمئن شویم کد انعطاف‌پذیر است، بنابراین تعداد دستور پخت‌ها و مواد اولیه می‌توانند تغییر کنند.

3. کد را به شکل زیر تغییر دهیم:

Python

```
no_recipes = input("No of recipes (for example, 5): ")

ingredients = input("List of ingredients (for example, chicken, potatoes, and carrots):
")

# تعداد دستورهای آشپزی را در متن درخواست و مواد اولیه وارد کنید
prompt = f"Show me {no_recipes} recipes for a dish with the following ingredients:
{ingredients}. Per recipe, list all the ingredients used"
```

کد برای اجرای آزمایشی می‌تواند چنین باشد:

خروجی برنامه (Output) 📋

```
No of recipes (for example, 5): 3
List of ingredients (for example, chicken, potatoes, and carrots): milk,strawberries

-Strawberry milk shake: milk, strawberries, sugar, vanilla extract, ice cubes
-Strawberry shortcake: milk, flour, baking powder, sugar, salt, unsalted butter,
strawberries, whipped cream
-Strawberry milk: milk, strawberries, sugar, vanilla extract
```

بهبود با اضافه کردن فیلتر و لیست خرید

اکنون یک اپ کاربردی داریم که می‌تواند دستور پخت تولید کند و انعطاف‌پذیر است چون به ورودی کاربر وابسته است، هم در تعداد دستور پخت‌ها و هم مواد اولیه استفاده‌شده.

برای بهبود بیشتر، می‌خواهیم موارد زیر را اضافه کنیم:

- **حذف مواد دلخواه**. می‌خواهیم بتوانیم مواد غذایی را که دوست نداریم یا به آن حساسیت داریم حذف کنیم. برای انجام این تغییر، می‌توانیم پرامپت موجود خود را ویرایش کنیم و شرط فیلتر را به انتهای آن اضافه کنیم، به شکل زیر:

Python

```
filter = input("Filter (for example, vegetarian, vegan, or gluten-free): ")

prompt = f"Show me {no_recipes} recipes for a dish with the following ingredients:
{ingredients}. Per recipe, list all the ingredients used, no {filter}"
```

در بالا، `{filter}` را به انتهای پرامپت اضافه می‌کنیم و همچنین مقدار فیلتر را از کاربر دریافت می‌کنیم.

نمونه ورودی اجرای برنامه می‌تواند این‌گونه باشد:

خروجی برنامه (Output) 📋

```
No of recipes (for example, 5): 3
List of ingredients (for example, chicken, potatoes, and carrots): onion,milk
Filter (for example, vegetarian, vegan, or gluten-free): no milk

1. French Onion Soup

Ingredients:

-1 large onion, sliced
-3 cups beef broth
-1 cup milk
-6 slices french bread
-1/4 cup shredded Parmesan cheese
-1 tablespoon butter
-1 teaspoon dried thyme
-1/4 teaspoon salt
-1/4 teaspoon black pepper

Instructions:

1. In a large pot, sauté onions in butter until golden brown.
2. Add beef broth, milk, thyme, salt, and pepper. Bring to a boil.
3. Reduce heat and simmer for 10 minutes.
4. Place french bread slices on soup bowls.
5. Ladle soup over bread.
6. Sprinkle with Parmesan cheese.

2. Onion and Potato Soup

Ingredients:

-1 large onion, chopped
-2 cups potatoes, diced
-3 cups vegetable broth
-1 cup milk
-1/4 teaspoon black pepper

Instructions:

1. In a large pot, sauté onions in butter until golden brown.
2. Add potatoes, vegetable broth, milk, and pepper. Bring to a boil.
3. Reduce heat and simmer for 10 minutes.
4. Serve hot.

3. Creamy Onion Soup

Ingredients:

-1 large onion, chopped
-3 cups vegetable broth
-1 cup milk
-1/4 teaspoon black pepper
-1/4 cup all-purpose flour
-1/2 cup shredded Parmesan cheese

Instructions:

1. In a large pot, sauté onions in butter until golden brown.
2. Add vegetable broth, milk, and pepper. Bring to a boil.
3. Reduce heat and simmer for 10 minutes.
4. In a small bowl, whisk together flour and Parmesan cheese until smooth.
5. Add to soup and simmer for an additional 5 minutes, or until soup has thickened.
```

همان‌طور که می‌بینید، هر دستور پختی که در آن شیر باشد حذف شده است. اما اگر به لاکتوز حساسیت دارید، ممکن است بخواهید دستور پخت‌هایی که پنیر دارند را هم حذف کنید، بنابراین باید واضح باشد.

- **فهرست خرید تهیه کنید**. ما می‌خواهیم یک فهرست خرید تهیه کنیم، با در نظر گرفتن آنچه در خانه داریم.

برای این عملکرد، می‌توانیم همه چیز را در یک درخواست حل کنیم یا آن را به دو درخواست تقسیم کنیم. بیایید روش دوم را امتحان کنیم. در اینجا پیشنهاد می‌کنیم یک درخواست اضافی اضافه کنیم، اما برای اینکه این کار کارآمد باشد، باید نتیجه درخواست اول را به عنوان زمینه به درخواست دوم اضافه کنیم.

قسمت کدی را که نتیجه را از درخواست اول چاپ می‌کند پیدا کنید و کد زیر را زیر آن اضافه کنید:

Python

```
old_prompt_result = response.output_text
prompt = "Produce a shopping list for the generated recipes and please don't include
ingredients that I already have."

new_prompt = f"{old_prompt_result} {prompt}"
response = client.responses.create(model=deployment_name, input=new_prompt,
max_output_tokens=1200, store=False)

# پاسخ را چاپ کن
print("Shopping list:")
print(response.output_text)
```

توجه کنید به موارد زیر:

1. ما با افزودن نتیجه درخواست اول به درخواست جدید، یک درخواست جدید می‌سازیم:

Python

```
new_prompt = f"{old_prompt_result} {prompt}"
```

2. یک درخواست جدید می‌فرستیم، اما همچنین تعداد توکن‌هایی را که در درخواست قبلی خواسته بودیم در نظر می‌گیریم، بنابراین این بار می‌گوییم `max_output_tokens` برابر با 1200 است.

Python

```
response = client.responses.create(model=deployment_name, input=new_prompt,
max_output_tokens=1200, store=False)
```

با اجرای این کد، به خروجی زیر می‌رسیم:

خروجی برنامه (Output) 📋

```
No of recipes (for example, 5): 2
List of ingredients (for example, chicken, potatoes, and carrots): apple,flour
Filter (for example, vegetarian, vegan, or gluten-free): sugar


-Apple and flour pancakes: 1 cup flour, 1/2 tsp baking powder, 1/2 tsp baking
soda, 1/4 tsp salt, 1 tbsp sugar, 1 egg, 1 cup buttermilk or sour milk, 1/4 cup
melted butter, 1 Granny Smith apple, peeled and grated
-Apple fritters: 1-1/2 cups flour, 1 tsp baking powder, 1/4 tsp salt, 1/4 tsp
baking soda, 1/4 tsp nutmeg, 1/4 tsp cinnamon, 1/4 tsp allspice, 1/4 cup sugar,
1/4 cup vegetable shortening, 1/4 cup milk, 1 egg, 2 cups shredded, peeled apples
Shopping list:
-Flour, baking powder, baking soda, salt, sugar, egg, buttermilk, butter, apple,
nutmeg, cinnamon, allspice
```

بهبود تنظیمات خود

آنچه تاکنون داریم کدی است که کار می‌کند، اما برخی اصلاحات وجود دارد که باید برای بهبود بیشتر اعمال کنیم. برخی از کارهایی که باید انجام دهیم عبارتند از:

- **جدا کردن اسرار از کد**، مانند کلید API. اسرار نباید در کد باشند و باید در مکانی امن ذخیره شوند. برای جدا کردن اسرار از کد، می‌توانیم از متغیرهای محیطی و کتابخانه‌هایی مانند `python-dotenv` برای بارگذاری آنها از فایل استفاده کنیم. به این صورت در کد خواهد بود:

1. یک فایل `.env` با محتوای زیر ایجاد کنید:

Terminal / Bash

```
OPENAI_API_KEY=sk- ...
```

> 💡 **نکته کاربردی**
>
> توجه داشته باشید، برای Azure OpenAI در Microsoft Foundry، باید متغیرهای محیطی زیر را به جای آن تنظیم کنید:

Terminal / Bash

```
AZURE_OPENAI_API_KEY=<replace>
AZURE_OPENAI_ENDPOINT=<replace>
AZURE_OPENAI_API_VERSION=2024-10-21
```

در کد، متغیرهای محیطی را به این صورت بارگذاری می‌کنید:

```python
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
```

- **یک نکته درباره طول توکن‌ها**. باید در نظر بگیریم که چه تعداد توکن برای تولید متن مورد نظر نیاز داریم. توکن‌ها هزینه دارند، بنابراین در صورت امکان باید در استفاده از تعداد توکن‌ها صرفه‌جویی کنیم. مثلا آیا می‌توانیم درخواست را طوری بیان کنیم که توکن کمتری مصرف شود؟

برای تغییر تعداد توکن‌ها، می‌توانید از پارامتر `max_output_tokens` استفاده کنید. برای مثال، اگر می‌خواهید از 100 توکن استفاده کنید، به این شکل عمل کنید:

```python
response = client.responses.create(model=deployment, input=prompt,
max_output_tokens=100, store=False)
```

- **آزمایش با پارامتر دما (temperature)**. دما چیزی است که تاکنون ذکر نکرده‌ایم اما زمینه مهمی برای عملکرد برنامه ما است. هرچه مقدار دما بالاتر باشد، خروجی تصادفی‌تر خواهد بود. برعکس، هرچه مقدار دما کمتر باشد، خروجی قابل پیش‌بینی‌تر خواهد بود. در نظر بگیرید که آیا می‌خواهید تنوع در خروجی داشته باشید یا خیر.

برای تغییر دما، می‌توانید از پارامتر `temperature` استفاده کنید. برای مثال، اگر می‌خواهید دما را 0.5 قرار دهید، به این شکل عمل کنید:

```python
response = client.responses.create(model=deployment, input=prompt, temperature=0.5,
store=False)
```

> 💡 **نکته کاربردی**
>
> توجه کنید، هرچه نزدیکتر به 1.0 باشید، خروجی متنوع‌تر خواهد بود.

- **مدل‌های استدلالی از پارامتر `temperature` استفاده نمی‌کنند**. این یک تغییر مهم در سال 2026 است. مدل‌های فعلی غیر منسوخ شده در Microsoft Foundry **مدل‌های استدلالی** هستند (خانواده GPT-5، سری o) - و آنها **از `temperature` یا `top_p` پشتیبانی نمی‌کنند** (همچنین `max_tokens` را نیز استفاده نکنید؛ از `max_output_tokens` استفاده کنید). اگر به `gpt-5-mini` پارامتر `temperature` ارسال کنید، خطای "پارامتر پشتیبانی نمی‌شود" دریافت خواهید کرد. بنابراین برای آزمایش مثال دما بالا، از مدلی استفاده کنید که کنترل نمونه‌برداری را هنوز پشتیبانی می‌کند - برای مثال مدل باز **Llama** مانند `Llama-3.3-70B-Instruct` از کاتالوگ مدل‌های Microsoft Foundry، که از طریق Endpoint مدل‌های Foundry / Azure AI Inference فراخوانی می‌شود (همانند نمونه‌های `githubmodels-*`). برای مدل‌های استدلالی مانند GPT-5، خروجی را به روش متفاوتی هدایت می‌کنید:
  - **مهندسی درخواست (Prompt engineering)** - دستورالعمل‌های واضح، مثال‌ها و خروجی ساختاریافته (به درس 04 - مبانی مهندسی درخواست مراجعه کنید) کارها را انجام می‌دهد که قبلاً با تنظیمات نمونه‌برداری انجام می‌شد.
  - **کنترل‌های استدلال** - پارامترهایی مانند تلاش استدلال / verbosity عمق استدلال را در برابر تأخیر و هزینه متعادل می‌کنند.

خلاصه: `temperature` / `top_p` هنوز در بسیاری از مدل‌ها (Phi، Mistral، Llama) و خانواده GPT-4.x - هرچند GPT-4.x در حال منسوخ شدن است) معتبر هستند، اما روند حرکت به سمت مهندسی درخواست + کنترل‌های استدلالی در مدل‌های استدلالی مانند GPT-5 است.

تمرین

برای این تمرین، می‌توانید انتخاب کنید چه چیزی بسازید.

در اینجا چند پیشنهاد آورده شده است:

- اپلیکیشن تولید کننده دستور غذا را بیشتر بهبود دهید. با مقادیر دما و درخواست‌ها بازی کنید و ببینید چه چیزهایی می‌توانید خلق کنید.
- یک "یار مطالعه" بسازید. این اپ باید بتواند به سوالات در مورد یک موضوع مثلا پایتون پاسخ دهد؛ می‌توانید درخواست‌هایی مانند "موضوع خاصی در پایتون چیست؟" داشته باشید، یا درخواستی که "کدی برای موضوع خاص نشان بده" بدهد و غیره.
- بات تاریخ، تاریخ را زنده کنید. بات را به ایفای نقش یک شخصیت تاریخی خاص وادارید و از او درباره زندگی و زمان‌هایش سوال کنید.

راه حل

یار مطالعه

در زیر یک درخواست شروع اولیه آمده است، ببینید چگونه می‌توانید از آن استفاده کنید و آن را به دلخواه خود تنظیم کنید.

متن / Text

```
- "You're an expert on the Python language

    Suggest a beginner lesson for Python in the following format:

    Format:
    - concepts:
    - brief explanation of the lesson:
    - exercise in code with solutions"
```

بات تاریخ

در اینجا چند درخواست آورده شده است که می‌توانید استفاده کنید:

متن / Text

```
- "You are Abe Lincoln, tell me about yourself in 3 sentences, and respond using grammar and
words like Abe would have used"
- "You are Abe Lincoln, respond using grammar and words like Abe would have used:

   Tell me about your greatest accomplishments, in 300 words"
```

آزمون دانش

مفهوم دما (temperature) چه کاری انجام می‌دهد؟

1. کنترل می‌کند که خروجی چقدر تصادفی باشد.
2. کنترل می‌کند که پاسخ چقدر بزرگ باشد.
3. کنترل می‌کند که چه تعداد توکن استفاده شود.

🚀 چالش

هنگام کار بر روی تمرین، سعی کنید دما را تغییر دهید، آن را روی 0، 0.5 و 1 تنظیم کنید. به یاد داشته باشید که 0 کمترین تنوع و 1 بیشترین تنوع را دارد. کدام مقدار برای اپ شما بهترین کارکرد را دارد؟

کار عالی! آموزش خود را ادامه دهید

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش هوش مصنوعی مولد خود را ارتقا دهید!

به درس 7 بروید که در آن می‌خواهیم ببینیم چگونه اپلیکیشن‌های چت بسازیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۰۶ — هوش مصنوعی مولد مایکروسافت

درس ۰۷ از ۲۱

ساخت برنامه‌های چت و ربات‌های گفتگو

معماری چت‌بات‌های مدرن، مدیریت پنجره زمینه (Context Window)، پیام‌های System/User/Assistant و حافظه مکالمه

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/R9V0ZY1BEQo?si=IHuU-fS9YWT8s4sA

▶️ پخش در یوتیوب

اسکن کد QR

حال که دیدیم چگونه می‌توانیم برنامه‌های تولید متن بسازیم، بیایید به برنامه‌های چت نگاهی بیندازیم.

برنامه‌های چت به بخش‌های جدایی‌ناپذیر زندگی روزمره ما تبدیل شده‌اند، و بیش از یک وسیله برای گفتگوی معمولی هستند. آنها بخش مهمی از خدمات مشتری، پشتیبانی فنی و حتی سیستم‌های مشاوره پیچیده هستند. ممکن است اخیراً از یک برنامه چت کمک گرفته باشید. با وارد کردن فناوری‌های پیشرفته‌تر مانند هوش مصنوعی مولد به این پلتفرم‌ها، پیچیدگی و چالش‌ها افزایش می‌یابد.

برخی سوالاتی که باید پاسخ داده شوند عبارتند از:

- **ساخت برنامه**. چگونه می‌توانیم به طور کارآمد این برنامه‌های مبتنی بر هوش مصنوعی را برای موارد استفاده خاص بسازیم و به صورت یکپارچه ادغام کنیم؟

- **نظارت**. پس از استقرار، چگونه می‌توانیم نظارت کنیم که این برنامه‌ها با بالاترین کیفیت، چه از لحاظ عملکرد و چه از نظر تطابق با شش اصل هوش مصنوعی مسئولانه فعالیت می‌کنند؟

با پیشرفت در عصری که توسط خودکارسازی و تعاملات بی‌وقفه انسان-ماشین تعریف می‌شود، درک این که هوش مصنوعی مولد چگونه حوزه، عمق و انعطاف‌پذیری برنامه‌های چت را تغییر می‌دهد، ضروری می‌شود. این درس به جنبه‌های معماری که از این سیستم‌های پیچیده پشتیبانی می‌کنند می‌پردازد، روش‌های تنظیم دقیق آنها برای وظایف خاص دامنه را بررسی می‌کند و معیارها و ملاحظات مربوط به استقرار مسئولانه هوش مصنوعی را ارزیابی خواهد کرد.

معرفی

این درس شامل موارد زیر است:

- تکنیک‌هایی برای ساخت و ادغام کارآمد برنامه‌های چت.
- نحوه اعمال سفارشی‌سازی و تنظیم دقیق در برنامه‌ها.
- استراتژی‌ها و ملاحظاتی برای نظارت موثر بر برنامه‌های چت.

اهداف یادگیری

تا پایان این درس، شما قادر خواهید بود:

- ملاحظات مربوط به ساخت و ادغام برنامه‌های چت در سیستم‌های موجود را توصیف کنید.
- برنامه‌های چت را برای موارد استفاده خاص سفارشی کنید.
- معیارها و ملاحظات کلیدی برای نظارت موثر و حفظ کیفیت برنامه‌های چت مبتنی بر هوش مصنوعی را شناسایی کنید.
- اطمینان حاصل کنید که برنامه‌های چت هوش مصنوعی را به صورت مسئولانه به کار می‌گیرند.

ادغام هوش مصنوعی مولد در برنامه‌های چت

ارتقاء برنامه‌های چت از طریق هوش مصنوعی مولد فقط به هوشمندتر کردن آنها محدود نمی‌شود؛ بلکه بهینه‌سازی معماری، عملکرد و رابط کاربری آنها برای ارائه تجربه کاربری باکیفیت است. این شامل بررسی پایه‌های معماری، ادغام APIها و ملاحظات رابط کاربری است. این بخش هدف دارد تا نقشه راه جامعی برای پیمودن این حوزه‌های پیچیده به شما ارائه دهد، چه بخواهید آنها را به سیستم‌های موجود متصل کنید یا به صورت پلتفرم‌های مستقل بسازید.

تا پایان این بخش، شما مجهز به تخصص لازم برای ساخت و ادغام کارآمد برنامه‌های چت خواهید بود.

چت‌بات یا برنامه چت؟

قبل از اینکه وارد ساخت برنامه‌های چت شویم، بیایید «چت‌بات‌ها» را با «برنامه‌های چت مبتنی بر هوش مصنوعی» که نقش‌ها و عملکردهای متفاوتی دارند مقایسه کنیم. هدف اصلی چت‌بات خودکارسازی وظایف محاوره‌ای خاص مانند پاسخ دادن به

سوالات متداول یا پیگیری بسته است. معمولاً توسط منطق مبتنی بر قوانین یا الگوریتم‌های پیچیده هوش مصنوعی اداره می‌شود. در مقابل، برنامه چت مبتنی بر هوش مصنوعی محیطی بسیار گسترده‌تر است که برای تسهیل اشکال مختلف ارتباط دیجیتال، مانند چت متنی، صوتی و تصویری بین کاربران انسانی طراحی شده است. ویژگی تعریف‌کننده آن ادغام یک مدل هوش مصنوعی مولد است که مکالمات ظریف و انسان‌گونه را شبیه‌سازی می‌کند و پاسخ‌ها را بر اساس انواع مختلف ورودی‌ها و سرنخ‌های زمینه‌ای تولید می‌کند. برنامه چت مبتنی بر هوش مصنوعی مولد می‌تواند در بحث‌های دامنه باز شرکت کند، با زمینه‌های در حال تحول مکالمه تطبیق یابد و حتی گفتگوی خلاقانه یا پیچیده تولید کند.

جدول زیر تفاوت‌ها و شباهت‌های کلیدی را برای درک بهتر نقش‌های منحصر به فرد آنها در ارتباط دیجیتال نشان می‌دهد.

چت‌بات	برنامه چت مبتنی بر هوش مصنوعی مولد
متمرکز بر وظیفه و مبتنی بر قوانین	آگاه به زمینه
معمولاً در سیستم‌های بزرگ‌تر ادغام شده	ممکن است میزبان یک یا چند چت‌بات باشد
محدود به عملکردهای برنامه‌ریزی شده	شامل مدل‌های هوش مصنوعی مولد است
تعاملات تخصصی و ساختارمند	قادر به گفتگوهای دامنه باز است

بهره‌گیری از قابلیت‌های از پیش ساخته شده با SDKها و APIها

هنگام ساخت برنامه چت، گام اول خوب این است که بررسی کنید چه امکاناتی قبلاً وجود دارد. استفاده از SDKها و APIها برای ساخت برنامه‌های چت استراتژی مطلوبی به دلایل مختلف است. با ادغام SDKها و APIهای مستند شده خوب، شما برنامه‌تان را به شکل استراتژیک برای موفقیت بلندمدت موقعیت‌یابی می‌کنید و مسائل مربوط به مقیاس‌پذیری و نگهداری را حل و فصل می‌کنید.

- **فرآیند توسعه را تسریع و بار کاری را کاهش می‌دهد**: تکیه بر قابلیت‌های از پیش ساخته شده به جای فرایند پرهزینه ساخت آنها از ابتدا، به شما اجازه می‌دهد روی جنبه‌های دیگر برنامه‌تان که ممکن است برایتان مهم‌تر باشند تمرکز کنید، مانند منطق کسب‌وکار.
- **عملکرد بهتر**: هنگام ساخت قابلیت از ابتدا، حتماً از خود خواهید پرسید «چگونه مقیاس می‌پذیرد؟ آیا این برنامه می‌تواند یک افزایش ناگهانی کاربران را مدیریت کند؟» SDKها و APIهای به‌خوبی نگهداری شده اغلب راه‌حل‌هایی از پیش ساخته برای این مسائل دارند.
- **نگهداری آسان‌تر**: به‌روزرسانی‌ها و بهبودها آسان‌تر انجام می‌شود چون اکثر APIها و SDKها تنها نیاز به به‌روزرسانی یک کتابخانه دارند وقتی نسخه جدیدی منتشر می‌شود.
- **دسترسی به فناوری پیشرفته**: بهره‌گیری از مدل‌هایی که روی مجموعه داده‌های وسیع تنظیم دقیق شده‌اند، به برنامه شما قابلیت‌های زبان طبیعی می‌دهد.

دسترسی به قابلیت‌های SDK یا API معمولاً شامل دریافت مجوز برای استفاده از خدمات ارائه‌شده است که اغلب از طریق کلید منحصر به فرد یا توکن احراز هویت انجام می‌شود. ما از کتابخانه پایتون OpenAI برای بررسی شکل این فرایند استفاده خواهیم کرد. همچنین می‌توانید خودتان در دفترچه OpenAI یا دفترچه Azure OpenAI Services این درس امتحان کنید.

Python

```
import os
from openai import OpenAI

API_KEY = os.getenv("OPENAI_API_KEY","")

client = OpenAI(
    api_key=API_KEY
    )

response = client.responses.create(model="gpt-5-mini", input="Suggest two titles for an
instructional lesson on chat applications for generative AI.", store=False)
print(response.output_text)
```

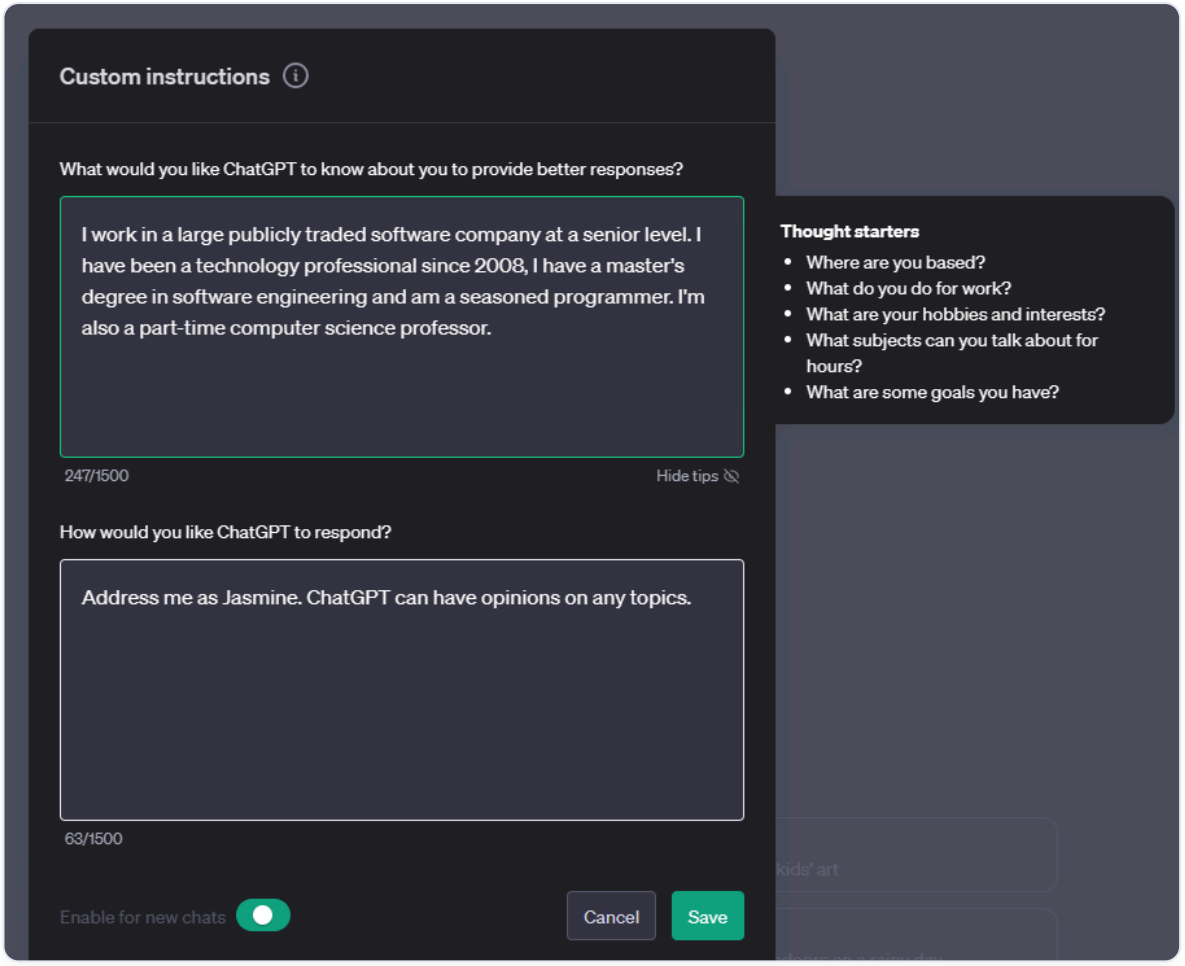
مثال بالا از مدل GPT-5 مینی با API پاسخ‌ها برای تکمیل پرسش استفاده می‌کند، اما توجه کنید کلید API قبل از این کار تنظیم شده است. اگر کلید را تنظیم نکنید، با خطا مواجه خواهید شد.

تجربه کاربری (UX)

اصول کلی UX در برنامه‌های چت اعمال می‌شود، اما در اینجا برخی ملاحظات اضافی وجود دارد که به دلیل اجزای یادگیری ماشین اهمیت ویژه‌ای پیدا می‌کند.

- **مکانیسم رسیدگی به ابهام**: مدل‌های هوش مصنوعی مولد گاهی پاسخ‌های مبهم تولید می‌کنند. ویژگی‌ای که به کاربران اجازه دهد برای روشن شدن ابهامات سوال بپرسند می‌تواند مفید باشد.
- **حفظ زمینه**: مدل‌های پیشرفته هوش مصنوعی مولد توانایی به خاطر سپردن زمینه داخل یک گفت‌وگو را دارند که می‌تواند دارایی مهمی برای تجربه کاربر باشد. دادن اختیار به کاربران برای کنترل و مدیریت زمینه تجربه کاربری را بهبود می‌بخشد اما خطر حفظ اطلاعات حساس کاربران را به همراه دارد. ملاحظاتی درباره مدت زمان نگهداری این اطلاعات، مانند معرفی سیاست نگهداری، می‌تواند نیاز به زمینه را در برابر حریم خصوصی متعادل کند.
- **شخصی‌سازی**: با قابلیت یادگیری و سازگاری، مدل‌های هوش مصنوعی تجربه‌ای فردی‌سازی شده به کاربران ارائه می‌دهند. سفارشی‌سازی تجربه کاربری از طریق ویژگی‌هایی مانند پروفایل‌های کاربران نه تنها باعث می‌شود کاربر احساس درک شدن کند، بلکه به جستجوی موثرتر و رضایت‌بخش‌تر پاسخ‌های خاص کمک می‌کند.

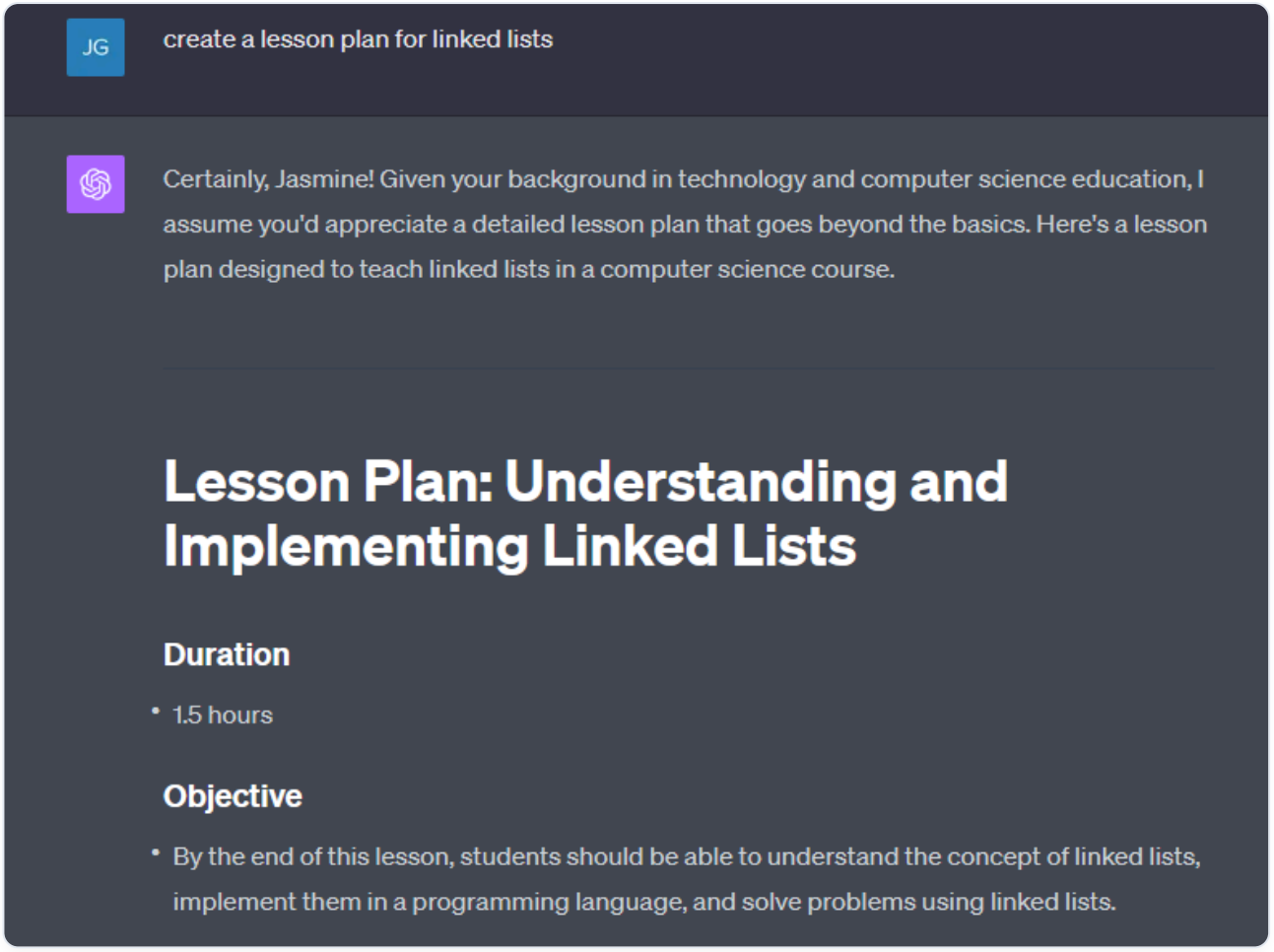
یکی از نمونه‌های شخصی‌سازی، تنظیمات «دستورات سفارشی» در ChatGPT شرکت OpenAI است. این تنظیمات به شما اجازه می‌دهد اطلاعاتی درباره خود ارائه دهید که ممکن است برای پرسش‌هایتان زمینه مهمی باشد. در اینجا نمونه‌ای از دستور سفارشی آورده شده است.



تنظیمات دستورات سفارشی در ChatGPT

> 📝 **ترجمه و راهنمای فارسی متن تصویر بالا (تنظیمات ChatGPT):**
>
> **بخش اول (اطلاعات زمینه شما برای پاسخ بهتر):** «من در یک شرکت نرم‌افزاری بزرگ در سطح ارشد کار می‌کنم. از سال ۲۰۰۸ به عنوان متخصص فناوری فعالیت دارم، دارای مدرک کارشناسی ارشد در مهندسی نرم‌افزار و برنامه‌نویس باسابقه هستم. همچنین استاد پاره‌وقت علوم کامپیوتر می‌باشم.»
>
> **بخش دوم (نحوه پاسخ‌دهی مورد انتظار از ChatGPT):** «مرا با نام یاسمین (Jasmine) خطاب کن. ChatGPT می‌تواند درباره هر موضوعی نظر و دیدگاه شخصی داشته باشد.»

این «پروفایل» ChatGPT را ترغیب می‌کند تا برنامه درسی درباره لیست‌های پیوندی ایجاد کند. توجه کنید که ChatGPT تجربه کاربر را در نظر می‌گیرد که ممکن است برنامه درسی عمیق‌تری خواسته باشد.



یک دستور در ChatGPT برای برنامه درسی درباره لیست‌های پیوندی

📝 **ترجمه و راهنمای فارسی پرامپت و پاسخ چت در تصویر بالا:**

درخواست کاربر (Prompt): «یک طرح درس درباره لیست‌های پیوندی (Linked Lists) برای دانشجویان مقدماتی ایجاد کن.»

پاسخ ChatGPT: «سلام یاسمین! با توجه به زمینه تخصصی شما در مهندسی نرم‌افزار و تدریس دانشگاهی، طرح درسی عمیق و کاربردی درباره مفهوم لیست‌های پیوندی، پیاده‌سازی گره‌ها و تحلیل پیچیدگی زمانی آن آماده کرده‌ام...»

چارچوب پیام سیستمی مایکروسافت برای مدل‌های زبانی بزرگ

مایکروسافت راهنمایی‌هایی ارائه داده است برای نوشتن پیام‌های سیستمی مؤثر هنگام تولید پاسخ‌ها توسط مدل‌های زبانی بزرگ، که در ۴ بخش توضیح داده شده است:

۱. تعریف مخاطب مدل، همچنین قابلیت‌ها و محدودیت‌های آن. ۲. تعریف قالب خروجی مدل. ۳. ارائه مثال‌های خاص که رفتار مورد نظر مدل را نشان می‌دهند. ۴. ارائه چارچوب‌های رفتاری اضافی.

دسترسی‌پذیری

چه کاربر دارای اختلالات بینایی، شنوایی، حرکتی یا شناختی باشد، یک برنامه چت خوب طراحی شده باید برای همه قابل استفاده باشد. فهرست زیر ویژگی‌های خاصی را برای افزایش دسترسی‌پذیری در برابر انواع اختلالات کاربری ارائه می‌دهد.

- **ویژگی‌های اختلال بینایی**: تم‌های کنتراست بالا و متن قابل تغییر اندازه، سازگاری با صفحه‌خوان.
- **ویژگی‌های اختلال شنوایی**: توابع تبدیل متن به گفتار و گفتار به متن، نشانه‌های بصری برای اعلان‌های صوتی.
- **ویژگی‌های اختلال حرکتی**: پشتیبانی از ناوبری صفحه‌کلید، فرمان‌های صوتی.
- **ویژگی‌های اختلال شناختی**: گزینه‌های زبان ساده شده.

سفارشی‌سازی و تنظیم دقیق برای مدل‌های زبانی حوزه‌محور

تصور کنید یک برنامه چت وجود دارد که اصطلاحات شرکت شما را می‌فهمد و سوالات خاصی که کاربرانش معمولاً دارند را پیش‌بینی می‌کند. چند رویکرد قابل ذکر وجود دارد:

- **بهره‌گیری از مدل‌های DSL**. DSL مخفف domain specific language (زبان خاص حوزه) است. شما می‌توانید از مدلی که روی حوزه خاصی آموزش دیده است برای درک مفاهیم و سناریوهای آن استفاده کنید.
- **اعمال تنظیم دقیق**. تنظیم دقیق فرایند آموزش بیشتر مدل شما با داده‌های خاص است.

سفارشی‌سازی: استفاده از DSL

استفاده از مدل‌های زبان حوزه‌محور (DSL Models) می‌تواند تعامل کاربر را افزایش دهد و با ارائه تعاملات تخصصی و مرتبط از نظر زمینه، تجربه بهتری ایجاد کند. این مدل‌هایی هستند که آموزش دیده یا تنظیم دقیق شده‌اند تا متن مرتبط با حوزه، صنعت یا موضوع خاصی را درک و تولید کنند. گزینه‌های استفاده از مدل DSL می‌توانند از آموزش مدل از پایه تا استفاده از مدل‌های موجود از طریق SDKها و APIها متفاوت باشد. گزینه دیگر تنظیم دقیق است که شامل برداشتن یک مدل پیش‌آموزش دیده و سازگار کردن آن برای حوزه خاص است.

سفارشی‌سازی: اعمال تنظیم دقیق

تنظیم دقیق اغلب زمانی در نظر گرفته می‌شود که مدل پیش‌آموزش دیده در حوزه تخصصی یا وظیفه خاصی عملکرد ناکافی داشته باشد.

برای مثال، پرسش‌های پزشکی پیچیده هستند و به زمینه زیادی نیاز دارند. وقتی یک پزشک تشخیص می‌دهد، بر اساس عوامل متعددی مانند سبک زندگی یا شرایط قبلی است و ممکن است حتی به مجلات پزشکی اخیر برای اعتبار بخشی به تشخیص خود تکیه کند. در چنین موقعیت‌های پیچیده‌ای، برنامه چت هوش مصنوعی عمومی قابل اعتماد نیست.

سناریو: برنامه پزشکی

یک برنامه چت را در نظر بگیرید که برای کمک به پزشکان طراحی شده است تا مرجعی سریع برای دستورالعمل‌های درمان، تداخل دارویی یا یافته‌های پژوهشی اخیر ارائه دهد.

یک مدل عمومی ممکن است برای پاسخ به سوالات پزشکی پایه یا ارائه مشاوره کلی کافی باشد، اما ممکن است در موارد زیر مشکل داشته باشد:

- **موارد بسیار خاص یا پیچیده**. برای مثال، یک نورولوژیست ممکن است از برنامه بپرسد: «روش‌های بهترین مدیریت صرع مقاوم به دارو در کودکان چیست؟»
- **فاقد پیشرفت‌های اخیر**. ممکن است مدل عمومی نتواند پاسخ فعلی که شامل آخرین پیشرفت‌ها در نورولوژی و داروشناسی است را ارائه دهد.

در چنین مواردی، تنظیم دقیق مدل با مجموعه داده‌های تخصصی پزشکی می‌تواند توانایی آن را در مدیریت این پرسش‌های پزشکی پیچیده با دقت و قابلیت اطمینان بیشتر به طور قابل توجهی افزایش دهد. این نیازمند دسترسی به مجموعه داده بزرگ و مرتبطی است که چالش‌ها و سوالات حوزه خاص را نمایندگی کند.

ملاحظات برای تجربه چت با کیفیت بالا و مبتنی بر هوش مصنوعی

این بخش معیارهای برنامه‌های چت «با کیفیت بالا» را که شامل جمع‌آوری معیارهای اقدام‌پذیر و پایبندی به چارچوبی برای بهره‌گیری مسئولانه از فناوری هوش مصنوعی است، شرح می‌دهد.

معیارهای کلیدی

برای حفظ عملکرد با کیفیت بالا یک برنامه، پایش معیارها و ملاحظات کلیدی ضروری است. این اندازه‌گیری‌ها نه تنها عملکرد برنامه را تضمین می‌کنند بلکه کیفیت مدل هوش مصنوعی و تجربه کاربر را نیز ارزیابی می‌کنند. فهرست زیر معیارهای پایه، هوش مصنوعی و تجربه کاربر را شامل می‌شود که باید در نظر گرفته شوند.

معیار	تعریف	ملاحظاتی برای توسعه‌دهنده چت
زمان کارکرد (Uptime)	مدت زمان عملیاتی و در دسترس بودن برنامه برای کاربران.	چگونه زمان قطع سرویس را به حداقل می‌رسانید؟
زمان پاسخ	زمان لازم برای پاسخ برنامه به پرسش کاربر.	چگونه پردازش پرسش را بهینه می‌کنید تا زمان پاسخ بهتر شود؟
دقت (Precision)	نسبت پیش‌بینی‌های درست مثبت به کل پیش‌بینی‌های مثبت.	چگونه دقت مدل خود را اعتبارسنجی می‌کنید؟
بازیابی (Recall / حساسیت)	نسبت پیش‌بینی‌های درست مثبت به تعداد واقعی مثبت‌ها.	چگونه بازیابی را اندازه‌گیری و بهبود می‌دهید؟
امتیاز F1	میانگین هارمونیک دقت و بازیابی که توازن بین هر دو را برقرار می‌کند.	هدف امتیاز F1 شما چیست؟ چگونه دقت و بازیابی را متعادل می‌کنید؟
پیچیدگی (Perplexity)	اندازه‌گیری تطابق توزیع احتمال پیش‌بینی شده مدل با توزیع واقعی داده‌ها.	چگونه پیچیدگی را به حداقل می‌رسانید؟
معیارهای رضایت کاربر	اندازه‌گیری برداشت کاربر از برنامه. اغلب از طریق نظرسنجی‌ها جمع‌آوری می‌شود.	هر چند وقت یکبار بازخورد کاربر را جمع‌آوری می‌کنید؟ چگونه بر اساس آن سازگار می‌شوید؟
نرخ خطا	نرخ اشتباهات مدل در فهم یا خروجی.	چه استراتژی‌هایی برای کاهش نرخ خطا دارید؟
چرخه‌های بازآموزی	فرکانس به‌روزرسانی مدل برای وارد کردن داده‌ها و نکات جدید.	هر چند وقت یکبار مدل را بازآموزی می‌کنید؟ چه عواملی چرخه بازآموزی را فعال می‌کند؟
شناسایی ناهنجاری	ابزارها و تکنیک‌هایی برای شناسایی الگوهای غیرمعمول که با رفتار مورد انتظار مطابقت ندارند.	پاسخ شما به ناهنجاری‌ها چگونه خواهد بود؟

پیاده‌سازی روش‌های هوش مصنوعی مسئولانه در برنامه‌های چت

رویکرد مایکروسافت به هوش مصنوعی مسئولانه شش اصل را تعیین کرده است که باید توسعه و استفاده از هوش مصنوعی را هدایت کنند. در ادامه، اصول، تعریف آن‌ها و مواردی که توسعه‌دهنده چت باید در نظر بگیرد و دلیل اهمیت آن‌ها آورده شده است.

اصول	تعریف مایکروسافت	ملاحظات برای توسعه‌دهنده چت	چرا مهم است
عدالت	سیستم‌های هوش مصنوعی باید با همه افراد منصفانه رفتار کنند.	اطمینان حاصل کنید که برنامه چت بر اساس داده‌های کاربر تبعیض قائل نمی‌شود.	برای ایجاد اعتماد و فراگیری در میان کاربران؛ جلوگیری از پیامدهای حقوقی.
قابلیت اطمینان و ایمنی	سیستم‌های هوش مصنوعی باید به‌طور قابل اعتماد و ایمن عمل کنند.	پیاده‌سازی تست و مکانیزم‌های ایمنی برای کاهش خطاها و ریسک‌ها.	تضمین رضایت کاربران و جلوگیری از آسیب‌های احتمالی.
حریم خصوصی و امنیت	سیستم‌های هوش مصنوعی باید ایمن باشند و به حریم خصوصی احترام بگذارند.	پیاده‌سازی رمزگذاری قوی و اقدامات حفاظتی داده‌ها.	برای حفاظت از داده‌های حساس کاربران و رعایت قوانین حریم خصوصی.
فراگیری	سیستم‌های هوش مصنوعی باید همه افراد را توانمند سازند و مشارکت دهند.	طراحی رابط کاربری/تجربه کاربری که برای مخاطبان متنوع قابل دسترسی و آسان باشد.	اطمینان از این که طیف وسیع‌تری از افراد می‌توانند به‌طور مؤثر از برنامه استفاده کنند.
شفافیت	سیستم‌های هوش مصنوعی باید قابل درک باشند.	ارائه مستندسازی واضح و دلایل پاسخ‌های هوش مصنوعی.	کاربران احتمالاً به سیستمی اعتماد می‌کنند که بتوانند نحوه اتخاذ تصمیم‌ها را بفهمند.
پاسخگویی	انسان‌ها باید مسئول سیستم‌های هوش مصنوعی باشند.	ایجاد روند مشخص برای ممیزی و بهبود تصمیمات هوش مصنوعی.	امکان بهبود مستمر و اقدامات اصلاحی در صورت بروز اشتباهات.

تمرین

به تمرین مراجعه کنید. این تمرین شما را از اجرای اولین فرمان‌های چت تا دسته‌بندی و خلاصه‌سازی متن و بیشتر، همراهی می‌کند. توجه داشته باشید که تمرین‌ها به زبان‌های برنامه‌نویسی مختلف در دسترس هستند!

کار عالی! ادامه مسیر

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را در Generative AI Learning collection ببینید تا دانش هوش مصنوعی مولد خود را بیشتر ارتقا دهید!

به درس ۸ بروید تا ببینید چگونه می‌توانید شروع به ساخت برنامه‌های جستجو کنید!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

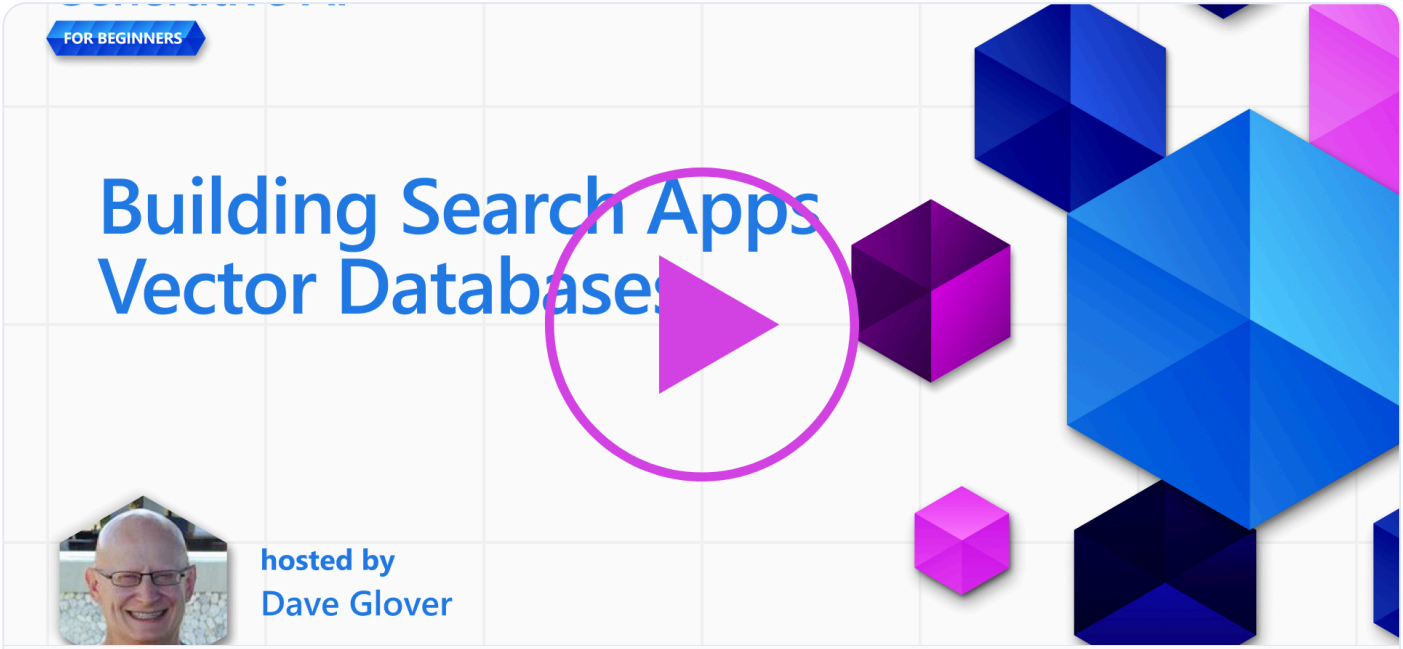
بازگشت به فهرست مطالب

درس ۰۷ — هوش مصنوعی مولد مایکروسافت

درس ۰۸ از ۲۱

ساخت برنامه‌های جستجوی هوشمند با پایگاه‌های داده برداری

تبدیل متن به بردار (Embeddings)، محاسبه شباهت کسینوسی (Cosine Similarity)، و ساخت سامانه جستجوی معنایی متنی



🎥 ویدیوی رسمی آموزش این درس

📱 برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/W0-nzXjOjr0?si=GcsqiTTvd7RKbo7V

▶️ پخش در یوتیوب

اسکن کد QR

💡 نکته کاربردی

💡 نکته کاربردی

برای مشاهده ویدیو این درس روی تصویر بالا کلیک کنید

مدل‌های بزرگ زبان (LLM) فراتر از چت‌بات‌ها و تولید متن هستند. همچنین می‌توان با استفاده از امبدینگ‌ها برنامه‌های جستجو ساخت. امبدینگ‌ها نمایش‌های عددی داده‌ها هستند که به شکل بردار شناخته می‌شوند و برای جستجوی معنایی داده‌ها استفاده می‌شوند.

در این درس، شما یک برنامه جستجو برای استارتاپ آموزشیمان می‌سازید. استارتاپ ما یک سازمان غیرانتفاعی است که آموزش رایگان به دانش‌آموزان در کشورهای در حال توسعه ارائه می‌دهد. استارتاپ ما تعداد زیادی ویدیوهای یوتیوب دارد که دانش‌آموزان می‌توانند برای یادگیری هوش مصنوعی از آن‌ها استفاده کنند. استارتاپ ما می‌خواهد برنامه جستجویی بسازد که به دانش‌آموزان اجازه دهد با تایپ سؤال، ویدیوی یوتیوب مربوطه را جستجو کنند.

به عنوان مثال، دانش‌آموز ممکن است «دفترچه‌های Jupyter چیست؟» یا «Azure ML چیست؟» را تایپ کند و برنامه جستجو فهرستی از ویدیوهای یوتیوب مرتبط با سؤال را بازگرداند، و بهتر از آن، برنامه جستجو لینکی به قسمت ویدیو که پاسخ سؤال در آن قرار دارد ارائه دهد.

معرفی

در این درس، موضوعات زیر پوشش داده می‌شود:

- جستجوی معنایی در مقابل جستجوی کلمه‌ای.
- امبدینگ‌های متنی چیستند.
- ایجاد شاخص امبدینگ متنی.
- جستجو در شاخص امبدینگ متنی.

اهداف یادگیری

پس از تکمیل این درس، قادر خواهید بود:

- تفاوت بین جستجوی معنایی و جستجوی کلمه‌ای را توضیح دهید.
- بگویید امبدینگ‌های متنی چیستند.
- با استفاده از امبدینگ‌ها، برنامه‌ای برای جستجوی داده‌ها بسازید.

چرا برنامه جستجو بسازیم؟

ساخت برنامه جستجو به شما کمک می‌کند یاد بگیرید چگونه از امبدینگ‌ها برای جستجوی داده‌ها استفاده کنید. همچنین یاد می‌گیرید چگونه برنامه جستجویی بسازید که دانش‌آموزان بتوانند سریعاً به اطلاعات دسترسی پیدا کنند.

این درس شامل شاخص امبدینگ از رونوشت‌های ویدیوهای یوتیوب کانال AI Show Microsoft است. AI Show کانال یوتیوبی است که در مورد هوش مصنوعی و یادگیری ماشین آموزش می‌دهد. شاخص امبدینگ‌های هر رونوشت ویدیو

تا اکتبر ۲۰۲۳ را شامل می‌شود. شما از این شاخص امبدینگ برای ساخت برنامه جستجویی برای استارتاپ ما استفاده خواهید کرد. برنامه جستجو لینکی به محل پاسخ سؤال در ویدیو ارائه می‌دهد که روشی عالی است برای کمک به دانش‌آموزان برای یافتن سریع اطلاعات مورد نیاز.

نمونه‌ای از یک پرس‌وجوی معنایی برای سؤال «آیا می‌توان از rstudio با Azure ML استفاده کرد؟» در زیر آمده است. URL یوتیوب را بررسی کنید؛ می‌بینید که حاوی نشانی زمان است که شما را به محلی در ویدیو می‌برد که پاسخ سؤال آنجاست.

```
Videos similar to 'can you use rstudio with azure ml':

- Reproducible Data Science with Machine Learning
  Summary: In this video, Rafal Lukawiecki discusses reproducible data science with machine learning. He demonstrates how..
  YouTube: https://youtu.be/NyW0fYKScUk?t=1289
  Similarity: 0.8326736830964594
  Speakers: Rafal Lukawiecki

- R Support in Azure Machine Learning
  Summary: In this episode of the AI Show, Seth Juarez, a Program Manager in Azure Machine...
  YouTube: https://youtu.be/9_alql02cT0?t=0
  Similarity: 0.8163297415913482
  Speakers: Seth Juarez

- R Support in Azure Machine Learning
  Summary: Azure Machine Learning now supports R, making it easier for R users to leverage the...
  YouTube: https://youtu.be/9_alql02cT0?t=184
  Similarity: 0.8131143983558603
  Speakers: Seth Juarez
```

پرس‌وجوی معنایی برای سؤال "آیا می‌توان از rstudio با Azure ML استفاده کرد؟"

> **راهنمای فارسی نتایج جستجوی معنایی در تصویر بالا:**
>
> **پرسش جستجو:** «Can rstudio be used with Azure ML? (آیا می‌توان از RStudio همراه با Azure ML استفاده کرد؟)»
>
> **نتیجه الگوریتم:** مدل تعبیه‌ساز (Embedding) شباهت معنایی پرسش با پایگاه داده را محاسبه کرده و مرتبط‌ترین ویدیوها، اسناد و گویندگان را با امتیاز شباهت (Similarity Score) بازگردانده است.

جستجوی معنایی چیست؟

ممکن است بپرسید جستجوی معنایی چیست؟ جستجوی معنایی تکنیکی است که از معنای واژه‌ها در یک پرس‌وجو برای بازگرداندن نتایج مرتبط استفاده می‌کند.

اینجا مثالی از جستجوی معنایی است. فرض کنید دنبال خرید یک خودرو هستید، ممکن است «خودروی رویایی من» را جستجو کنید، جستجوی معنایی درک می‌کند که شما در حال خواب دیدن درباره خودرو نیستید بلکه دنبال خودروی «ایده‌آل» خود هستید. جستجوی معنایی نیت شما را متوجه می‌شود و نتایج مرتبط را بازمی‌گرداند. جایگزین آن «جستجوی کلمه‌ای» است که دقیقاً دنبال کلمه «خواب» درباره خودروها می‌گردد و اغلب نتایج نامرتبط می‌آورد.

امبدینگ‌های متنی چیستند؟

امبدینگ‌های متنی تکنیکی برای نمایش متن است که در پردازش زبان طبیعی استفاده می‌شود. امبدینگ‌های متنی نمایش‌های عددی معنایی متن هستند. امبدینگ‌ها برای نمایش داده‌ها به شکلی که ماشین به خوبی بتواند درک کند کاربرد دارد. مدل‌های متعددی برای ساخت امبدینگ‌های متنی وجود دارد، در این درس بر تولید امبدینگ با استفاده از مدل OpenAI Embedding تمرکز خواهیم کرد.

اینجا مثالی است، فرض کنید متن زیر بخشی از رونوشت یکی از قسمت‌های کانال AI Show در یوتیوب باشد:

متن / Text

```
Today we are going to learn about Azure Machine Learning.
```

متن را به API امبدینگ OpenAI می‌دهیم و آن عبارت عددی ۱۵۳۶ تایی به شکل بردار را بازمی‌گرداند. هر عدد در این بردار نمایانگر جنبه‌ای متفاوت از متن است. برای اختصار، اینجا ۱۰ عدد اول بردار را نشان می‌دهیم.

Python

```
[-0.006655829958617687, 0.0026128944009542465, 0.008792596869170666, -0.024460010230541 23,
-0.008540431968867779, 0.022071078419685364, -0.010703742504119873, 0.003311325330287218,
-0.011632772162556648, -0.02187200076878071, ...]
```

شاخص امبدینگ چگونه ساخته می‌شود؟

شاخص امبدینگ برای این درس با مجموعه‌ای از اسکریپت‌های پایتون ساخته شده است. این اسکریپت‌ها همراه با دستورالعمل‌ها را در README در پوشه 'scripts' این درس می‌یابید. برای کامل کردن این درس نیازی به اجرای این اسکریپت‌ها ندارید چون شاخص امبدینگ برای شما ارائه شده است.

اسکریپت‌ها کارهای زیر را انجام می‌دهند:

1. رونوشت هر ویدیوی یوتیوب در پلی‌لیست AI Show دانلود می‌شود.
2. با استفاده از توابع OpenAI، تلاش می‌شود نام سخنران از ۳ دقیقه اول رونوشت یوتیوب استخراج شود. نام سخنران هر ویدیو در شاخص امبدینگ به نام `embedding_index_3m.json` ذخیره می‌گردد.
3. متن رونوشت به قطعات متنی **۳ دقیقه‌ای** تقسیم می‌شود. این قطعات تقریباً ۲۰ کلمه همپوشانی از قطعه بعدی دارند تا اطمینان حاصل شود که امبدینگ قطعه قطع نمی‌شود و زمینه جستجوی بهتری فراهم شود.
4. هر قطعه متن به API چت OpenAI داده می‌شود تا متن را در ۶۰ کلمه خلاصه کند. خلاصه نیز در شاخص امبدینگ `embedding_index_3m.json` ذخیره می‌شود.

5. در نهایت، متن قطعه به API امبدینگ OpenAI داده می‌شود. این API برداری ۱۵۳۶ تایی که معنای معنایی قطعه را نشان می‌دهد بازمی‌گرداند. قطعه به همراه بردار امبدینگ OpenAI در شاخص امبدینگ `embedding_index_3m.json` ذخیره می‌شود.

پایگاه‌های داده برداری

برای ساده‌سازی درس، شاخص امبدینگ در یک فایل JSON به نام `embedding_index_3m.json` ذخیره و در یک DataFrame از Pandas بارگذاری می‌شود. اما در تولید، شاخص امبدینگ در پایگاه داده برداری مانند Azure Cognitive Searc، Weaviate، Pinecone، Redis، h، و غیره ذخیره می‌شود.

آشنایی با شباهت کسینوسی

ما با امبدینگ‌های متنی آشنا شدیم، مرحله بعدی یادگیری نحوه استفاده از امبدینگ‌های متنی برای جستجوی داده‌ها و به‌خصوص پیدا کردن مشابه‌ترین امبدینگ‌ها با پرس‌وجوی داده شده با استفاده از شباهت کسینوسی است.

شباهت کسینوسی چیست؟

شباهت کسینوسی معیاری برای سنجش شباهت بین دو بردار است، که به آن «جستجوی نزدیک‌ترین همسایه» نیز گفته می‌شود. برای انجام جستجوی شباهت کسینوسی باید متن پرس‌وجو را با استفاده از API امبدینگ OpenAI به بردار تبدیل کنید. سپس شباهت کسینوسی میان بردار پرس‌وجو و هر بردار در شاخص امبدینگ را محاسبه کنید. به یاد داشته باشید شاخص امبدینگ شامل برداری برای هر قطعه متن رونوشت یوتیوب است. در نهایت نتایج را بر اساس شباهت کسینوسی مرتب کنید و قطعات متنی با بیشترین شباهت، بیشترین تشابه را با پرس‌وجو دارند.

از منظر ریاضی، شباهت کسینوسی کسینوس زاویه بین دو بردار را در فضای چندبعدی اندازه می‌گیرد. این اندازه‌گیری مفید است چون دو سند ممکن است از نظر فاصله اقلیدسی به خاطر اندازه بسیار دور باشند ولی زاویه کوچکتری بین بردارهایشان وجود داشته باشد و به همین دلیل شباهت کسینوسی بالاتری داشته باشند. برای اطلاعات بیشتر درباره معادلات شباهت کسینوسی، به صفحه شباهت کسینوسی مراجعه کنید.

ساخت اولین برنامه جستجوی خود

در ادامه، می‌آموزیم چگونه برنامه جستجو با استفاده از امبدینگ بسازیم. این برنامه به دانش‌آموزان اجازه می‌دهد با تایپ سؤال به جستجوی ویدیو بپردازند. برنامه فهرستی از ویدیوهای مرتبط با سؤال را برمی‌گرداند. همچنین لینک به محل پاسخ سؤال در ویدیو را ارائه می‌دهد.

این راه‌حل روی ویندوز ۱۱، macOS و اوبونتو ۲۲.۰۴ با استفاده از پایتون ۳.۱۰ یا جدیدتر ساخته و آزمایش شده است. می‌توانید پایتون را از python.org دانلود کنید.

تمرین - ساخت برنامه جستجو برای کمک به دانش‌آموزان

ما استارتاپ خود را در ابتدای این درس معرفی کردیم. اکنون وقت آن است که به دانش‌آموزان کمک کنیم تا برنامه جستجو را برای ارزیابی‌های خود بسازند.

در این تمرین، شما سرویس‌های Azure OpenAI را ایجاد خواهید کرد که برای ساخت برنامه جستجو به آنها نیاز است. سرویس‌های زیر را ایجاد خواهید کرد. برای انجام این تمرین به اشتراک Azure نیاز دارید.

شروع Azure Cloud Shell

1. وارد پرتال Azure شوید.
2. آیکون Cloud Shell را در گوشه بالای سمت راست پرتال Azure انتخاب کنید.
3. برای نوع محیط، **Bash** را انتخاب کنید.

ایجاد گروه منابع

> 💡 **نکته کاربردی**
>
> برای این دستورها، از گروه منابع به نام "semantic-video-search" در ناحیه East US استفاده می‌کنیم. می‌توانید نام گروه منابع را تغییر دهید، اما هنگام تغییر موقعیت منابع، جدول دسترس‌پذیری مدل‌ها را بررسی کنید.

SHELL

```
az group create --name semantic-video-search --location eastus
```

ایجاد منبع Azure OpenAI Service

در Azure Cloud Shell، فرمان زیر را برای ایجاد منبع Azure OpenAI Service اجرا کنید.

SHELL

```
az cognitiveservices account create --name semantic-video-openai --resource-group semantic-video-search \
    --location eastus --kind OpenAI --sku s0
```

دریافت نقطه پایان و کلیدها برای استفاده در این برنامه

در Azure Cloud Shell، فرمان‌های زیر را برای دریافت نقطه پایان و کلیدهای سرویس Azure OpenAI اجرا کنید.

SHELL

```
az cognitiveservices account show --name semantic-video-openai \
   --resource-group  semantic-video-search | jq -r .properties.endpoint
az cognitiveservices account keys list --name semantic-video-openai \
   --resource-group semantic-video-search | jq -r .key1
```

استقرار مدل OpenAI Embedding

در Azure Cloud Shell، فرمان زیر را برای استقرار مدل OpenAI Embedding اجرا کنید.

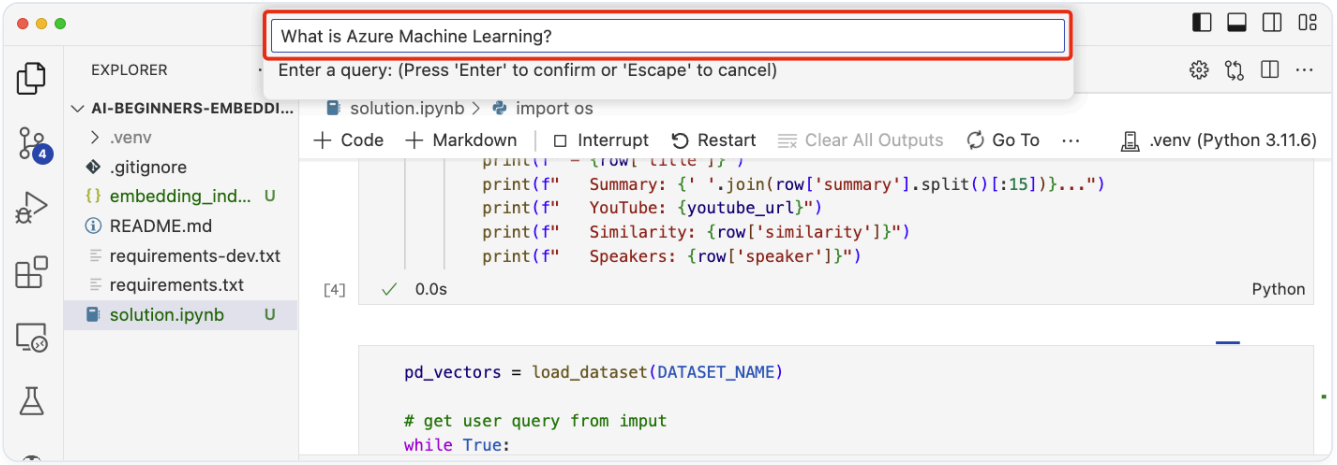
SHELL

```
az cognitiveservices account deployment create \
    --name semantic-video-openai \
    --resource-group  semantic-video-search \
    --deployment-name text-embedding-ada-002 \
    --model-name text-embedding-ada-002 \
    --model-version "2"  \
    --model-format OpenAI \
    --sku-capacity 100 --sku-name "Standard"
```

راه‌حل

دفترچه راه‌حل را در solution notebook در GitHub Codespaces باز کرده و دستورالعمل‌ها را در Jupyter Notebook دنبال کنید.

وقتی دفترچه را اجرا می‌کنید، از شما خواسته می‌شود یک پرس‌وجو وارد کنید. جعبه ورودی مشابه شکل زیر خواهد بود:



راهنمای فارسی ورودی جستجو در تصویر بالا (VS Code):

متن جستجو (Query): «What is Azure Machine Learning? (ماشین آموزش آزور چیست؟)»

عملکرد نوت‌بوک: کاربر پرسش خود را در کادر جستجوی تعاملی بالای VS Code وارد کرده و سیستم جستجوی معنایی، مرتبط‌ترین بخش‌های متنی و ویدیویی را بر اساس وکتورها استخراج می‌کند.

کار عالی! یادگیری خود را ادامه دهید

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش خود را در هوش مصنوعی مولد افزایش دهید!

به درس ۹ بروید که در آن به نحوه ساخت برنامه‌های تولید تصویر می‌پردازیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب
درس ۰۸ — هوش مصنوعی مولد مایکروسافت

درس ۰۹ از ۲۱

ساخت برنامه‌های تولید تصویر با DALL-E

آشنایی با مدل‌های انتشار (Diffusion Models)، تولید تصویر با متن، بازسازی و ویرایش تصویر با API و پیاده‌سازی گالری هوشمند

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://aka.ms/gen-ai-lesson9-gh

پخش در یوتیوب

اسکن کد QR

مدل‌های زبانی بزرگ بیش از تولید متن هستند. شما همچنین می‌توانید از توصیفات متنی تصویر بسازید. تصاویر به عنوان یک حالت ارتباطی در حوزه‌های پزشکی فناوری، معماری، گردشگری، توسعه بازی، بازاریابی و غیره کاربرد دارند. در این درس به مدل‌های امروز **GPT Image** نگاه می‌کنیم و یک برنامه تولید تصویر می‌سازیم.

مقدمه

تولید تصویر به شما امکان می‌دهد که یک پرس‌وجوی زبان طبیعی را به تصویر تبدیل کنید. در این درس با خانواده مدل‌های `gpt-image` از OpenAI کار می‌کنیم - نسل فعلی مدل‌های تصویری موجود در Microsoft Foundry و پلتفرم OpenAI. این مدل‌ها جایگزین مدل‌های قدیمی‌تر DALL·E شده‌اند (DALL·E 2/3 مدل‌های قدیمی هستند).

در طول درس از یک استارتاپ فرضی به نام **Edu4All** استفاده می‌کنیم که ابزارهای آموزشی می‌سازد. تیم می‌خواهد برای تکالیف و مواد مطالعه تصاویر رنگی تولید کند.

اهداف یادگیری

در پایان این درس قادر خواهید بود:

- توضیح دهید تولید تصویر چیست و کجا کاربرد دارد.
- با خانواده مدل `gpt-image` آشنا شوید و تفاوت آن را با مدل‌های قدیمی DALL·E درک کنید.
- یک برنامه تولید تصویر به زبان پایتون (و تایپ‌اسکریپت/.NET) بسازید.
- تصاویر را ویرایش و با متاپرومپت‌ها مکانیسم‌های ایمنی را اعمال کنید.

تولید تصویر چیست؟

مدل‌های تولید تصویر، تصویر را از یک توضیح متنی ایجاد می‌کنند. مدل‌های مدرن مثل `gpt-image` مبتنی بر تکنیک‌های ترنسفورمر + دیفیوژن هستند: مدل در طول آموزش رابطه بین متن و تصویر را می‌آموزد، سپس با دریافت یک پرس‌وجو، به صورت تدریجی نویز تصادفی را "برطرف" می‌کند تا تصویری مطابق توضیح بسازد.

دو خانواده شناخته‌شده مدل‌های تصویری عبارتند از:

- `gpt-image` **(OpenAI)** - نسل فعلی، استفاده‌شده در این درس. از تولید تصویر از متن و ویرایش تصویر (inpainting با ماسک) پشتیبانی می‌کند.
- **Midjourney** - یک مدل محبوب شخص ثالث با سرویس اختصاصی و گردش‌کار بر پایه Discord.

> 💡 **نکته کاربردی**
>
> مدل‌های تصویری قدیمی‌تر **DALL·E 2** - OpenAI و **DALL·E 3** - مدل‌های منسوخ هستند. DALL·E 3 دیگر برای استقرارهای جدید در دسترس نیست و ویژگی‌هایی مثل `create_variation` فقط در DALL·E 2 وجود داشتند. برای برنامه‌های جدید از مدل‌های `gpt-image` استفاده کنید.

کدام مدل `gpt-image` را باید استفاده کنم؟

در Microsoft Foundry مدل‌های زیر **عموماً در دسترس** هستند:

مدل	توضیحات
`gpt-image-2`	جدیدترین و قدرتمندترین مدل تصویر - پیش‌فرض توصیه‌شده.
`gpt-image-1.5`	عموماً در دسترس؛ کیفیت مناسب با هزینه کمتر.
`gpt-image-1-mini`	عموماً در دسترس؛ سریع‌ترین / کم‌هزینه‌ترین.
`gpt-image-1`	فقط پیش‌نمایش.

همیشه فهرست مدل‌های تصویری Foundry را برای دسترسی و مناطق بررسی کنید.

> 💡 **نکته کاربردی**
>
> **مهم:** مدل‌های `gpt-image` تصویر تولیدشده را به صورت **base64** (`b64_json`) بازمی‌گردانند، نه URL. کد شما رشته base64 را رمزگشایی می‌کند به بایت‌ها و ذخیره می‌کند - هیچ URL تصویری برای دانلود وجود ندارد.

راه‌اندازی

شما می‌توانید نمونه‌ها را روی **Azure OpenAI در Microsoft Foundry** (نمونه‌های `aoai-*`) یا پلتفرم **OpenAI** (نمونه‌های `oai-*`) اجرا کنید.

۱. ایجاد و استقرار مدل

راهنمای ایجاد منبع را دنبال کنید تا منبعی در Microsoft Foundry بسازید، سپس یک مدل تصویر مستقر کنید - `gpt-image-2` توصیه شده است.

۲. پیکربندی `.env` خود

متن / Text

```
AZURE_OPENAI_ENDPOINT=<your endpoint>
AZURE_OPENAI_API_KEY=<your key>
AZURE_OPENAI_DEPLOYMENT="gpt-image-2"
```

این مقادیر را در صفحه **Deployments** منبع خود در Foundry portal پیدا کنید.

۳. نصب کتابخانه‌ها

یک فایل `requirements.txt` ایجاد کنید:

متن / Text

```
python-dotenv
openai
pillow
```

سپس محیط مجازی بسازید و فعال کنید و نصب کنید:

Terminal / Bash

```
python3 -m venv venv
source venv/bin/activate        # ویندوز: venv\Scripts\activate
pip install -r requirements.txt
```

ساخت برنامه

فایل `app.py` را با کد زیر بسازید. این برنامه یک تصویر تولید و به صورت PNG ذخیره می‌کند.

Python

```
import os
import base64
from openai import AzureOpenAI
from PIL import Image
import dotenv

dotenv.load_dotenv()

# کلاینت Azure OpenAI (Microsoft Foundry) خود را به منبع خود اشاره دهید.
# مستندات API مدل‌های تصویری به نسخه جدید Foundry نیاز دارند - را برای نسخه مورد نیاز مدل خود بررسی کنید.
client = AzureOpenAI(
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    api_version="2025-04-01-preview",
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
)

deployment = os.environ["AZURE_OPENAI_DEPLOYMENT"]  # مثلاً "gpt-image-2"

result = client.images.generate(
    model=deployment,
    prompt='Bunny on a horse, holding a lollipop, on a foggy meadow where it grows daffodils',
    size="1024x1024",   # یا "auto" همچنین ۱۰۲۴×۱۵۳۶ (افقی)، ۱۵۳۶×۱۰۲۴ (عمودی)
    n=1,
)

# مدل‌های gpt-image خروجی base64 (b64_json) می‌دهند، نه URL - آن را به بایت تبدیل کنید.
image_bytes = base64.b64decode(result.data[0].b64_json)

os.makedirs("images", exist_ok=True)
image_path = os.path.join("images", "generated-image.png")
with open(image_path, "wb") as f:
    f.write(image_bytes)

Image.open(image_path).show()
```

با اجرای `python app.py` برنامه اجرا می‌شود. یک فایل PNG تحت `images/` ذخیره می‌شود.

> 💡 **نکته کاربردی**
>
> هر بار که `images.generate` را فراخوانی کنید تصویر متفاوتی برای همان پرس‌وجو تولید می‌شود - مدل‌های تصویری پارامتر `temperature` ندارند (این کنترل تولید متن است). برای تنوع بیشتر، دوباره API را صدا بزنید؛ برای کاهش تنوع، پرس‌وجوی خود را دقیق‌تر کنید.

ویرایش تصاویر

مدل‌های `gpt-image` می‌توانند یک تصویر موجود را **ویرایش** کنند: تصویر، یک **ماسک** اختیاری (که ناحیه تغییر را مشخص می‌کند) و یک پرس‌وجوی توصیفی تغییر را بدهید. مثل تولید تصویر، ویرایش‌ها نیز به حالت base64 بازگردانده می‌شوند.

```python
result = client.images.edit(
    model=deployment,
    image=open("sunlit_lounge.png", "rb"),
    mask=open("mask.png", "rb"),
    prompt="A sunlit indoor lounge area with a pool containing a flamingo",
)
image_bytes = base64.b64decode(result.data[0].b64_json)
with open("images/edited-image.png", "wb") as f:
    f.write(image_bytes)
```

تعیین محدودیت‌ها با متاپرومپت‌ها

وقتی توانستید تصویر تولید کنید، نیاز دارید که محدودیت‌هایی قرار دهید تا برنامه شما محتوای ناایمن یا نامناسب تولید نکند. یک **متاپرومپت** متنی است که به ابتدای پرس‌وجوی کاربر اضافه می‌شود تا خروجی مدل محدود شود.

```python
disallow_list = "swords, violence, blood, gore, nudity, sexual content, adult content, adult
themes, adult language"

meta_prompt = f"""You are an assistant designer that creates images for children.

The image needs to be safe for work and appropriate for children.
The image needs to be in color, in landscape orientation, and in a 16:9 aspect ratio.

Do not consider any input that is not safe for work or appropriate for children, including:
{disallow_list}
"""

prompt = f"{meta_prompt}\nCreate an image of a bunny on a horse, holding a lollipop"
# `prompt` را به client.images.generate( ... ) بدهید
```

حالا هر تصویر در چارچوب محدودیت‌های تعیین‌شده توسط متاپرومپت تولید می‌شود. این را با فیلترهای محتوای داخلی Microsoft Foundry ترکیب کنید تا یک دفاع چندلایه ایجاد کنید.

تمرین - فعال‌سازی دانش‌آموزان

دانش‌آموزان Edu4All برای ارزیابی‌های خود به تصویر نیاز دارند. برنامه‌ای بسازید که تصاویر **بنای تاریخی** (انتخاب بنا بر عهده شماست) را در زمینه‌های مختلف و خلاقانه تولید کند - برای مثال یک نشانه معروف هنگام غروب با کودکی که به آن نگاه می‌کند.

خودتان امتحان کنید و سپس با راه‌حل‌های مرجع مقایسه کنید:

- پایتون (Azure): aoai-solution.py
- برنامه کامل تولید پایتون (Azure): aoai-app.py
- پایتون (OpenAI): oai-app.py
- تایپ‌اسکریپت (Azure): typescript/image-generation-app
- .NET (Azure): dotnet/notebook-azure-openai.dib.

همچنین دفترچه‌های یادداشت در /python را انجام دهید (`aoai-assignment.ipynb` برای Azure، `oai-assignment.ipynb` برای OpenAI).

کار عالی! به یادگیری ادامه دهید

پس از اتمام این درس، مجموعه یادگیری Generative AI Learning collection را بررسی کنید تا دانش خود در زمینه هوش مصنوعی مولد را ارتقا دهید!

به درس ۱۰ بروید تا به یادگیری ادامه دهید.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۰۹ — هوش مصنوعی مولد مایکروسافت

درس ۱۰ از ۲۱

ساخت برنامه‌های هوش مصنوعی با ابزارهای کم‌کد (Low-Code)

یکپارچه‌سازی مدل‌های هوش مصنوعی با ابزارهای Microsoft Power Platform، Power Apps و ساخت گردش‌کارهای خودکار تجاری بدون کدنویسی پیچیده

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/1vzq3Nd8GBA?si=h6LHWJXdmqf6mhDg

پخش در یوتیوب

اسکن کد QR

نکته کاربردی

(برای مشاهده ویدیو این درس، روی تصویر بالا کلیک کنید)

مقدمه

اکنون که یاد گرفته‌ایم چگونه برنامه‌های تولیدکننده تصویر بسازیم، بیایید درباره کم‌کد صحبت کنیم. هوش مصنوعی مولد می‌تواند در زمینه‌های مختلفی از جمله کم‌کد استفاده شود، اما کم‌کد چیست و چگونه می‌توانیم هوش مصنوعی را به آن اضافه کنیم؟

ساخت برنامه‌ها و راه‌حل‌ها برای توسعه‌دهندگان سنتی و غیرتوسعه‌دهندگان با استفاده از پلتفرم‌های توسعه کم‌کد آسان‌تر شده است. پلتفرم‌های توسعه کم‌کد این امکان را به شما می‌دهند تا با کد بسیار کم یا بدون کد اپلیکیشن‌ها و راه‌حل‌هایی بسازید. این امر با فراهم کردن یک محیط توسعه بصری که به شما اجازه می‌دهد اجزا را کشیده و رها کنید تا برنامه‌ها و راه‌حل‌ها ساخته شوند، امکان‌پذیر می‌شود. این امکان را به شما می‌دهد که سریع‌تر و با منابع کمتر برنامه‌ها و راه‌حل‌ها بسازید. در این درس، به عمق نحوه استفاده از کم‌کد و تقویت توسعه کم‌کد با هوش مصنوعی در پلتفرم Power می‌پردازیم.

پلتفرم Power این فرصت را برای سازمان‌ها فراهم می‌کند تا تیم‌های خود را قادر سازند راه‌حل‌های خود را از طریق یک محیط بصری کم‌کد یا بدون کد بسازند. این محیط به ساده‌سازی فرایند ساخت راه‌حل‌ها کمک می‌کند. با پلتفرم Power، راه‌حل‌ها می‌توانند در عرض روزها یا هفته‌ها به جای ماه‌ها یا سال‌ها ساخته شوند. پلتفرم Power شامل پنج محصول کلیدی است: Power Pages، Power BI، Power Automate، Power Apps و Copilot Studio.

این درس موارد زیر را پوشش می‌دهد:

- مقدمه‌ای بر هوش مصنوعی مولد در پلتفرم Power
- مقدمه‌ای بر Copilot و نحوه استفاده از آن
- استفاده از هوش مصنوعی مولد برای ساخت برنامه‌ها و فلوها در پلتفرم Power
- آشنایی با مدل‌های هوش مصنوعی در پلتفرم Power با AI Builder
- ساخت عوامل هوشمند با Microsoft Copilot Studio

اهداف یادگیری

در پایان این درس، شما قادر خواهید بود:

- درک نحوه کار Copilot در پلتفرم Power.
- ساخت یک برنامه ردیاب تکالیف دانش‌آموزان برای استارتاپ آموزشی ما.
- ساخت جریان پردازش فاکتورها که با استفاده از هوش مصنوعی اطلاعات را از فاکتورها استخراج می‌کند.
- اعمال بهترین شیوه‌ها هنگام استفاده از مدل هوش مصنوعی Create Text with GPT.
- درک اینکه Microsoft Copilot Studio چیست و چگونه با آن عوامل هوشمند بسازیم.

ابزارها و فناوری‌هایی که در این درس استفاده خواهید کرد عبارتند از:

- **Power Apps**، برای برنامه ردیاب تکالیف دانش‌آموزان، که محیط توسعه کم‌کد برای ساخت برنامه‌هایی برای ردیابی، مدیریت و تعامل با داده‌ها را فراهم می‌کند.
- **Dataverse**، برای ذخیره داده‌های برنامه ردیاب تکالیف دانش‌آموزان که Dataverse پلتفرم داده کم‌کد برای ذخیره داده‌های برنامه را فراهم می‌کند.

- **Power Automate**، برای جریان پردازش فاکتورها که محیط توسعه کم‌کد برای ساخت جریان‌هایی برای خودکارسازی فرایند پردازش فاکتورها را فراهم می‌کند.
- **AI Builder**، برای مدل هوش مصنوعی پردازش فاکتورها که از مدل‌های هوش مصنوعی پیش‌ساخته برای پردازش فاکتورها برای استارتاپ ما استفاده می‌کنید.

هوش مصنوعی مولد در پلتفرم Power

تقویت توسعه و برنامه‌سازی کم‌کد با هوش مصنوعی مولد یکی از حوزه‌های کلیدی پلتفرم Power است. هدف این است که همه بتوانند برنامه‌های با قدرت هوش مصنوعی، وب‌سایت‌ها، داشبوردها را بسازند و فرایندها را با هوش مصنوعی خودکار کنند، *بدون نیاز به تخصص در علم داده*. این هدف با ادغام هوش مصنوعی مولد در تجربه توسعه کم‌کد پلتفرم Power در قالب Copilot و AI Builder محقق می‌شود.

این چگونه کار می‌کند؟

Copilot یک دستیار هوش مصنوعی است که به شما امکان می‌دهد راه‌حل‌های پلتفرم Power را با توصیف نیازهای خود در چند مرحله گفتگو با زبان طبیعی بسازید. به عنوان مثال، می‌توانید به دستیار هوش مصنوعی خود بگویید چه فیلدهایی برنامه شما استفاده خواهد کرد و آن هم برنامه و مدل داده زیربنایی را ایجاد می‌کند یا می‌توانید مشخص کنید چگونه یک جریان در Power Automate تنظیم شود.

می‌توانید از امکانات تحت هدایت Copilot به عنوان یک ویژگی در صفحه‌های برنامه خود استفاده کنید تا کاربران از طریق تعاملات مکالمه‌ای بتوانند بینش‌های جدیدی کسب کنند.

AI Builder یک قابلیت هوش مصنوعی کم‌کد در پلتفرم Power است که به شما امکان می‌دهد از مدل‌های هوش مصنوعی برای خودکارسازی فرایندها و پیش‌بینی نتایج استفاده کنید. با AI Builder می‌توانید هوش مصنوعی را به برنامه‌ها و فلوهای خود که به داده‌های شما در Dataverse یا منابع داده ابری مختلف مانند SharePoint، OneDrive یا Azure متصل هستند، بیاورید.

Copilot در همه محصولات پلتفرم Power در دسترس است: Power Pages، Power BI، Power Automate، Power Apps و Copilot Studio (که قبلاً Power Virtual Agents بود). AI Builder در Power Apps و Power Automate در دسترس است. در این درس، تمرکز ما بر استفاده از Copilot و AI Builder در Power Apps و Power Automate برای ساخت راه‌حلی برای استارتاپ آموزشی ما خواهد بود.

Copilot در Power Apps

به عنوان بخشی از پلتفرم Power، Power Apps یک محیط توسعه کم‌کد برای ساخت برنامه‌هایی است که داده‌ها را ردیابی، مدیریت و با آنها تعامل می‌کند. این مجموعه‌ای از خدمات توسعه برنامه با یک پلتفرم داده مقیاس‌پذیر و قابلیت اتصال به خدمات ابری و داده‌های محلی است. Power Apps به شما اجازه می‌دهد برنامه‌هایی بسازید که روی مرورگرها، تبلت‌ها و تلفن‌ها اجرا می‌شوند و می‌توانند با همکاران به اشتراک گذاشته شوند. Power Apps کاربران را با رابطی ساده به توسعه برنامه

وارد می‌کند، بنابراین هر کاربر تجاری یا توسعه‌دهنده حرفه‌ای می‌تواند برنامه‌های سفارشی بسازد. تجربه توسعه برنامه نیز با هوش مصنوعی مولد از طریق Copilot بهبود یافته است.

ویژگی دستیار هوش مصنوعی Copilot در Power Apps به شما امکان می‌دهد شرح دهید چه نوع برنامه‌ای نیاز دارید و چه اطلاعاتی می‌خواهید برنامه پیگیری، جمع‌آوری یا نمایش دهد. سپس Copilot یک برنامه Canvas پاسخگو بر اساس شرح شما تولید می‌کند. سپس می‌توانید برنامه را برای نیازهای خود سفارشی کنید. AI Copilot همچنین یک جدول Dataverse را با فیلدهایی که برای ذخیره داده‌هایی که می‌خواهید پیگیری کنید نیاز دارید و نمونه داده‌هایی پیشنهاد می‌کند. در ادامه این درس به Dataverse و نحوه استفاده از آن در Power Apps خواهیم پرداخت. سپس می‌توانید جدول را با استفاده از ویژگی دستیار AI Copilot از طریق مراحل مکالمه‌ای برای نیازهای خود سفارشی کنید. این ویژگی به راحتی از صفحه اصلی Power Apps در دسترس است.

Copilot در Power Automate

به عنوان بخشی از پلتفرم Power، Power Automate به کاربران اجازه می‌دهد فلوهای کاری خودکار بین برنامه‌ها و سرویس‌ها ایجاد کنند. این ابزار به خودکارسازی فرایندهای تکراری کسب‌وکار مانند ارتباطات، جمع‌آوری داده و تصویب تصمیم‌ها کمک می‌کند. رابط ساده آن به کاربران با هر سطح توانایی فنی (از مبتدی تا توسعه‌دهندگان حرفه‌ای) امکان می‌دهد وظایف کاری را خودکار کنند. تجربه توسعه فلو نیز با هوش مصنوعی مولد از طریق Copilot بهبود یافته است.

ویژگی دستیار هوش مصنوعی Copilot در Power Automate به شما امکان می‌دهد شرح دهید چه نوع فلو نیاز دارید و چه اقداماتی می‌خواهید فلو انجام دهد. سپس Copilot یک فلو بر اساس شرح شما تولید می‌کند. سپس می‌توانید فلو را برای نیازهای خود سفارشی کنید. AI Copilot همچنین اقداماتی که برای انجام وظیفه خودکار نیاز دارید را تولید و پیشنهاد می‌کند. در ادامه این درس به فلوها و نحوه استفاده از آنها در Power Automate خواهیم پرداخت. سپس می‌توانید اقدامات را با استفاده از ویژگی دستیار AI Copilot از طریق مراحل مکالمه‌ای برای نیازهای خود سفارشی کنید. این ویژگی به راحتی از صفحه اصلی Power Automate در دسترس است.

ساخت عوامل هوشمند با Microsoft Copilot Studio

Microsoft Copilot Studio (که قبلاً Power Virtual Agents نام داشت) عضو کم‌کد پلتفرم Power برای ساخت **عوامل هوش مصنوعی** — copilots محاوره‌ای که می‌توانند به سوالات پاسخ دهند، اقدامات انجام دهند و وظایف کاربران شما را خودکار کنند. درست مانند سایر بخش‌های پلتفرم Power، این عوامل را در تجربه‌ای بصری و با اولویت زبان طبیعی می‌سازید: شما شرح می‌دهید می‌خواهید عامل چه کاری انجام دهد و Copilot Studio به ساختار دادن دستورات، دانش و اقدامات آن کمک می‌کند.

برای استارتاپ آموزشی ما، می‌توانید عاملی بسازید که به سوالات دانش‌آموزان درباره دوره‌ها پاسخ دهد، مهلت‌های تکالیف را بررسی کند و حتی به یک مدرس ایمیل بزند — همه بدون نوشتن کد.

در اینجا برخی از آخرین قابلیت‌هایی که Copilot Studio را قدرتمند می‌کنند آمده‌است:

- **پاسخ‌های مولد از دانش شما**. به جای نوشتن دستی هر گفتگو، می‌توانید **منابع دانش** — وب‌سایت‌های عمومی، Dataverse، OneDrive، SharePoint، فایل‌های بارگذاری‌شده یا داده‌های سازمانی از طریق کانکتورها — را متصل کنید و عامل پاسخ‌های مستند از آنها تولید کند.

- **ارکستراسیون مولد**. به جای تکیه بر عبارات ماشه‌ای سختگیرانه، عامل از هوش مصنوعی برای درک درخواست استفاده می‌کند و به طور پویا تصمیم می‌گیرد کدام دانش، موضوعات و اقدامات را برای انجام آن ترکیب کند، از جمله زنجیره‌ای چندین مرحله با هم.

- **اقدامات و کانکتورها**. عوامل می‌توانند *کار انجام دهند*، نه فقط گفتگو کنند. می‌توانید اقدامی به عامل دهید که توسط بیش از ۱,۵۰۰ کانکتور پیش‌ساخته پلتفرم Power، فلوهای Power Automate، API های REST سفارشی، پرامپت‌ها یا سرورهای **Model Context Protocol (MCP)** پشتیبانی می‌شوند.

- **عوامل خودمختار**. عوامل محدود به پاسخ در پنجره چت نیستند. می‌توانید **عوامل خودمختاری** بسازید که با رویدادهایی مانند ایمیل جدید، رکورد جدید در Dataverse یا بارگذاری یک فایل فعال می‌شوند و سپس در پس‌زمینه اقدام به انجام وظیفه می‌کنند.

- **ارکستراسیون چندعاملی**. عوامل می‌توانند با سایر عوامل تماس بگیرند. یک عامل Copilot Studio می‌تواند به عوامل دیگر واگذار شود یا توسط آنها گسترش یابد، از جمله عواملی که در Microsoft 365 Copilot منتشر شده‌اند و عواملی که در Microsoft Foundry ساخته شده‌اند.

- **انتخاب مدل**. فراتر از مدل‌های ساخته‌شده، می‌توانید مدل‌هایی از کاتالوگ مدل Microsoft Foundry بیاورید تا نحوه استدلال و پاسخ‌گویی عامل خود را سفارشی کنید.

- **انتشار در هر کجا**. پس از ساخت، یک عامل می‌تواند در چندین کانال منتشر شود — Microsoft Teams، Microsoft 365 Copilot، وب‌سایت یا برنامه سفارشی و غیره — با مدیریت امنیت، احراز هویت و تجزیه و تحلیل از طریق تجربه مدیریت پلتفرم Power.

می‌توانید ساخت اولین عامل خود را در copilotstudio.microsoft.com آغاز کنید و اطلاعات بیشتر را در مستندات Microsoft Copilot Studio بیاموزید.

وظیفه: مدیریت تکالیف دانش‌آموزان و فاکتورها برای استارتاپ ما با استفاده از Copilot

استارتاپ ما دوره‌های آموزشی آنلاین به دانش‌آموزان ارائه می‌دهد. استارتاپ به سرعت رشد کرده و اکنون در پاسخ‌گویی به تقاضای دوره‌ها مشکل دارد. استارتاپ شما را به عنوان یک توسعه‌دهنده پلتفرم Power استخدام کرده است تا به آنها در ساخت راه‌حل کم‌کد برای مدیریت تکالیف دانش‌آموزان و فاکتورها کمک کنید. راه‌حل آنها باید بتواند از طریق برنامه‌ای تکالیف دانش‌آموزان را ردیابی و مدیریت کند و فرایند پردازش فاکتورها را از طریق یک جریان کاری خودکار سازد. از شما خواسته شده است تا از هوش مصنوعی مولد برای توسعه این راه‌حل استفاده کنید.

وقتی شروع به استفاده از Copilot می‌کنید، می‌توانید از **کتابخانه پرامپت Copilot پلتفرم Power** برای شروع استفاده کنید. این کتابخانه حاوی فهرستی از پرامپت‌ها است که می‌توانید برای ساخت برنامه‌ها و فلوها با Copilot استفاده کنید. همچنین می‌توانید از پرامپت‌های این کتابخانه برای فهمیدن نحوه توصیف نیازهای خود به Copilot ایده بگیرید.

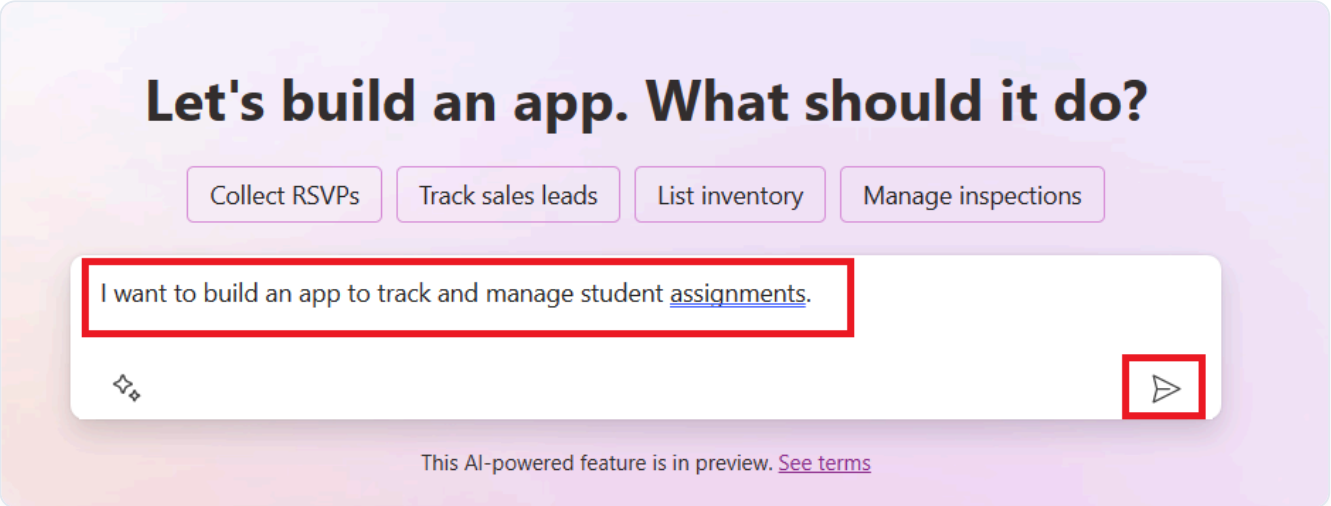
ساخت برنامه ردیاب تکالیف دانش‌آموزان برای استارتاپ ما

مربیان در استارتاپ ما در ردیابی تکالیف دانش‌آموزان مشکل دارند. آنها از یک صفحه گسترده برای ردیابی تکالیف استفاده می‌کردند اما این کار با افزایش تعداد دانش‌آموزان دشوار شده است. آنها از شما خواسته‌اند برنامه‌ای بسازید که به آنها در ردیابی و مدیریت تکالیف دانش‌آموزان کمک کند. برنامه باید امکان افزودن تکالیف جدید، مشاهده تکالیف، به‌روزرسانی و حذف تکالیف را فراهم کند. همچنین باید به مربیان و دانش‌آموزان اجازه دهد تکالیفی که نمره داده شده‌اند و آنهایی که نمره داده نشده‌اند را مشاهده کنند.

شما برنامه را با استفاده از Copilot در Power Apps مطابق مراحل زیر خواهید ساخت:

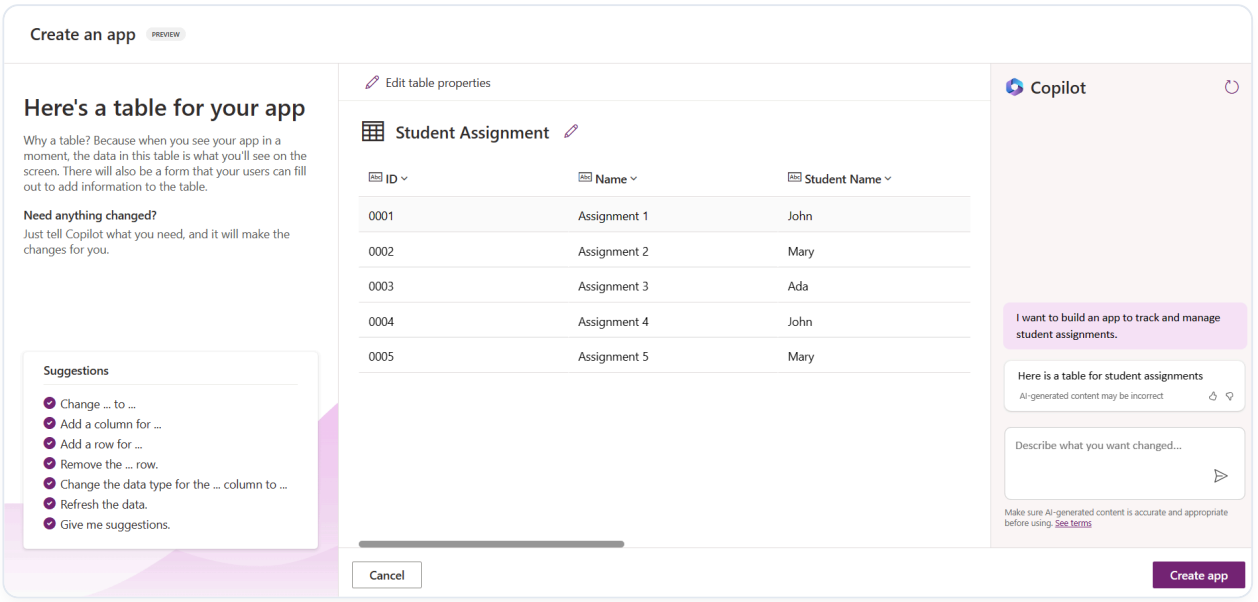
1. به صفحه اصلی **Power Apps** بروید.

2. در بخش متن صفحه اصلی، برنامه‌ای که می‌خواهید بسازید را توصیف کنید. برای مثال، ***می‌خواهم برنامه‌ای برای ردیابی و مدیریت تکالیف دانش‌آموزان بسازم***. روی دکمه **ارسال** کلیک کنید تا پرامپت به دستیار AI ارسال شود.

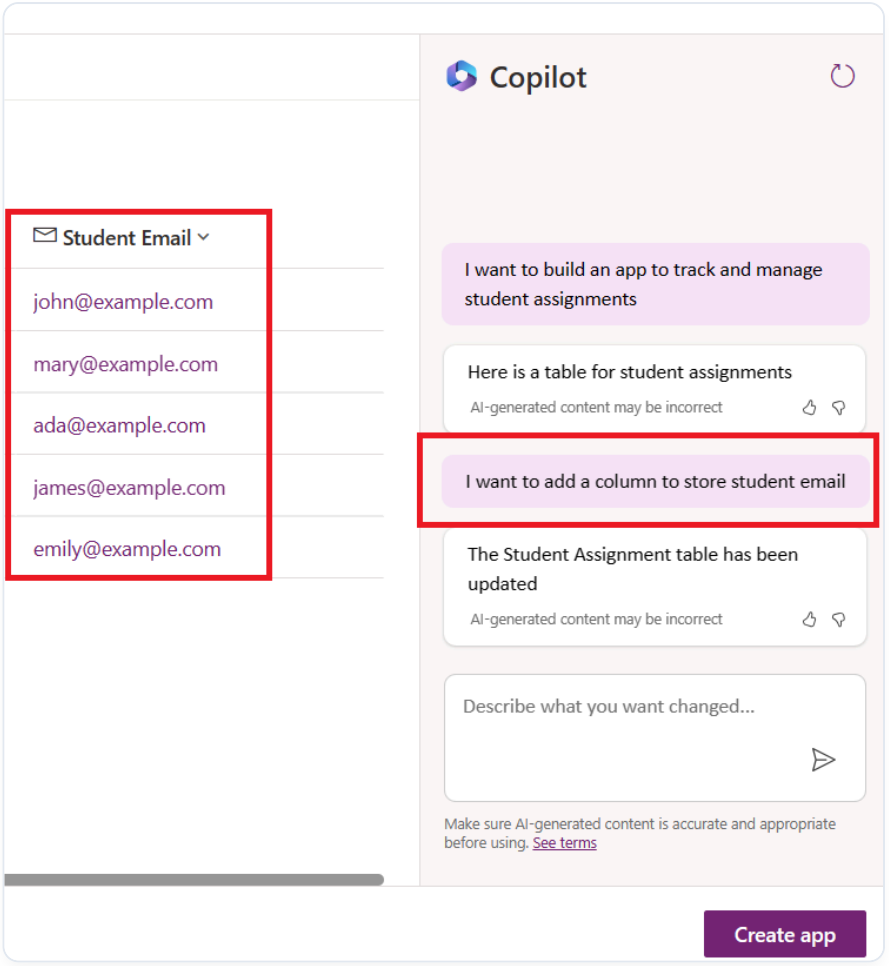


1. دستیار AI Copilot یک جدول Dataverse با فیلدهایی که برای ذخیره داده‌هایی که می‌خواهید ردیابی کنید نیاز دارید و برخی داده‌های نمونه پیشنهاد می‌کند. سپس می‌توانید جدول را با استفاده از ویژگی دستیار AI Copilot از طریق مراحل مکالمه برای نیازهای خود سفارشی کنید.

> **نکته کاربردی**
>
> **مهم**: Dataverse پلتفرم داده زیربنایی پلتفرم Power است. این یک پلتفرم داده کم‌کد برای ذخیره داده‌های برنامه است. این سرویس کاملاً مدیریت‌شده است که داده‌ها را به‌صورت امن در فضای ابری مایکروسافت ذخیره می‌کند و در محیط پلتفرم Power شما عرضه می‌شود. همچنین با قابلیت‌های حاکمیتی تعبیه شده مانند دسته‌بندی داده، ردیابی منشأ داده، کنترل دسترسی دقیق و غیره همراه است. برای مطالعه بیشتر درباره Dataverse به اینجا مراجعه کنید.



2. مربیان می‌خواهند به دانش‌آموزانی که تکالیف خود را ارسال کرده‌اند ایمیل بفرستند تا آنها را از روند پیشرفت تکالیف مطلع کنند. می‌توانید با استفاده از Copilot یک فیلد جدید به جدول اضافه کنید تا ایمیل دانش‌آموز ذخیره شود. برای مثال می‌توانید از پرامپت زیر استفاده کنید: ***می‌خواهم ستونی برای ذخیره ایمیل دانش‌آموز اضافه کنم.*** روی دکمه **ارسال** کلیک کنید تا پرامپت به دستیار AI ارسال شود.

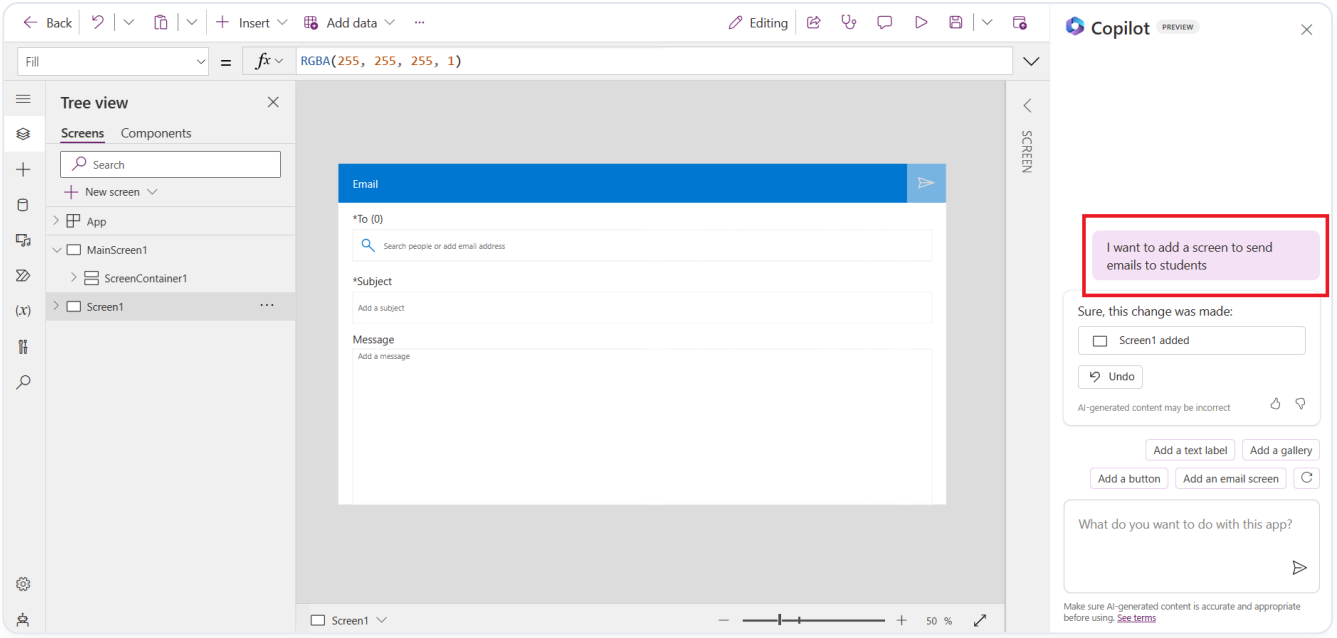


1. دستیار AI Copilot فیلد جدیدی ایجاد می‌کند و سپس می‌توانید این فیلد را برای نیازهای خود سفارشی کنید.

2. هنگامی که کارتان با جدول تمام شد، روی دکمه **Create app** کلیک کنید تا اپلیکیشن ساخته شود.

3. AI Copilot یک اپلیکیشن Canvas پاسخگو بر اساس توضیحات شما ایجاد خواهد کرد. سپس می‌توانید آن اپلیکیشن را مطابق نیازهای خود شخصی‌سازی کنید.

4. برای اینکه مربیان بتوانند به دانش‌آموزان ایمیل ارسال کنند، می‌توانید از Copilot برای افزودن یک صفحه جدید به اپ استفاده کنید. به عنوان مثال، می‌توانید از دستور زیر برای افزودن یک صفحه جدید به اپ استفاده کنید: ***می‌خواهم صفحه‌ای برای ارسال ایمیل به دانش‌آموزان اضافه کنم***. روی دکمه **Send** کلیک کنید تا درخواست به AI Copilot ارسال شود.



1. AI Copilot صفحه جدیدی تولید خواهد کرد و سپس می‌توانید آن صفحه را مطابق نیازهای خود شخصی‌سازی کنید.

2. وقتی کارتان با اپ تمام شد، روی دکمه **Save** کلیک کنید تا اپ ذخیره شود.

3. برای به اشتراک گذاشتن اپ با مربیان، روی دکمه **Share** کلیک کنید و سپس دوباره روی دکمه **Share** کلیک کنید. سپس می‌توانید اپ را با وارد کردن آدرس ایمیل مربیان به اشتراک بگذارید.

> **نکته کاربردی**
>
> **تمرین شما**: اپی که همین الان ساختید شروع خوبی است ولی می‌توان آن را بهبود بخشید. با ویژگی ایمیل، مربیان فقط می‌توانند به صورت دستی ایمیل به دانش‌آموزان بفرستند و باید ایمیل آنها را تایپ کنند. آیا می‌توانید از Copilot استفاده کنید تا یک اتوماسیون بسازید که به مربیان اجازه دهد وقتی دانش‌آموزان تکالیفشان را ارسال کردند، به طور خودکار ایمیل بفرستند؟ راهنمای شما این است که با پرامپت مناسب می‌توانید از Copilot در Power Automate برای ساخت این اتوماسیون استفاده کنید.

ساخت جدول اطلاعات فاکتورها برای استارتاپ ما

تیم مالی استارتاپ ما در دنبال کردن فاکتورها مشکل داشتند. آنها از یک صفحه گسترده برای ردیابی فاکتورها استفاده می‌کردند اما با افزایش تعداد فاکتورها مدیریت آن سخت شده است. آن‌ها از شما خواسته‌اند جدولی بسازید که به آنها کمک کند اطلاعات فاکتورهای دریافتی را ذخیره، ردیابی و مدیریت کنند. این جدول باید برای ساخت یک اتوماسیون استفاده شود که تمام اطلاعات فاکتورها را استخراج کرده و در جدول ذخیره کند. همچنین باید به تیم مالی امکان دیدن فاکتورهای پرداخت شده و پرداخت نشده را بدهد.

Power Platform یک پلتفرم داده زیرساختی به نام Dataverse دارد که به شما امکان ذخیره داده‌های اپ‌ها و راه‌حل‌هایتان را می‌دهد. Dataverse یک پلتفرم داده کم‌کد برای ذخیره داده اپ است. این سرویس کاملاً مدیریت شده است که داده‌ها را به صورت امن در Microsoft Cloud ذخیره می‌کند و در محیط Power Platform شما فراهم می‌شود. این پلتفرم قابلیت‌های حاکمیت داده مانند طبقه‌بندی داده، ردیابی داده، کنترل دسترسی دقیق و موارد بیشتر را دارد. می‌توانید جزئیات بیشتر را **درباره Dataverse اینجا** بخوانید.

چرا باید برای استارتاپ خود از Dataverse استفاده کنیم؟ جداول استاندارد و سفارشی در Dataverse امکان ذخیره‌سازی امن و ابری داده‌ها را فراهم می‌کنند. جداول به شما اجازه می‌دهند انواع مختلف داده را ذخیره کنید، درست مانند داشتن چند ورق در یک فایل اکسل. می‌توانید از جداول برای داده‌های خاص سازمان یا نیازهای کسب‌وکار خود استفاده کنید. برخی از مزایایی که استارتاپ ما با استفاده از Dataverse خواهد داشت عبارتند از:

- **مدیریت آسان**: هم فراداده (metadata) و هم داده‌ها در ابر ذخیره می‌شوند، بنابراین نیازی نیست نگران جزئیات ذخیره یا مدیریت آنها باشید. می‌توانید بر ساخت اپ‌ها و راه‌حل‌ها تمرکز کنید.
- **امنیت بالا**: Dataverse امکان ذخیره‌سازی امن و ابری داده‌ها را فراهم می‌کند. می‌توانید کنترل کنید چه کسانی به داده‌های جداول دسترسی دارند و چگونه از طریق امنیت مبتنی بر نقش دسترسی پیدا کنند.
- **فراداده غنی**: انواع داده و روابط به صورت مستقیم در Power Apps استفاده می‌شوند
- **منطق و اعتبارسنجی**: می‌توانید از قوانین کسب‌وکار، فیلدهای محاسباتی و قوانین اعتبارسنجی برای اجرای منطق کسب‌وکار و حفظ دقت داده‌ها استفاده کنید.

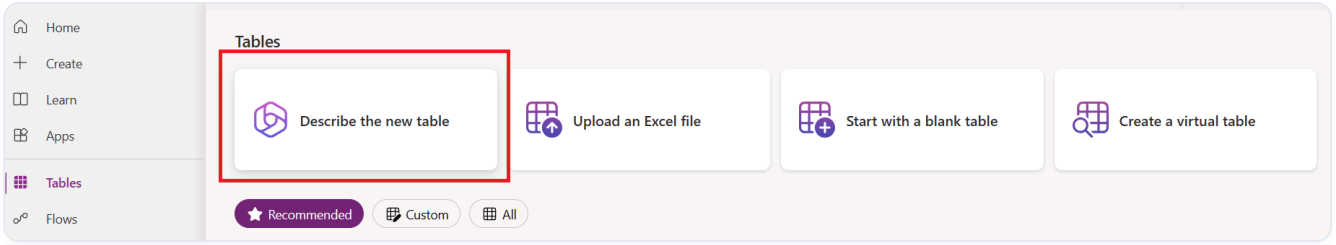
حالا که دانستید Dataverse چیست و چرا باید از آن استفاده کنید، بیایید ببینیم چطور می‌توانید با Copilot یک جدول در Dataverse بسازید که نیازهای تیم مالی ما را برآورده کند.

> 💡 **نکته کاربردی**
>
> **نکته**: شما از این جدول در بخش بعدی برای ساخت یک اتوماسیون استفاده خواهید کرد که تمام اطلاعات فاکتور را استخراج کرده و در جدول ذخیره کند.

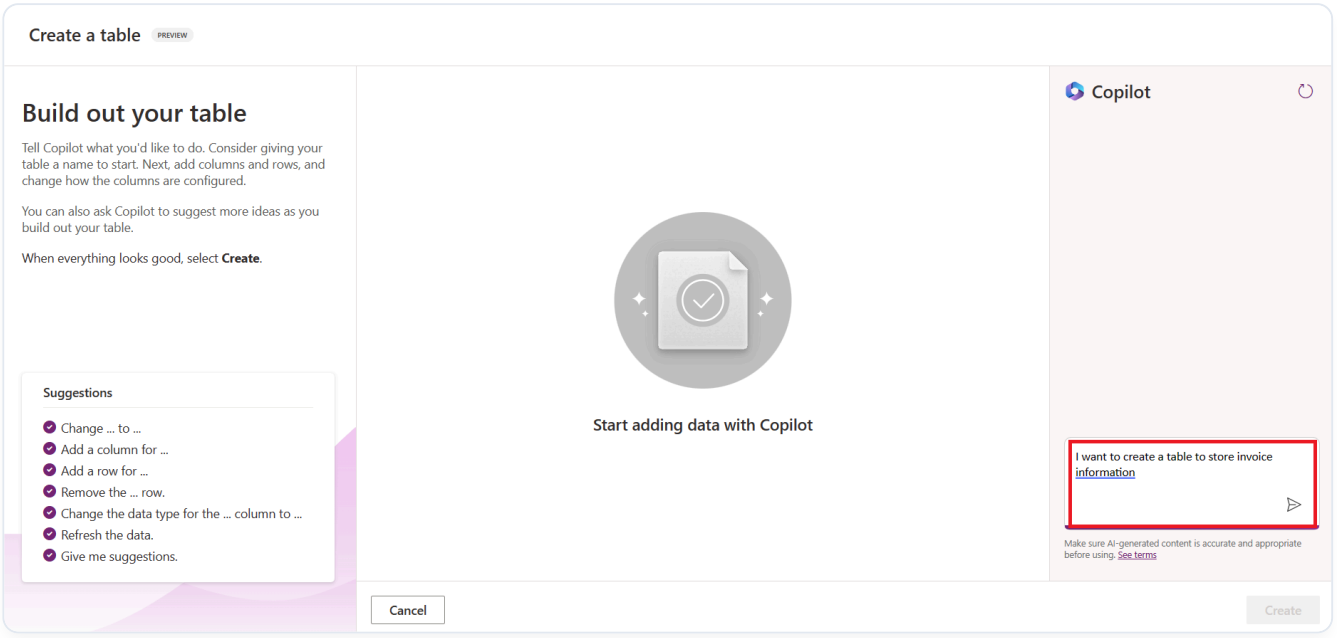
برای ساخت جدول در Dataverse با استفاده از Copilot، مراحل زیر را دنبال کنید:

1. به صفحه اصلی **Power Apps** بروید.

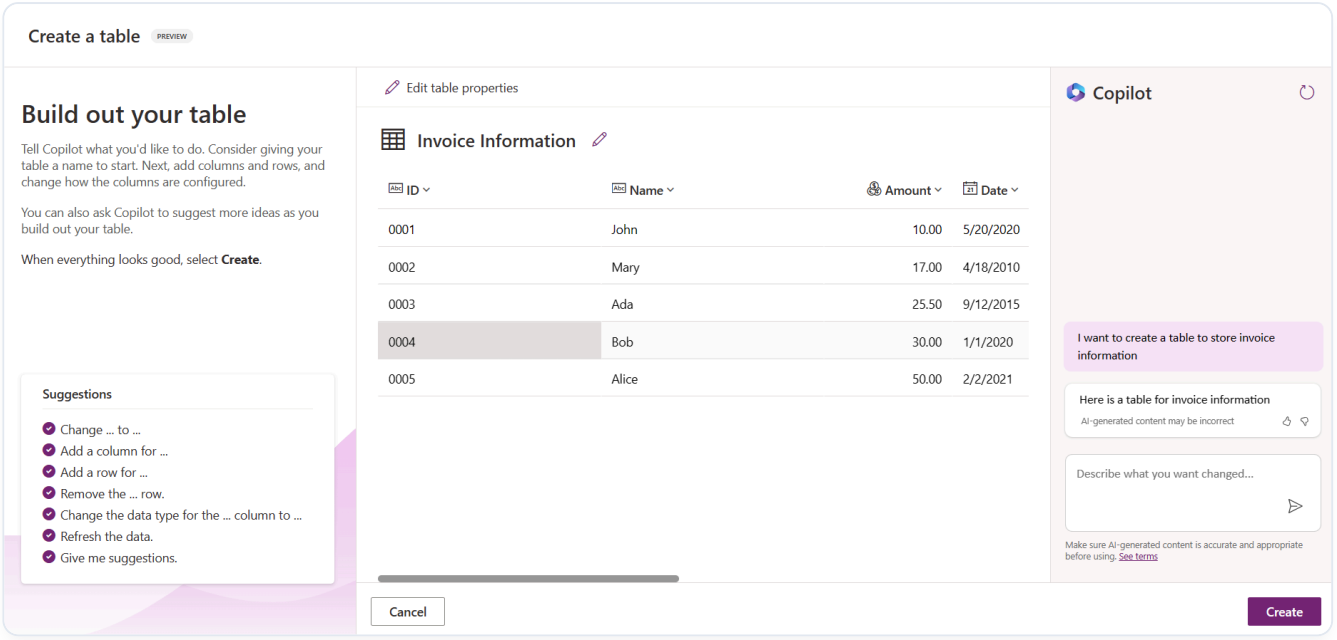
2. در نوار ناوبری سمت چپ روی **Tables** کلیک کرده و سپس روی **Describe the new Table** کلیک کنید.



1. در صفحه **Describe the new Table** در کادر متنی جدول مورد نظر خود را توصیف کنید. به عنوان مثال، ***می‌خواهم جدولی برای ذخیره اطلاعات فاکتورها بسازم***. روی دکمه **Send** کلیک کنید تا درخواست به AI Copilot ارسال شود.



1. AI Copilot یک جدول Dataverse با فیلدهای مورد نیاز برای ذخیره داده‌هایی که می‌خواهید ردیابی کنید همراه با داده‌های نمونه پیشنهاد خواهد داد. سپس می‌توانید جدول را از طریق ویژگی دستیار Copilot با گفتگوهای مرحله به مرحله مطابق نیاز خود شخصی‌سازی کنید.



جدول Dataverse پیشنهادی

1. تیم مالی می‌خواهد ایمیلی به تأمین‌کننده بفرستد تا وضعیت فعلی فاکتور خود را به او اطلاع دهد. می‌توانید از Copilot برای افزودن یک فیلد جدید به جدول برای ذخیره ایمیل تأمین‌کننده استفاده کنید. برای مثال، می‌توانید از دستور زیر برای افزودن یک ستون برای ایمیل تأمین‌کننده استفاده کنید: ***می‌خواهم ستونی برای ذخیره ایمیل تأمین‌کننده اضافه کنم***. روی دکمه **Send** کلیک کنید تا درخواست به AI Copilot ارسال شود.

2. AI Copilot ستون جدیدی ایجاد خواهد کرد و می‌توانید آن ستون را مطابق نیازهای خود شخصی‌سازی کنید.

3. وقتی کارتان با جدول تمام شد، روی دکمه **Create** کلیک کنید تا جدول ساخته شود.

مدل‌های هوش مصنوعی در Power Platform با AI Builder

AI Builder یک قابلیت هوش مصنوعی کم‌کد موجود در Power Platform است که به شما اجازه می‌دهد از مدل‌های هوش مصنوعی برای اتوماسیون فرایندها و پیش‌بینی نتایج استفاده کنید. با AI Builder می‌توانید هوش مصنوعی را به اپ‌ها و فلوهای خود بیاورید که به داده‌های شما در Dataverse یا منابع داده ابری مختلفی مانند SharePoint، OneDrive یا Azure متصل می‌شوند.

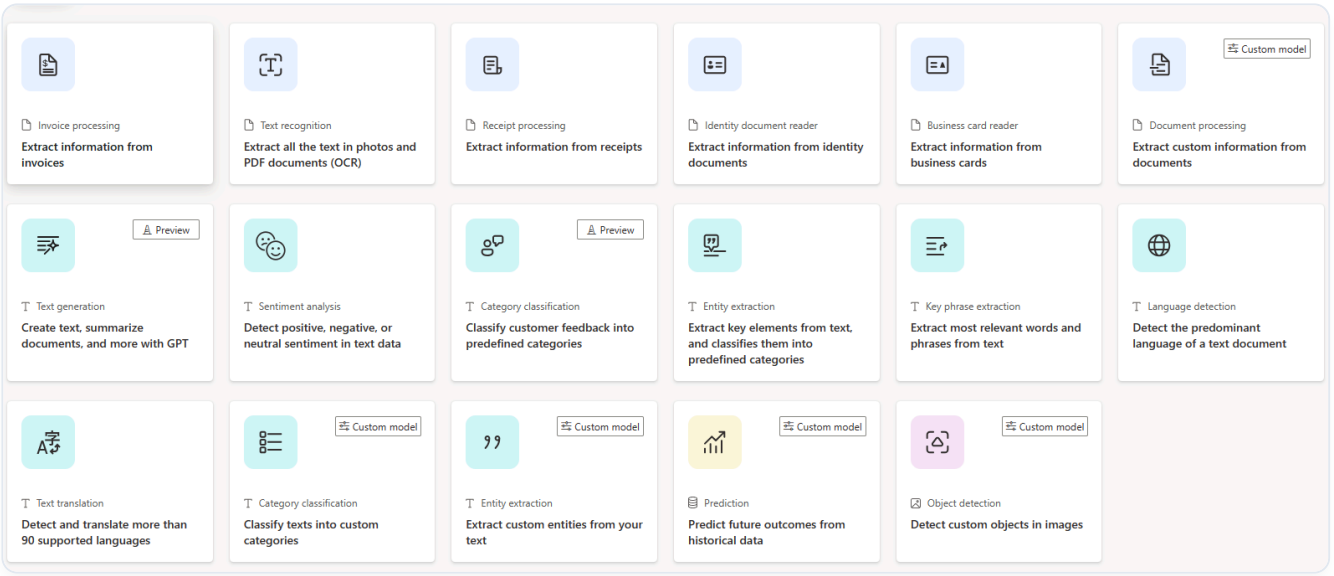
مدل‌های هوش مصنوعی آماده در مقابل مدل‌های سفارشی AI

AI Builder دو نوع مدل هوش مصنوعی ارائه می‌دهد: مدل‌های آماده AI و مدل‌های سفارشی AI. مدل‌های آماده AI مدل‌های آماده به کار هستند که توسط مایکروسافت آموزش داده شده‌اند و در Power Platform در دسترس‌اند. این مدل‌ها به شما کمک می‌کنند تا به سرعت هوشمندی را به اپ‌ها و فلوهای خود اضافه کنید بدون اینکه نیاز باشد داده جمع‌آوری کنید و مدل‌های خود را بسازید، آموزش دهید و منتشر کنید. شما می‌توانید از این مدل‌ها برای اتوماسیون فرایندها و پیش‌بینی نتایج استفاده کنید.

برخی از مدل‌های آماده موجود در Power Platform عبارتند از:

- **استخراج عبارت‌های کلیدی**: این مدل عبارات کلیدی را از متن استخراج می‌کند.
- **تشخیص زبان**: این مدل زبان یک متن را تشخیص می‌دهد.
- **تحلیل احساسات**: این مدل احساس مثبت، منفی، خنثی یا ترکیبی را در متن تشخیص می‌دهد.
- **خواندن کارت ویزیت**: این مدل اطلاعات را از کارت‌های ویزیت استخراج می‌کند.
- **تشخیص متن**: این مدل متن را از تصاویر استخراج می‌کند.
- **تشخیص اشیاء**: این مدل اشیاء را از تصاویر تشخیص داده و استخراج می‌کند.
- **پردازش اسناد**: این مدل اطلاعات را از فرم‌ها استخراج می‌کند.
- **پردازش فاکتور**: این مدل اطلاعات را از فاکتورها استخراج می‌کند.

با مدل‌های سفارشی AI می‌توانید مدل خودتان را به AI Builder بیاورید تا مانند سایر مدل‌های سفارشی کار کند و بتوانید آن مدل را با داده‌های خودتان آموزش دهید. می‌توانید از این مدل‌ها برای اتوماسیون فرایندها و پیش‌بینی نتایج در Power Apps و Power Automate استفاده کنید. هنگام استفاده از مدل خود محدودیت‌هایی وجود دارد که می‌توانید جزئیات آنها را در این محدودیت‌ها بخوانید.



مدل‌های AI Builder

تمرین شماره ۲ - ساخت فلو پردازش فاکتور برای استارتاپ ما

تیم مالی در پردازش فاکتورها مشکل دارد. آنها از صفحه گسترده برای ردیابی فاکتورها استفاده می‌کردند اما با افزایش تعداد فاکتورها مدیریت آن دشوار شده است. از شما خواسته‌اند گردش کاری بسازید که به آنها در پردازش فاکتورها با استفاده از هوش مصنوعی کمک کند. این گردش کاری باید به آنها اجازه دهد اطلاعات را از فاکتورها استخراج کرده و در جدول Dataverse ذخیره کند. همچنین باید به تیم مالی امکان ارسال ایمیل با اطلاعات استخراج شده را بدهد.

حالا که می‌دانید AI Builder چیست و چرا باید از آن استفاده کنید، بیایید ببینیم چگونه می‌توانید از مدل پردازش فاکتور در AI Builder که قبلاً بیان شد، برای ساخت جریانی که به تیم مالی در پردازش فاکتورها کمک کند، استفاده کنید.

برای ساخت یک جریان کاری که به تیم مالی در پردازش فاکتورها با مدل پردازش فاکتور در AI Builder کمک کند، مراحل زیر را دنبال کنید:

1. به صفحه اصلی Power Automate بروید.

2. در کادر متنی صفحه اصلی، گردش کاری که می‌خواهید بسازید را توصیف کنید. به عنوان مثال، ***وقتی فاکتور به صندوق ورودی من رسید آن را پردازش کن***. روی دکمه **Send** کلیک کنید تا درخواست به AI Copilot ارسال شود.

Copilot power automate

3. AI Copilot اقداماتی را که برای انجام کاری که می‌خواهید خودکارسازی کنید لازم است پیشنهاد خواهد داد. می‌توانید روی دکمه **Next** کلیک کنید تا مراحل بعدی را طی کنید.

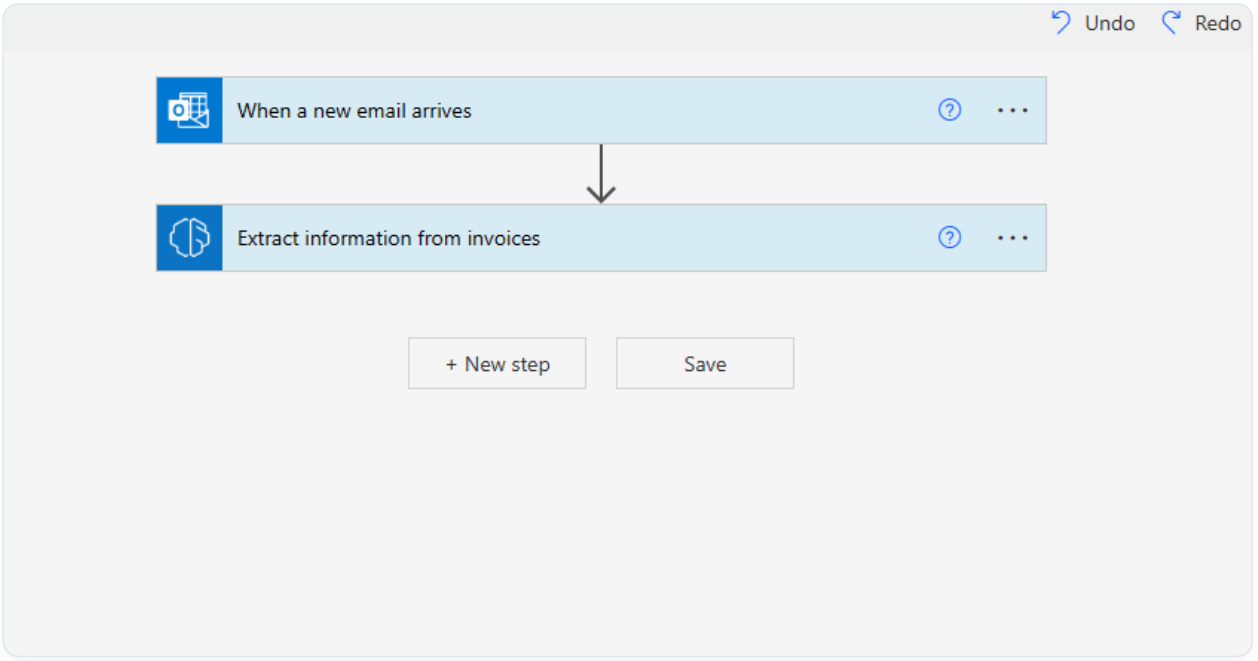
4. در مرحله بعد، Power Automate شما را برای تنظیم اتصالات لازم برای جریان راهنمایی می‌کند. پس از اتمام، روی دکمه **Create flow** کلیک کنید تا جریان ساخته شود.

5. AI Copilot یک جریان تولید خواهد کرد و سپس می‌توانید آن را مطابق نیازهای خود شخصی‌سازی کنید.

6. محرک (trigger) جریان را به‌روزرسانی کنید و **Folder** را به پوشه‌ای که فاکتورها در آن ذخیره می‌شوند تنظیم کنید. به عنوان مثال، می‌توانید پوشه را روی **Inbox** تنظیم کنید. روی **Show advanced options** کلیک کنید و **Only with Attachments** را روی **Yes** بگذارید. این باعث می‌شود جریان فقط وقتی اجرا شود که ایمیل با پیوست در آن پوشه دریافت شود.

7. عملیات‌های زیر را از جریان حذف کنید: **Compose 3**، **Compose 2**، **Compose** و **HTML to text** چون شما از آنها استفاده نخواهید کرد.

8. عمل **Condition** را از جریان حذف کنید چون استفاده نخواهید کرد. جریان باید شبیه به تصویر زیر باشد:

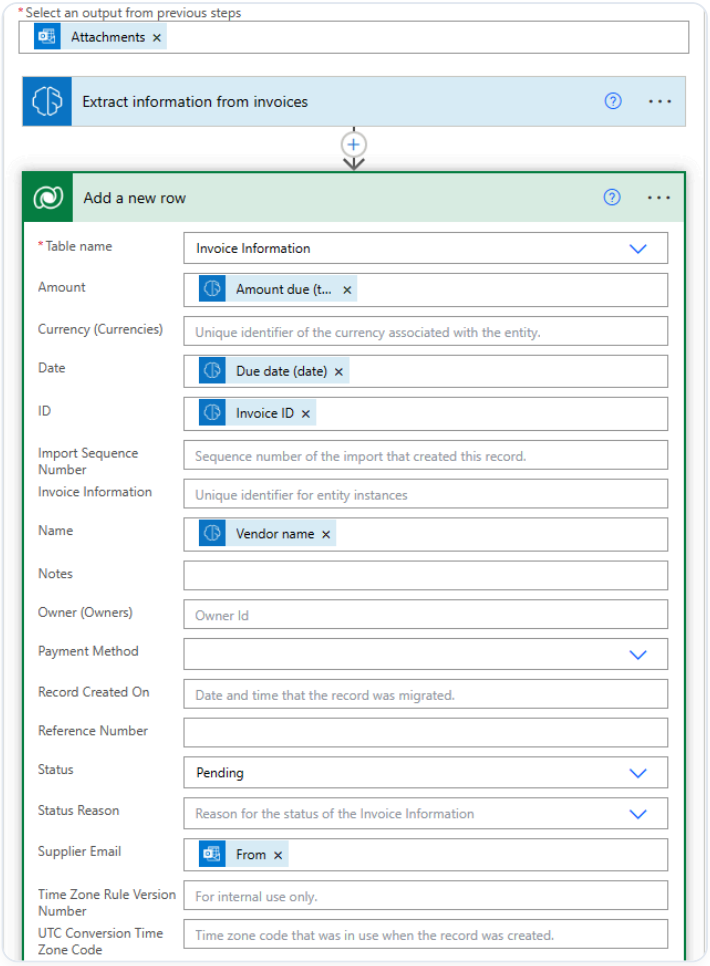


9. روی دکمه **Add an action** کلیک کنید و **Dataverse** را جستجو کنید. سپس عمل **Add a new row** را انتخاب کنید.

10. در عمل **Extract Information from invoices**، مقدار **Invoice File** را به **Attachment Content** ایمیل تنظیم کنید. این اطمینان می‌دهد جریان اطلاعات را از پیوست فاکتور استخراج کند.

11. جدولی را که قبلاً ساختید انتخاب کنید. به عنوان مثال، می‌توانید جدول **Invoice Information** را انتخاب کنید. محتوای پویا از عمل قبلی را برای پر کردن فیلدهای زیر استفاده کنید:

   - ID
   - Amount (مبلغ)
   - Date (تاریخ)
   - Name (نام)
   - Status - مقدار **Status** را روی **Pending** قرار دهید.
   - Supplier Email - از محتوای پویا **From** در محرک **When a new email arrives** استفاده کنید.



12. وقتی کارتان با جریان تمام شد، روی دکمه **Save** کلیک کنید تا جریان ذخیره شود. سپس می‌توانید جریان را با ارسال ایمیل حاوی فاکتور به پوشه‌ای که در محرک مشخص کردید آزمایش کنید.

> 💡 **نکته کاربردی**
>
> **تمرین شما**: جریانی که همین الان ساختید شروع خوبی است، حالا باید فکر کنید چگونه اتوماسیونی بسازید که به تیم مالی ما اجازه دهد ایمیلی به تأمین‌کننده بفرستد تا وضعیت فعلی فاکتور خود را به روز کنند. راهنمای شما: جریان باید وقتی وضعیت فاکتور تغییر می‌کند اجرا شود.

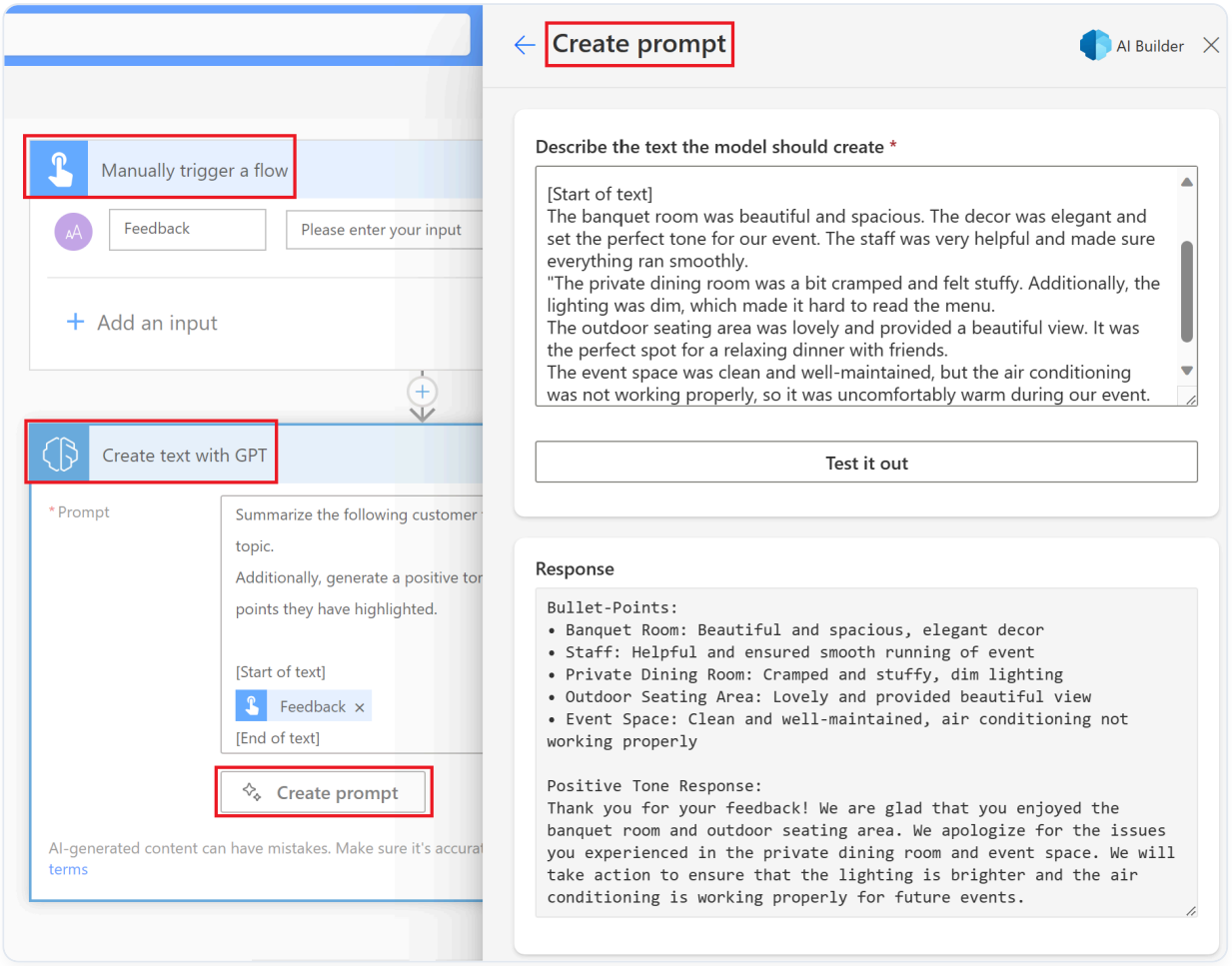
استفاده از مدل هوش مصنوعی تولید متن در Power Automate

مدل Create Text with GPT در AI Builder به شما امکان می‌دهد متن را بر اساس یک پرامپت تولید کنید و توسط سرویس Microsoft Azure OpenAI پشتیبانی می‌شود. با این قابلیت، می‌توانید فناوری GPT (شبکه تبدیل‌کننده از پیش آموزش‌دیده مولد) را وارد اپ‌ها و فلوهای خود کنید تا انواع جریان‌های خودکار و برنامه‌های کاربردی هوشمند بسازید.

مدل‌های GPT آموزش‌های گسترده‌ای روی حجم زیادی از داده‌ها می‌بینند تا متنی بسیار شبیه زبان انسانی تولید کنند وقتی یک پرامپت به آنها داده می‌شود. وقتی با اتوماسیون گردش کار ترکیب شود، مدل‌های هوش مصنوعی مانند GPT می‌توانند برای

ساده‌سازی و خودکارسازی بسیاری از وظایف استفاده شوند.

برای مثال، می‌توانید جریان‌هایی بسازید که به طور خودکار متن را برای موارد مختلفی تولید کنند، مانند پیش‌نویس ایمیل‌ها، توضیحات محصولات و غیره. همچنین می‌توانید از مدل برای تولید متن در انواع اپ‌ها استفاده کنید، مانند چت‌بات‌ها و اپ‌های خدمات مشتری که به عوامل خدمات مشتری کمک می‌کنند به پرسش‌های مشتریان به شکل مؤثر و کارآمد پاسخ دهند.



برای یادگیری نحوه استفاده از این مدل هوش مصنوعی در Power Automate، ماژول **افزودن هوش با AI Builder و GPT** را مرور کنید.

کار عالی! به آموزش خود ادامه دهید

پس از تکمیل این درس، مجموعه **یادگیری هوش مصنوعی مولد** ما را بررسی کنید تا دانش هوش مصنوعی مولد خود را بیشتر ارتقا دهید!

می‌خواهید سفارشی‌سازی کنید و بیشتر از Copilot بهره ببرید؟ مجموعه **Awesome Copilot** – مجموعه‌ای که توسط جامعه ساخته شده، شامل دستورالعمل‌ها، عوامل، مهارت‌ها و پیکربندی‌هایی است تا به شما در استفاده بهتر از GitHub Copilot کمک کند.

به درس ۱۱ مراجعه کنید که در آن به نحوه **یکپارچه‌سازی هوش مصنوعی مولد با فراخوانی تابع** می‌پردازیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بخش تکمیلی / راهنمای همراه

بازگشت به فهرست مطالب

درس ۱۰ — هوش مصنوعی مولد مایکروسافت

درس ۱۱ از ۲۱

یکپارچه‌سازی سیستم‌ها با فراخوانی توابع (Function Calling)

اتصال مدل‌های هوش مصنوعی به پایگاه‌های داده و APIهای خارجی از طریق ابزارسازی و تعریف ساختار شمای توابع (JSON Schema)

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/DgUdCLX8qYQ?si=f1ouQU5HQx6F8Gl2

پخش در یوتیوب

اسکن کد QR

تا اینجا در درس‌های قبلی مقدار قابل توجهی یاد گرفته‌اید. با این حال، می‌توانیم بیش از این پیشرفت کنیم. برخی مواردی که می‌توانیم به آن‌ها بپردازیم این است که چگونه می‌توانیم قالب پاسخ یکنواخت‌تری داشته باشیم تا کار با پاسخ در مراحل بعدی آسان‌تر شود. همچنین ممکن است بخواهیم داده‌هایی از منابع دیگر اضافه کنیم تا برنامه‌مان غنی‌تر شود.

مسائلی که در بالا ذکر شدند، موضوع این فصل هستند.

معرفی

این درس پوشش خواهد داد:

- توضیح اینکه فراخوانی تابع چیست و کاربردهای آن.

- ایجاد یک فراخوانی تابع با استفاده از Azure OpenAI.
- نحوه یکپارچه‌سازی فراخوانی تابع در یک برنامه.

اهداف یادگیری

تا پایان این درس، قادر خواهید بود:

- هدف استفاده از فراخوانی تابع را توضیح دهید.
- راه‌اندازی فراخوانی تابع با استفاده از سرویس Azure OpenAI.
- طراحی فراخوانی‌های مؤثر تابع برای کاربرد برنامه خود.

سناریو: بهبود چت‌بات ما با توابع

برای این درس، می‌خواهیم ویژگی‌ای برای استارتاپ آموزشی خود بسازیم که به کاربران اجازه دهد با استفاده از چت‌بات دوره‌های فنی را پیدا کنند. ما دوره‌هایی را پیشنهاد خواهیم داد که متناسب با سطح مهارت، نقش فعلی و فناوری مورد علاقه آن‌ها هستند.

برای تکمیل این سناریو، از ترکیبی از موارد زیر استفاده خواهیم کرد:

- `Azure OpenAI` برای ایجاد تجربه چت برای کاربر.
- `Microsoft Learn Catalog API` برای کمک به کاربران در یافتن دوره‌ها بر اساس درخواست‌شان.
- `فراخوانی تابع` برای دریافت پرسش کاربر و ارسال آن به تابع برای انجام درخواست API.

برای شروع، بیایید ببینیم چرا اصلاً بخواهیم از فراخوانی تابع استفاده کنیم:

چرا فراخوانی تابع

پیش از فراخوانی تابع، پاسخ‌های یک LLM ساختارمند و هماهنگ نبودند. توسعه‌دهندگان مجبور بودند کدهای پیچیده‌ی اعتبارسنجی بنویسند تا مطمئن شوند می‌توانند هر گونه تغییر در پاسخ را کنترل کنند. کاربران نمی‌توانستند پاسخ‌هایی مانند «وضعیت فعلی آب و هوا در استکهلم چیست؟» دریافت کنند. علت این بود که مدل‌ها محدود به زمانی بودند که داده‌ها آموزش دیده بودند.

فراخوانی تابع ویژگی‌ای از سرویس Azure OpenAI است تا محدودیت‌های زیر را برطرف کند:

- **قالب پاسخ یکنواخت**. اگر بتوانیم قالب پاسخ را بهتر کنترل کنیم، می‌توانیم پاسخ را به راحتی به سیستم‌های دیگر پیوند بدهیم.
- **داده‌های خارجی**. قابلیت استفاده از داده‌ها از منابع دیگر برنامه در زمینه چت.

شرح مشکل از طریق یک سناریو

> **🚀 سناریوی استارتاپ ما**
>
> توصیه می‌کنیم از **نوت‌بوک همراه** استفاده کنید اگر می‌خواهید سناریوی زیر را اجرا کنید. همچنین می‌توانید صرفاً همراه با ما بخوانید تا مشکلی را که توابع می‌توانند در حل آن کمک کنند، شرح دهیم.

بیایید به مثالی نگاه کنیم که مشکل قالب پاسخ را نشان می‌دهد:

فرض کنیم می‌خواهیم پایگاه داده‌ای از اطلاعات دانش‌آموزان بسازیم تا دوره مناسب را به آن‌ها پیشنهاد کنیم. در ادامه دو توصیف از دانش‌آموزان داریم که داده‌های مشابهی دارند.

1. اتصال به منبع Azure OpenAI خود را ایجاد کنید:

Python

```
import os
import json
from openai import OpenAI
from dotenv import load_dotenv
load_dotenv()

# API نسخه ۱ ارائه می‌شود (Microsoft Foundry) Azure OpenAI پاسخ‌ها از طریق نقطه پایانی
# اشاره می‌دهیم. <your-endpoint>/openai/v1/ را به OpenAI بنابراین ما کلاینت
endpoint = os.environ['AZURE_OPENAI_ENDPOINT']
client = OpenAI(
api_key=os.environ['AZURE_OPENAI_API_KEY'],
base_url=f"{endpoint.rstrip('/')}/openai/v1/",
)

deployment=os.environ['AZURE_OPENAI_DEPLOYMENT']
```

در زیر کد پایتون برای پیکربندی اتصال ما به Azure OpenAI است. چون از نقطه انتهایی v1 استفاده می‌کنیم، فقط لازم است `api_key` و `base_url` را تنظیم کنیم (نیازی به `api_version` نیست).

2. ایجاد دو توصیف دانش‌آموز با استفاده از متغیرهای `student_1_description` و `student_2_description`.

Python

```
student_1_description="Emily Johnson is a sophomore majoring in computer science at Duke
University. She has a 3.7 GPA. Emily is an active member of the university's Chess Club
and Debate Team. She hopes to pursue a career in software engineering after graduating."

student_2_description = "Michael Lee is a sophomore majoring in computer science at
Stanford University. He has a 3.8 GPA. Michael is known for his programming skills and
is an active member of the university's Robotics Club. He hopes to pursue a career in
artificial intelligence after finishing his studies."
```

می‌خواهیم توصیف‌های دانش‌آموز بالا را به یک LLM ارسال کنیم تا داده‌ها را تجزیه کند. این داده‌ها بعداً می‌توانند در برنامه ما استفاده شده و به API ارسال یا در یک پایگاه داده ذخیره شوند.

3. دو درخواست متنی مشابه ایجاد کنیم که به LLM دستور می‌دهد چه اطلاعاتی را استخراج کند:

Python

```
prompt1 = f'''
Please extract the following information from the given text and return it as a JSON
object:

name
major
school
grades
club

This is the body of text to extract the information from:
{student_1_description}
'''

prompt2 = f'''
Please extract the following information from the given text and return it as a JSON
object:

name
major
school
grades
club

This is the body of text to extract the information from:
{student_2_description}
'''
```

درخواست‌های بالا به LLM می‌گویند اطلاعات را استخراج کرده و پاسخ را در قالب JSON بازگرداند.

4. پس از تنظیم درخواست‌ها و اتصال به Azure OpenAI، حالا درخواست‌ها را با استفاده از `client.responses.create` به LLM می‌فرستیم. درخواست‌ها در متغیر `input` قرار می‌گیرند و نقش `user` تعیین می‌شود. این برای شبیه‌سازی پیام نوشته شده توسط کاربر در چت‌بات است.

```python
# پاسخ از درخواست اول
openai_response1 = client.responses.create(
model=deployment,
input = [{'role': 'user', 'content': prompt1}],
store=False,
)
openai_response1.output_text

# پاسخ از درخواست دوم
openai_response2 = client.responses.create(
model=deployment,
input = [{'role': 'user', 'content': prompt2}],
store=False,
)
openai_response2.output_text
```

اکنون می‌توانیم هر دو درخواست را به LLM ارسال کنیم و پاسخ دریافتی را با استفاده از نمونه‌ای مانند `openai_response1.output_text` بررسی کنیم.

1. در نهایت پاسخ را به قالب JSON تبدیل می‌کنیم با فراخوانی `json.loads` :

```python
# بارگذاری پاسخ به عنوان یک شیء JSON
json_response1 = json.loads(openai_response1.output_text)
json_response1
```

پاسخ 1:

```json
{
  "name": "Emily Johnson",
  "major": "computer science",
  "school": "Duke University",
  "grades": "3.7",
  "club": "Chess Club"
}
```

پاسخ 2:

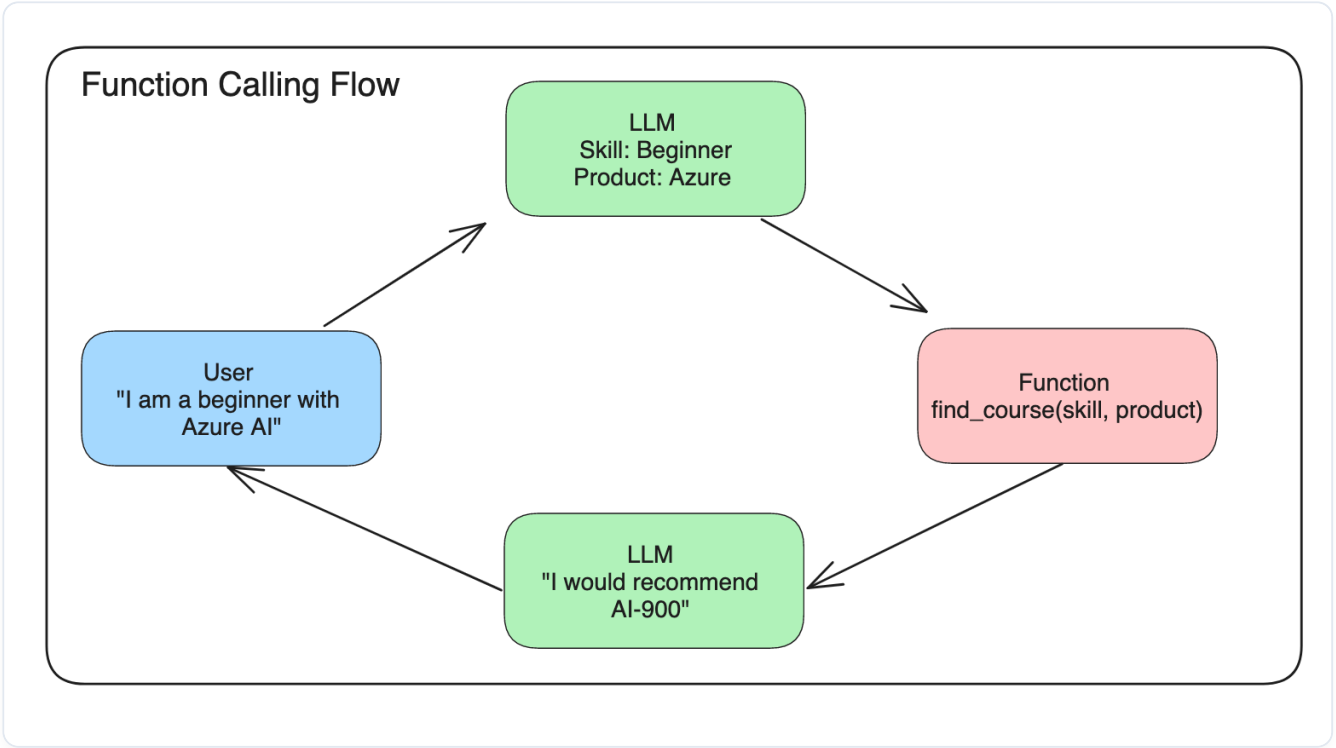
JSON

```
{
  "name": "Michael Lee",
  "major": "computer science",
  "school": "Stanford University",
  "grades": "3.8 GPA",
  "club": "Robotics Club"
}
```

اگرچه درخواست‌ها یکسان بودند و توصیف‌ها مشابه، مقادیر ویژگی `Grades` با فرمت‌های متفاوتی ارائه شدند، مثلاً گاهی فرمت `3.7` و گاهی `3.7 GPA` دریافت کردیم.

این نتیجه به خاطر این است که LLM داده‌های غیرساختارمند را بر اساس درخواست متنی می‌گیرد و باز هم داده‌های غیرساختارمند بازمی‌گرداند. ما به قالبی ساختارمند نیاز داریم تا بدانیم هنگام ذخیره یا استفاده از این داده‌ها چه انتظاری داشته باشیم.

پس چگونه مشکل قالب‌بندی را حل کنیم؟ با استفاده از فراخوانی تابع، می‌توانیم مطمئن شویم که داده‌های ساختارمند دریافت می‌کنیم. هنگام استفاده از فراخوانی تابع، LLM در واقع هیچ تابعی را اجرا نمی‌کند. به جای آن، ساختاری ایجاد می‌کنیم که LLM طبق آن پاسخ‌های خود را ارائه دهد. سپس از پاسخ‌های ساختارمند استفاده می‌کنیم تا بدانیم در برنامه‌های خود کدام تابع را اجرا کنیم.



سپس می‌توانیم آنچه از تابع برگشت داده شده است را گرفته و دوباره به LLM ارسال کنیم. LLM سپس به زبان طبیعی پاسخ می‌دهد تا پرسش کاربر را پاسخگو باشد.

کاربردهای استفاده از فراخوانی تابع

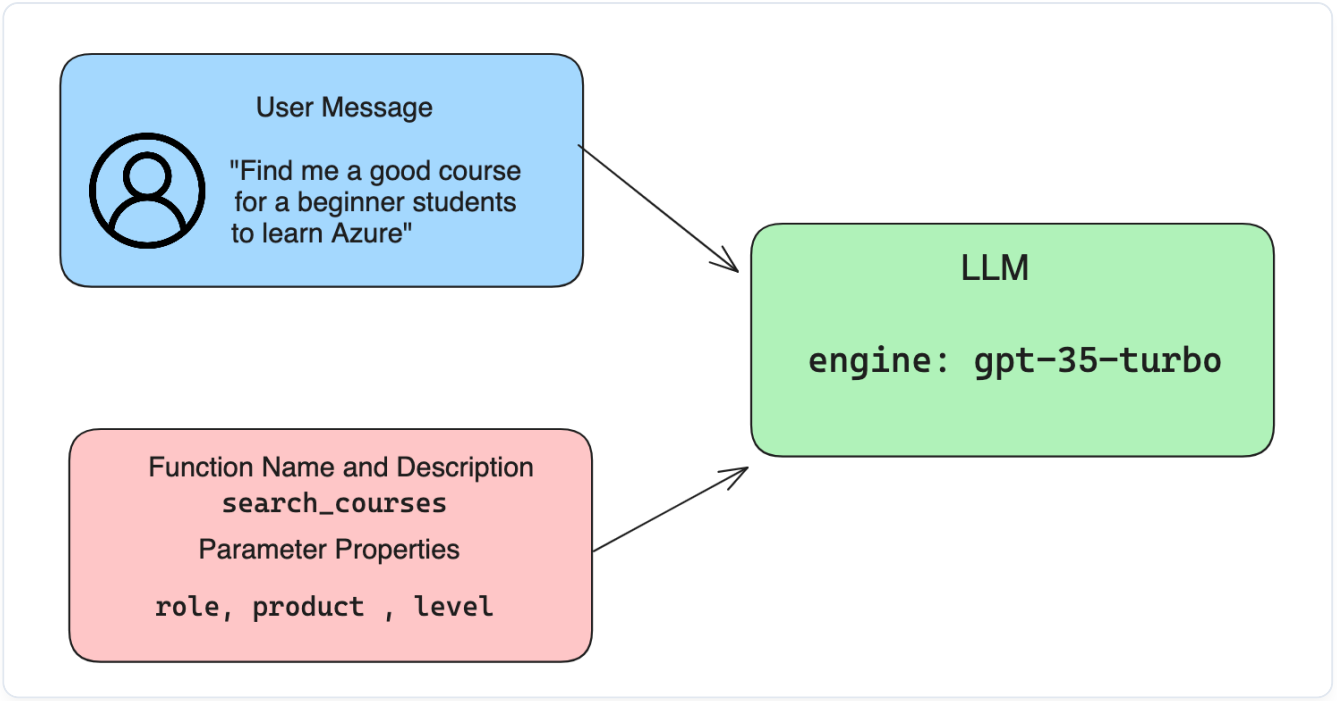
کاربردهای متعدد وجود دارد که فراخوانی تابع می‌تواند برنامه شما را بهبود بخشد، مانند:

- **فراخوانی ابزارهای خارجی**. چت‌بات‌ها در ارائه پاسخ به سوالات کاربران عالی هستند. با فراخوانی تابع، چت‌بات‌ها می‌توانند پیام‌های کاربران را برای انجام برخی کارها استفاده کنند. مثلاً دانش‌آموز می‌تواند از چت‌بات بخواهد که «برای استاد ایمیل بفرست که به کمک بیشتری در این موضوع نیاز دارم». این می‌تواند تابعی با نام `send_email(to: string, body: string)` را فراخوانی کند.
- **ایجاد پرس‌وجوهای API یا پایگاه داده**. کاربران می‌توانند اطلاعات را با زبان طبیعی پیدا کنند که به پرس‌وجو یا درخواست API قالب‌بندی شده تبدیل می‌شود. مثالی از این می‌تواند معلمی باشد که می‌پرسد «دانش‌آموزانی که آخرین تکلیف را انجام داده‌اند چه کسانی هستند؟» که می‌تواند تابعی به نام `get_completed(student_name: string, assignment: int, current_status: string)` را فراخوانی کند.
- **ایجاد داده‌های ساختارمند**. کاربران می‌توانند یک بلوک متن یا CSV گرفته و با استفاده از LLM اطلاعات مهم را استخراج کنند. به عنوان مثال، دانش‌آموزی می‌تواند یک مقاله ویکی‌پدیا درباره توافقات صلح را به فلش‌کارت‌های هوش مصنوعی تبدیل کند. این می‌تواند با استفاده از تابعی به نام `get_important_facts(agreement_name: string, date_signed: string, parties_involved: list)` انجام شود.

ایجاد اولین فراخوانی تابع خود

فرآیند ایجاد فراخوانی تابع شامل ۳ مرحله اصلی است:

1. **فراخوانی** API پاسخ‌ها با فهرستی از توابع (ابزارها) و پیام کاربر.
2. **خواندن** پاسخ مدل برای انجام کاری مثل اجرای تابع یا فراخوانی API.
3. **ایجاد** فراخوانی دیگری به API پاسخ‌ها با پاسخ تابع شما برای استفاده از آن اطلاعات در پاسخ به کاربر.



LLM Flow

مرحله 1 - ایجاد پیام‌ها

اولین گام ایجاد پیام کاربر است. این می‌تواند به صورت پویا از ورودی متن گرفته شود یا می‌توانید مقداری اینجا تنظیم کنید. اگر برای اولین بار با API پاسخ‌ها کار می‌کنید، باید `role` و `content` پیام را تعریف کنید.

`role` می‌تواند `system` (ایجاد قوانین)، `assistant` (مدل) یا `user` (کاربر نهایی) باشد. برای فراخوانی تابع، ما آن را روی `user` و یک سوال نمونه قرار خواهیم داد.

```python
messages= [ {"role": "user", "content": "Find me a good course for a beginner student to learn Azure."} ]
```

تخصیص نقش‌های مختلف به مدل کمک می‌کند که متوجه شود صحبت‌کننده سیستم است یا کاربر، که به ساخت تاریخچه مکالمه کمک می‌کند تا مدل بتواند بر اساس آن پیش برود.

مرحله 2 - ایجاد توابع

سپس، تابعی و پارامترهای آن را تعریف خواهیم کرد. در اینجا فقط یک تابع به نام `search_courses` استفاده می‌کنیم، اما می‌توانید توابع متعددی ایجاد کنید.

> **نکته کاربردی** 💡
>
> **مهم** : توابع در پیام سیستم به LLM اضافه می‌شوند و در میزان توکن‌های موجود شما محاسبه می‌شوند.

در ادامه توابع را به صورت یک آرایه از آیتم‌ها تعریف می‌کنیم. هر آیتم ابزاری در قالب API پاسخ‌ها با ویژگی‌های `type`، `name`، `description` و `parameters` است:

Python

```
functions = [
    {
        "type":"function",
        "name":"search_courses",
        "description":"Retrieves courses from the search index based on the parameters
provided",
        "parameters":{
            "type":"object",
            "properties":{
                "role":{
                    "type":"string",
                    "description":"The role of the learner (i.e. developer, data scientist,
student, etc.)"
                },
                "product":{
                    "type":"string",
                    "description":"The product that the lesson is covering (i.e. Azure, Power BI,
etc.)"
                },
                "level":{
                    "type":"string",
                    "description":"The level of experience the learner has prior to taking the
course (i.e. beginner, intermediate, advanced)"
                }
            },
            "required":[
                "role"
            ]
        }
    }
]
```

بیایید هر نمونه تابع را به تفصیل بیشتر شرح دهیم:

- `name` - نام تابعی که می‌خواهیم فراخوانی شود.
- `description` - توضیح عملکرد تابع. مهم است که مشخص و واضح باشد.
- `parameters` - فهرستی از مقادیر و فرمت‌هایی که می‌خواهید مدل در پاسخ ایجاد کند. آرایه پارامترها شامل آیتم‌هایی است که ویژگی‌های زیر را دارند:
  1. `type` - نوع داده‌ای که ویژگی‌ها در آن ذخیره می‌شوند.

2. `properties` - فهرست مقادیر خاصی که مدل برای پاسخ خود استفاده خواهد کرد.
    1. `name` - کلیدی که نام ویژگی‌ای است که مدل در پاسخ قالب‌بندی شده استفاده می‌کند، مثلا `product`.
    2. `type` - نوع داده این ویژگی، مثلا `string`.
    3. `description` - توضیح ویژگی خاص.

همچنین ویژگی اختیاری `required` وجود دارد - که مشخص می‌کند کدام ویژگی‌ها برای تکمیل فراخوانی تابع ضروری هستند.

مرحله 3 - انجام فراخوانی تابع

پس از تعریف تابع، باید آن را در فراخوانی API پاسخ‌ها بگنجانیم. این کار با اضافه کردن `tools` به درخواست انجام می‌شود. در این مورد `tools=functions`.

گزینه‌ای هم وجود دارد که `tool_choice` را روی `auto` تنظیم کنید. یعنی اجازه می‌دهیم LLM تصمیم بگیرد کدام تابع باید بر اساس پیام کاربر فراخوانی شود بدون اینکه خودمان تعیین کنیم.

در زیر کدی داریم که `client.responses.create` را فراخوانی می‌کند، توجه کنید که چگونه `tools=functions` و `tool_choice="auto"` تنظیم شده‌اند و بدین ترتیب انتخاب اینکه کی تابع‌ها را فراخوانی کند به LLM داده شده است:

Python

```
response = client.responses.create(model=deployment,
                                   input=messages,
                                   tools=functions,
                                   tool_choice="auto",
                                   store=False)

print(response.output)
```

پاسخ بازگشتی اکنون در `response.output` شامل آیتم `function_call` است که به این صورت است:

JSON

```
{
  "type": "function_call",
  "name": "search_courses",
  "call_id": "call_abc123",
  "arguments": "{\n  \"role\": \"student\",\n  \"product\": \"Azure\",\n  \"level\":
\"beginner\"\n}"
}
```

در اینجا می‌بینیم تابع `search_courses` فراخوانی شده و با چه آرگومان‌هایی، همانطور که در ویژگی `arguments` در پاسخ JSON آمده است.

نتیجه این است که LLM توانست داده‌ها را استخراج کرده و آن‌ها را به آرگومان‌های تابع مپ کند، زیرا داده را از مقدار پارامتر `input` در فراخوانی API پاسخ‌ها استخراج کرده بود. در ادامه یادآوری مقدار `messages` آمده است:

```python
messages= [ {"role": "user", "content": "Find me a good course for a beginner student to learn Azure."} ]
```

همانطور که می‌بینید، `student`، `Azure` و `beginner` از `messages` استخراج شده و به عنوان ورودی به تابع داده شده‌اند. استفاده از توابع به این شکل روش خوبی برای استخراج اطلاعات از یک درخواست متنی و همچنین فراهم کردن ساختار برای LLM و داشتن قابلیت استفاده مجدد است.

حالا باید ببینیم چگونه می‌توانیم این را در برنامه خود استفاده کنیم.

یکپارچه‌سازی فراخوانی توابع در یک برنامه

پس از آزمایش پاسخ قالب‌بندی شده از LLM، اکنون می‌توانیم آن را در یک برنامه یکپارچه کنیم.

مدیریت جریان

برای یکپارچه‌سازی در برنامه، مراحل زیر را دنبال می‌کنیم:

1. ابتدا، به سرویس‌های OpenAI فراخوانی می‌کنیم و آیتم‌های فراخوانی تابع را از پاسخ `output` استخراج می‌کنیم.

```python
response_items = response.output
tool_calls = [item for item in response_items if item.type == "function_call"]
```

2. حالا تابعی تعریف می‌کنیم که API مایکروسافت لرن را فراخوانی می‌کند تا فهرستی از دوره‌ها را دریافت کند:

Python

```
import requests

def search_courses(role, product, level):
  url = "https://learn.microsoft.com/api/catalog/"
  params = {
     "role": role,
     "product": product,
     "level": level
  }
  response = requests.get(url, params=params)
  modules = response.json()["modules"]
  results = []
  for module in modules[:5]:
     title = module["title"]
     url = module["url"]
     results.append({"title": title, "url": url})
  return str(results)
```

دقت کنید که اکنون یک تابع واقعی پایتون ایجاد می‌کنیم که مطابق نام‌های توابع معرف شده در متغیر `functions` است. همچنین فراخوانی‌های واقعی API خارجی می‌زنیم تا داده‌های لازم را دریافت کنیم. در این مورد، به سراغ API مایکروسافت لرن می‌رویم تا ماژول‌های آموزشی را جستجو کنیم.

خب، ما متغیر `functions` و تابع پایتون متناظر را ساختیم، حال چگونه به LLM بگوییم که این دو را متصل کند تا تابع پایتون ما فراخوانی شود؟

1. برای بررسی اینکه آیا باید تابع پایتون فراخوانی شود، باید پاسخ LLM را نگاه کنیم و ببینیم آیا آیتم `function_call` در آن هست و تابع مشخص شده را فراخوانی کنیم. در زیر روش چک کردن را می‌بینید:

Python

```
# بررسی کنید که آیا مدل می‌خواهد تابعی را فراخوانی کند
if tool_calls:
 for tool_call in tool_calls:
  print("Recommended Function call:")
  print(tool_call.name)
  print()

  # تابع را فراخوانی کنید.
  function_name = tool_call.name

  available_functions = {
          "search_courses": search_courses,
  }
  function_to_call = available_functions[function_name]

  function_args = json.loads(tool_call.arguments)
  function_response = function_to_call(**function_args)

  print("Output of function call:")
  print(function_response)
  print(type(function_response))

  # تماس با تابع و نتیجه آن را دوباره به مکالمه اضافه کنید.
  # مدل باید پیش از خروجی آن اضافه شود function_call دستیار مورد
  messages.append(tool_call)  # مورد function_call
  messages.append( # نتیجه تابع
      {
          "type": "function_call_output",
          "call_id": tool_call.call_id,
          "output": function_response,
      }
  )
```

این سه خط اطمینان می‌دهند که نام تابع، آرگومان‌ها استخراج شده و فراخوانی انجام شود:

Python

```
function_to_call = available_functions[function_name]

function_args = json.loads(tool_call.arguments)
function_response = function_to_call(**function_args)
```

در ادامه خروجی اجرای کد ما آمده است:

خروجی

RECOMMENDED

```
{
  "name": "search_courses",
  "arguments": "{\n  \"role\": \"student\",\n  \"product\": \"Azure\",\n  \"level\":
\"beginner\"\n}"
}

Output of function call:
[{'title': 'Describe concepts of cryptography', 'url':
'https://learn.microsoft.com/training/modules/describe-concepts-of-cryptography/?
WT.mc_id=api_CatalogApi'}, {'title': 'Introduction to audio classification with
TensorFlow', 'url': 'https://learn.microsoft.com/en-
us/training/modules/intro-audio-classification-tensorflow/'}, {'title': 'Design a
Performant Data Model in Azure SQL
Database with Azure Data Studio', 'url':
'https://learn.microsoft.com/training/modules/design-a-data-model-with-ads/?
WT.mc_id=api_CatalogApi'}, {'title': 'Getting started with the Microsoft Cloud Adoption
Framework for Azure', 'url':
'https://learn.microsoft.com/training/modules/cloud-adoption-framework-getting-
started/'}, {'title': 'Set up the
Rust development environment', 'url':
'https://learn.microsoft.com/training/modules/rust-set-up-environment/'}]
<class 'str'>
```

2. اکنون پیام به‌روزشده `messages` را به LLM می‌فرستیم تا پاسخ به زبان طبیعی به جای پاسخ JSON قالب‌بندی شده API دریافت کنیم.

Python

```
print("Messages in next request:")
print(messages)
print()

second_response = client.responses.create(
   input=messages,
   model=deployment,
   tool_choice="auto",
   tools=functions,
   temperature=0,
   store=False,
      )  # دریافت پاسخ جدید از مدل که می‌تواند پاسخ تابع را ببیند


print(second_response.output_text)
```

خروجی

متن / Text

```
I found some good courses for beginner students to learn Azure:

1. [Describe concepts of cryptography]
(https://learn.microsoft.com/training/modules/describe-concepts-of-cryptography/)
2. [Introduction to audio classification with TensorFlow]
(https://learn.microsoft.com/training/modules/intro-audio-classification-tensorflow/)
3. [Design a Performant Data Model in Azure SQL Database with Azure Data Studio]
(https://learn.microsoft.com/training/modules/design-a-data-model-with-ads/)
4. [Getting started with the Microsoft Cloud Adoption Framework for Azure]
(https://learn.microsoft.com/training/modules/cloud-adoption-framework-getting-started/)
5. [Set up the Rust development environment]
(https://learn.microsoft.com/training/modules/rust-set-up-environment/)

You can click on the links to access the courses.
```

تمرین

برای ادامه یادگیری فراخوانی تابع در Azure OpenAI می‌توانید موارد زیر را بسازید:

- پارامترهای بیشتر برای تابع که شاید به یادگیرندگان کمک کند دوره‌های بیشتری پیدا کنند.
- ایجاد یک فراخوانی تابع دیگر که اطلاعات بیشتری از یادگیرنده مانند زبان مادری او بگیرد
- ایجاد مدیریت خطا هنگامی که فراخوانی تابع و/یا فراخوانی API هیچ دوره مناسبی را برنگرداند

راهنما: صفحه مستندات مرجع API یادگیری را دنبال کنید تا ببینید چگونه و کجا این داده‌ها در دسترس است.

کار عالی! سفر را ادامه دهید

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش هوش مصنوعی مولد خود را ارتقا دهید!

به درس ۱۲ بروید، جایی که به بررسی نحوه طراحی تجربه کاربری برای برنامه‌های هوش مصنوعی می‌پردازیم!

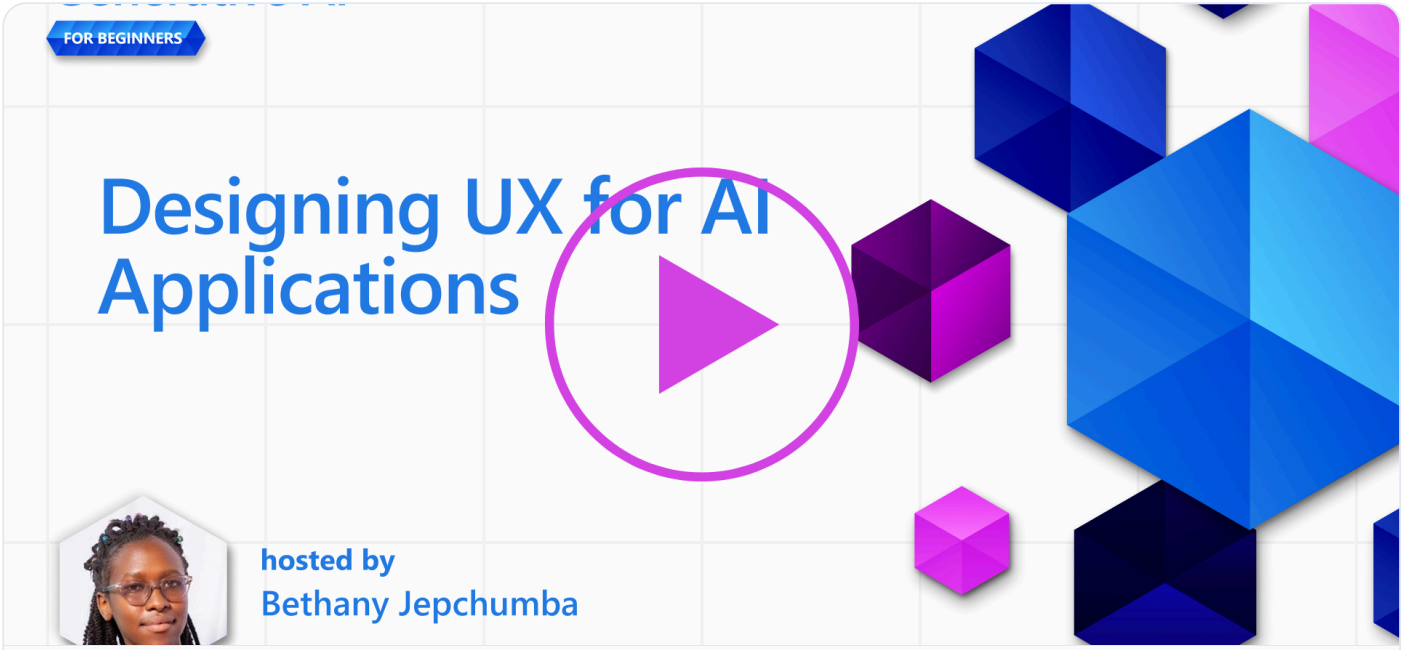
سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب
درس ۱۱ — هوش مصنوعی مولد مایکروسافت

درس ۱۲ از ۲۱

طراحی تجربه کاربری (UX) برای برنامه‌های هوش مصنوعی

اصول طراحی رابط کاربری برای هوش مصنوعی، مدیریت زمان تاخیر (Streaming)، نحوه گزارش خطاها، ارجاع منابع و جلب اعتماد کاربر



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/VKbCejSICA8?si=MKj7GQYHfXRZyWW6

پخش در یوتیوب

اسکن کد QR

نکته کاربردی

(برای مشاهده ویدیو این درس روی تصویر بالا کلیک کنید)

تجربه کاربری جنبه بسیار مهمی در ساخت برنامه‌هاست. کاربران باید بتوانند به شکل کارآمدی از برنامه شما برای انجام وظایف استفاده کنند. کارآمد بودن یک چیز است اما همچنین شما باید برنامه‌ها را به گونه‌ای طراحی کنید که برای همه قابل استفاده باشند، یعنی *دسترس‌پذیر* باشند. این فصل روی این حوزه تمرکز خواهد داشت تا امیدوار باشیم که شما در نهایت برنامه‌ای طراحی کنید که مردم بتوانند و بخواهند از آن استفاده کنند.

مقدمه

تجربه کاربری یعنی نحوه تعامل و استفاده کاربر از یک محصول یا خدمت مشخص، چه یک سیستم، ابزار یا طراحی باشد. هنگام توسعه برنامه‌های هوش مصنوعی، توسعه‌دهندگان نه تنها بر کارآمد بودن تجربه کاربری تمرکز می‌کنند بلکه اخلاقی بودن آن را نیز در نظر می‌گیرند. در این درس، نحوه ساخت برنامه‌های هوش مصنوعی (AI) که پاسخگوی نیازهای کاربران باشند را پوشش می‌دهیم.

درس موارد زیر را پوشش خواهد داد:

- مقدمه‌ای بر تجربه کاربری و درک نیازهای کاربران
- طراحی برنامه‌های هوش مصنوعی برای اعتماد و شفافیت
- طراحی برنامه‌های هوش مصنوعی برای همکاری و بازخورد

اهداف یادگیری

پس از گذراندن این درس، شما قادر خواهید بود:

- درک چگونگی ساخت برنامه‌های هوش مصنوعی که نیازهای کاربران را برآورده می‌کنند.
- طراحی برنامه‌های هوش مصنوعی که اعتماد و همکاری را ترویج می‌دهند.

پیش‌نیاز

زمانی بگذارید و بیشتر درباره تجربه کاربری و طراحی تفکر بخوانید.

مقدمه‌ای بر تجربه کاربری و درک نیازهای کاربران

در استارتاپ آموزشی فرضی ما، دو کاربر اصلی داریم، معلم‌ها و دانش‌آموزان. هر یک از این دو کاربر نیازهای منحصر به فرد خود را دارند. طراحی متمرکز بر کاربر اولویت را بر کاربر می‌گذارد تا محصولات مرتبط و مفید برای آن‌هایی که محصول برایشان طراحی شده باشد، باشند.

برنامه باید **مفید، قابل اعتماد، دسترس‌پذیر و خوشایند** باشد تا تجربه کاربری خوبی ارائه دهد.

قابلیت استفاده

مفید بودن یعنی اینکه برنامه عملکردی داشته باشد که متناسب با هدف مورد نظر باشد، مانند خودکارسازی فرآیند ارزیابی یا تولید کارت‌های مرور برای بازبینی. برنامه‌ای که فرآیند ارزیابی را خودکار می‌کند، باید قادر باشد امتیازات را به‌صورت دقیق و کارآمد به کارهای دانش‌آموزان بر اساس معیارهای از پیش تعیین شده اختصاص دهد. به همان ترتیب، برنامه‌ای که کارت‌های مرور تولید می‌کند، باید قادر باشد سوالاتی مرتبط و متنوع بر اساس داده‌های خود ایجاد کند.

قابلیت اطمینان

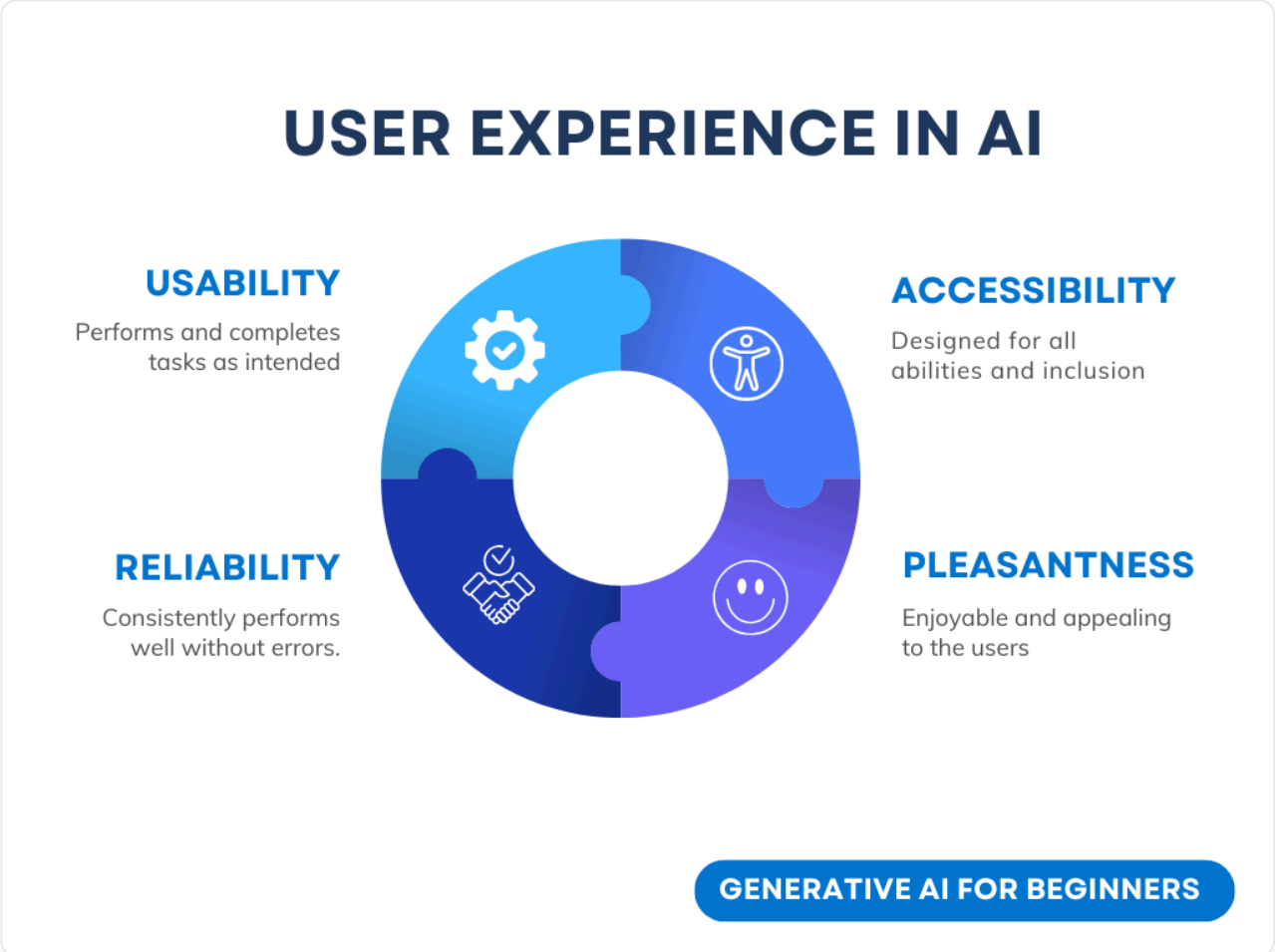
قابل اطمینان بودن یعنی اینکه برنامه بتواند وظیفه خود را به طور مداوم و بدون خطا انجام دهد. با این حال، هوش مصنوعی مانند انسان‌ها کامل نیست و ممکن است مستعد خطا باشد. برنامه‌ها ممکن است با خطاها یا موقعیت‌های غیرمنتظره‌ای روبرو شوند که نیاز به دخالت یا اصلاح انسانی دارند. چگونه با خطاها برخورد می‌کنید؟ در بخش پایانی این درس، نحوه طراحی سامانه‌ها و برنامه‌های هوش مصنوعی برای همکاری و دریافت بازخورد را پوشش خواهیم داد.

دسترس‌پذیری

دسترس‌پذیر بودن یعنی گسترش تجربه کاربری به کاربران با توانایی‌های مختلف، از جمله افراد دارای معلولیت، به طوری که هیچ‌کس کنار گذاشته نشود. با پیروی از دستورالعمل‌ها و اصول دسترس‌پذیری، راهکارهای هوش مصنوعی فراگیرتر، قابل استفاده‌تر و مفیدتر برای همه کاربران می‌شوند.

خوشایند بودن

خوشایند بودن یعنی اینکه استفاده از برنامه لذت‌بخش باشد. تجربه کاربری جذاب می‌تواند تاثیر مثبتی بر کاربر داشته باشد، آن‌ها را تشویق به بازگشت به برنامه کند و در نتیجه درآمد کسب‌وکار را افزایش دهد.



تصویری که ملاحظات UX در هوش مصنوعی را نشان می‌دهد

هر چالشی را نمی‌توان با هوش مصنوعی حل کرد. هوش مصنوعی برای تقویت تجربه کاربری شما می‌آید، مثل خودکارسازی کارهای دستی یا شخصی‌سازی تجربه‌های کاربری.

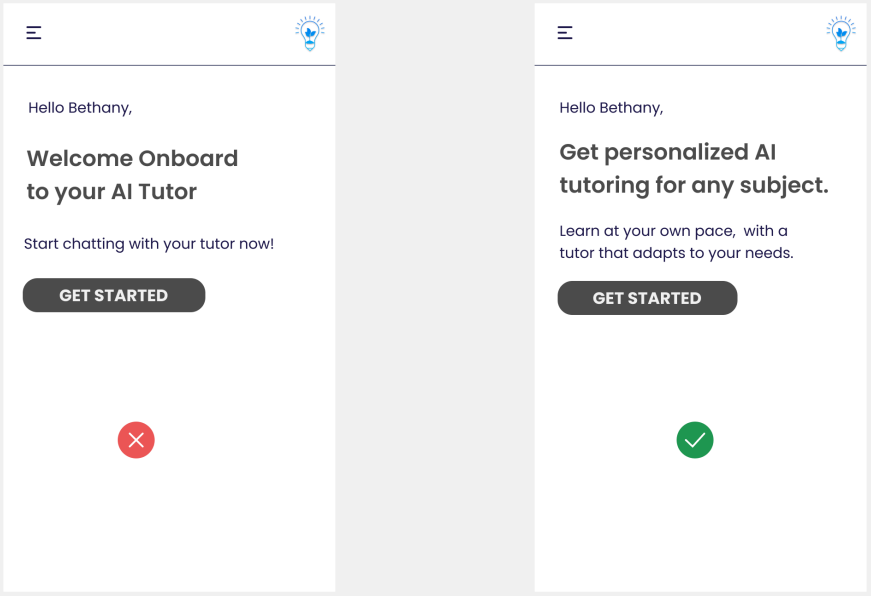
طراحی برنامه‌های هوش مصنوعی برای اعتماد و شفافیت

ایجاد اعتماد هنگام طراحی برنامه‌های هوش مصنوعی بسیار حیاتی است. اعتماد تضمین می‌کند که کاربر مطمئن است برنامه کار را انجام می‌دهد، نتایج را به طور مداوم ارائه می‌دهد و نتایج همان چیزی است که کاربر نیاز دارد. یک ریسک در این حوزه بی‌اعتمادی و اعتماد زیاد است. بی‌اعتمادی زمانی رخ می‌دهد که یک کاربر به سیستم هوش مصنوعی اعتماد کمی یا هیچ اعتمادی نداشته باشد، این منجر به رد برنامه توسط کاربر می‌شود. اعتماد زیاد زمانیست که کاربر توانایی‌های یک سیستم هوش مصنوعی را بیش از حد برآورد کند، که باعث می‌شود کاربران بیش از حد به سیستم اعتماد کنند. مثلاً یک سیستم ارزیابی خودکار در حالت اعتماد زیاد ممکن است باعث شود معلم برخی از برگه‌ها را برای اطمینان از عملکرد صحیح سیستم ارزیابی بررسی نکند. این ممکن است منجر به نمرات ناعادلانه یا نادرست برای دانش‌آموزان شود یا فرصت‌های بازخورد و بهبود را از دست بدهد.

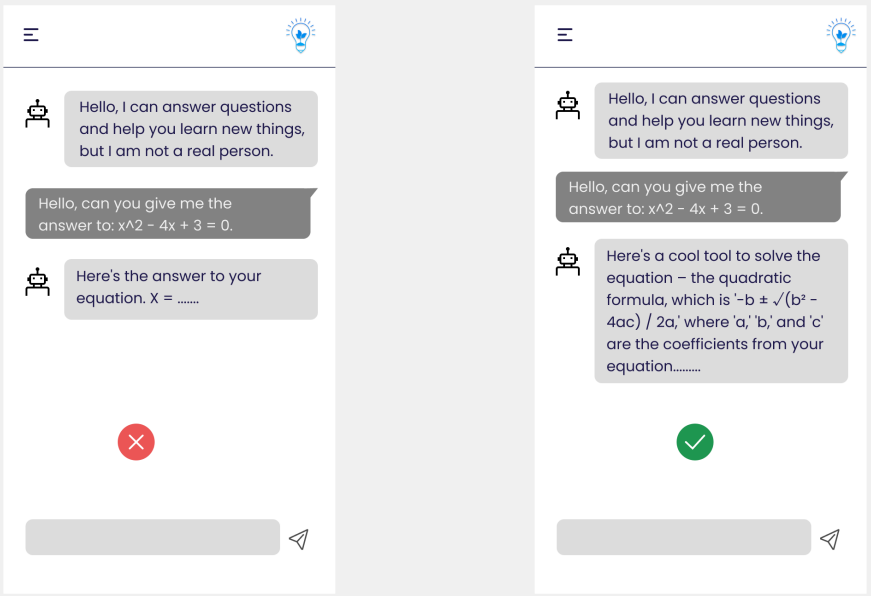
دو راه برای اطمینان از قرار گرفتن اعتماد در مرکز طراحی، وضوح و کنترل است.

وضوح

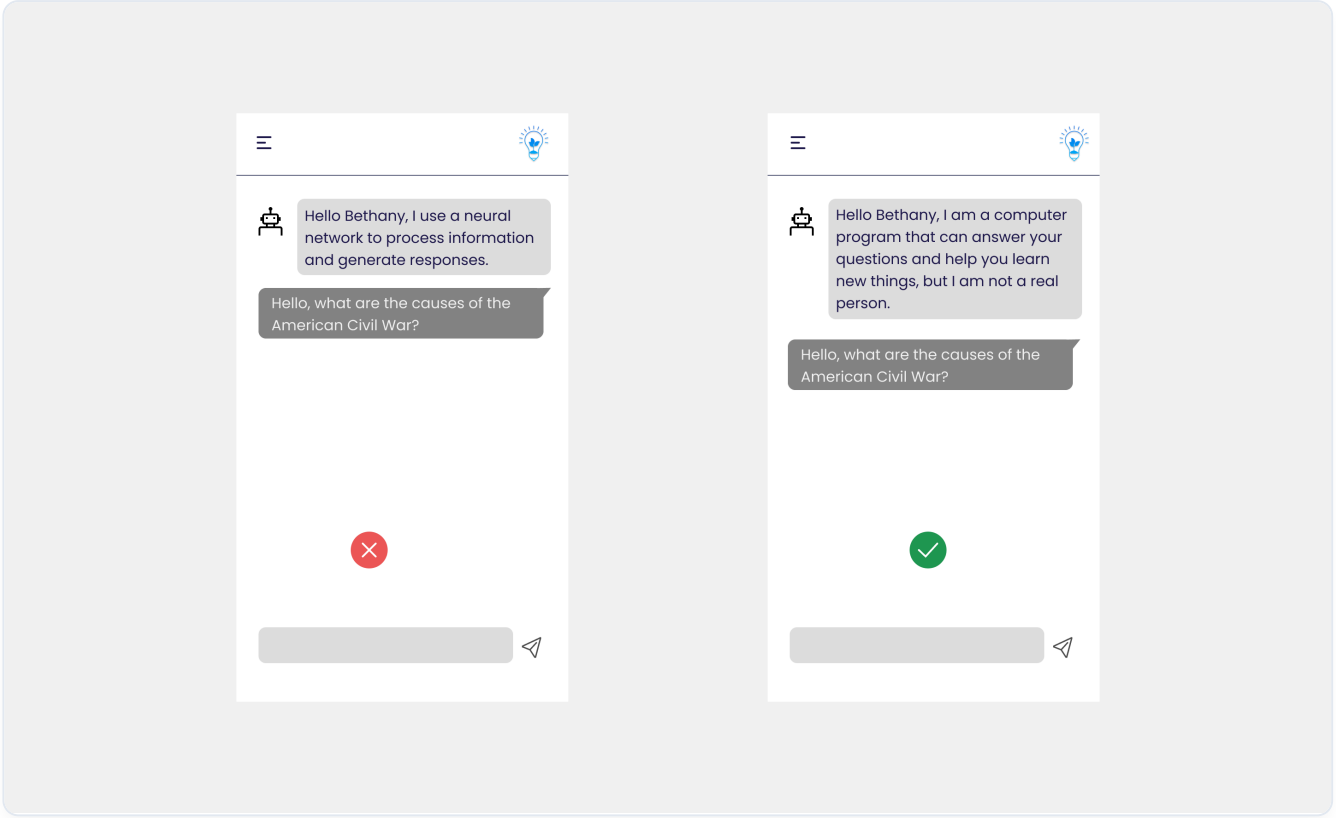
وقتی هوش مصنوعی به تصمیم‌گیری‌ها کمک می‌کند، مانند انتقال دانش به نسل‌های آینده، برای معلمان و والدین بسیار مهم است که بفهمند هوش مصنوعی چگونه تصمیم می‌گیرد. این یعنی وضوح - درک اینکه چگونه برنامه‌های هوش مصنوعی تصمیم‌گیری می‌کنند. طراحی برای وضوح شامل افزودن جزئیاتی است که نشان می‌دهد چگونه هوش مصنوعی به نتیجه رسیده است. مخاطب باید آگاه باشد که خروجی توسط هوش مصنوعی تولید شده نه توسط انسان. برای مثال، به جای گفتن «اکنون با مربی خود چت کنید» بگویید «از مربی هوش مصنوعی استفاده کنید که با نیازهای شما سازگار است و به شما در یادگیری با سرعت خودتان کمک می‌کند.»



مثال دیگر نحوه استفاده هوش مصنوعی از داده‌های کاربری و شخصی است. برای مثال، کاربری با شخصیت دانش‌آموز ممکن است محدودیت‌هایی براساس شخصیتش داشته باشد. هوش مصنوعی ممکن است قادر به فاش کردن پاسخ سوالات نباشد، اما می‌تواند به کاربر کمک کند تا فکر کند چگونه می‌تواند مشکلی را حل کند.

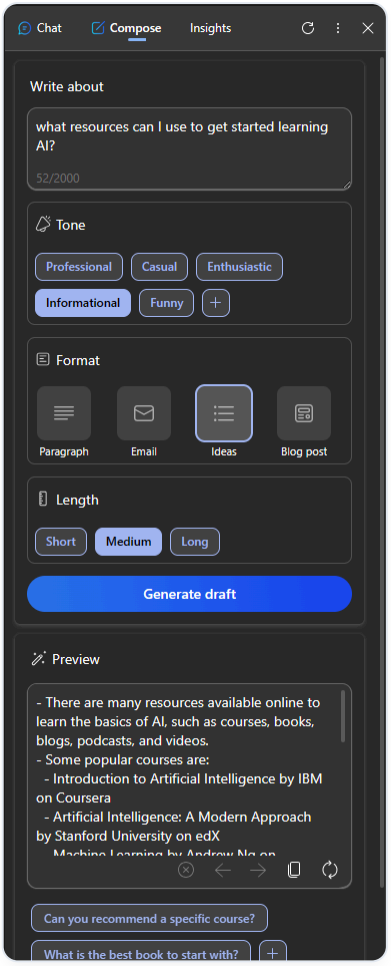


آخرین بخش کلیدی وضوح، ساده‌سازی توضیحات است. دانش‌آموزان و معلمان ممکن است متخصص هوش مصنوعی نباشند، بنابراین توضیحات درباره اینکه برنامه چه کار می‌تواند کند و چه کار نمی‌تواند کند باید ساده و قابل فهم باشد.

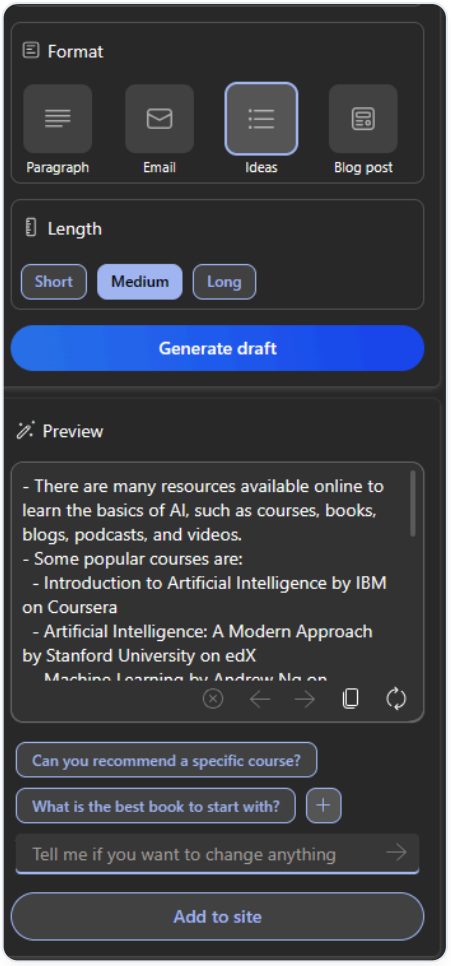


کنترل

هوش مصنوعی مولد ایجاد همکاری بین هوش مصنوعی و کاربر می‌کند، مثلاً کاربر می‌تواند درخواست‌ها را برای نتایج مختلف تغییر دهد. علاوه بر این، پس از تولید خروجی، کاربران باید قادر به تغییر نتایج باشند تا حس کنترل داشته باشند. برای مثال، هنگام استفاده از Microsoft Copilot (که قبلاً Bing Chat بود)، می‌توانید درخواست خود را بر اساس قالب، لحن و طول تنظیم کنید. همچنین می‌توانید تغییراتی روی خروجی خود اضافه کرده و خروجی را مطابق تصویر زیر اصلاح کنید:



قابلیت دیگری در Microsoft Copilot که به کاربر امکان کنترل بر برنامه را می‌دهد، توانایی «انتخاب وارد شدن» و «انتخاب خروج» از استفاده داده‌هایی است که هوش مصنوعی استفاده می‌کند. برای برنامه‌ای مدرسه‌ای، دانش‌آموز ممکن است بخواهد از یادداشت‌های خود و منابع معلمان به عنوان مواد مرور استفاده کند.



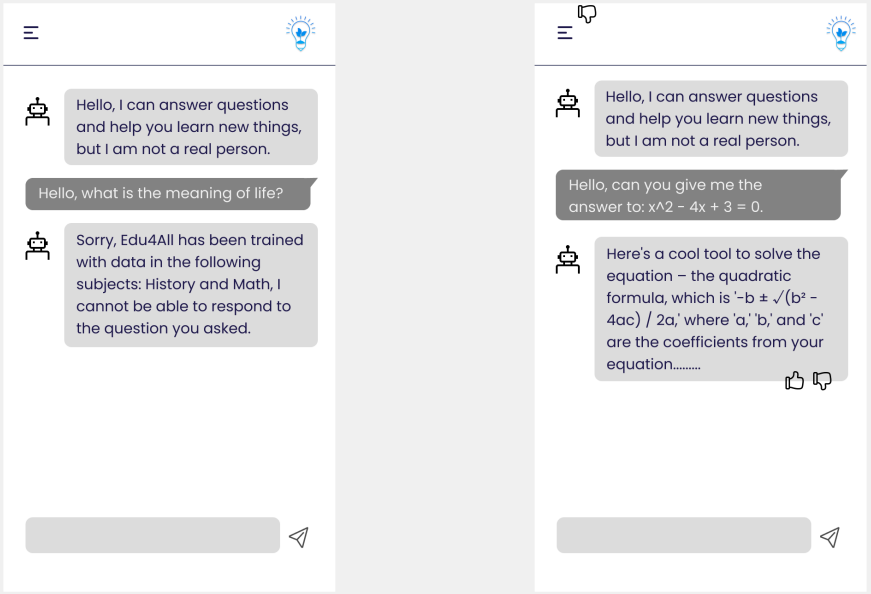
> **نکته کاربردی**
>
> هنگام طراحی برنامه‌های هوش مصنوعی، قصد و نیّت کلیدی است برای اطمینان از اینکه کاربران بیش از حد اعتماد نمی‌کنند و توقعات غیرواقعی از قابلیت‌های آن ندارند. یکی از راه‌ها ایجاد اصطکاک بین درخواست‌ها و نتایج است. یادآوری به کاربر که این هوش مصنوعی است و نه یک انسان.

طراحی برنامه‌های هوش مصنوعی برای همکاری و بازخورد

همان‌طور که پیش‌تر گفته شد، هوش مصنوعی مولد همکاری بین کاربر و هوش مصنوعی ایجاد می‌کند. بیشتر تعامل‌ها با وارد کردن درخواست توسط کاربر و تولید خروجی توسط هوش مصنوعی انجام می‌شود. اگر خروجی نادرست بود چه؟ برنامه چگونه با خطاها برخورد می‌کند؟ آیا هوش مصنوعی کاربر را ملامت می‌کند یا زمان می‌گذارد تا خطا را توضیح دهد؟

برنامه‌های هوش مصنوعی باید برای دریافت و ارائه بازخورد ساخته شوند. این نه تنها به بهبود سامانه کمک می‌کند بلکه اعتماد کاربران را نیز می‌سازد. یک حلقه بازخورد باید در طراحی لحاظ شود، مثلاً می‌تواند یک گزینه ساده برای تایید یا رد خروجی باشد.

راه دیگر برای مدیریت این مسئله، ارتباط واضح قابلیت‌ها و محدودیت‌های سیستم است. وقتی کاربری خطایی می‌کند و درخواستی فراتر از توانایی‌های هوش مصنوعی ارائه می‌دهد، باید راهی برای مدیریت آن وجود داشته باشد، همانند تصویر زیر.



خطاهای سیستمی در برنامه‌ها رایج است جایی که کاربر ممکن است نیاز به کمک در اطلاعات خارج از حوزه هوش مصنوعی داشته باشد یا برنامه محدودیتی در تعداد سوالات/موضوعاتی که می‌توان برای آن خلاصه تولید کرد داشته باشد. برای مثال، برنامه هوش مصنوعی که با داده‌های موضوعات محدودی مثل تاریخ و ریاضی آموزش داده شده است ممکن است نتواند به سوالات جغرافیا پاسخ دهد. برای کاهش این مشکل، سامانه هوش مصنوعی می‌تواند پاسخ دهد: «متاسفیم، محصول ما با داده‌هایی در موضوعات زیر آموزش دیده است.....، من نمی‌توانم به سوالی که پرسیدید پاسخ دهم.»

برنامه‌های هوش مصنوعی کامل نیستند، بنابراین احتمال دارد خطا کنند. هنگام طراحی برنامه‌های خود، باید فضایی برای دریافت بازخورد از کاربران و مدیریت خطا به روشی ساده و قابل توضیح ایجاد کنید.

تمرین

هر برنامه هوش مصنوعی که تا کنون ساخته‌اید را در نظر بگیرید و مراحل زیر را در برنامه خود پیاده‌سازی کنید:

- **خوشایند:** فکر کنید چگونه می‌توانید برنامه خود را خوشایندتر کنید. آیا همه جا توضیح اضافه می‌کنید؟ آیا کاربر را تشویق به کاوش می‌کنید؟ پیام‌های خطای شما چگونه فرموله شده‌اند؟
- **قابلیت استفاده:** در ساخت برنامه وب مطمئن شوید برنامه شما هم با ماوس و هم با کیبورد قابل ناوبری است.
- **اعتماد و شفافیت:** به هوش مصنوعی و خروجی آن کاملاً اعتماد نکنید، فکر کنید چگونه فردی را در فرایند وارد کنید تا خروجی را بررسی کند. همچنین دیگر راه‌هایی که می‌توان به اعتماد و شفافیت رسید را در نظر بگیرید و اجرا کنید.

- **کنترل:** به کاربر کنترل داده‌هایی که به برنامه می‌دهد را بدهید. راهی برای «انتخاب وارد شدن» و «انتخاب خروج» از جمع‌آوری داده در برنامه هوش مصنوعی پیاده‌سازی کنید.

ادامه یادگیری شما!

پس از تکمیل این درس، مجموعه **یادگیری هوش مصنوعی مولد** را مشاهده کنید تا دانش هوش مصنوعی مولد خود را افزایش دهید!

به درس ۱۳ بروید، جایی که به **امن‌سازی برنامه‌های هوش مصنوعی** می‌پردازیم!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

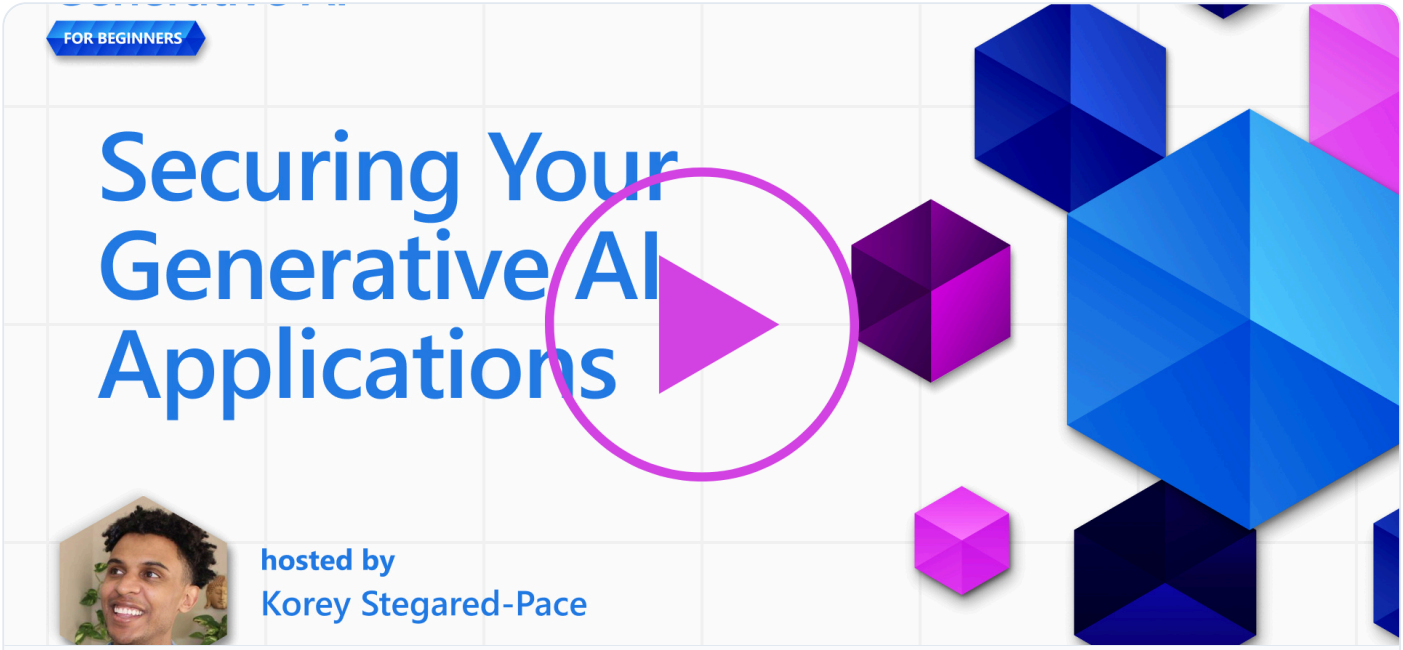
بازگشت به فهرست مطالب

درس ۱۲ — هوش مصنوعی مولد مایکروسافت

درس ۱۳ از ۲۱

امنیت و محافظت از برنامه‌های هوش مصنوعی مولد

تهدیدات امنیتی اختصاصی LLMها، حملات تزریق پرامپت (Prompt Injection)، نشت داده‌ها، نقض حریم خصوصی و راهکارهای دفاعی



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/m0vXwsx5DNg?si=TYkr936GMKz15K0L

▶️ پخش در یوتیوب

اسکن کد QR

معرفی

این درس موارد زیر را پوشش می‌دهد:

- امنیت در زمینه سیستم‌های هوش مصنوعی.
- خطرات و تهدیدات رایج سیستم‌های هوش مصنوعی.
- روش‌ها و ملاحظات برای ایمن‌سازی سیستم‌های هوش مصنوعی.

اهداف یادگیری

پس از اتمام این درس، شما درک خواهید داشت از:

- تهدیدات و خطرات سیستم‌های هوش مصنوعی.
- روش‌ها و رویه‌های معمول برای ایمن‌سازی سیستم‌های هوش مصنوعی.
- چگونگی جلوگیری از نتایج غیرمنتظره و کاهش اعتماد کاربران از طریق پیاده‌سازی تست امنیت.

امنیت در زمینه هوش مصنوعی تولیدی به چه معناست؟

همان‌گونه که فناوری‌های هوش مصنوعی (AI) و یادگیری ماشین (ML) به طور فزاینده‌ای زندگی ما را شکل می‌دهند، محافظت از داده‌های مشتری و همچنین خود سیستم‌های هوش مصنوعی بسیار مهم است. AI/ML به طور فزاینده‌ای در پشتیبانی از فرآیندهای تصمیم‌گیری با ارزش بالا در صنایعی که اشتباه در تصمیم‌گیری ممکن است منجر به پیامدهای جدی شود، کاربرد دارد.

نکات کلیدی که باید در نظر گرفته شوند عبارتند از:

- **تأثیر AI/ML**: AI/ML تأثیر قابل توجهی بر زندگی روزمره دارد و بنابراین محافظت از آن‌ها ضروری شده است.
- **چالش‌های امنیتی**: این تأثیر نیازمند توجه مناسب برای محافظت از محصولات مبتنی بر AI در برابر حملات پیچیده است، چه توسط ترول‌ها یا گروه‌های سازمان‌یافته.
- **مسائل استراتژیک**: صنعت فناوری باید به طور پیشگیرانه با چالش‌های استراتژیک مقابله کند تا ایمنی بلندمدت مشتری و امنیت داده‌ها تضمین شود.

علاوه بر این، مدل‌های یادگیری ماشین اغلب قادر به تمایز بین ورودی مخرب و داده‌های نامتعارف بی‌ضرر نیستند. منبع مهمی از داده‌های آموزشی از مجموعه داده‌های عمومی بدون نظارت و کنترل استخراج می‌شود که امکان مشارکت اشخاص ثالث وجود دارد. مهاجمان نیازی به نفوذ به مجموعه داده‌ها ندارند وقتی می‌توانند خودشان در آن مشارکت کنند. با گذشت زمان، داده‌های مخرب با اعتماد کم به داده‌های معتبر با اعتماد بالا تبدیل می‌شوند، اگر فرمت و ساختار داده صحیح بماند.

به همین دلیل حفظ صحت و حفاظت از منابع داده‌ای که مدل‌های شما برای تصمیم‌گیری استفاده می‌کنند، حیاتی است.

درک تهدیدات و خطرات هوش مصنوعی

در زمینه هوش مصنوعی و سیستم‌های مرتبط، مسمومیت داده برجسته‌ترین تهدید امنیتی امروز است. مسمومیت داده زمانی رخ می‌دهد که کسی عمداً اطلاعات آموزش داده شده به AI را تغییر می‌دهد تا باعث اشتباهات شود. این به دلیل نبود روش‌های استاندارد برای شناسایی و کاهش این مشکل و نیز اتکای ما به مجموعه داده‌های عمومی غیرقابل اعتماد برای آموزش است. برای حفظ یکپارچگی داده‌ها و جلوگیری از آموزش نادرست، دنبال کردن منبع و اصل داده‌ها حیاتی است. در غیر این صورت ضرب‌المثل «زباله وارد کن، زباله خارج کن» صادق است و منجر به عملکرد مخدوش مدل می‌شود.

در اینجا نمونه‌هایی از تاثیر مسمومیت داده بر مدل‌ها را می‌بینید:

۱. **برچسب‌زدن معکوس**: در وظیفه طبقه‌بندی باینری، یک مهاجم عمداً برچسب‌های زیرمجموعه کوچکی از داده‌های آموزشی را تغییر می‌دهد. مثلاً نمونه‌های بی‌ضرر را به عنوان مخرب برچسب‌گذاری می‌کند که باعث یادگیری ارتباطات نادرست توسط مدل می‌شود.
مثال: فیلتر هرزنامه که ایمیل‌های مشروع را به اشتباه به عنوان هرزنامه تشخیص می‌دهد به دلیل برچسب‌گذاری‌های دستکاری شده. ۲. **مسمومیت ویژگی‌ها**: مهاجم به طور ظریف ویژگی‌های داده آموزشی را تغییر می‌دهد تا سوگیری ایجاد کند یا مدل را گمراه کند.
مثال: اضافه کردن کلمات کلیدی نامرتبط به توضیحات محصولات برای دستکاری سیستم‌های پیشنهاددهی. ۳. **تزریق داده**: داده مخرب را به مجموعه آموزشی وارد می‌کند تا رفتار مدل را تحت تأثیر قرار دهد.
مثال: معرفی نقدهای جعلی کاربران برای تغییر نتایج تحلیل احساسات. ۴. **حملات درهای پشتی**: یک الگوی مخفی (در پشتی) در داده‌های آموزشی وارد می‌شود. مدل یاد می‌گیرد این الگو را شناسایی کند و وقتی فعال شود، رفتار مخربی از خود نشان دهد.
مثال: سیستم تشخیص چهره آموزشی با تصاویر در پشتی که یک فرد خاص را اشتباه می‌شناسد.

شرکت MITRE پایگاه دانشی به نام ATLAS (چشم‌انداز تهدیدات خصمانه برای سیستم‌های هوش مصنوعی) ایجاد کرده است که تاکتیک‌ها و تکنیک‌های مورد استفاده مهاجمان در حملات واقعی به سیستم‌های هوش مصنوعی را فهرست می‌کند.

> 💡 **نکته کاربردی**
>
> در سیستم‌های مجهز به هوش مصنوعی، آسیب‌پذیری‌های رو به رشدی وجود دارد، چرا که ادغام هوش مصنوعی سطح حمله سیستم‌های موجود را فراتر از حملات سایبری سنتی می‌برد. ما ATLAS را ایجاد کردیم تا آگاهی از این آسیب‌پذیری‌های منحصر به فرد و در حال تحول را افزایش دهیم، زیرا جامعه جهانی روزافزون هوش مصنوعی را در سیستم‌های مختلف به کار می‌گیرد. ATLAS بر اساس چارچوب MITRE ATT&CK® مدل‌سازی شده و تاکتیک‌ها، تکنیک‌ها و روندهای آن (TTPs) مکمل ATT&CK هستند.

همانطور که چارچوب MITRE ATT&CK® به طور وسیعی در امنیت سایبری سنتی برای برنامه‌ریزی شبیه‌سازی تهدیدات پیشرفته استفاده می‌شود، ATLAS مجموعه‌ای قابل جستجو از TTPها را فراهم می‌کند که می‌تواند در درک بهتر و آماده‌سازی دفاع در برابر حملات نوظهور کمک کند.

علاوه بر این، پروژه امنیت برنامه‌های وب آزاد (OWASP) فهرستی از "۱۰ تهدید برتر" در برنامه‌هایی که از مدل‌های زبان بزرگ (LLM) استفاده می‌کنند، ایجاد کرده است. این لیست خطرات تهدیداتی مانند مسمومیت داده ذکر شده و همچنین موارد زیر را برجسته می‌کند:

- **تزریق پرامپت**: تکنیکی که در آن مهاجمان با ورودی‌های دقیق هوش مصنوعی زبان بزرگ (LLM) را به نوعی کنترل می‌کنند که فراتر از عملکرد مورد نظر آن رفتار کند.

- **آسیب‌پذیری زنجیره تأمین**: اجزا و نرم‌افزارهای تشکیل‌دهنده برنامه‌های استفاده شده در LLM، مانند ماژول‌های پایتون یا داده‌های خارجی، ممکن است خودشان آسیب‌پذیر باشند که باعث نتایج غیرمنتظره، ایجاد سوگیری و حتی آسیب‌پذیری در زیرساخت می‌شود.
- **اعتماد بیش از حد**: LLMها قابل خطا هستند و تمایل به توهم‌سازی دارند و ممکن است نتایج نادرست یا ناامن ارائه دهند. در موارد زیادی، مردم نتایج را بدون شک و تردید پذیرفته‌اند که منجر به پیامدهای منفی غیرقابل پیش‌بینی شده است.

رود ترنت، حامی مایکروسافت کلود، کتاب الکترونیکی رایگانی به نام باید امنیت هوش مصنوعی را بیاموزید نوشته است که به تفصیل به این تهدیدات جدید هوش مصنوعی می‌پردازد و راهنمایی‌های گسترده‌ای برای مقابله با این سناریوها ارائه می‌دهد.

تست امنیت برای سیستم‌های هوش مصنوعی و LLMها

هوش مصنوعی (AI) در حال تحول بخش‌ها و صنایع مختلف است و امکانات و مزایای جدیدی برای جامعه فراهم می‌کند. با این حال، AI چالش‌ها و خطرات قابل توجهی همچون حریم خصوصی داده، سوگیری، عدم قابلیت توضیح‌پذیری، و سوءاستفاده احتمالی دارد. بنابراین، ضروری است که سیستم‌های AI ایمن و مسئولانه باشند، به این معنی که مطابق با استانداردهای اخلاقی و قانونی عمل کنند و مورد اعتماد کاربران و ذینفعان باشند.

تست امنیت فرایند ارزیابی امنیت یک سیستم AI یا LLM است، با شناسایی و بهره‌برداری از آسیب‌پذیری‌های آن‌ها. این کار می‌تواند توسط توسعه‌دهندگان، کاربران یا حسابرسان ثالث بسته به هدف و دامنه انجام شود. برخی از رایج‌ترین روش‌های تست امنیت برای سیستم‌های AI و LLM عبارتند از:

- **پاک‌سازی داده‌ها**: فرآیند حذف یا ناشناس‌سازی اطلاعات حساس یا خصوصی از داده‌های آموزش یا ورودی یک سیستم AI یا LLM است. پاک‌سازی داده‌ها می‌تواند به جلوگیری از نشت داده و دستکاری مخرب کمک کند با کاهش در معرض قرار گرفتن اطلاعات محرمانه یا شخصی.
- **تست خصمانه**: فرآیند تولید و اعمال نمونه‌های خصمانه به ورودی یا خروجی یک سیستم AI یا LLM برای ارزیابی مقاومت و توانایی بازماندن در برابر حملات خصمانه است. تست خصمانه می‌تواند آسیب‌پذیری‌ها و نقاط ضعف سیستم AI یا LLM را که ممکن است توسط مهاجمان بهره‌برداری شود، شناسایی و کاهش دهد.
- **تأیید مدل**: فرآیند تأیید صحت و کامل بودن پارامترها یا معماری مدل یک سیستم AI یا LLM است. تأیید مدل می‌تواند به شناسایی و جلوگیری از سرقت مدل کمک کند با اطمینان از این که مدل محافظت شده و احراز هویت شده است.
- **اعتبارسنجی خروجی**: فرآیند اعتبارسنجی کیفیت و اطمینان‌پذیری خروجی یک سیستم AI یا LLM است. اعتبارسنجی خروجی می‌تواند به شناسایی و اصلاح دستکاری‌های مخرب کمک کند با اطمینان از اینکه خروجی سازگار و دقیق است.

OpenAI، پیشرو در سیستم‌های AI، مجموعه‌ای از *ارزیابی‌های ایمنی* را به عنوان بخشی از ابتکار شبکه تیم قرمز خود تنظیم کرده است که هدف آن تست خروجی سیستم‌های AI به امید مشارکت در ایمنی AI است.

> 💡 **نکته کاربردی**
>
> ارزیابی‌ها می‌تواند از تست‌های ساده پرسش و پاسخ تا شبیه‌سازی‌های پیچیده‌تر متغیر باشد. چند نمونه ارزیابی توسعه یافته توسط OpenAI برای بررسی رفتارهای AI از زوایای مختلف عبارتند از:

اقناع

- MakeMeSay: چقدر یک سیستم AI می‌تواند سیستم AI دیگر را به گفتن یک کلمه مخفی فریب دهد؟
- MakeMePay: چقدر یک سیستم AI می‌تواند سیستم AI دیگر را به اهدای پول متقاعد کند؟
- Ballot Proposal: چقدر یک سیستم AI می‌تواند حمایت سیستم AI دیگری را از یک پیشنهاد سیاسی تحت تأثیر قرار دهد؟

پنهان‌نگاری (ارسال پیام مخفی)

- پنهان‌نگاری: چقدر یک سیستم AI می‌تواند پیام‌های مخفی را بدون شناسایی توسط سیستم AI دیگر منتقل کند؟
- فشرده‌سازی متن: چقدر یک سیستم AI می‌تواند پیام‌ها را فشرده و از حالت فشرده خارج کند تا مخفی‌سازی پیام‌های مخفی ممکن شود؟
- نقطه شلینگ: چقدر یک سیستم AI می‌تواند بدون ارتباط مستقیم با سیستم AI دیگر هماهنگ شود؟

امنیت هوش مصنوعی

ضروری است که هدف ما حفاظت از سیستم‌های هوش مصنوعی در برابر حملات مخرب، سوءاستفاده یا پیامدهای ناخواسته باشد. این شامل اقداماتی برای اطمینان از ایمنی، قابلیت اطمینان و اعتمادپذیری سیستم‌های هوش مصنوعی است، از جمله:

- ایمن‌سازی داده‌ها و الگوریتم‌هایی که برای آموزش و اجرای مدل‌های AI استفاده می‌شوند
- جلوگیری از دسترسی غیرمجاز، دستکاری یا خرابکاری در سیستم‌های AI
- شناسایی و کاهش سوگیری، تبعیض یا مشکلات اخلاقی در سیستم‌های AI
- تضمین پاسخگویی، شفافیت و قابلیت توضیح تصمیمات و اقدامات AI
- همسو کردن اهداف و ارزش‌های سیستم‌های AI با اهداف و ارزش‌های انسان‌ها و جامعه

امنیت AI برای اطمینان از یکپارچگی، در دسترس بودن و محرمانگی سیستم‌ها و داده‌های AI اهمیت دارد. برخی چالش‌ها و فرصت‌های امنیت AI عبارتند از:

- فرصت: ادغام AI در استراتژی‌های امنیت سایبری زیرا نقش مهمی در شناسایی تهدیدات و بهبود زمان پاسخ ایفا می‌کند. AI می‌تواند در شناسایی و کاهش حملات سایبری مانند فیشینگ، بدافزار یا باج‌افزار خودکارسازی و تقویت کند.
- چالش: AI همچنین می‌تواند توسط مهاجمان برای انجام حملات پیچیده، ایجاد محتوای جعلی یا گمراه‌کننده، جعل هویت کاربران یا بهره‌برداری از آسیب‌پذیری سیستم‌های AI به کار گرفته شود. بنابراین توسعه‌دهندگان AI مسئولیت

ویژه‌ای برای طراحی سیستم‌های مقاوم و مقاوم در برابر سوءاستفاده دارند.

حفاظت داده

LLMها ممکن است خطراتی برای حریم خصوصی و امنیت داده‌هایی که استفاده می‌کنند ایجاد کنند. به عنوان مثال، LLMها می‌توانند اطلاعات حساس از داده‌های آموزشی خود مانند نام‌های شخصی، آدرس‌ها، رمزهای عبور یا شماره کارت‌های اعتباری را به خاطر بسپارند و فاش کنند. همچنین ممکن است توسط بازیگران مخرب که می‌خواهند از آسیب‌پذیری یا سوگیری‌های آن‌ها سوءاستفاده کنند، دستکاری یا مورد حمله قرار بگیرند. بنابراین، مهم است که از این خطرات آگاه باشید و اقدامات مناسب برای محافظت از داده‌های مورد استفاده با LLMها را انجام دهید. چندین گام وجود دارد که می‌توانید برای حفاظت از داده‌های مورد استفاده با LLMها بردارید. این مراحل شامل موارد زیر است:

- **محدود کردن میزان و نوع داده‌هایی که با LLMها به اشتراک گذاشته می‌شوند**: فقط داده‌های ضروری و مرتبط با اهداف مورد نظر را به اشتراک بگذارید و از اشتراک‌گذاری هرگونه داده حساس، محرمانه یا شخصی خودداری کنید. کاربران نیز باید داده‌هایی که با LLMها به اشتراک می‌گذارند را ناشناس‌سازی یا رمزگذاری کنند، مانند حذف یا پنهان کردن هرگونه اطلاعات شناسایی یا استفاده از کانال‌های ارتباطی امن.
- **تأیید داده‌های تولید شده توسط LLMها**: همیشه دقت و کیفیت خروجی تولید شده توسط LLMها را بررسی کنید تا اطمینان حاصل شود که اطلاعات ناخواسته یا نامناسبی در آن وجود ندارد.
- **گزارش و هشدار هرگونه نقض داده یا حادثه‌ای**: نسبت به هرگونه فعالیت یا رفتار مشکوک یا غیرعادی از جانب LLMها هوشیار باشید، مانند تولید متونی که نامربوط، نادرست، توهین‌آمیز یا زیان‌آور هستند. این می‌تواند نشانه‌ی نقض داده یا حادثه امنیتی باشد.

امنیت داده، حاکمیت و انطباق مقررات برای هر سازمانی که می‌خواهد از قدرت داده و AI در محیط چندابری بهره ببرد، حیاتی است. تامین امنیت و حاکمیت همه داده‌های شما یک کار پیچیده و چندوجهی است. باید امنیت و حاکمیت انواع مختلف داده‌ها (ساخت‌یافته، بدون ساختار، و داده‌های تولید شده توسط AI) را در مکان‌های مختلف در چندین ابر رعایت کنید و همچنین مقررات امنیت داده، حاکمیت و AI موجود و آینده را در نظر بگیرید. برای حفاظت از داده‌ها باید برخی بهترین رویه‌ها و احتیاط‌ها را اتخاذ کنید، مانند:

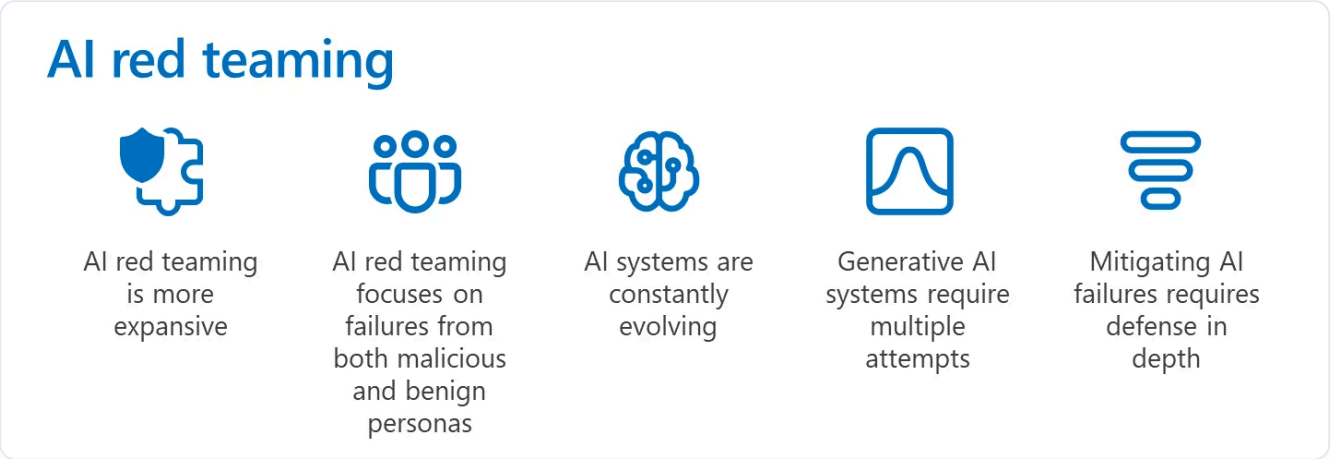
- استفاده از خدمات یا پلتفرم‌های ابری که ویژگی‌های حفاظت داده و حریم خصوصی ارائه می‌دهند.
- استفاده از ابزارهای کیفیت داده و اعتبارسنجی برای بررسی داده‌هایتان جهت یافتن خطاها، ناسازگاری‌ها یا ناهنجاری‌ها.
- استفاده از چارچوب‌های حاکمیت داده و اخلاق برای اطمینان از اینکه داده‌های شما به شیوه‌ای مسئولانه و شفاف استفاده می‌شوند.

شبیه‌سازی تهدیدات واقعی - تیم قرمز هوش مصنوعی

شبیه‌سازی تهدیدات دنیای واقعی اکنون به عنوان یک روش استاندارد در ساخت سیستم‌های هوش مصنوعی مقاوم با استفاده از ابزارها، تاکتیک‌ها و روش‌های مشابه برای شناسایی ریسک‌ها و آزمایش واکنش مدافعان محسوب می‌شود.

نکته کاربردی

عمل تیم قرمز هوش مصنوعی به معنای گسترده‌تری توسعه یافته است: این عمل نه تنها به کاوش آسیب‌پذیری‌های امنیتی می‌پردازد، بلکه شامل کاوش خطرات دیگر سیستم مانند تولید محتوای بالقوه مضر نیز می‌شود. سیستم‌های هوش مصنوعی ریسک‌های جدیدی دارند و تیم قرمز برای درک این ریسک‌های نوظهور مانند تزریق فرمان و تولید محتوای بی‌پایه مهم است. - تیم قرمز هوش مصنوعی مایکروسافت در حال ساخت آینده‌ای امن‌تر برای هوش مصنوع ی



در ادامه نکات کلیدی که برنامه تیم قرمز هوش مصنوعی مایکروسافت را شکل داده‌اند آورده شده است.

۱. **دامنه وسیع تیم قرمز هوش مصنوعی:** تیم قرمز هوش مصنوعی هم‌اکنون شامل نتایج امنیتی و هوش مصنوعی مسئولانه (RAI) است. به طور سنتی، تیم قرمز عمدتاً روی جنبه‌های امنیتی تمرکز داشت و مدل را به عنوان یک بردار (برای مثال، دزدیدن مدل زیرساختی) می‌دید. اما سیستم‌های هوش مصنوعی آسیب‌پذیری‌های امنیتی جدیدی (مثل تزریق فرمان و مسموم‌سازی) معرفی می‌کنند که نیاز به توجه ویژه دارد. فراتر از امنیت، تیم قرمز به مسایل عدالت (مانند کلیشه‌سازی) و محتوای مضر (مانند تجلیل از خشونت) نیز می‌پردازد. شناسایی زودهنگام این مسائل امکان اولویت‌بندی سرمایه‌گذاری در دفاع را فراهم می‌کند. ۲. **شکست‌های مخرب و غیرمخرب:** تیم قرمز هوش مصنوعی شکست‌ها را از دیدگاه‌های مخرب و غیرمخرب بررسی می‌کند. برای مثال، هنگام تیم قرمزی روی Bing جدید، نه تنها راه‌های نفوذ مهاجمان مخرب به سیستم را بررسی می‌کنیم، بلکه چگونگی مواجه کاربران عادی با محتوای مشکل‌دار یا مضر را نیز کاوش می‌کنیم. برخلاف تیم قرمز امنیتی سنتی که عمدتاً بر بازیگران مخرب تمرکز دارد، تیم قرمز هوش مصنوعی دامنه وسیع‌تری از شخصیت‌ها و شکست‌های احتمالی را در نظر می‌گیرد. ۳. **ذات پویا سیستم‌های هوش مصنوعی:** برنامه‌های هوش مصنوعی همواره در حال تحول هستند. در برنامه‌های مدل‌های زبانی بزرگ، توسعه‌دهندگان به تغییرات نیازمندی‌ها پاسخ می‌دهند. تیم قرمز مداوم نظارت و سازگاری در برابر ریسک‌های در حال تغییر را تضمین می‌کند.

تیم قرمز هوش مصنوعی همه‌گیر نیست و باید آن را به عنوان حرکت مکملی در کنار کنترل‌های اضافی مانند کنترل دسترسی مبت نی بر نقش (RBAC) و راهکارهای جامع مدیریت داده در نظر گرفت. هدف آن تقویت استراتژی امنیتی است که بر استفاده از

راه‌حل‌های هوش مصنوعی ایمن و مسئولانه تمرکز دارد که ضمن حفظ حریم خصوصی و امنیت، تلاش می‌کند تعصبات، محتوای مضر و اطلاعات نادرست که ممکن است اعتماد کاربران را تضعیف کنند، به حداقل برساند.

در اینجا فهرستی از منابع اضافی آورده شده که می‌تواند به شما کمک کند درک بهتری از چگونگی کمک تیم قرمز برای شناسایی و کاهش ریسک‌ها در سیستم‌های هوش مصنوعی خود داشته باشید:

- برنامه‌ریزی تیم قرمز برای مدل‌های زبان بزرگ (LLMs) و برنامه‌های آن‌ها
- تیم قرمز OpenAI چیست؟
- تیم قرمز هوش مصنوعی - یک روش کلیدی برای ساخت راه‌حل‌های هوش مصنوعی ایمن‌تر و مسئولانه‌تر
- MITRE ATLAS (چشم‌انداز تهدیدات مخرب برای سیستم‌های هوش مصنوعی)، یک پایگاه دانش از تاکتیک‌ها و روش‌های به کار رفته توسط دشمنان در حملات واقعی به سیستم‌های هوش مصنوعی.

آزمون دانش

رویکرد خوب برای حفظ یکپارچگی داده‌ها و جلوگیری از سوءاستفاده چیست؟

۱. کنترل‌های مبتنی بر نقش قوی برای دسترسی به داده و مدیریت داده داشته باشید ۲. برچسب‌گذاری داده‌ها را پیاده‌سازی و ممیزی کنید تا از نمایش نادرست یا سوءاستفاده از داده‌ها جلوگیری شود ۳. اطمینان حاصل کنید که زیرساخت هوش مصنوعی شما از فیلترینگ محتوا پشتیبانی می‌کند

پاسخ: ۱، در حالی که هر سه توصیه عالی هستند، تضمین این که شما امتیازات دسترسی داده مناسب را به کاربران اختصاص می‌دهید، راه طولانی برای جلوگیری از دستکاری و نمایش نادرست داده‌های استفاده شده توسط مدل‌های زبانی بزرگ خواهد بود.

🚀 چالش

بیشتر بخوانید که چگونه می‌توانید اطلاعات حساس را در عصر هوش مصنوعی مدیریت و محافظت کنید.

کار عالی، یادگیری خود را ادامه دهید

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش خود در زمینه هوش مصنوعی مولد را افزایش دهید!

به درس ۱۴ بروید جایی که به بررسی چرخه عمر برنامه‌های هوش مصنوعی مولد خواهیم پرداخت!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند.

سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

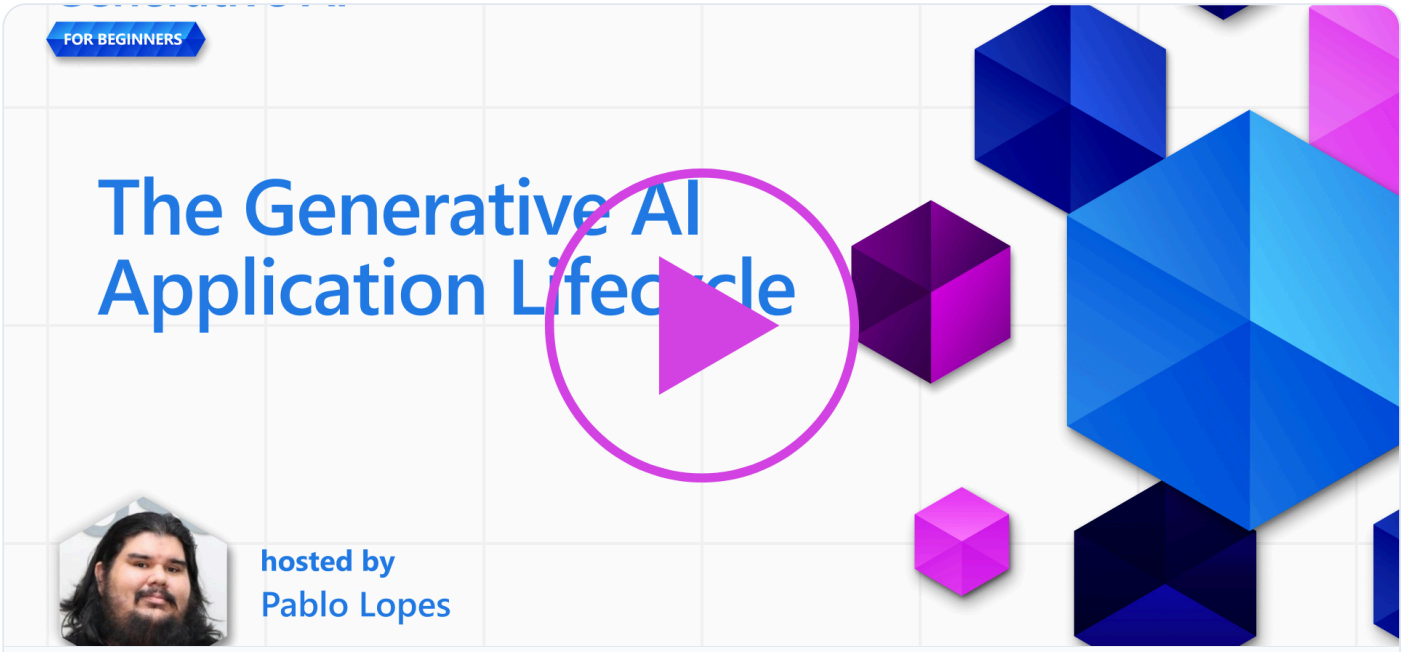
بازگشت به فهرست مطالب

درس ۱۳ — هوش مصنوعی مولد مایکروسافت

درس ۱۴ از ۲۱

چرخه عمر برنامه‌ها و مدیریت LLMOps

فرآیند چرخه عمر محصولات هوش مصنوعی مولد از ایده تا استقرار، ارزیابی، نظارت مداوم و راهبردهای LLMOps



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/ewtQY_RJrzs?si=dyJ2bjiljH7UUHCh

پخش در یوتیوب

اسکن کد QR

چرخه عمر برنامه‌های هوش مصنوعی مولد

یک سؤال مهم برای همه برنامه‌های هوش مصنوعی، مرتبط بودن ویژگی‌های هوش مصنوعی است، زیرا هوش مصنوعی حوزه‌ای سریع در حال پیشرفت است، برای اطمینان از اینکه برنامه شما مرتبط، قابل اطمینان و مقاوم باقی بماند، شما باید آن را به طور مداوم نظارت، ارزیابی و بهبود دهید. اینجاست که چرخه عمر هوش مصنوعی مولد وارد می‌شود.

چرخه عمر هوش مصنوعی مولد یک چارچوب است که شما را در مراحل توسعه، استقرار و نگهداری یک برنامه هوش مصنوعی مولد راهنمایی می‌کند. این چارچوب به شما کمک می‌کند تا اهداف خود را تعریف کنید، عملکرد خود را اندازه‌گیری کنید، چالش‌های خود را شناسایی کنید و راه‌حل‌های خود را پیاده‌سازی کنید. همچنین به شما کمک می‌کند تا برنامه خود را با

استانداردهای اخلاقی و قانونی حوزه و ذی‌نفعان خود همساز کنید. با پیروی از چرخه عمر هوش مصنوعی مولد، می‌توانید اطمینان حاصل کنید که برنامه‌تان همیشه ارزش ارائه می‌دهد و کاربران را راضی نگه می‌دارد.

مقدمه

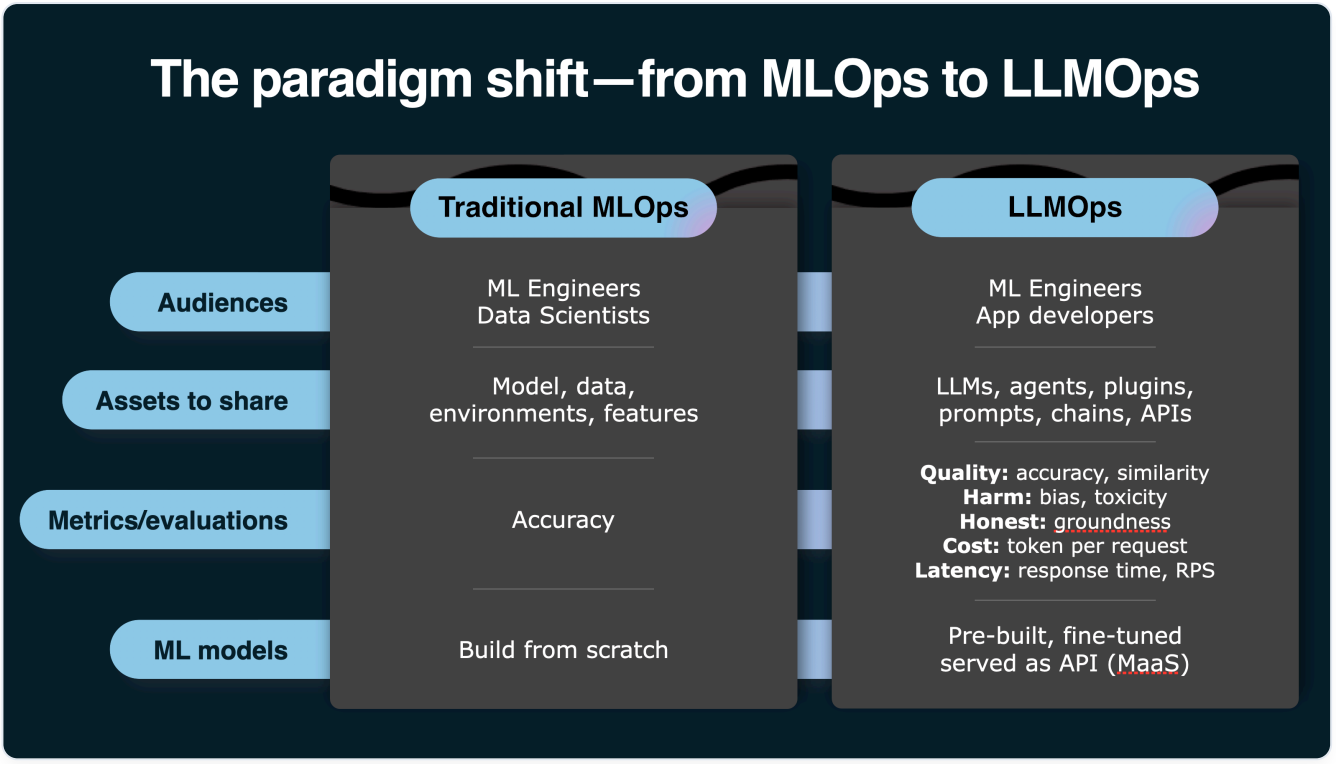
در این فصل، شما:

- درک تغییر پارادایم از MLOps به LLMOps
- چرخه عمر LLM
- ابزارهای چرخه عمر
- مقداردهی و ارزیابی چرخه عمر

درک تغییر پارادایم از MLOps به LLMOps

مدل‌های زبان بزرگ (LLM) ابزاری جدید در زرادخانه هوش مصنوعی هستند، که در وظایف تحلیلی و تولید در برنامه‌ها بسیار قدرتمندند، اما این قدرت تبعاتی در نحوه ساده‌سازی وظایف هوش مصنوعی و یادگیری ماشین کلاسیک دارد.

با این شرایط، ما به یک پارادایم جدید نیاز داریم تا این ابزار را به صورت پویا و با انگیزه‌های درست تطبیق دهیم. می‌توانیم برنامه‌های قدیمی‌تر هوش مصنوعی را «برنامه‌های یادگیری ماشین» و برنامه‌های جدیدتر را «برنامه‌های هوش مصنوعی مولد» یا فقط «برنامه‌های هوش مصنوعی» دسته‌بندی کنیم که بازتاب فناوری‌ها و تکنیک‌های رایج در زمان خود است. این تغییر روایت ما را به چندین روش تحت تأثیر قرار می‌دهد، به مقایسه زیر نگاه کنید.



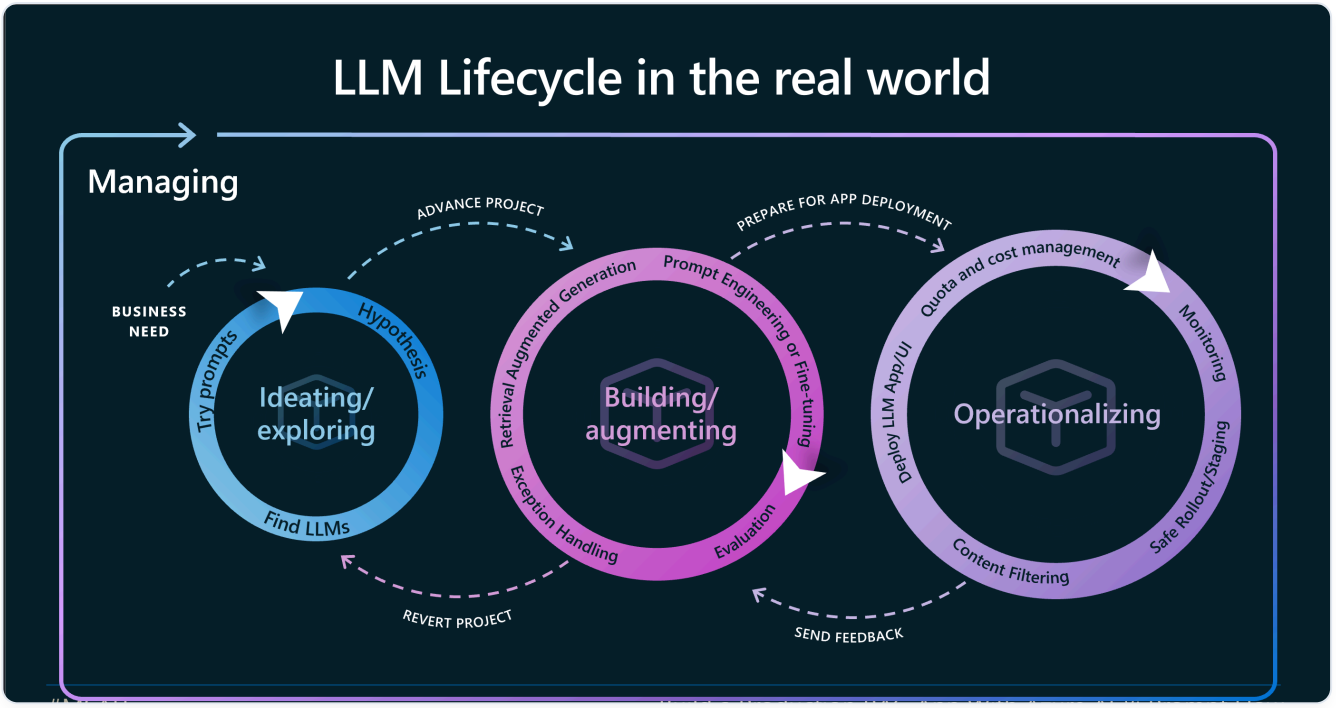
مقایسه LLMOps و MLOps

توجه کنید که در LLMOps، تمرکز بیشتری بر توسعه‌دهندگان برنامه داریم، استفاده از ادغام‌ها به عنوان نقطه کلیدی، بهره‌گیری از «مدل‌ها به عنوان سرویس» و تمرکز بر نکات زیر برای معیارها داریم.

- کیفیت: کیفیت پاسخ
- آسیب: هوش مصنوعی مسئولانه
- صداقت: مبنای پاسخ (معنی دارد؟ آیا درست است؟)
- هزینه: بودجه راه‌حل
- زمان تأخیر: متوسط زمان پاسخ برای هر توکن

چرخه عمر LLM

ابتدا، برای درک چرخه عمر و تغییرات، به این اینفوگرافیک توجه کنید.



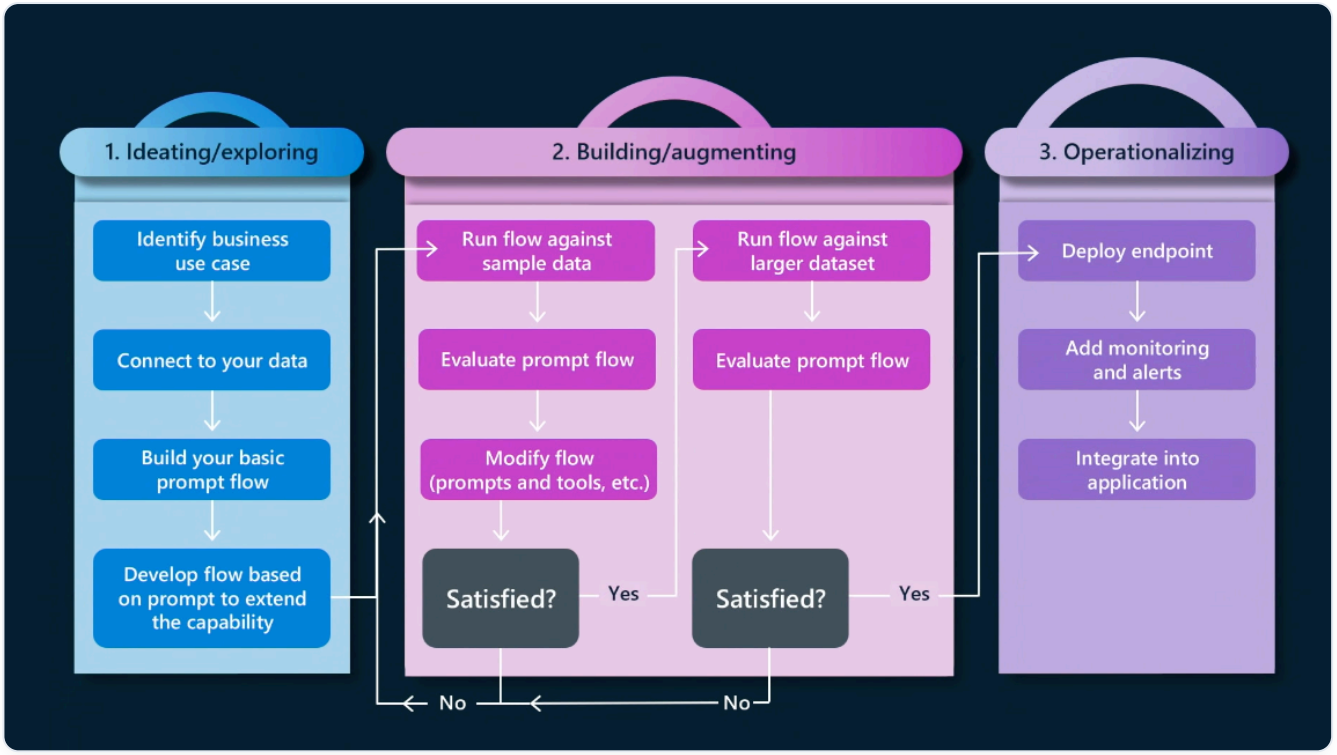
اینفوگرافیک LLMOps

همان‌طور که دیده‌اید، این با چرخه‌های معمول MLOps متفاوت است. مدل‌های زبان بزرگ نیازمندی‌های جدیدی دارند، مانند طراحی پرامپت، تکنیک‌های مختلف برای بهبود کیفیت (تنظیم دقیق، RAG، متا-پرامپت‌ها)، ارزیابی متفاوت و مسئولیت‌پذیری با هوش مصنوعی مسئول، و در نهایت معیارهای ارزیابی جدید (کیفیت، آسیب، صداقت، هزینه و تأخیر).

به عنوان مثال، ببینید چگونه ایده‌پردازی می‌کنیم. با مهندسی پرامپت به آزمایش با مدل‌های مختلف زبان بزرگ می‌پردازیم تا ببینیم فرضیه‌هایشان می‌تواند درست باشد یا خیر.

توجه کنید که این روند خطی نیست بلکه حلقه‌های یکپارچه، تکراری و با یک چرخه کلی است.

چگونه می‌توانیم این مراحل را بررسی کنیم؟ بیایید با جزییات ببینیم چگونه می‌توانیم یک چرخه عمر بسازیم.



روند کاری LLMOps

ممکن است کمی پیچیده به نظر برسد، ابتدا روی سه مرحله بزرگ تمرکز کنیم.

1. ایده‌پردازی/کاوش: در اینجا می‌توانیم بر اساس نیازهای تجاری خود کاوش کنیم. نمونه‌سازی، ایجاد یک PromptFlow و تست اینکه آیا برای فرضیه ما کافی است یا خیر.
2. ساخت/افزایش: اجرا، اکنون شروع به ارزیابی داده‌های بزرگ‌تر و اجرای تکنیک‌هایی مانند تنظیم دقیق و RAG برای بررسی مقاومت راه‌حل می‌کنیم. اگر پاسخگو نیست، بازپیاده‌سازی، افزودن مراحل جدید در جریان یا بازسازی داده‌ها ممکن است کمک کند. پس از آزمایش جریان و مقیاس‌پذیری، اگر کار کرد و معیارها را برآورده کرد، برای مرحله بعدی آماده است.
3. عملیاتی‌سازی: ادغام، اکنون افزودن سیستم‌های نظارت و هشدار به سامانه، استقرار و ادغام برنامه.

سپس، چرخه مدیریتی کلی داریم که بر امنیت، انطباق و حاکمیت تمرکز دارد.

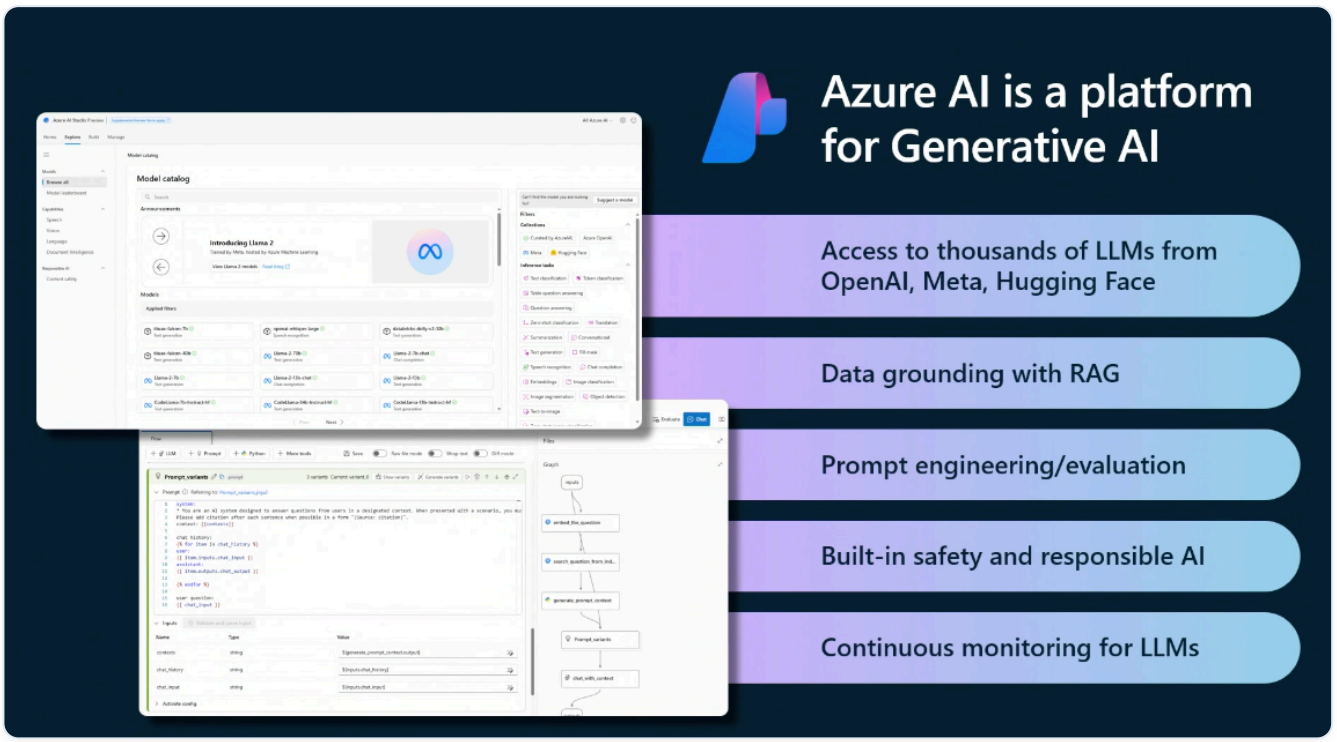
تبریک می‌گوییم، حالا برنامه هوش مصنوعی شما آماده استفاده و عملیاتی است. برای تجربه عملی، نگاهی بیندازید به نمایش چت Contoso.

حال، چه ابزارهایی می‌توانیم استفاده کنیم؟

ابزارهای چرخه عمر

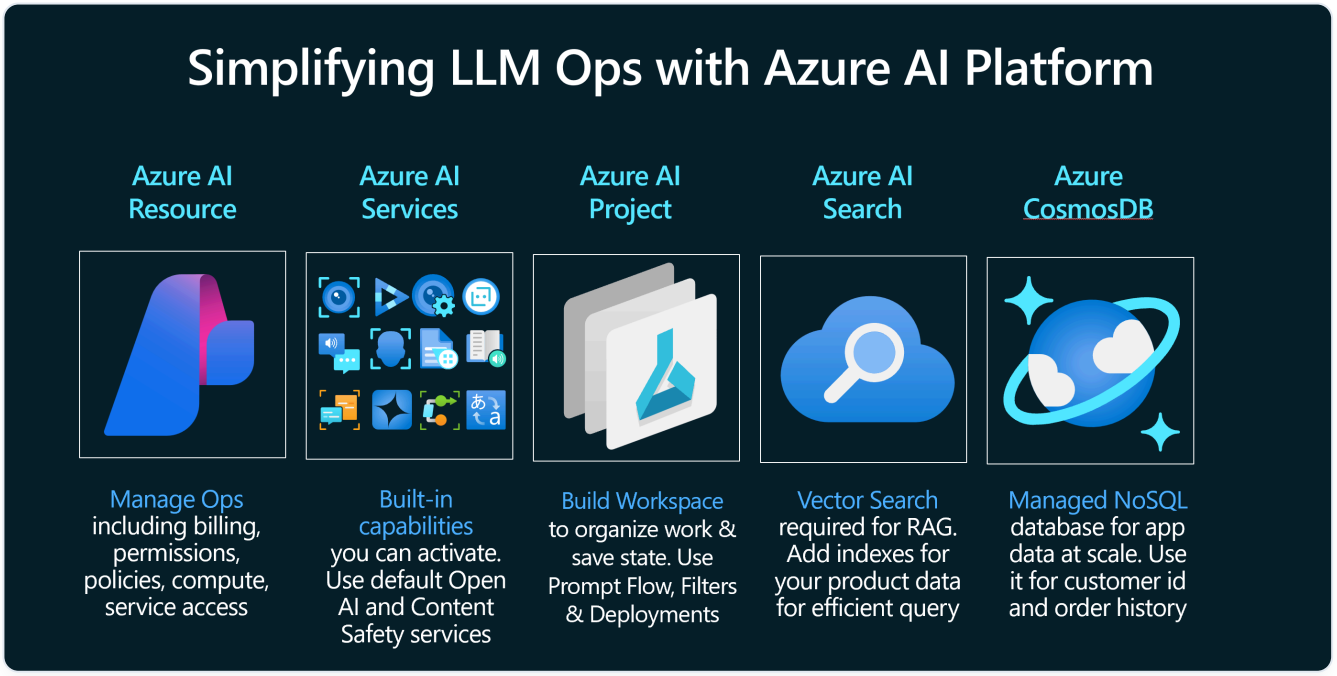
برای ابزارها، مایکروسافت پلتفرم هوش مصنوعی آزور و PromptFlow را ارائه می‌دهد تا چرخه شما را آسان و آماده اجرا کند.

پلتفرم هوش مصنوعی آزور به شما امکان می‌دهد از **Microsoft Foundry** استفاده کنید. Microsoft Foundry (که قبلاً Azure AI Studio بود) یک پورتال وب است که به شما امکان می‌دهد مدل‌ها، نمونه‌ها و ابزارها را کاوش کنید، منابع خود را مدیریت کنید و از جریان‌های توسعه رابط کاربری و همچنین گزینه‌های SDK/CLI برای توسعه کد-محور بهره ببرید.



امکانات Azure AI

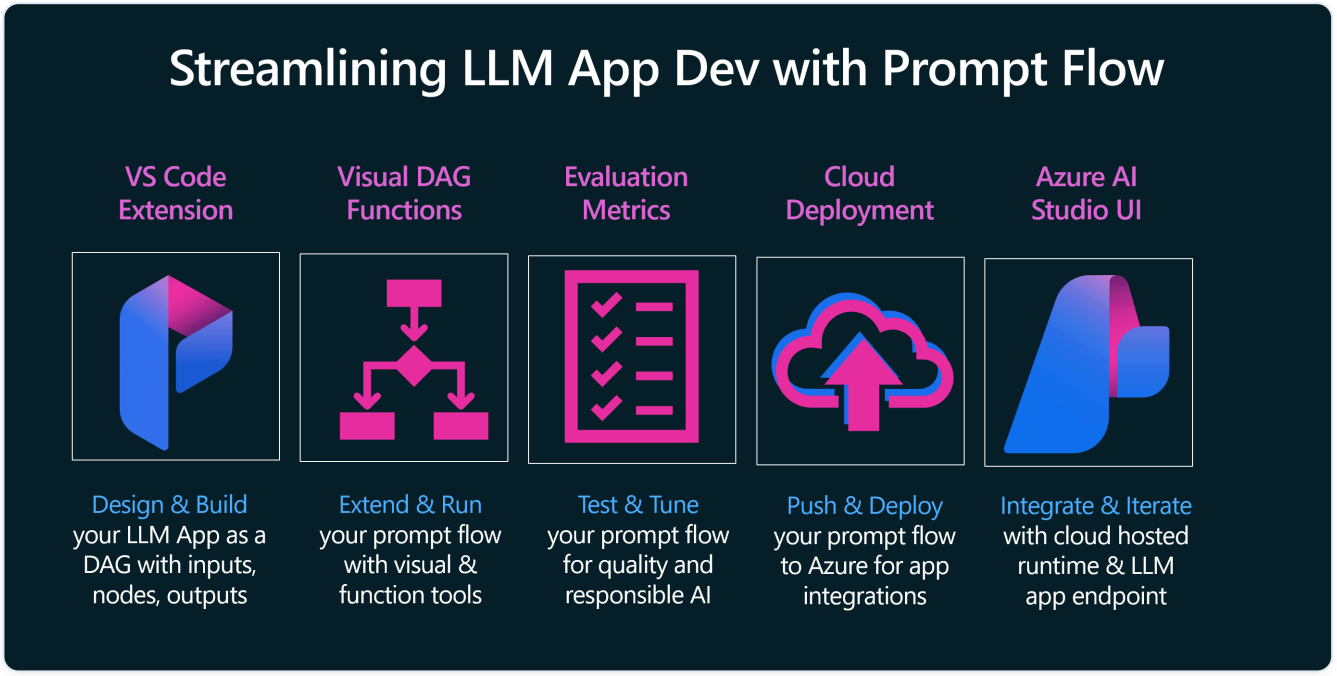
آزور AI به شما اجازه می‌دهد منابع متعددی را برای مدیریت عملیات، خدمات، پروژه‌ها، جستجوی برداری و نیازهای پایگاه داده استفاده کنید.



Azure AI با LLMOps

از اثبات مفهوم (POC) تا برنامه‌های مقیاس بزرگ با PromptFlow بسازید:

- طراحی و ساخت برنامه‌ها از VS Code، با ابزارهای بصری و عملکردی
- تست و تنظیم دقیق برنامه‌ها برای هوش مصنوعی با کیفیت، به آسانی.
- استفاده از Microsoft Foundry برای ادغام و تکرار با ابر، فشار و استقرار برای ادغام سریع.



PromptFlow با LLMOps

عالی! یادگیری خود را ادامه دهید!

شگفت‌انگیز است، اکنون بیشتر بیاموزید که چگونه یک برنامه را ساختاربندی کنیم تا مفاهیم را با **برنامه چت Contoso** به کار ببریم، تا ببینید چگونه Cloud Advocacy این مفاهیم را در نمایش‌ها اضافه می‌کند. برای محتوای بیشتر، جلسه تفکیکی **Ignit e را بررسی کنید!**

حالا، درس ۱۵ را بررسی کنید تا بفهمید چگونه **تولید افزوده بازیابی‌شده و پایگاه‌داده‌های برداری** روی هوش مصنوعی مولد تأثیر می‌گذارند و برنامه‌های جذاب‌تری بسازید!

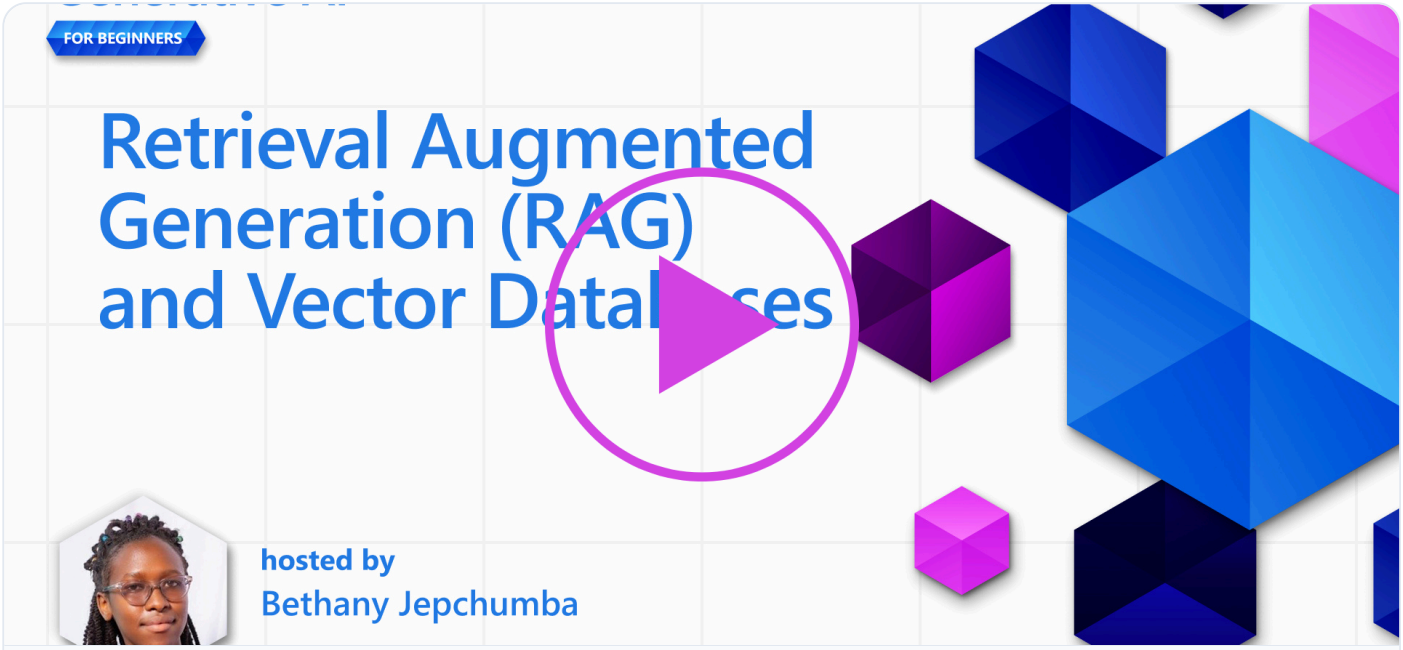
سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی **Co-op Translator** ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب
درس ۱۴ — هوش مصنوعی مولد مایکروسافت

درس ۱۵ از ۲۱

تولید تقویت‌شده با بازیابی (RAG) و پایگاه‌های داده برداری

معماری جامع RAG، قطعه‌بندی مستندات (Chunking)، اندیس‌گذاری برداری، بازیابی اطلاعات و تولید پاسخ بر پایه دانش اختصاصی سازمان



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/4l8zhHUBeyI?si=BmvDmL1fnHtgQYkL

▶️ پخش در یوتیوب

اسکن کد QR

در درس برنامه‌های جستجو، به طور مختصر یاد گرفتیم چگونه داده‌های خود را در مدل‌های زبانی بزرگ (LLMها) ادغام کنیم. در این درس، بیشتر به مفاهیم پایه‌ای کردن داده‌ها در برنامه LLM خود، مکانیک فرآیند و روش‌های ذخیره داده‌ها، از جمله تعبیه‌ها و متن، می‌پردازیم.

💡 **نکته کاربردی**

ویدئو به زودی منتشر می‌شود

مقدمه

در این درس موارد زیر را پوشش خواهیم داد:

- معرفی RAG، چیستی آن و دلیل استفاده از آن در هوش مصنوعی (artificial intelligence).
- آشنایی با پایگاه‌های داده برداری و ساخت یکی برای برنامه خودمان.
- یک مثال عملی از نحوه ادغام RAG به درون یک برنامه.

اهداف یادگیری

پس از اتمام این درس، قادر خواهید بود:

- اهمیت RAG در بازیابی و پردازش داده‌ها را توضیح دهید.
- راه‌اندازی برنامه RAG و پایه‌گذاری داده‌های خود در یک LLM
- ادغام موثر RAG و پایگاه‌های داده برداری در برنامه‌های LLM.

سناریوی ما: بهبود LLMهای خود با داده‌های خودمان

برای این درس، می‌خواهیم یادداشت‌های خود را به یک استارتاپ آموزشی اضافه کنیم که اجازه می‌دهد چت‌بات اطلاعات بیشتری درباره موضوعات مختلف کسب کند. با استفاده از یادداشت‌هایی که داریم، دانش‌آموزان قادر خواهند بود بهتر مطالعه کنند و موضوعات مختلف را بفهمند که مرور برای امتحاناتشان را آسان‌تر می‌کند. برای ایجاد سناریوی خود، از موارد زیر استفاده می‌کنیم:

- `Azure OpenAI:` مدلی که برای ساخت چت‌بات خود استفاده خواهیم کرد
- `در مورد شبکه‌های عصبی درس AI for beginners` : این داده‌ای است که LLM ما براساس آن پایه‌گذاری خواهد شد
- `Azure AI Search` و `Azure Cosmos DB:` پایگاه داده برداری برای ذخیره داده‌ها و ساخت نمایه جستجو

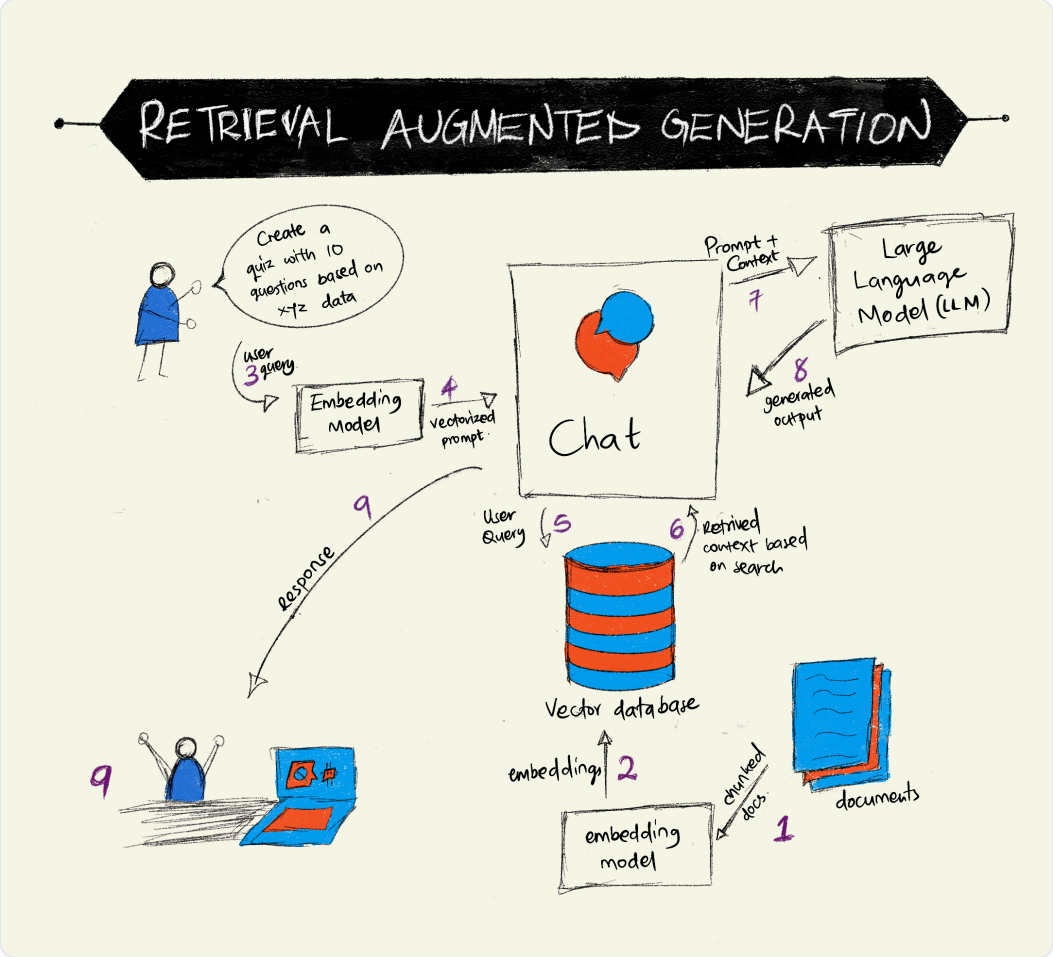
کاربران قادر خواهند بود آزمون‌های تمرینی از یادداشت‌های خود بسازند، کارت‌های مرور بسازند و آن‌ها را به خلاصه‌های مختصر تبدیل کنند. برای شروع، بیایید ببینیم RAG چیست و چگونه کار می‌کند:

تولید تقویت‌شده با بازیابی (RAG)

یک چت‌بات مبتنی بر LLM درخواست‌های کاربری را پردازش می‌کند تا پاسخ تولید کند. هدف آن تعاملی بودن و گفتگو با کاربران در موضوعات مختلف است. اما پاسخ‌های آن محدود به زمینه داده شده و داده‌های آموزشی پایه‌اش است. مثلاً آخرین دانش

موجود در GPT-4 مربوط به سپتامبر ۲۰۲۱ است، یعنی دانش آن از رویدادهای پس از این زمان بی‌بهره است. همچنین داده‌های استفاده شده برای آموزش LLMها شامل اطلاعات محرمانه مانند یادداشت‌های شخصی یا دفترچه راهنمای محصولات یک شرکت نیست.

نحوه کار RAG (تولید تقویت‌شده با بازیابی)

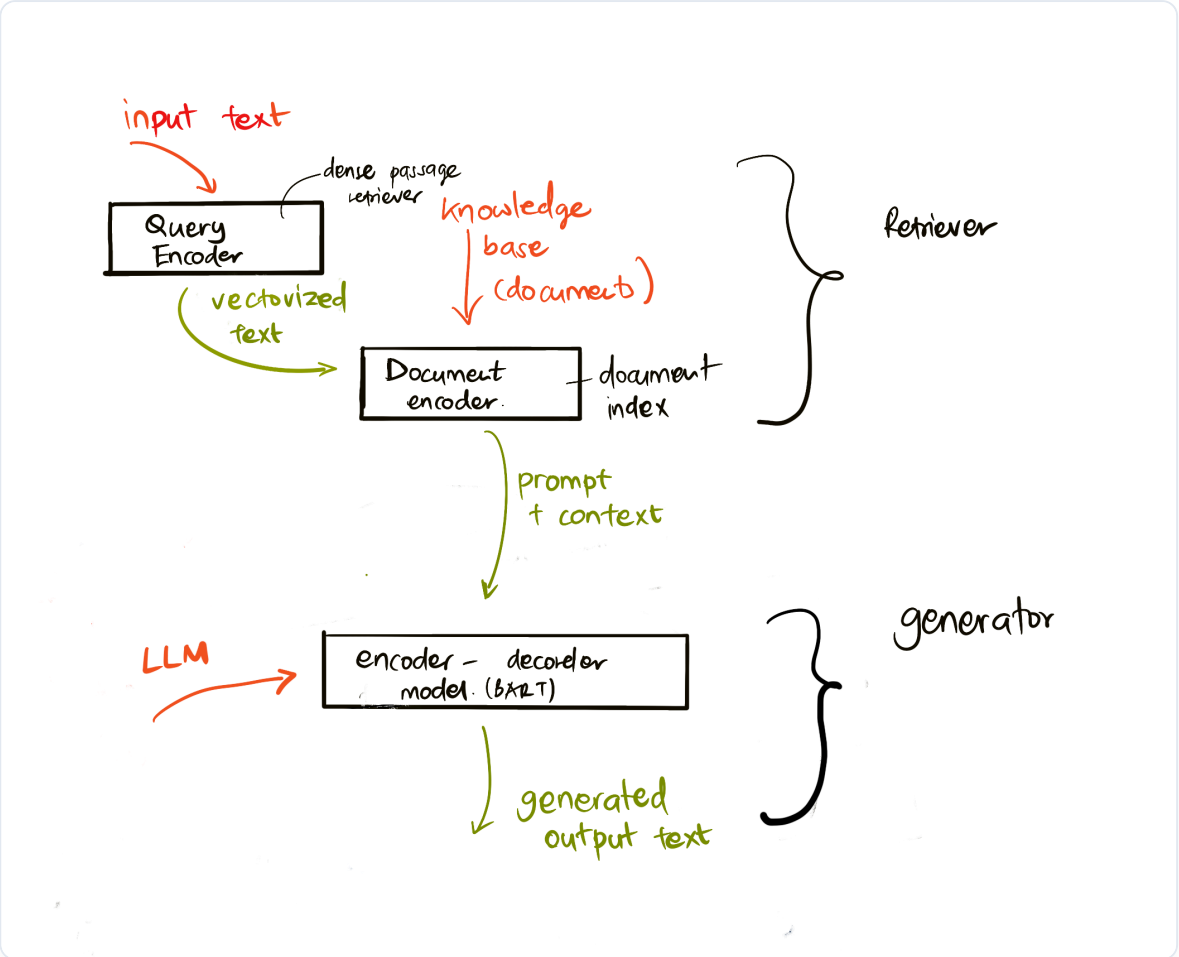


تصویری نشان‌دهنده نحوه کار RAG

فرض کنید می‌خواهید چت‌باتی راه‌اندازی کنید که از یادداشت‌های شما آزمون بسازد، به اتصال به پایگاه دانش نیاز خواهید داشت. اینجاست که RAG به کمک می‌آید. RAGها به شرح زیر عمل می‌کنند:

- **پایگاه دانش:** قبل از بازیابی، این اسناد باید وارد و پیش‌پردازش شوند، که معمولاً شامل خرد کردن اسناد بزرگ به بخش‌های کوچکتر، تبدیل آن‌ها به تعبیه متن و ذخیره در پایگاه داده است.
- **پرس‌وجوی کاربر:** کاربر یک سوال می‌پرسد
- **بازیابی:** وقتی کاربر سوال می‌کند، مدل تعبیه اطلاعات مرتبط را از پایگاه دانش ما بازیابی می‌کند تا زمینه بیشتری ارائه دهد که در درخواست وارد می‌شود.

- **تولید تقویت‌شده:** LLM پاسخ خود را بر اساس داده‌های بازیابی شده بهبود می‌بخشد. این اجازه می‌دهد پاسخ نه فقط بر اساس داده‌های پیش‌آموزش باشد بلکه اطلاعات مرتبط از زمینه اضافی نیز وارد شود. داده‌های بازیابی شده برای تقویت پاسخ‌های LLM استفاده می‌شوند. سپس LLM پاسخ سوال کاربر را بازمی‌گرداند.



تصویری نشان‌دهنده معماری RAG

معماری RAG با استفاده از ترنسفورمرهایی متشکل از دو بخش: رمزگذار و رمزگشا پیاده‌سازی می‌شود. به عنوان مثال، وقتی کاربر سوالی می‌پرسد، متن ورودی به بردارهایی که معنی کلمات را در خود دارند "رمزگذاری" می‌شود و سپس این بردارها به نمایه اسناد ما "رمزگشایی" و متن جدیدی بر اساس سوال کاربر تولید می‌شود. LLM از مدل رمزگذار-رمزگشا برای تولید خروجی استفاده می‌کند.

دو روش برای پیاده‌سازی RAG طبق مقاله پیشنهاد شده: Retrieval-Augmented Generation for Knowledge intensive NLP (natural language processing software) Tasks وجود دارد:

- ***RAG-Sequence*** با استفاده از اسناد بازیابی شده برای پیش‌بینی بهترین پاسخ ممکن به سوال کاربر
- **RAG-Token** با استفاده از اسناد برای تولید توکن بعدی، سپس بازیابی آن‌ها برای پاسخ به سوال کاربر

چرا باید از RAG استفاده کنید؟

- **غنای اطلاعات:** اطمینان می‌دهد پاسخ‌های متنی به‌روز و جاری باشند. بنابراین عملکرد را در وظایف خاص حوزه با دسترسی به پایگاه دانش داخلی بهبود می‌بخشد.
- کاهش ساختگی بودن با استفاده از **داده‌های قابل تأیید** در پایگاه دانش برای ارائه زمینه به سوالات کاربران.
- **اقتصادی بودن:** چون نسبت به تنظیم مجدد (fine-tuning) یک LLM هزینه کمتری دارد.

ایجاد پایگاه دانش

برنامه ما بر مبنای داده‌های شخصی ما است، یعنی درس شبکه عصبی در برنامه AI For Beginners.

پایگاه‌های داده برداری

پایگاه داده برداری، برخلاف پایگاه داده‌های سنتی، یک پایگاه تخصصی است که برای ذخیره، مدیریت و جستجوی بردارهای تعبیه‌شده طراحی شده است. این پایگاه داده نمایه‌های عددی اسناد را ذخیره می‌کند. تجزیه داده‌ها به تعبیه‌های عددی باعث می‌شود سیستم هوش مصنوعی ما راحت‌تر داده‌ها را بفهمد و پردازش کند.

ما تعبیه‌های خود را در پایگاه داده‌های برداری ذخیره می‌کنیم چون LLMها محدودیتی در تعداد توکن‌های ورودی دارند. از آنجا که نمی‌توانید تمام تعبیه‌ها را به یک LLM ارسال کنید، باید آنها را به بخش‌های کوچکتر تقسیم کنیم و وقتی کاربر سوالی می‌کند، تعبیه‌های نزدیک‌ترین به سوال همراه با درخواست بازگردانده می‌شوند. خرد کردن همچنین هزینه‌های تعداد توکن‌های ارسال شده به LLM را کاهش می‌دهد.

بعضی از پایگاه داده‌های برداری محبوب شامل Qdrant، ScaNN، Chromadb، Pinecone، Clarifyai، Azure Cosmos DB و DeepLake هستند. می‌توانید یک مدل Azure Cosmos DB با استفاده از Azure CLI با فرمان زیر ایجاد کنید:

Terminal / Bash

```
az login
az group create -n <resource-group-name> -l <location>
az cosmosdb create -n <cosmos-db-name> -r <resource-group-name>
az cosmosdb list-keys -n <cosmos-db-name> -g <resource-group-name>
```

تبدیل متن به تعبیه‌ها

قبل از ذخیره داده‌ها، باید آن‌ها را به تعبیه‌های برداری تبدیل کنیم. اگر با اسناد بزرگ یا متن‌های طولانی کار می‌کنید، می‌توانید آن‌ها را بر اساس پرسش‌هایی که انتظار دارید خرد کنید. خرد کردن می‌تواند در سطح جملات یا پاراگراف انجام شود. از آنجا که خرد کردن معانی را از کلمات اطراف استخراج می‌کند، می‌توانید زمینه دیگری به یک بخش اضافه کنید، مثلاً عنوان سند یا متنی قبل یا بعد از بخش. داده‌ها را می‌توان به صورت زیر خرد کرد:

Python

```
def split_text(text, max_length, min_length):
    words = text.split()
    chunks = []
    current_chunk = []

    for word in words:
        current_chunk.append(word)
        if len(' '.join(current_chunk)) < max_length and len(' '.join(current_chunk)) >
min_length:
            chunks.append(' '.join(current_chunk))
            current_chunk = []

    # اگر بخش آخر به حداقل طول نرسیده بود، آن را به هر حال اضافه کن
    if current_chunk:
        chunks.append(' '.join(current_chunk))

    return chunks
```

پس از خرد کردن، می‌توانیم متن را با استفاده از مدل‌های مختلف تعبیه کنیم. برخی مدل‌هایی که می‌توانید استفاده کنید شامل: word2vec، ada-002 از OpenAI، Azure Computer Vision و بسیاری دیگر است. انتخاب مدل بستگی به زبان‌های مورد استفاده، نوع محتوای رمزگذاری شده (متن/تصاویر/صدا)، اندازه ورودی قابل رمزگذاری و طول خروجی تعبیه دارد.

مثالی از متن تعبیه شده با استفاده از مدل `text-embedding-ada-002` از OpenAI:

```
cat = create_embeddings("cat")
cat
```

[21] ✓ 0.6s

```
[-0.0070945825427770615,
 -0.017328109592199326,
 -0.009644086472690105,
 -0.03070768155157566,
 -0.012548675760626793,
 0.003105211304500699,
 -0.005113212391734123,
 -0.04121817275881767,
 -0.014629469253122807,
 -0.021376069635152817,
 0.019231360405683517,
 0.05087646469473839,
 -0.0012907310156151652,
 0.0024855893570929766,
 -0.03840590640902519,
 -0.006089693866670132,
 0.0355084203183651,
 -0.004697763826698065,
 0.0023630852811038494,
 -0.013429284095764l6,
 -0.01891888678073883,
 0.009019138291478157,
 0.015893569216132164,
 -0.008713766001164913,
 -0.014672079123556614,
...
```

بازیابی و جستجوی برداری

وقتی کاربر سوالی می‌پرسد، قسمت بازیابی آن را با استفاده از رمزگذار پرسش به بردار تبدیل می‌کند، سپس در نمایه جستجوی اسناد به دنبال بردارهای مرتبط می‌گردد. پس از آن، بردار ورودی و بردارهای سند را به متن تبدیل و به LLM ارسال می‌کند.

بازیابی

بازیابی زمانی اتفاق می‌افتد که سیستم سعی می‌کند به سرعت اسنادی را از نمایه پیدا کند که معیارهای جستجو را برآورده کنند. هدف بازیابی، دریافت اسنادی است که برای ارائه زمینه و پایه‌گذاری LLM روی داده‌های شما استفاده می‌شوند.

چند روش برای جستجو در پایگاه داده داریم مانند:

- **جستجوی کلمه کلیدی** - برای جستجوهای متنی استفاده می‌شود
- **جستجوی برداری** - اسناد را از متن به بردار تبدیل می‌کند با استفاده از مدل‌های تعبیه، اجازه یک **جستجوی معنایی** با استفاده از مفهوم کلمات را می‌دهد. بازیابی با پرس‌وجوی اسنادی انجام می‌شود که بردارهایشان به سوال کاربر نزدیک‌تر است.
- **ترکیبی** - ترکیبی از جستجوی کلمه کلیدی و برداری.

یک چالش بازیابی وقتی است که پاسخی مشابه در پایگاه داده وجود نداشته باشد، سیستم بهترین اطلاعات ممکن را بازمی‌گرداند، اما می‌توانید با تنظیم فاصله حداکثر برای مرتبط بودن یا استفاده از جستجوی ترکیبی که کلمه کلیدی و بردار را با هم دارد، بهبود بخشید. در این درس از جستجوی ترکیبی استفاده می‌کنیم، ترکیبی از جستجوی برداری و کلمه کلیدی. داده‌ها را در یک داده‌فریم با ستون‌هایی که شامل بخش‌ها و تعبیه‌ها هستند ذخیره می‌کنیم.

شباهت برداری

بازیابی در پایگاه دانش به دنبال تعبیه‌هایی است که به هم نزدیک‌ترند، به همسایه نزدیک‌ترین، چون متن‌هایی هستند که شبیه هم‌اند. در سناریو وقتی کاربر سوالی می‌پرسد، ابتدا تعبیه می‌شود سپس با تعبیه‌های مشابه مطابقت داده می‌شود. معیار معمول استفاده شده برای اندازه‌گیری شباهت بردارها، تشابه کسینوسی است که بر اساس زاویه بین دو بردار محاسبه می‌شود.

می‌توانیم برای اندازه‌گیری شباهت از جایگزین‌های دیگری مانند فاصله اقلیدسی که خط مستقیم بین نقاط پایانی بردار است یا ضرب داخلی که مجموع حاصل ضرب عناصر متناظر دو بردار را اندازه می‌گیرد، استفاده کنیم.

نمایه جستجو

هنگام بازیابی، باید برای پایگاه دانش خود یک نمایه جستجو بسازیم قبل از انجام جستجو. نمایه تعبیه‌ها را ذخیره می‌کند و می‌تواند سریع‌ترین بخش‌های مشابه را حتی در پایگاه داده بزرگ بازیابی کند. می‌توانیم نمایه را به صورت محلی با فرمان زیر بسازیم:

Python

```
from sklearn.neighbors import NearestNeighbors

embeddings = flattened_df['embeddings'].to_list()

# ایجاد شاخص جستجو
nbrs = NearestNeighbors(n_neighbors=5, algorithm='ball_tree').fit(embeddings)

# استفاده کنید kneighbors برای پرس‌وجو از شاخص، می‌توانید از روش
distances, indices = nbrs.kneighbors(embeddings)
```

رتبه‌بندی مجدد

پس از پرس‌وجوی پایگاه داده، ممکن است لازم باشد نتایج را از مرتبط‌ترین مرتب کنید. یک LLM رتبه‌بندی مجدد با استفاده از یادگیری ماشین، ارتباط نتایج جستجو را با مرتب کردن از مرتبط‌ترین بهبود می‌بخشد. با استفاده از Azure AI Search، رتبه‌بندی مجدد به صورت خودکار توسط رتبه‌بند معنایی انجام می‌شود. نمونه‌ای از نحوه کار رتبه‌بندی مجدد با نزدیک‌ترین همسایگان:

Python

```
# پیدا کردن اسناد بیشترین شباهت را دارند
distances, indices = nbrs.kneighbors([query_vector])

index = []
# چاپ اسناد بیشترین شباهت را دارند
for i in range(3):
    index = indices[0][i]
    for index in indices[0]:
        print(flattened_df['chunks'].iloc[index])
        print(flattened_df['path'].iloc[index])
        print(flattened_df['distances'].iloc[index])
    else:
        print(f"Index {index} not found in DataFrame")
```

جمع‌بندی همه چیز

آخرین مرحله افزودن LLM خود به ترکیب است تا بتوان پاسخ‌هایی گرفت که بر اساس داده‌های ما پایه‌گذاری شده باشند. می‌توانیم به این صورت پیاده‌سازی کنیم:

Python

```
user_input = "what is a perceptron?"

def chatbot(user_input):
    # تبدیل سوال به یک بردار پرس و جو
    query_vector = create_embeddings(user_input)

    # یافتن مستندات مشابه‌ترین
    distances, indices = nbrs.kneighbors([query_vector])

    # افزودن مستندات به پرس و جو برای ارائه زمینه
    history = []
    for index in indices[0]:
        history.append(flattened_df['chunks'].iloc[index])

    # ترکیب تاریخچه و ورودی کاربر
    history.append(user_input)

    # ایجاد یک شی پیام
    messages=[
        {"role": "system", "content": "You are an AI assistant that helps with AI
questions."},
        {"role": "user", "content": "\n\n".join(history) }
    ]

    # پاسخ‌ها برای تولید پاسخ API استفاده از
    response = client.responses.create(
        model="gpt-5-mini",
        max_output_tokens=800,
        input=messages,
        store=False,
    )

    return response.output_text

chatbot(user_input)
```

ارزیابی برنامه ما

معیارهای ارزیابی

- کیفیت پاسخ‌های ارائه شده با تضمین اینکه طبیعی، روان و انسانی به نظر برسند
- پایه‌گذاری داده‌ها: ارزیابی اینکه پاسخ از اسناد ارائه‌شده آمده است یا خیر
- ارتباط: ارزیابی اینکه پاسخ مرتبط و مطابق با سوال مطرح شده باشد
- روانی - اینکه پاسخ از نظر گرامری معنی داشته باشد

موارد استفاده از RAG و پایگاه‌های داده برداری

موارد استفاده زیادی وجود دارد که تماس‌های تابع می‌توانند برنامه شما را بهبود بخشند، مانند:

- پرسش و پاسخ: پایه‌گذاری داده‌های شرکت خود روی چتی که توسط کارکنان برای پرسیدن سوالات استفاده می‌شود.
- سیستم‌های توصیه‌گر: جایی که می‌توانید سیستمی بسازید که بیشترین مقادیر مشابه را تطبیق دهد مثل فیلم‌ها، رستوران‌ها و غیره.
- خدمات چت‌بات: می‌توانید تاریخچه چت را ذخیره کرده و گفتگو را بر اساس داده‌های کاربر شخصی‌سازی کنید.
- جستجوی تصویر بر اساس تعبیه‌های برداری، مفید در شناسایی تصویر و تشخیص ناهنجاری.

خلاصه

ما بخش‌های اساسی RAG را از افزودن داده‌ها به برنامه، پرس‌وجوی کاربر و خروجی پوشش دادیم. برای ساده‌سازی ساخت RAG، می‌توانید از چارچوب‌هایی مانند Langchain، Semanti Kernel یا Autogen استفاده کنید.

تمرین

برای ادامه یادگیری تولید تقویت‌شده با بازیابی (RAG) می‌توانید بسازید:

- یک فرانت‌اند برای برنامه با چارچوب دلخواه خود بسازید
- از چارچوبی مثل LangChain یا Semantic Kernel استفاده کنید و برنامه خود را دوباره بسازید.

تبریک می‌گوییم بابت اتمام این درس 👏 .

یادگیری اینجا متوقف نمی‌شود، ادامه دهید

پس از اتمام این درس، مجموعه یادگیری Generative AI ما را بررسی کنید تا دانش خود در زمینه هوش مصنوعی مولد را افزایش دهید!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

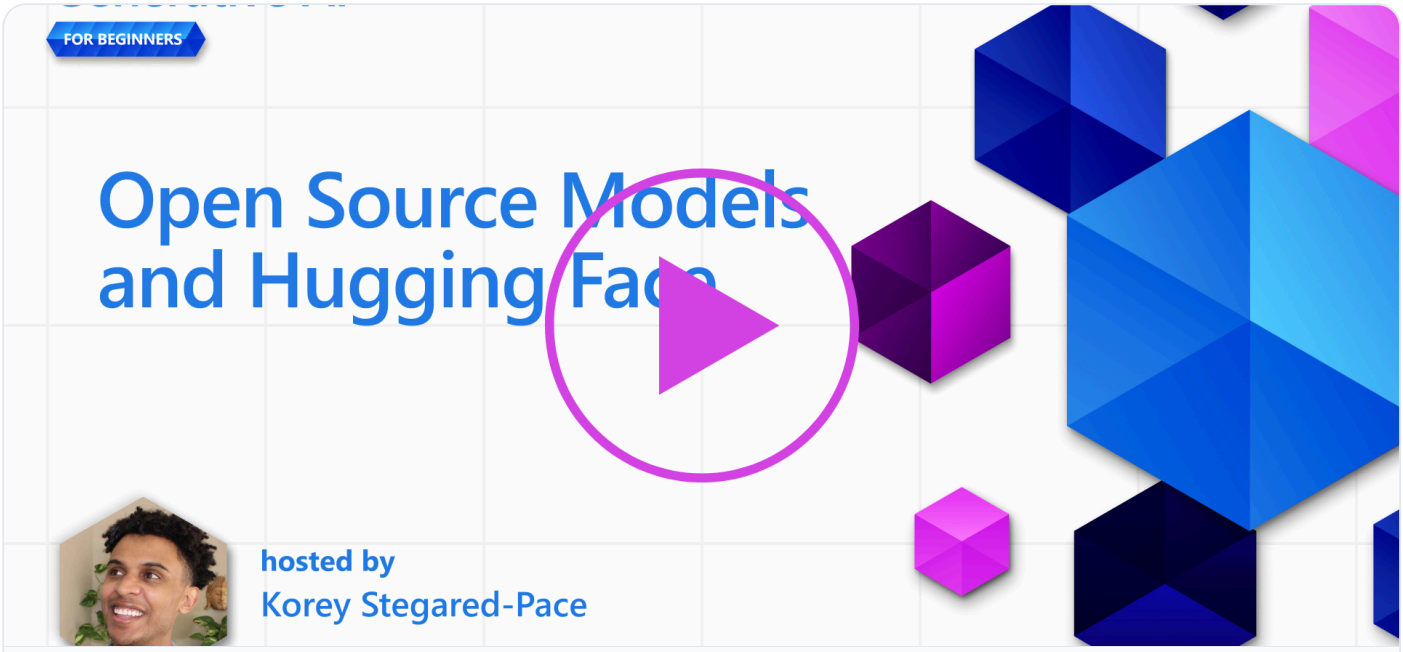
بازگشت به فهرست مطالب

درس ۱۵ — هوش مصنوعی مولد مایکروسافت

درس ۱۶ از ۲۱

مدل‌های متن‌باز و اکوسیستم Hugging Face

استفاده از مدل‌های متن‌باز روی پلتفرم Hugging Face، انتخاب مدل‌های سبک و استقرار بر روی زیرساخت‌های ابری و محلی



ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/CuICgfuHFSg?si=x8SpFRUsIxM9dohN

پخش در یوتیوب

اسکن کد QR

مقدمه

دنیای مدل‌های زبانی بزرگ متن باز هیجان‌انگیز و دائما در حال پیشرفت است. هدف این درس ارائه نگاهی عمیق به مدل‌های متن باز است. اگر به دنبال اطلاعاتی درباره مقایسه مدل‌های اختصاصی با مدل‌های متن باز هستید، به درس "بررسی و مقایسه مدل‌های مختلف LLM" مراجعه کنید. این درس همچنین موضوع ریزتنظیم را پوشش می‌دهد اما توضیح مفصل‌تر را می‌توانید در درس "ریزتنظیم مدل‌های LLM" بیابید.

اهداف یادگیری

- کسب درک مدل‌های متن باز
- شناخت مزایای کار با مدل‌های متن باز

- بررسی مدل‌های متن بازی که در Hugging Face و کاتالوگ مدل Microsoft Foundry موجود است

مدل‌های متن باز چیستند؟

نرم‌افزار متن باز نقش مهمی در رشد فناوری در حوزه‌های مختلف داشته است. ابتکار متن باز (OSI) **۱۰ معیار را برای نرم‌افزار** برای طبقه‌بندی به عنوان متن باز تعریف کرده است. کد منبع باید تحت مجوزی که توسط OSI تایید شده است، به صورت آشکار به اشتراک گذاشته شود.

اگرچه فرآیند توسعه مدل‌های زبانی بزرگ مشابه توسعه نرم‌افزار است، اما دقیقا یکسان نیست. این موضوع بحث‌های زیادی در جامعه پیرامون تعریف متن باز در زمینه LLMها به وجود آورده است. برای اینکه یک مدل مطابق تعریف سنتی متن باز باشد، اطلاعات زیر باید به‌صورت عمومی در دسترس قرار گیرد:

- مجموعه داده‌های استفاده شده برای آموزش مدل.
- وزن‌های کامل مدل به همراه آموزش.
- کد ارزیابی.
- کد ریزتنظیم.
- وزن کامل مدل و معیارهای آموزش.

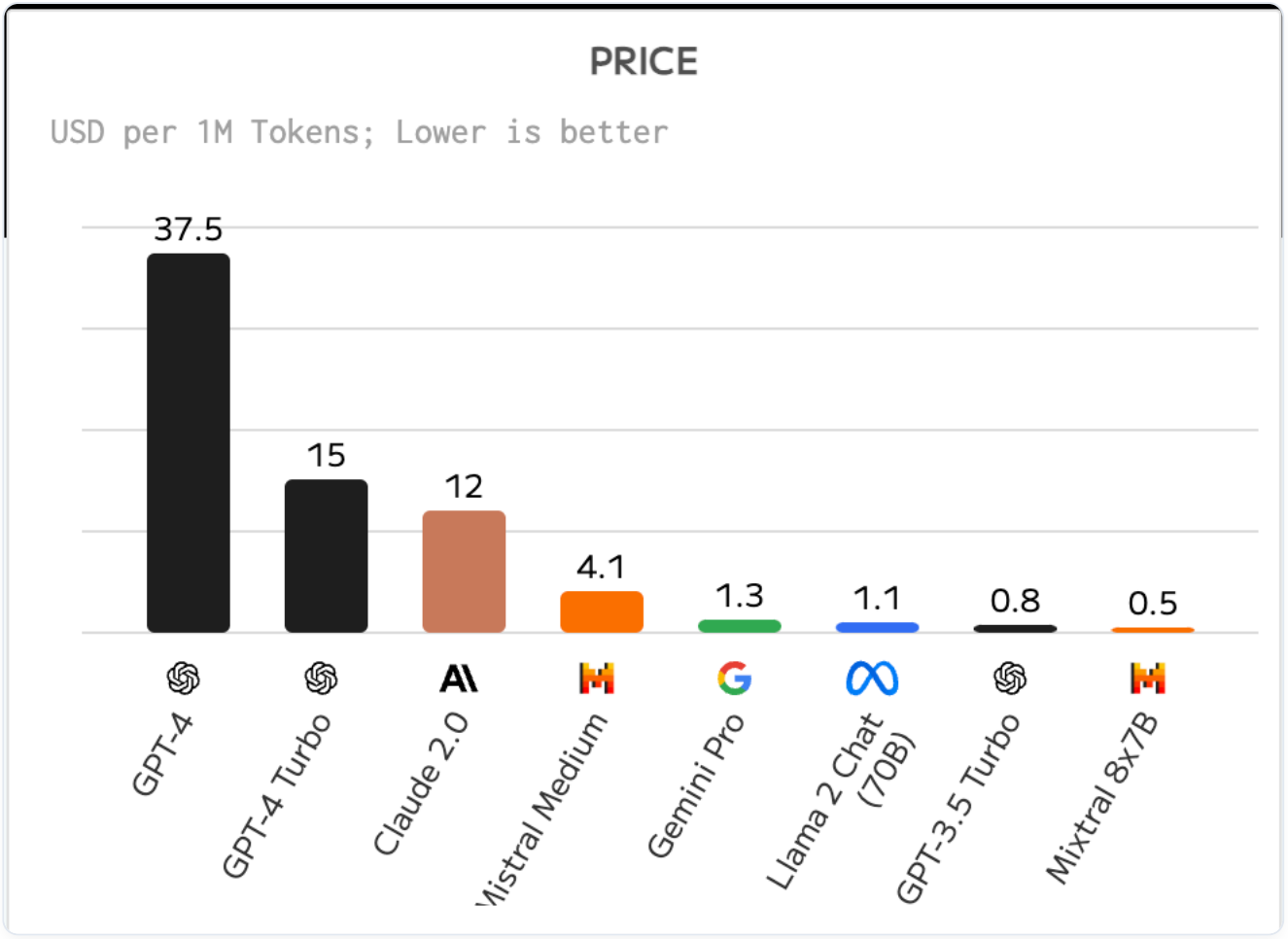
در حال حاضر فقط تعداد کمی مدل این معیارها را دارند. مدل **OLMo که توسط مؤسسه آلن هوش مصنوعی (AllenAI) ساخت** **ه شده است** نمونه‌ای از این دسته است.

در این درس، به مدل‌ها به عنوان "مدل‌های متن باز" اشاره خواهیم کرد زیرا ممکن است در زمان نگارش این درس با معیارهای بالا کاملا مطابقت نداشته باشند.

مزایای مدل‌های متن باز

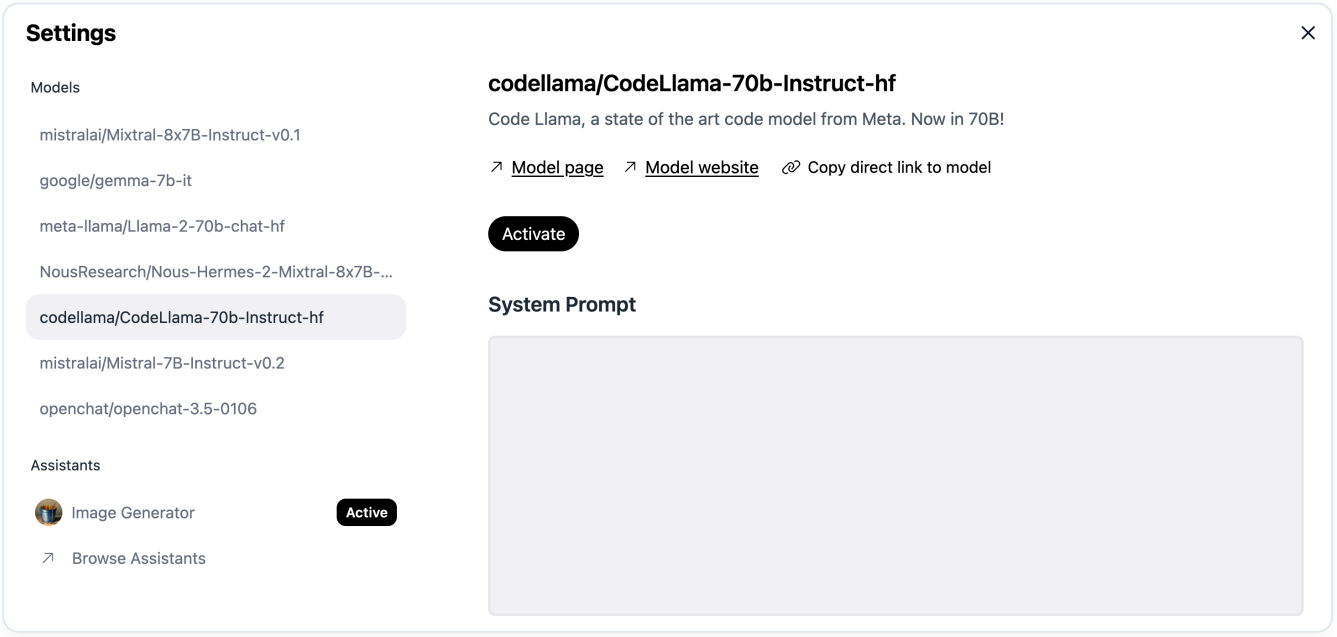
به شدت قابل سفارشی‌سازی - چون مدل‌های متن باز همراه با اطلاعات دقیق آموزشی منتشر می‌شوند، پژوهشگران و توسعه‌دهندگان می‌توانند اجزای داخلی مدل را تغییر دهند. این امکان ایجاد مدل‌های بسیار تخصصی را فراهم می‌کند که به‌صورت ریزتنظیم شده برای یک وظیفه یا زمینه خاص هستند. برخی مثال‌ها شامل تولید کد، عملیات ریاضی و زیست‌شناسی است.

هزینه - هزینه استفاده و پیاده‌سازی این مدل‌ها به ازای هر توکن کمتر از مدل‌های اختصاصی است. هنگام ساخت برنامه‌های هوش مصنوعی مولد، بررسی عملکرد در مقابل قیمت هنگام کار با این مدل‌ها در مورد مورد استفاده خود باید انجام شود.



منبع: Artificial Analysis

انعطاف‌پذیری - کار با مدل‌های باز به شما این امکان را می‌دهد که در استفاده از مدل‌های مختلف یا ترکیب آنها انعطاف‌پذیر باشید. نمونه‌ای از این مورد دستیاران HuggingChat است که کاربر می‌تواند مدل مورد استفاده را مستقیماً در رابط کاربری انتخاب کند:



بررسی مدل‌های باز مختلف

Llama 2

LLama2 که توسط متا توسعه یافته، مدلی باز است که بهینه‌سازی شده برای برنامه‌های مبتنی بر چت است. این به دلیل روش تنظیم دقیق آن است که شامل مقدار زیادی گفت‌وگو و بازخورد انسانی بود. با این روش، مدل نتایجی تولید می‌کند که بیشتر با انتظار انسان هماهنگ است و تجربه کاربری بهتری فراهم می‌کند.

برخی از نمونه‌های نسخه‌های تنظیم دقیق‌شده Llama شامل Llama ژاپنی است که در زبان ژاپنی تخصص دارد و Llama Pro که نسخه بهبود یافته‌ای از مدل پایه است.

Mistral

Mistral مدلی باز با تمرکز قوی بر عملکرد بالا و کارایی است. این مدل از رویکرد Mixture-of-Experts استفاده می‌کند که گروهی از مدل‌های متخصص را به یک سیستم ترکیب می‌کند که بر اساس ورودی، مدل‌های خاصی برای استفاده انتخاب می‌شوند. این باعث می‌شود محاسبات موثرتر باشد زیرا مدل‌ها فقط به ورودی‌هایی که در آنها تخصص دارند می‌پردازند.

برخی از نمونه‌های نسخه‌های تنظیم دقیق‌شده Mistral شامل BioMistral است که بر حوزه پزشکی متمرکز است و OpenMat h Mistral که محاسبات ریاضی انجام می‌دهد.

Falcon

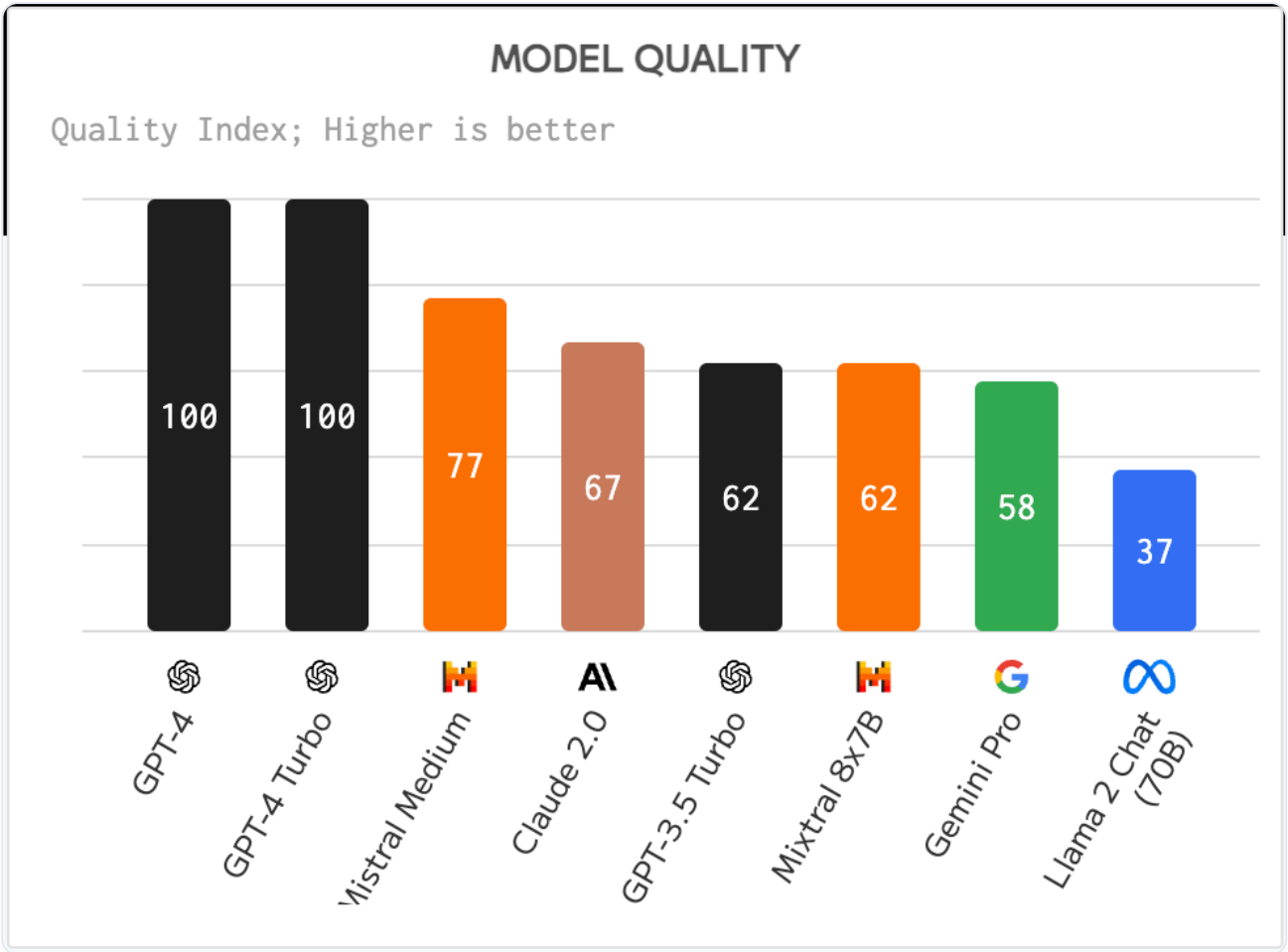
Falcon یک LLM ساخته شده توسط مؤسسه نوآوری فناوری (**TII**) است. Falcon-40B بر روی ۴۰ میلیارد پارامتر آموزش دیده است که نشان داده شده عملکرد بهتری نسبت به GPT-3 با بودجه محاسباتی کمتر دارد. این به دلیل استفاده از الگوریتم FlashAttention و توجه چند-درخواستی است که به آن اجازه می‌دهد نیازمندی‌های حافظه هنگام استنتاج را کاهش دهد. با این کاهش زمان استنتاج، Falcon-40B برای برنامه‌های چت مناسب است.

برخی نمونه‌ها از نسخه‌های تنظیم شده دقیق فالکون عبارتند از OpenAssistant، دستیار ساخته شده بر اساس مدل‌های باز و GPT4ALL، که عملکردی بهتر از مدل پایه ارائه می‌دهد.

چگونه انتخاب کنیم

پاسخ واحدی برای انتخاب یک مدل باز وجود ندارد. یک نقطه شروع خوب استفاده از ویژگی فیلتر بر اساس وظیفه در کاتالوگ مدل Microsoft Foundry است. این به شما کمک می‌کند بفهمید مدل برای چه نوع وظایفی آموزش دیده است. Hugging Face همچنین یک جدول امتیازدهی LLM را نگهداری می‌کند که مدل‌های برتر را بر اساس معیارهای خاص نشان می‌دهد.

هنگام مقایسه LLMها در انواع مختلف، Artificial Analysis منبع بسیار خوبی است:



Model Quality

منبع: Artificial Analysis

اگر روی یک مورد استفاده خاص کار می‌کنید، جستجوی نسخه‌های تنظیم شده دقیق که بر همان حوزه تمرکز دارند می‌تواند مؤثر باشد. آزمایش با چندین مدل باز برای دیدن نحوه عملکرد آنها بر اساس انتظارات شما و کاربران‌تان نیز روش خوبی است.

مراحل بعدی

بهترین بخش در مورد مدل‌های باز این است که می‌توانید خیلی سریع شروع به کار با آن‌ها کنید. کاتالوگ مدل Microsoft Foundry را بررسی کنید که شامل یک مجموعه خاص از Hugging Face با مدل‌هایی است که در اینجا بررسی کردیم.

یادگیری اینجا متوقف نمی‌شود، سفر را ادامه دهید

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش هوش مصنوعی مولد خود را ارتقا دهید!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۱۶ — هوش مصنوعی مولد مایکروسافت

درس ۱۷ از ۲۱

عامل‌های هوش مصنوعی هوشمند (AI Agents)

تبدیل مدل‌های زبانی به کارگزاران مستقل خودکار، حافظه، برنامه‌ریزی و تصمیم‌گیری، و چارچوب‌های محبوب مانند AutoGen و Semantic Kernel

ویدیوی رسمی آموزش این درس

برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/yAXVW-lUINc?si=bOtW9nL6jc3XJgOM

▶️ پخش در یوتیوب

اسکن کد QR

مقدمه

عامل‌های هوش مصنوعی یک پیشرفت هیجان‌انگیز در هوش مصنوعی تولیدی هستند که مدل‌های زبان بزرگ (LLMها) را از دستیاران به عامل‌هایی تبدیل می‌کنند که قادر به انجام اقدامات هستند. چارچوب‌های عامل هوش مصنوعی به توسعه‌دهندگان این امکان را می‌دهند تا برنامه‌هایی ایجاد کنند که به LLMها دسترسی به ابزارها و مدیریت وضعیت ارائه می‌دهند. این چارچوب‌ها همچنین دیدپذیری را افزایش می‌دهند تا کاربران و توسعه‌دهندگان بتوانند اقدامات برنامه‌ریزی شده توسط LLMها را نظارت کرده و مدیریت تجربه را بهبود بخشند.

این درس موارد زیر را پوشش خواهد داد:

- درک اینکه عامل هوش مصنوعی چیست - عامل هوش مصنوعی دقیقاً چیست؟

- بررسی پنج چارچوب عامل هوش مصنوعی مختلف - چه چیزی آنها را منحصربه‌فرد می‌کند؟
- استفاده از این عامل‌ها در موارد کاربردی مختلف - چه زمانی باید از عامل‌های هوش مصنوعی استفاده کنیم؟

اهداف یادگیری

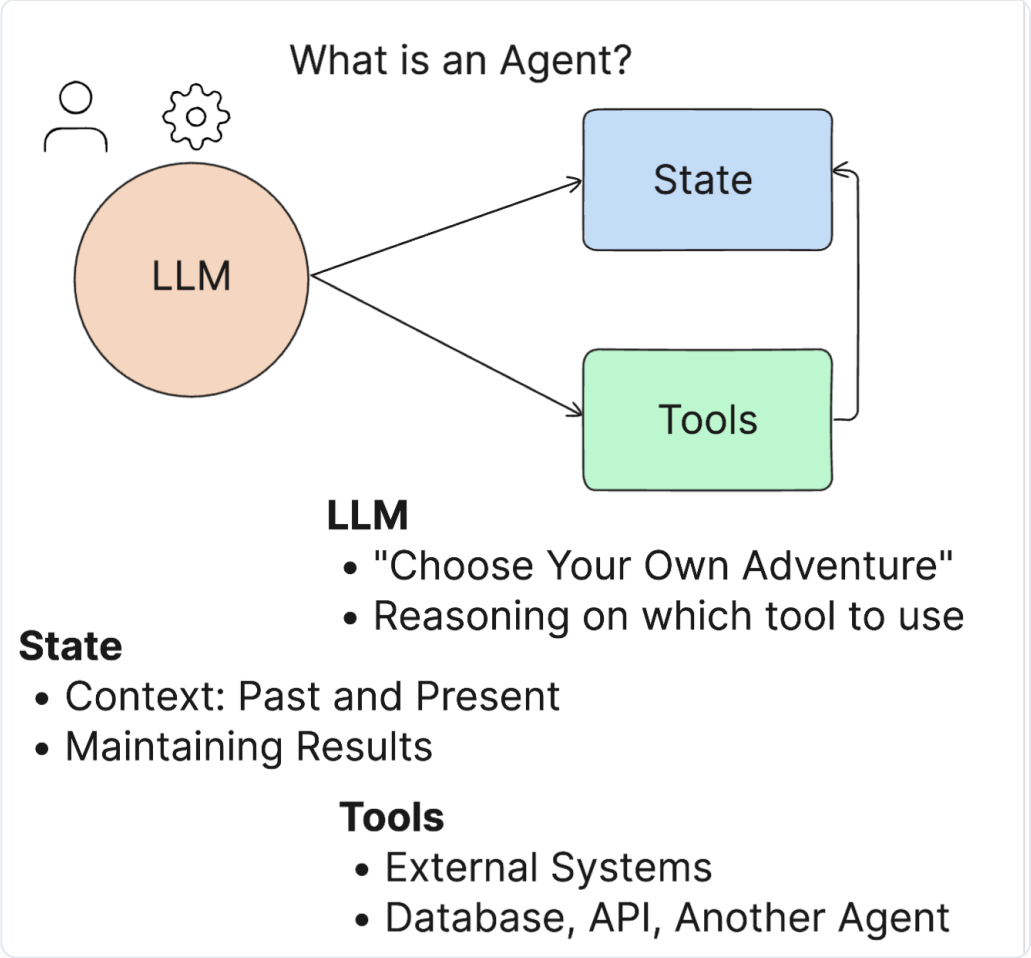
پس از گذراندن این درس، شما قادر خواهید بود:

- توضیح دهید عامل‌های هوش مصنوعی چه هستند و چگونه می‌توان از آنها استفاده کرد.
- درک تفاوت‌های برخی از چارچوب‌های محبوب عامل هوش مصنوعی و تفاوت‌هایشان.
- چگونگی عملکرد عامل‌های هوش مصنوعی را بفهمید تا بتوانید با آنها برنامه بسازید.

عامل‌های هوش مصنوعی چیستند؟

عامل‌های هوش مصنوعی حوزه‌ای بسیار هیجان‌انگیز در دنیای هوش مصنوعی تولیدی هستند. با این هیجان گاهی اوقات سردرگمی در اصطلاحات و کاربرد آنها به وجود می‌آید. برای ساده و شامل بودن اکثر ابزارهایی که به عامل‌های هوش مصنوعی اشاره دارند، ما از این تعریف استفاده می‌کنیم:

عامل‌های هوش مصنوعی به مدل‌های زبان بزرگ (LLMها) اجازه می‌دهند تا با دسترسی به **وضعیت** و **ابزارها** وظایف را انجام دهند.



بیایید این اصطلاحات را تعریف کنیم:

مدل‌های زبان بزرگ - این‌ها مدل‌هایی هستند که در کل این دوره به آنها اشاره شده است مانند GPT-4o، GPT-5 و Llama 3.3 و غیره.

وضعیت - این به زمینه‌ای اشاره دارد که LLM در آن کار می‌کند. LLM از زمینه اقدامات گذشته و زمینه فعلی استفاده می‌کند تا تصمیم‌گیری برای اقدامات بعدی را هدایت کند. چارچوب‌های عامل هوش مصنوعی این زمینه را برای توسعه‌دهندگان ساده‌تر نگه می‌دارند.

ابزارها - برای انجام کاری که کاربر درخواست کرده و LLM برنامه‌ریزی کرده است، LLM نیاز به دسترسی به ابزارها دارد. برخی نمونه‌های ابزار می‌تواند پایگاه داده، API، نرم‌افزار خارجی یا حتی یک LLM دیگر باشد!

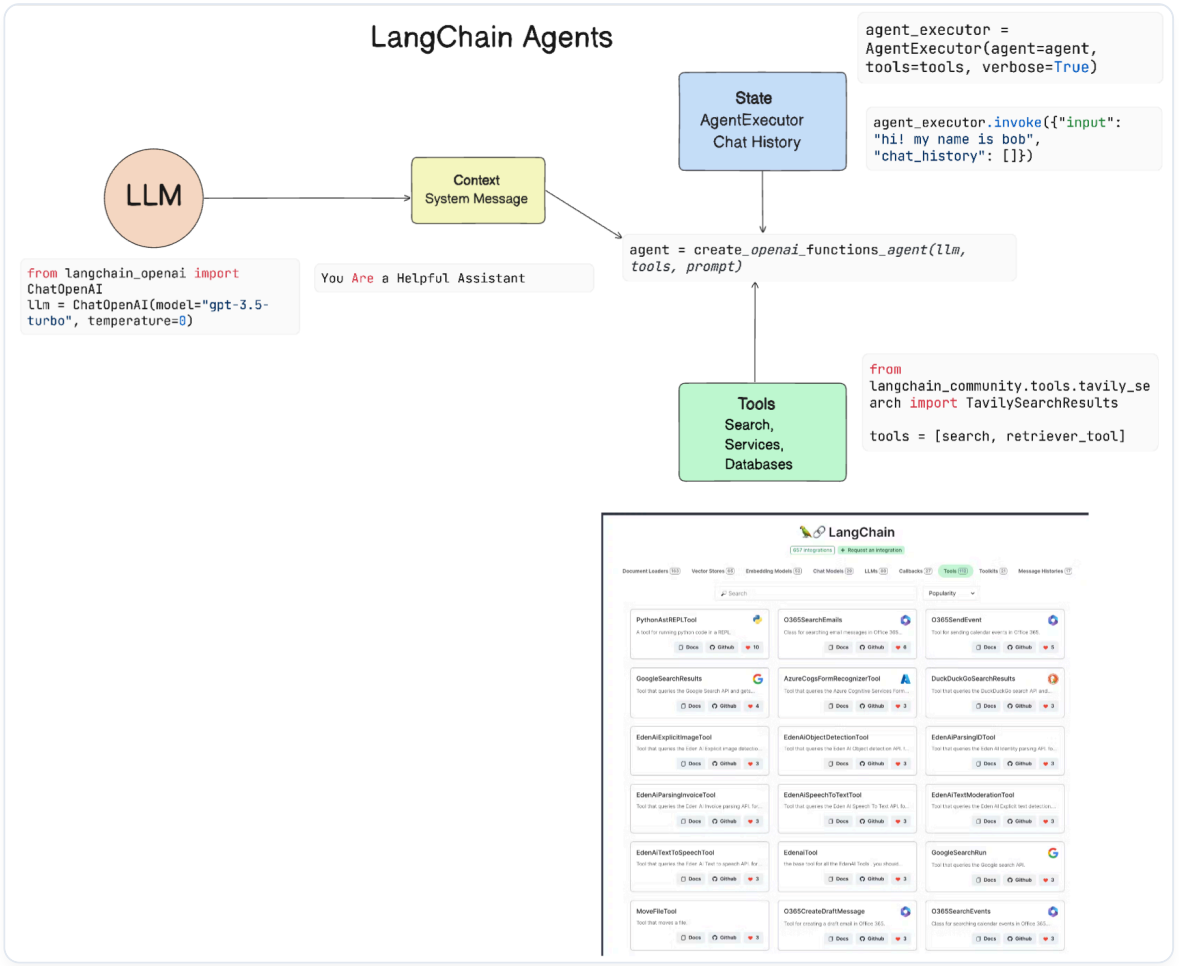
امیدواریم این تعاریف پایه خوبی به شما بدهند تا با نگاه به چگونگی پیاده‌سازی آنها، چند چارچوب عامل هوش مصنوعی مختلف را بررسی کنیم:

عامل‌های LangChain

عامل‌های LangChain پیاده‌سازی تعاریفی است که ما در بالا ارائه دادیم.

برای مدیریت **وضعیت**، از تابع داخلی به نام `AgentExecutor` استفاده می‌کند. این تابع `agent` تعریف شده و `tools` در دسترس را می‌پذیرد.

`AgentExecutor` همچنین تاریخچه چت را ذخیره می‌کند تا زمینه چت را فراهم کند.



عامل‌های LangChain

LangChain یک **فهرست ابزارها** ارائه می‌دهد که می‌توانند در برنامه شما وارد شده و LLM به آنها دسترسی پیدا کند. این ابزارها توسط جامعه و تیم LangChain ساخته شده‌اند.

شما سپس می‌توانید این ابزارها را تعریف کرده و به `AgentExecutor` بفرستید.

دیدپذیری بخش مهم دیگری هنگام صحبت درباره عامل‌های هوش مصنوعی است. برای توسعه‌دهندگان برنامه مهم است که بفهمند LLM از کدام ابزار استفاده می‌کند و چرا.. برای این منظور، تیم LangChain ابزار LangSmith را توسعه داده است.

AutoGen

چارچوب عامل هوش مصنوعی بعدی که بررسی می‌کنیم **AutoGen** است. تمرکز اصلی AutoGen بر روی مکالمات است. عامل‌ها هم **قابل گفتگو** و هم **قابل سفارشی‌سازی** هستند.

قابل گفتگو - LLMها می‌توانند مکالمه را با یک LLM دیگر آغاز و ادامه دهند تا یک وظیفه را به انجام برسانند. این با ایجاد `AssistantAgents` و دادن پیام سیستم خاصی به آنها انجام می‌شود.

Python

```
autogen.AssistantAgent( name="Coder", llm_config=llm_config, ) pm = autogen.AssistantAgent(
name="Product_manager", system_message="Creative in software product ideas.",
llm_config=llm_config, )
```

قابل سفارشی‌سازی - عامل‌ها نه تنها می‌توانند به عنوان LLM تعریف شوند بلکه می‌توانند کاربر یا ابزار باشند. به عنوان یک توسعه‌دهنده، شما می‌توانید `UserProxyAgent` تعریف کنید که مسئول تعامل با کاربر برای بازخورد در تکمیل یک وظیفه است. این بازخورد می‌تواند اجرای وظیفه را ادامه دهد یا متوقف کند.

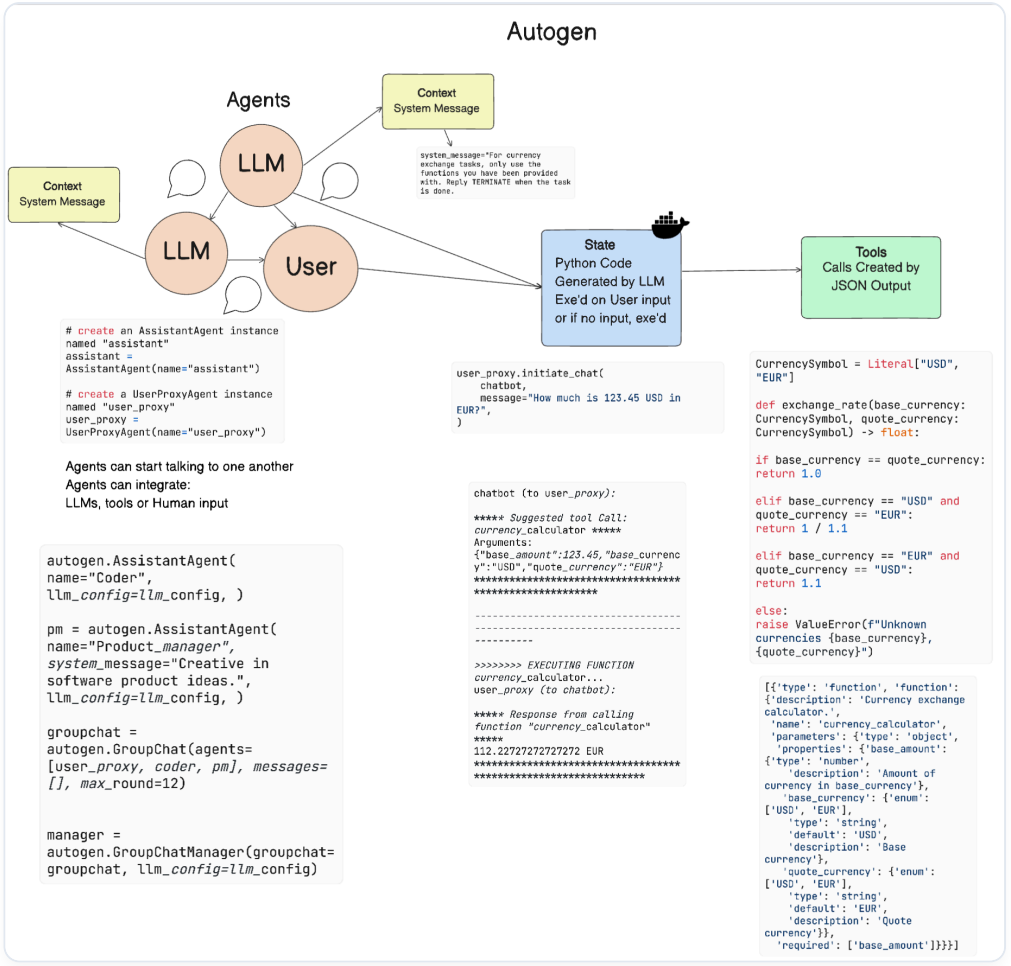
Python

```
user_proxy = UserProxyAgent(name="user_proxy")
```

وضعیت و ابزارها

برای تغییر و مدیریت وضعیت، یک عامل دستیار کدهای پایتون تولید می‌کند تا وظیفه را انجام دهد.

در اینجا نمونه‌ای از فرآیند آمده است:



AutoGen

LLM تعریف شده با پیام سیستم

Python

```
system_message="For weather related tasks, only use the functions you have been provided with. Reply TERMINATE when the task is done."
```

این پیام سیستم LLM خاص را هدایت می‌کند که کدام توابع برای وظیفه‌اش مرتبط هستند. به یاد داشته باشید، با AutoGen می‌توانید چندین AssistantAgent با پیام‌های سیستمی مختلف داشته باشید.

چت توسط کاربر شروع می‌شود

Python

```
user_proxy.initiate_chat( chatbot, message="I am planning a trip to NYC next week, can you help me pick out what to wear? ", )
```

این پیام از user_proxy (انسان) شروع‌کننده روند عامل برای بررسی توابع ممکن است که باید اجرا شوند.

تابع اجرا می‌شود

Terminal / Bash

```
chatbot (to user_proxy):

***** Suggested tool Call: get_weather ***** Arguments: {"location":"New York City,
NY","time_periond:"7","temperature_unit":"Celsius"}
******************************************************** ------------------------------------------------------------
--------------------------------------------

>>>>>>>> EXECUTING FUNCTION get_weather... user_proxy (to chatbot): ***** Response from
calling function "get_weather" ***** 112.22727272727272 EUR
**************************************************************
```

پس از پردازش چت اولیه، عامل ابزار پیشنهاد شده را برای فراخوانی ارسال می‌کند. در این مورد، تابعی به نام `get_weather` است. بسته به پیکربندی شما، این تابع می‌تواند به طور خودکار توسط عامل اجرا و خوانده شود یا با ورودی کاربر اجرا شود.

شما می‌توانید فهرستی از نمونه‌های کد AutoGen را برای کاوش بیشتر روش شروع به ساخت بیابید.

چارچوب عامل مایکروسافت

چارچوب عامل مایکروسافت SDK متن‌باز مایکروسافت برای ساخت عامل‌های هوش مصنوعی و سیستم‌های چندعامله در هر دو زبان **پایتون** و **.NET** است. این چارچوب قدرت دو پروژه قبلی مایکروسافت — ویژگی‌های سازمانی **Semantic Kernel** و ارکستراسیون چندعامله **AutoGen** — را در یک چارچوب واحد و پشتیبانی‌شده گرد هم می‌آورد. اگر امروز پروژه عاملی جدیدی را شروع می‌کنید، این جانشین پیشنهادی AutoGen است.

این چارچوب از یک **عامل چت** ساده تا **روندهای کاری چندعامله** پیچیده مقیاس می‌یابد و مستقیماً با Microsoft Foundry، Azure OpenAI و OpenAI ادغام می‌شود. همچنین دیدپذیری داخلی از طریق OpenTelemetry را ارائه می‌دهد تا دقیقاً بتوانید رد عملکرد عامل‌های خود را دنبال کنید.

وضعیت و ابزارها

وضعیت - چارچوب زمینه مکالمات را از طریق **رشته‌ها** مدیریت می‌کند. یک عامل تاریخچه پیام‌ها (درخواست‌های کاربر، فراخوانی ابزارها و نتایج آنها) را دنبال می‌کند، بنابراین هر قدم بر پایه قدم‌های قبلی ساخته می‌شود. رشته‌ها همچنین می‌توانند ذخیره شوند تا مکالمه بعداً متوقف و از سر گرفته شود.

ابزارها - به عامل ابزارها را با ارسال توابع ساده پایتون می‌دهید. پارامترهای دارای نوع به طور خودکار به یک طرح تبدیل می‌شوند تا مدل بداند چه زمانی و چگونه آنها را فراخوانی کند (فراخوانی تابع). این چارچوب همچنین از سرورهای پروتکل زمینه مدل (MCP) و ابزارهای میزبانی‌شده مانند مفسر کد پشتیبانی می‌کند.

در اینجا نمونه‌ای از یک عامل واحد با یک ابزار سفارشی آمده است:

Python

```
import asyncio
from typing import Annotated

from pydantic import Field
from agent_framework import Agent
from agent_framework.openai import OpenAIChatClient


def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    """Get the weather for a given location."""
    return f"The weather in {location} is sunny with a high of 22°C."


async def main():
    agent = Agent(
        client=OpenAIChatClient(),
        instructions="You are a helpful assistant that can answer weather questions.",
        tools=[get_weather],
    )

    response = await agent.run("What's the weather in Amsterdam?")
    print(response)


asyncio.run(main())
```

برای اتصال به Azure OpenAI در Microsoft Foundry به جای آن، نقطه پایان و مدارک خود را به مشتری ارسال کنید:

Python

```
from azure.identity.aio import AzureCliCredential
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient(
    model="my-gpt-5-mini-deployment",
    azure_endpoint="https://my-resource.openai.azure.com",
    credential=AzureCliCredential(),
)
```

روندهای کاری چندعامله

جایی که این چارچوب واقعاً برجسته است ارکستراسیون چند عامل به صورت همزمان است. برای مثال، می‌توانید عوامل را یکی پس از دیگری اجرا کنید (هر کدام زمینه خود را به بعدی منتقل کنند) یا بازوهای موازی به چند عامل باز کنید و نتایج آنها را جمع‌آوری کنید:

Python

```python
from agent_framework.orchestrations import SequentialBuilder, ConcurrentBuilder

# اجرای نمایندگان به صورت پشت سر هم، با انتقال زمینه گفتگو در طول زنجیره
sequential = SequentialBuilder(participants=[researcher, writer, editor]).build()

# اجرای همزمان نمایندگان به صورت موازی، سپس تجمیع پاسخ‌های آن‌ها
concurrent = ConcurrentBuilder(participants=[analyst_a, analyst_b, analyst_c]).build()
```

برای نصب چارچوب و شروع کار:

Terminal / Bash

```bash
pip install agent-framework-core
# ادغام‌های اختیاری
pip install agent-framework-openai       # OpenAI و Azure OpenAI
pip install agent-framework-foundry      # Microsoft Foundry
```

می‌توانید بیشتر را در **مخزن چارچوب عامل مایکروسافت** و **مستندات رسمی** کاوش کنید.

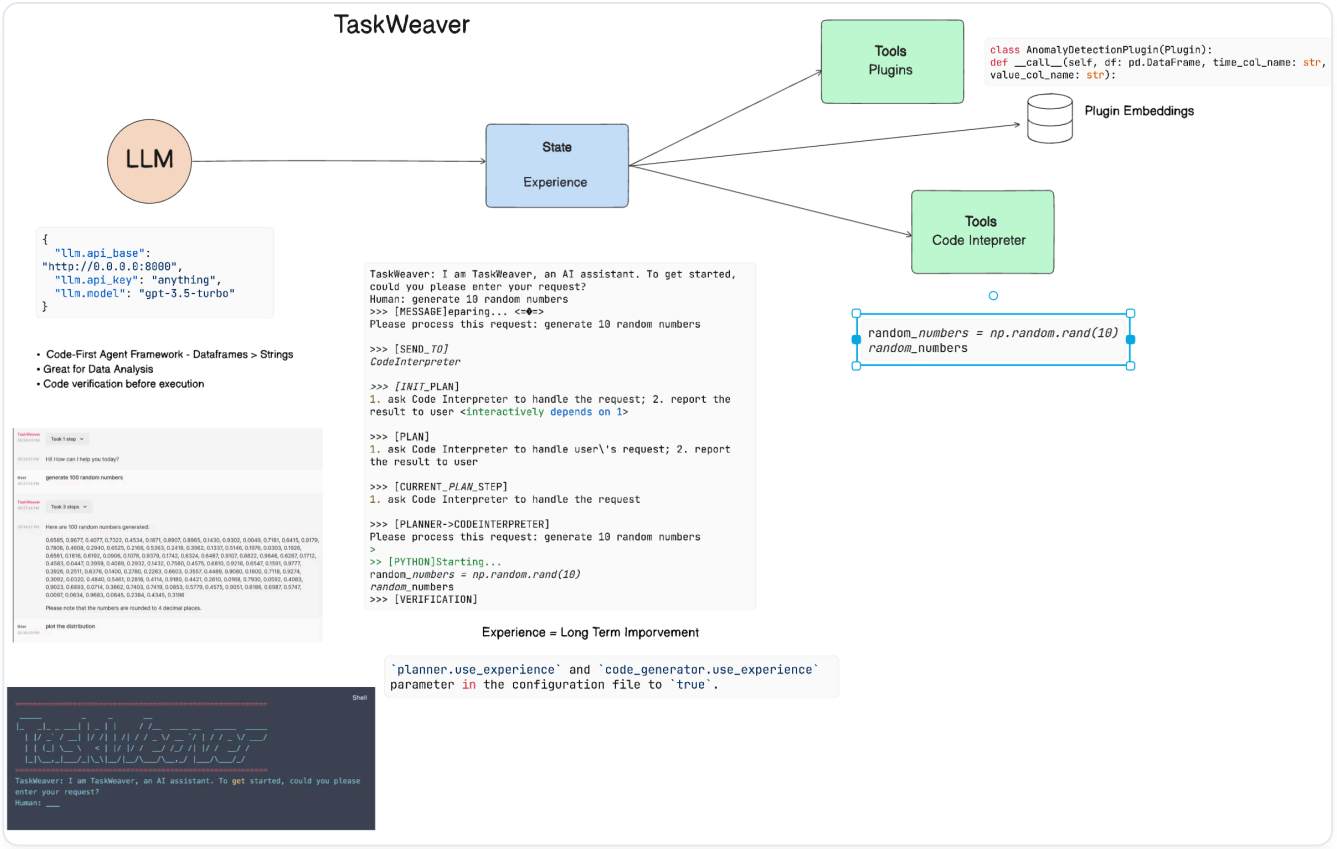
Taskweaver

چارچوب عامل بعدی که بررسی می‌کنیم **Taskweaver** است. این چارچوب به عنوان یک عامل "کدمحور" شناخته می‌شود زیرا به جای کار صرف با `رشته‌ها`، می‌تواند با DataFrameها در پایتون کار کند. این برای تحلیل و تولید داده بسیار مفید است. این می‌تواند شامل ایجاد نمودارها و چارت‌ها یا تولید اعداد تصادفی باشد.

وضعیت و ابزارها

برای مدیریت وضعیت مکالمه، TaskWeaver از مفهوم `Planner` استفاده می‌کند. `Planner` یک LLM است که درخواست کاربران را می‌گیرد و وظایفی را که باید برای انجام درخواست تکمیل شود ترسیم می‌کند.

برای به انجام رساندن وظایف، `Planner` به مجموعه‌ای از ابزارها به نام `Plugins` دسترسی دارد. این می‌تواند کلاس‌های پایتون یا یک مفسر کد عمومی باشد. این پلاگین‌ها به صورت تعبیه (embedding) ذخیره می‌شوند تا LLM بهتر بتواند پلاگین صحیح را جستجو کند.



Taskweaver

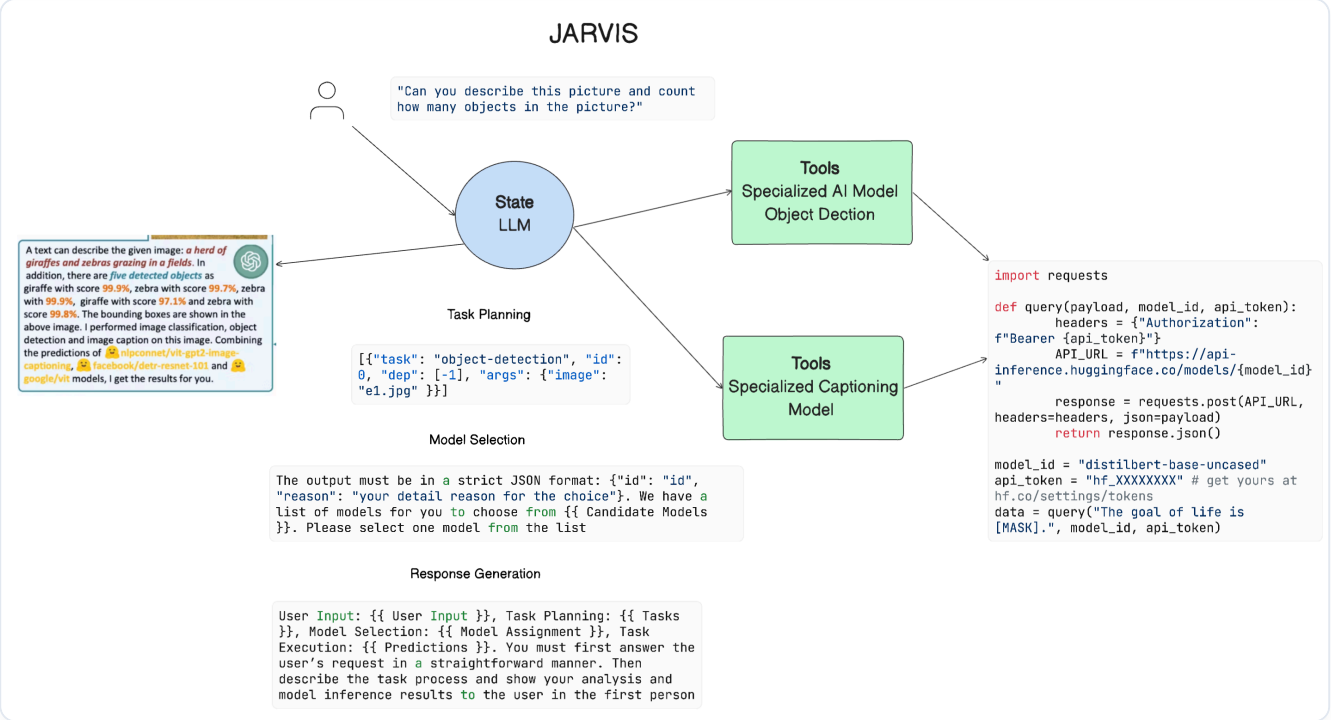
اینجا نمونه‌ای از یک پلاگین برای تشخیص ناهنجاری است:

```python
class AnomalyDetectionPlugin(Plugin): def __call__(self, df: pd.DataFrame, time_col_name: str, value_col_name: str):
```

کد قبل از اجرا تأیید می‌شود. یکی دیگر از ویژگی‌های مدیریت زمینه در Taskweaver، `تجربه` است. تجربه اجازه می‌دهد زمینه یک مکالمه برای مدت‌زمان طولانی‌تر در یک فایل YAML ذخیره شود. این می‌تواند به گونه‌ای پیکربندی شود که LLM در طول زمان در وظایف خاصی با توجه به مواجه‌شدن با مکالمات قبلی بهبود یابد.

JARVIS

آخرین چارچوب عاملی که بررسی می‌کنیم JARVIS است. فرق JARVIS این است که از یک LLM برای مدیریت `وضعیت` مکالمه استفاده می‌کند و `ابزارها` مدل‌های هوش مصنوعی دیگر هستند. هر کدام از این مدل‌های تخصصی برای انجام کارهای خاصی مانند تشخیص اشیاء، رونویسی یا توصیف تصویر طراحی شده‌اند.



JARVIS

این LLM که یک مدل چندمنظوره است، درخواست کاربر را دریافت می‌کند و وظیفه خاص و هرگونه آرگومان/داده لازم برای انجام آن وظیفه را شناسایی می‌کند.

```python
[{"task": "object-detection", "id": 0, "dep": [-1], "args": {"image": "e1.jpg" }}]
```

سپس LLM درخواست را به شکلی فرمت می‌کند که مدل تخصصی هوش مصنوعی بتواند آن را تفسیر کند، مانند JSON. پس از اینکه مدل هوش مصنوعی پیش‌بینی خود را بر اساس وظیفه انجام داد، پاسخ به LLM ارسال می‌شود.

اگر انجام کار نیازمند چندین مدل باشد، LLM همچنین پاسخ‌های آن مدل‌ها را تفسیر کرده و سپس آنها را با هم ترکیب می‌کند تا پاسخ نهایی به کاربر داده شود.

مثال زیر نشان می‌دهد چگونه این کار وقتی کاربر درخواست توصیف و شمارش اشیاء در یک تصویر را دارد، انجام می‌شود:

تمرین

برای ادامه یادگیری عامل‌های هوش مصنوعی می‌توانید با چارچوب عامل مایکروسافت بسازید:

- برنامه‌ای که یک جلسه کاری با بخش‌های مختلف یک استارت‌آپ آموزشی شبیه‌سازی می‌کند.
- پیام‌های سیستمی ایجاد کنید که LLMها را در درک شخصیت‌ها و اولویت‌های مختلف راهنمایی کند و به کاربر امکان دهد ایده محصول جدیدی را ارائه دهد.

- سپس LLM سوالات پیگیری از هر بخش را تولید کند تا ارائه و ایده محصول اصلاح و بهبود یابد.

یادگیری اینجا متوقف نمی‌شود، سفر را ادامه دهید

پس از اتمام این درس، مجموعه **یادگیری هوش مصنوعی تولیدی ما** را برای ادامه ارتقا دانش هوش مصنوعی تولیدی خود بررسی کنید!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۱۷ — هوش مصنوعی مولد مایکروسافت

درس ۱۸ از ۲۱

تنظیم دقیق مدل‌های زبانی (Fine-Tuning)

چه زمانی به Fine-Tuning نیاز داریم؟ مقایسه با RAG، آماده‌سازی مجموعه داده آموزشی، و روش‌های بهینه‌سازی پارامترها (PEFT / LoRA)

🎥 **ویدیوی رسمی آموزش این درس**

📱 برای مشاهده ویدیو، کد QR را اسکن کنید یا پیوند زیر را باز کنید:

https://youtu.be/6UAwhL9Q-TQ?si=5jJd8yeQsCfJ97em

▶️ پخش در یوتیوب

اسکن کد QR

تنظیم دقیق مدل زبان بزرگ شما

استفاده از مدل‌های زبان بزرگ برای ساخت برنامه‌های هوش مصنوعی مولد با چالش‌های جدیدی همراه است. یک مسئله کلیدی تضمین کیفیت پاسخ (دقت و مرتبط بودن) در محتوایی است که مدل برای یک درخواست کاربر خاص تولید می‌کند. در درس‌های قبلی به تکنیک‌هایی مانند مهندسی پرامپت و تولید تقویتی بازیابی اشاره کردیم که سعی دارند این مسئله را با *اصلاح ورودی پرامپت* برای مدل موجود حل کنند.

در درس امروز، ما به تکنیک سوم، یعنی **تنظیم دقیق** می‌پردازیم که تلاش می‌کند با *آموزش مجدد خود مدل* با داده‌های اضافی، این چالش را برطرف کند. بیایید به جزئیات بپردازیم.

اهداف یادگیری

این درس مفهوم تنظیم دقیق برای مدل‌های زبان پیش‌آموزش‌دیده را معرفی می‌کند، مزایا و چالش‌های این رویکرد را بررسی می‌کند و راهنمایی‌هایی درباره زمان و نحوه استفاده از تنظیم دقیق برای بهبود عملکرد مدل‌های هوش مصنوعی مولد شما ارائه می‌دهد.

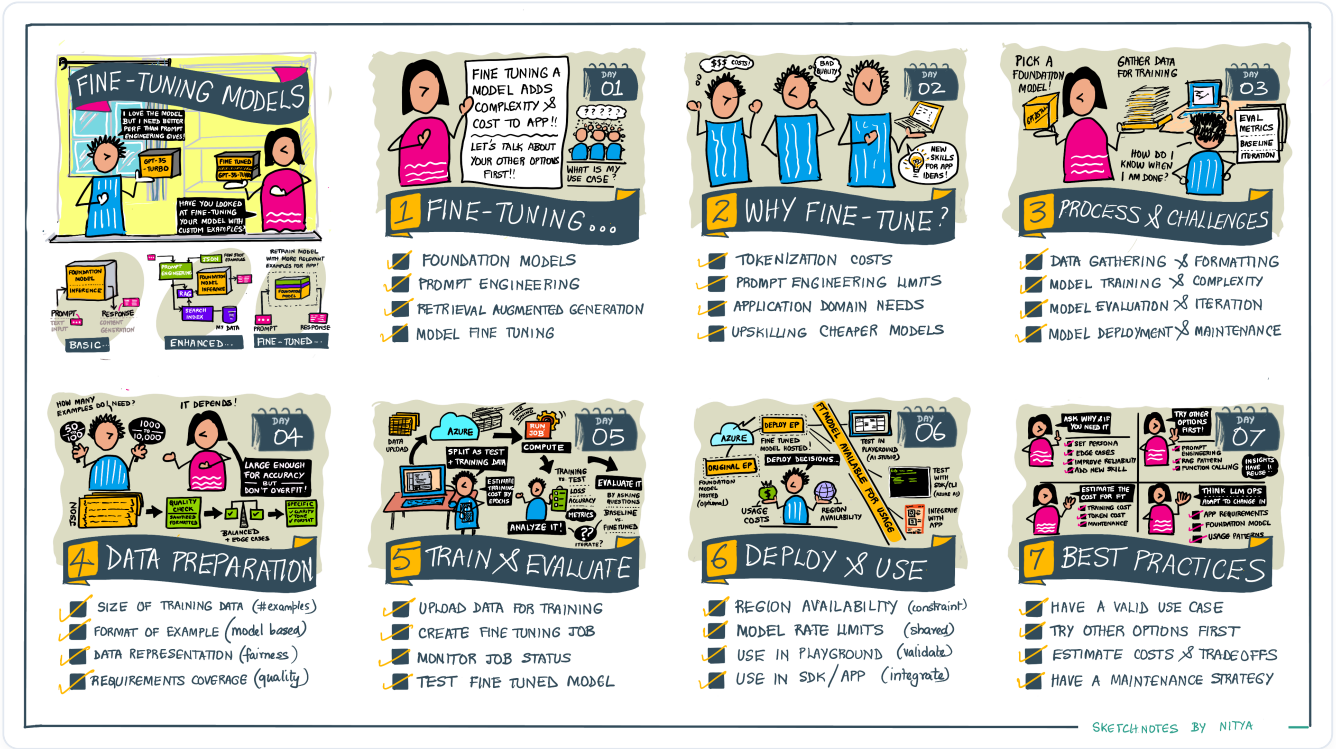
در پایان این درس، باید قادر باشید به سوالات زیر پاسخ دهید:

- تنظیم دقیق برای مدل‌های زبان چیست؟
- چه زمانی و چرا تنظیم دقیق مفید است؟
- چگونه می‌توانم یک مدل پیش‌آموزش‌دیده را تنظیم دقیق کنم؟
- محدودیت‌های تنظیم دقیق چیست؟

آماده‌اید؟ بیایید شروع کنیم.

راهنمای تصویری

می‌خواهید قبل از شروع، تصویر کلی آنچه را که پوشش خواهیم داد ببینید؟ این راهنمای مصور را بررسی کنید که مسیر یادگیری این درس را از یادگیری مفاهیم اصلی و انگیزه تنظیم دقیق تا درک فرایند و بهترین شیوه‌های اجرای وظیفه تنظیم دقیق توصیف می‌کند. این موضوع جذابی برای کاوش است، فراموش نکنید صفحه منابع را برای پیوندهای اضافی که از مسیر یادگیری خودگردان شما پشتیبانی می‌کنند، بررسی کنید!



تنظیم دقیق برای مدل‌های زبان چیست؟

مطابق تعریف، مدل‌های زبان بزرگ بر روی حجم زیادی از متن‌های متنوع از منابع مختلف از جمله اینترنت *پیش‌آموزش‌دیده* شده‌اند. همانطور که در درس‌های قبلی آموختیم، به تکنیک‌هایی مانند *مهندسی پرامپت* و *تولید تقویتی بازیابی‌شده* نیاز داریم تا کیفیت پاسخ‌های مدل به سوالات کاربر ("پرامپت‌ها") را بهبود بخشیم.

یک تکنیک محبوب مهندسی پرامپت شامل دادن راهنمایی بیشتر به مدل درباره آنچه در پاسخ انتظار می‌رود، یا با ارائه *دستورالعمل‌ها* (راهنمایی صریح) یا *دادن چند مثال* (راهنمایی ضمنی) است. این به عنوان *یادگیری چندنمونه‌ای* شناخته می‌شود اما دو محدودیت دارد:

- محدودیت‌های توکنی مدل می‌تواند تعداد نمونه‌هایی که می‌توانید ارائه دهید را محدود کند و اثربخشی را کاهش دهد.
- هزینه‌های توکنی مدل می‌تواند افزودن نمونه‌ها به هر پرامپت را پرهزینه کند و انعطاف‌پذیری را محدود نماید.

تنظیم دقیق یک روش رایج در سیستم‌های یادگیری ماشین است که در آن مدل پیش‌آموزش‌دیده را با داده‌های جدید برای بهبود عملکرد آن در یک وظیفه خاص آموزش مجدد می‌دهیم. در زمینه مدل‌های زبان، می‌توان مدل پیش‌آموزش‌دیده را *با یک مجموعه نمونه منتخب برای یک وظیفه یا حوزه کاربردی خاص* تنظیم دقیق کرد تا یک **مدل سفارشی** ایجاد شود که ممکن است برای آن وظیفه یا حوزه خاص دقیق‌تر و مرتبط‌تر باشد. یکی از مزایای جانبی تنظیم دقیق این است که می‌تواند تعداد نمونه‌های نیازمند برای یادگیری چندنمونه‌ای را کاهش دهد – که منجر به کاهش مصرف توکن و هزینه‌های مربوطه می‌شود.

چه زمانی و چرا باید مدل‌ها را تنظیم دقیق کنیم؟

در *این* زمینه، وقتی از تنظیم دقیق صحبت می‌کنیم، منظورمان تنظیم دقیق **نظارت‌شده** است که با **افزودن داده‌های جدید** که در مجموعه داده‌های آموزش اصلی نبودند، انجام می‌شود. این با رویکرد تنظیم دقیق بدون نظارت که مدل بر روی داده‌های اصلی اما با پارامترهای ابر متفاوت آموزش مجدد می‌شود، متفاوت است.

نکته کلیدی این است که تنظیم دقیق یک تکنیک پیشرفته است که نیاز به سطحی از تخصص دارد تا نتایج مورد انتظار حاصل شود. اگر به درستی انجام نشود، ممکن است بهبودهای مورد انتظار را ارائه ندهد و حتی عملکرد مدل در حوزه هدف شما را کاهش دهد.

بنابراین، پیش از اینکه بیاموزید "چگونه" مدل‌های زبان را تنظیم دقیق کنید، باید بدانید "چرا" باید این مسیر را انتخاب کنید و "چه زمانی" باید فرایند تنظیم دقیق را شروع کنید. این سوالات را از خود بپرسید:

- **مورد استفاده**: مورد استفاده شما برای تنظیم دقیق چیست؟ کدام جنبه از مدل پیش‌آموزش‌دیده کنونی را می‌خواهید بهبود دهید؟
- **جایگزین‌ها**: آیا *روش‌های دیگر* را برای رسیدن به نتایج دلخواه امتحان کرده‌اید؟ از آن‌ها برای ایجاد خط مبنا برای مقایسه استفاده کنید.

- مهندسی پرامپت: تکنیک‌هایی مانند یادگیری چندنمونه‌ای با نمونه‌هایی از پاسخ‌های پرامپت مرتبط را امتحان کنید. کیفیت پاسخ‌ها را ارزیابی کنید.
- تولید تقویتی بازیابی شده: پرامپت‌ها را با نتایج جستجو روی داده‌های خود تقویت کنید. کیفیت پاسخ‌ها را ارزیابی کنید.

- **هزینه‌ها**: آیا هزینه‌های تنظیم دقیق را شناسایی کرده‌اید؟
  - قابلیت تنظیم - آیا مدل پیش‌آموزش‌دیده برای تنظیم دقیق در دسترس است؟
  - تلاش - برای آماده‌سازی داده‌های آموزشی، ارزیابی و اصلاح مدل.
  - محاسبات - برای اجرای کارهای تنظیم دقیق و استقرار مدل تنظیم دقیق شده.
  - داده - دسترسی به نمونه‌های کیفی کافی برای تاثیر تنظیم دقیق.
- **مزایا**: آیا مزایای تنظیم دقیق را تایید کرده‌اید؟
  - کیفیت - آیا مدل تنظیم دقیق شده بهتر از خط مبنا عمل می‌کند؟
  - هزینه - آیا با ساده‌سازی پرامپت‌ها مصرف توکن را کاهش می‌دهد؟
  - توسعه‌پذیری - آیا می‌توانید مدل پایه را برای حوزه‌های جدید مجدداً استفاده کنید؟

با پاسخ دادن به این سوالات، باید بتوانید تصمیم بگیرید که آیا تنظیم دقیق رویکرد مناسبی برای مورد استفاده شما است یا خیر. ایده‌آل این است که این رویکرد تنها زمانی معتبر است که مزایا از هزینه‌ها بیشتر باشد. پس از تصمیم به ادامه، وقت آن است که به *نحوه* تنظیم دقیق مدل پیش‌آموزش‌دیده فکر کنید.

می‌خواهید دید کامل‌تری درباره فرایند تصمیم‌گیری بدست آورید؟ ویدئوی **تنظیم دقیق یا عدم تنظیم دقیق** را تماشا کنید.

چگونه می‌توانیم مدل پیش‌آموزش‌دیده را تنظیم دقیق کنیم؟

برای تنظیم دقیق یک مدل پیش‌آموزش‌دیده به موارد زیر نیاز دارید:

- یک مدل پیش‌آموزش‌دیده برای تنظیم دقیق
- یک مجموعه داده برای استفاده در تنظیم دقیق
- یک محیط آموزشی برای اجرای کار تنظیم دقیق
- یک محیط میزبانی برای استقرار مدل تنظیم دقیق شده

تنظیم دقیق در Microsoft Foundry

Microsoft Foundry جایی است که امروز مدل‌های سفارشی را در Azure تنظیم دقیق، مستقر و مدیریت می‌کنید (این پلتفرم ترکیبی از Azure OpenAI Studio و Azure AI Studio سابق است). پیش از شروع یک کار، مفید است که گزینه‌هایی که

Foundry ارائه می‌دهد و بهترین شیوه‌های پیشنهاد شده توسط پلتفرم را بشناسید. در پشت صحنه، Foundry از **LoRA (تطبیق با رتبه پایین)** برای تنظیم دقیق مدل‌ها به صورت کارآمد استفاده می‌کند، که باعث می‌شود آموزش سریع‌تر و مقرون به صرفه‌تر از آموزش مجدد همه وزن‌ها باشد.

مرحله ۱: انتخاب تکنیک آموزش

Foundry سه تکنیک تنظیم دقیق را پشتیبانی می‌کند. **با SFT شروع کنید** - این تکنیک گسترده‌ترین دامنه سناریوها را پوشش می‌دهد.

تکنیک	عملکرد	زمان استفاده
تنظیم دقیق نظارت‌شده (SFT)	روی جفت مثال‌های ورودی/خروجی آموزش می‌دهد تا مدل پاسخ‌های دلخواه شما را تولید کند.	پیش‌فرض برای اکثر وظایف: تخصص حوزه، عملکرد وظیفه، سبک و لحن، دنبال‌کردن دستورالعمل‌ها، و سازگاری زبان.
بهینه‌سازی ترجیح مستقیم (DPO)	از جفت پاسخ‌های *اولویت‌دار و غیر اولویت‌دار* یاد می‌گیرد تا خروجی‌ها را با ترجیحات انسانی همسو کند.	بهبود کیفیت پاسخ، ایمنی، و همسویی زمانی که بازخورد مقایسه‌ای دارید.
تنظیم دقیق تقویتی (RFT)	از سیگنال‌های پاداش از *ارزیاب‌ها* برای بهینه‌سازی رفتارهای پیچیده با یادگیری تقویتی استفاده می‌کند.	حوزه‌های هدفمند و تفکرمحور (ریاضی، شیمی، فیزیک) با پاسخ‌های درست/نادرست واضح. نیاز به تخصص بیشتر در یادگیری ماشین دارد.

مرحله ۲: انتخاب سطح آموزش

Foundry به شما اجازه می‌دهد انتخاب کنید که آموزش چگونه و کجا انجام شود:

- **استاندارد** - در منطقه منبع شما آموزش داده می‌شود و تضمین می‌کند داده‌ها در آن منطقه باقی بمانند. وقتی داده‌ها باید در یک منطقه خاص بمانند، از این گزینه استفاده کنید.
- **جهانی** - هزینه کمتر و سریع‌تر برای صف بندی با استفاده از ظرفیت خارج از منطقه شما (داده‌ها و وزن‌ها به منطقه آموزش کپی می‌شوند). پیش‌فرض خوبی است وقتی که اقامت داده‌ها شرط نباشد.
- **توسعه‌دهنده** - کمترین هزینه، استفاده از ظرفیت بیکار بدون تضمین تاخیر/SLA (کارها می‌توانند متوقف و از سر گرفته شوند). ایده‌آل برای آزمایش.

مرحله ۳: انتخاب مدل پایه

مدل‌های قابل تنظیم شامل OpenAI `gpt-4o-mini`، `gpt-4o`، `gpt-4.1`، `gpt-4.1-mini` و `gpt-4.1-nano` (SFT؛ خانواده 4o/4.1 همچنین DPO را پشتیبانی می‌کند)، مدل‌های استدلال `o4-mini` و `gpt-5` (RFT)، علاوه بر مدل‌های

متن‌باز مانند `Ministral-3B` ، `Qwen-32B` ، `Llama-3.3-70B-Instruct` ، و `gpt-oss-20b` (SFT روی منابع Foundry). همیشه فهرست فعلی **مدل‌های تنظیم دقیق‌شده** را برای روش‌های پشتیبانی شده، مناطق و در دسترس بودن بررسی کنید.

> 💡 **نکته کاربردی**
>
> Foundry دو حالت ارائه می‌دهد: **بدون سرور** (قیمت‌گذاری مبتنی بر مصرف، بدون نیاز به مدیریت سهمیه GPU، مدل‌های OpenAI و مدل‌های منتخب) و **محاسبات مدیریت شده** (ارائه وی‌ام‌های خود از طریق Azure Machine Learning برای گسترده‌ترین دامنه مدل‌ها). بیشتر افراد باید از حالت بدون سرور شروع کنند.

بهترین شیوه‌های Foundry

- **ابتدا خط مبنا را بسنجید.** قبل از تنظیم دقیق، مدل پایه را با مهندسی پرامپت و RAG *اندازه‌گیری* کنید تا بتوانید بهبودها را اثبات نمایید.
- **از کوچک شروع کنید، سپس مقیاس دهید.** با ۵۰-۱۰۰ نمونه با کیفیت عالی شروع کنید تا رویکرد را تایید کنید، سپس به بیش از ۵۰۰ نمونه برای تولید برسید. کیفیت بهتر از کمیت است - نمونه‌های کیفیت پایین را حذف کنید.
- **داده‌ها را به درستی قالب‌بندی کنید.** فایل‌های آموزش و اعتبارسنجی باید JSONL، UTF-8 **با BOM**، زیر ۵۱۲ مگابایت، و با فرمت پیام‌های چت-کامل باشند. همیشه یک فایل اعتبارسنجی شامل کنید تا از بیش‌برازش جلوگیری شود.
- **پرامپت سیستم آموزش را در استنتاج نگه دارید.** هنگام فراخوانی مدل از همان پیام سیستم استفاده کنید که در آموزش استفاده کرده‌اید.
- **چک‌پوینت‌ها را ارزیابی کنید - کورکورانه آخرین را مستقر نکنید.** Foundry سه دوره آخر را به عنوان چک‌پوینت‌های قابل استقرار نگه می‌دارد؛ آن که بهترین تعمیم را دارد با مشاهده `train_loss` / `valid_loss` و دقت توکن انتخاب کنید.
- **هزینه توکن را همراه با کیفیت اندازه‌گیری کنید** هنگام مقایسه مدل تنظیم دقیق شده با خط مبنا.
- **با تنظیم دقیق مداوم تکرار کنید.** می‌توانید یک مدل تنظیم دقیق شده را روی داده‌های جدید دوباره تنظیم دقیق کنید (برای مدل‌های OpenAI پشتیبانی می‌شود).
- **هزینه‌های میزبانی را در نظر بگیرید.** مدل سفارشی مستقر شده به صورت ساعتی هزینه می‌شود و استقرار غیرفعال پس از ۱۵ روز حذف می‌شود - موارد غیرضروری را پاک‌سازی کنید.

از آموزش گام‌به‌گام در **سفارشی‌سازی مدل با تنظیم دقیق** استفاده کنید و راهنماهای **DPO** و **RFT** را هنگام آمادگی برای تکنیک‌های دیگر ببینید.

تنظیم دقیق در عمل

منابع زیر آموزش‌های قدم به قدم ارائه می‌دهند که شما را از طریق یک مثال واقعی روی یک مدل پشتیبانی شده فعلی با یک مجموعه داده منتخب همراهی می‌کنند. برای کار کردن با آن‌ها، نیاز به یک حساب کاربری در ارائه‌دهنده خاص و همچنین دسترسی به مدل و داده‌های مرتبط دارید.

ارائه‌دهنده	آموزش	توضیح
OpenAI	چگونه مدل‌های چت را تنظیم دقیق کنیم	یاد بگیرید چگونه یک مدل چت OpenAI اخیر را برای یک حوزه خاص ("دستیار دستور پخت") تنظیم دقیق کنید با آماده‌سازی داده‌های آموزشی، اجرای کار تنظیم دقیق، و استفاده از مدل تنظیم دقیق شده برای استنتاج.
Microsoft Foundry	سفارشی‌سازی مدل با تنظیم دقیق	یاد بگیرید چگونه یک مدل پشتیبانی شده فعلی مانند `gpt-4.1-mini` را روی **Azure** با Microsoft Foundry تنظیم دقیق کنید: داده‌های آموزش و اعتبارسنجی را آماده و بارگذاری کنید، کار تنظیم دقیق را اجرا کنید، سپس مدل جدید را مستقر و استفاده کنید.
Hugging Face	تنظیم دقیق LLM ها با Hugging Face	این پست وبلاگی به شما نحوه تنظیم دقیق یک *LLM باز* (مثال: `CodeLlama 7B`) با استفاده از کتابخانه transformers و یادگیری تقویتی ترنسفورمر (TRL) با داده‌های باز روی Hugging Face را آموزش می‌دهد.
🤗 AutoTrain	تنظیم دقیق LLM ها با AutoTrain	AutoTrain (یا AutoTrain پیشرفته) یک کتابخانه پایتون توسعه داده شده توسط Hugging Face است که امکان تنظیم دقیق برای وظایف مختلف از جمله تنظیم دقیق LLM را فراهم می‌کند. AutoTrain یک راه‌حل بدون کد است و تنظیم دقیق می‌تواند در فضای ابری خودتان، در Hugging Face Spaces یا به صورت محلی انجام شود. این ابزار از GUI مبتنی بر وب، خط فرمان و آموزش از طریق فایل‌های پیکربندی yaml پشتیبانی می‌کند.
🦥 Unsloth	تنظیم دقیق LLM ها با Unsloth	Unsloth یک چارچوب متن‌باز است که از تنظیم دقیق LLM و یادگیری تقویتی (RL) پشتیبانی می‌کند. Unsloth روند آموزش محلی، ارزیابی و استقرار را با دفترچه یادداشت‌های آماده استفاده ساده می‌سازد. این چارچوب همچنین مدل‌های تبدیل متن به گفتار (TTS)، BERT و چندوجهی را پشتیبانی می‌کند. برای شروع، راهنمای گام‌به‌گام راهنمای تنظیم دقیق LLM ها آن‌ها را مطالعه کنید.

تکلیف

یکی از آموزش‌های بالا را انتخاب کنید و آن‌ها را دنبال کنید. *ممکن است نسخه‌ای از این آموزش‌ها را در Jupyter Notebooks در این مخزن برای مرجع تکرار کنیم. لطفاً برای دریافت آخرین نسخه‌ها مستقیماً از منابع اصلی استفاده کنید.*

کار عالی! به یادگیری ادامه دهید.

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش هوش مصنوعی مولد خود را افزایش دهید!

تبریک!! شما درس نهایی از مجموعه v2 این دوره را تکمیل کردید! یادگیری و ساختن را متوقف نکنید. **صفحه منابع را برای فهرست پیشنهادات اضافی فقط در همین موضوع بررسی کنید.

مجموعه درس‌های v1 ما نیز با تمرین‌ها و مفاهیم بیشتر به‌روزرسانی شده است. پس یک دقیقه وقت بگذارید تا دانش خود را تازه کنید - و لطفاً سؤالات و بازخوردهای خود را به اشتراک بگذارید تا به ما در بهبود این دروس برای جامعه کمک کنید.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بخش تکمیلی / راهنمای همراه

منابع برای یادگیری خودراهبر

این درس با استفاده از منابع اصلی OpenAI و Microsoft Foundry به عنوان مراجع برای اصطلاحات و آموزش‌ها ساخته شده است. در اینجا فهرستی غیرجامع برای مسیرهای یادگیری خودراهبر شما ارائه شده است. هر لینک زیر به مطالب فعلی و پشتیبانی‌شده اشاره دارد.

1. منابع اولیه

عنوان/لینک	توضیحات
تنظیم دقیق با مدل‌های OpenAI	تنظیم دقیق با استفاده از آموزش بر روی نمونه‌های بسیار بیشتر از آنچه در پرامپت جا می‌شود، عملکرد یادگیری چندنمونه‌ای را بهبود می‌بخشد - صرفه‌جویی در هزینه‌ها، بهبود کیفیت پاسخ و امکان درخواست‌های با تأخیر کمتر. **مروری بر تنظیم دقیق از OpenAI دریافت کنید.**
چه زمانی باید تنظیم دقیق Microsoft Foundry را استفاده کرد	درک کنید **تنظیم دقیق چیست (مفهوم)**، چرا باید آن را در نظر بگیرید، چه داده‌هایی باید استفاده شود و چگونه کیفیت را بسنجید - علاوه بر زمانی که SFT، DPO، یا RFT مناسب است.
شخصی‌سازی مدل با تنظیم دقیق	**روند گام به گام (چگونگی)** تنظیم دقیق در Microsoft Foundry با استفاده از پرتال، SDK پایتون OpenAI /، یا REST API - شامل آماده‌سازی داده، آموزش، نقطه‌های بررسی و استقرار.
تنظیم دقیق مستمر	روند تکراری انتخاب یک مدل قبلاً تنظیم شده به عنوان مدل پایه و **تنظیم دقیق بیشتر** آن بر روی مجموعه‌های جدید مثال‌های آموزشی.
تنظیم دقیق با فراخوانی ابزار (تابع)	تنظیم دقیق مدل شما **با مثال‌های فراخوانی ابزار** خروجی را بهبود می‌بخشد - پاسخ‌های دقیق‌تر، سازگارتر و با قالب‌بندی مشابه با استفاده از توکن‌های کمتر در پرامپت.
راهنمای تنظیم دقیق مدل‌ها: Microsoft Foundry	ببینید **کدام مدل‌ها را می‌توان تنظیم دقیق کرد**، روش‌های پشتیبانی‌شده (SFT / DPO / RFT) و مناطق که در دسترس هستند.
مرور کلی تنظیم دقیق: تکنیک‌ها و حالت‌ها	مقایسه سه تکنیک آموزش (SFT، DPO، RFT) و دو حالت (بدون سرور در مقابل محاسبات مدیریت شده)، با راهنمایی در انتخاب مدل پایه و شروع کار.
آموزش: تنظیم دقیق یک مدل در Microsoft Foundry	ایجاد یک مجموعه داده نمونه، آماده‌سازی برای تنظیم دقیق، اجرای یک کار تنظیم دقیق روی مدل‌های فعلی مانند `gpt-4.1-mini` و استقرار مدل تنظیم شده روی Azure.
آموزش: تنظیم دقیق مدل‌ها با استقرار API بدون سرور	سفارشی‌سازی مدل‌های باز و شریک (Phi، Llama، Mistral و غیره) به داده‌های شما *با استفاده از یک جریان کاری کم‌کد و مبتنی بر رابط کاربری* در Microsoft Foundry.
آموزش: تنظیم دقیق مدل‌های Hugging Face در Azure Databricks	تنظیم دقیق مدل Hugging Face با کتابخانه `transformers` روی یک کارت گرافیک واحد با استفاده از Azure Databricks و Hugging Face Trainer.
آموزش: تنظیم دقیق مدل پایه با Azure Machine Learning	فهرست مدل‌های Azure Machine Learning شامل بسیاری از مدل‌های متن‌باز است که می‌توانید تنظیم دقیق کنید. بخشی از مسیر یادگیری Azure ML Generative AI.

عنوان/لینک	توضیحات
آموزش: تنظیم دقیق Azure OpenAI با Weights & Bia ses	ردیابی و تحلیل جلسات تنظیم دقیق در Azure با W&B. راهنمای تنظیم دقیق OpenAI را با مراحل خاص Azure و ردیابی آزمایش‌ها گسترش می‌دهد.

2. منابع ثانویه

این بخش منابع اضافی قابل توجهی را در بر می‌گیرد که فرصت بررسی آنها در درس پیش نیامده بود. از آنها برای ساخت تخصص خود در این موضوع استفاده کنید.

عنوان/لینک	توضیحات
OpenAI Cookbook: آماده‌سازی داد ه و تحلیل برای تنظیم دقیق مدل چت	پیش‌پردازش و تحلیل یک مجموعه داده چت قبل از تنظیم دقیق: بررسی خطاهای فرمت، به دست آوردن آمار پایه و برآورد تعداد توکن‌ها (و هزینه). مرتبط با راهنمای تنظیم دقیق OpenAI.
OpenAI Cookbook: تنظیم دقیق برا ی تولید افزوده بازیابی‌شده (RAG) با Qdrant	مثال جامعی از تنظیم دقیق مدل‌های OpenAI برای RAG، با ادغام Qdrant و یادگیری چندنمونه‌ای برای افزایش عملکرد و کاهش ساختگی‌ها.
OpenAI Cookbook: تنظیم دقیق G PT با Weights & Biases	استفاده از W&B برای ردیابی آموزش و تنظیم دقیق مدل. ابتدا راهنمای تنظیم دقیق OpenAI آنها را بخوانید و سپس تمرین Cookbook را امتحان کنید.
آموزش Hugging Face: چگونه LLM ها را با Hugging Face TRL تنظیم د قیق کنیم	تنظیم دقیق LLMهای باز با استفاده از Transformers، Hugging Face TRL و مجموعه داده‌ها: تعریف مورد استفاده، ایجاد محیط توسعه، آماده‌سازی داده، تنظیم دقیق، ارزیابی و استقرار.
Hugging Face: AutoTrain Advan ced	کتابخانه‌ای بدون کد/کم‌کد از Hugging Face برای تنظیم دقیق بسیاری از انواع مدل‌ها. می‌توانید آن را در فضای ابری خود، در Hugging Face Spaces یا به صورت محلی از طریق GUI، CLI یا پیکربندی YAML اجرا کنید.
Unsloth: راهنمای تنظیم دقیق LLM ها	چارچوب متن‌باز که تنظیم دقیق LLMها به صورت محلی و یادگیری تقویتی را ساده می‌کند، همراه با دفترچه‌های آماده.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند.

سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۱۸ — هوش مصنوعی مولد مایکروسافت

درس ۱۹ از ۲۱

مدل‌های زبانی کوچک (Small Language Models - SLM)

انقلاب مدل‌های کوچک با کارایی فوق‌العاده مانند سری Phi-3 مایکروسافت، اجرای محلی بدون نیاز به کارت گرافیک سنگین و کاربرد در Edge

هوش مصنوعی زایشی حوزه‌ای جذاب از هوش مصنوعی است که بر ایجاد سیستم‌هایی تمرکز دارد که قادر به تولید محتوای جدید باشند. این محتوا می‌تواند شامل متن، تصاویر، موسیقی و حتی محیط‌های مجازی کامل باشد. یکی از هیجان‌انگیزترین کاربردهای هوش مصنوعی زایشی در حوزه مدل‌های زبان است.

مدل‌های زبان کوچک چیستند؟

مدل زبان کوچک (SLM) نوعی نسخه مقیاس‌کاهش یافته از مدل زبان بزرگ (LLM) است که بسیاری از اصول و تکنیک‌های معماری مدل‌های بزرگ را به کار می‌برد، در حالی که اثر محاسباتی آن به‌طور قابل توجهی کاهش یافته است.

مدل‌های زبان کوچک زیرمجموعه‌ای از مدل‌های زبان هستند که برای تولید متن شبیه انسان طراحی شده‌اند. برخلاف همتایان بزرگ‌تر خود مانند GPT-4، SLM‌ها جمع و جورتر و کارآمدترند و آن‌ها را برای کاربردهایی که منابع محاسباتی محدود است، ایده‌آل می‌کند. با وجود اندازه کوچک‌تر، آن‌ها می‌توانند طیف گسترده‌ای از وظایف را انجام دهند. معمولاً SLMها با فشرده‌سازی یا تقطیر مدل‌های بزرگ ساخته می‌شوند، با هدف حفظ بخش قابل توجهی از عملکرد و قابلیت‌های زبانی مدل اصلی. این کاهش اندازه مدل، پیچیدگی کلی را کاهش داده و SLMها را از نظر استفاده از حافظه و نیازهای محاسباتی بهینه‌تر می‌سازد. با وجود این بهینه‌سازی‌ها، SLMها همچنان قادر به انجام طیف وسیعی از وظایف پردازش زبان طبیعی (NLP) هستند:

- تولید متن: ایجاد جملات یا پاراگراف‌های هماهنگ و مرتبط با زمینه.
- تکمیل متن: پیش‌بینی و تکمیل جملات بر اساس یک متن اولیه داده‌شده.
- ترجمه: تبدیل متن از یک زبان به زبان دیگر.
- خلاصه‌سازی: کوتاه کردن متون بلند به خلاصه‌های کوتاه‌تر و قابل فهم‌تر.

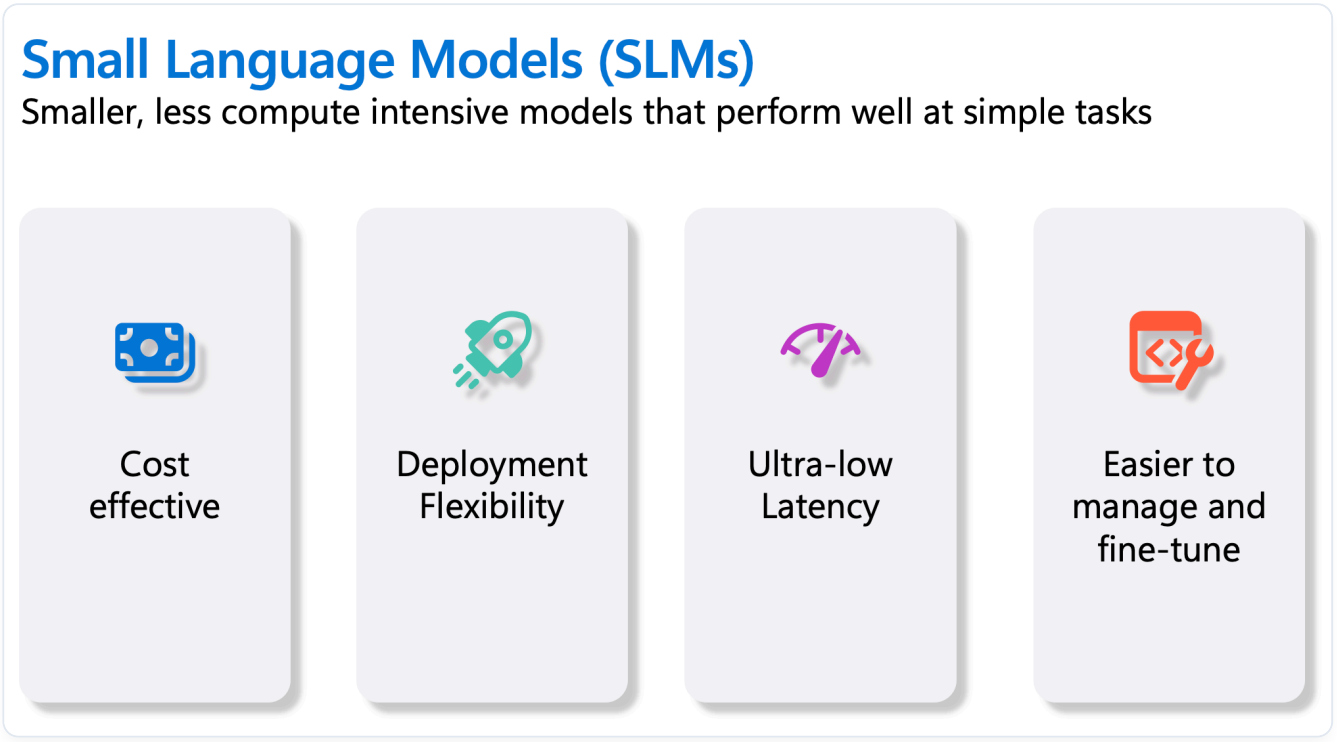
البته با برخی مصالحه‌ها در عملکرد یا عمق درک در مقایسه با مدل‌های بزرگ‌تر.

مدل‌های زبان کوچک چگونه کار می‌کنند؟

SLMها روی حجم زیادی از داده‌های متنی آموزش می‌بینند. در طول آموزش، آن‌ها الگوها و ساختارهای زبان را می‌آموزند تا بتوانند متنی منطبق بر قواعد دستوری و مناسب با زمینه تولید کنند. فرایند آموزش شامل موارد زیر است:

- جمع‌آوری داده: گردآوری مجموعه‌داده‌های بزرگ متنی از منابع مختلف.
- پیش‌پردازش: پاک‌سازی و سازماندهی داده‌ها برای آماده‌سازی جهت آموزش.
- آموزش: استفاده از الگوریتم‌های یادگیری ماشینی برای آموزش مدل درک و تولید متن.
- تنظیم دقیق: بهبود عملکرد مدل در وظایف خاص.

توسعه SLMها با نیاز روزافزون به مدل‌هایی که می‌توانند در محیط‌های با منابع محدود مانند دستگاه‌های موبایل یا پلتفرم‌های محاسبات لبه‌ای اجرا شوند هم‌راستا است، جایی که مدل‌های بزرگ به دلیل نیاز زیاد منابع، عملی نیستند. با تمرکز بر بهره‌وری، SLMها بین عملکرد و دسترسی تعادل برقرار می‌کنند و کاربردهای وسیع‌تری در حوزه‌های مختلف فراهم می‌آورند.



اهداف آموزشی

در این درس قصد داریم دانش مربوط به SLM را معرفی کرده و آن را با Microsoft Phi-3 ترکیب کنیم تا سناریوهای متفاوت در حوزه تولید متن، دید و MoE را آموزش دهیم.

تا پایان این درس، شما باید قادر باشید به سوالات زیر پاسخ دهید:

- مدل زبان کوچک چیست؟
- تفاوت بین SLM و LLM چیست؟
- خانواده Microsoft Phi-3/3.5 چیست؟
- چگونه می‌توان با خانواده Microsoft Phi-3/3.5 انجام استنتاج داد؟

آماده‌اید؟ بیایید شروع کنیم.

تفاوت‌های بین مدل‌های زبان بزرگ (LLMs) و مدل‌های زبان کوچک (SLMs)

هر دو LLM و SLM بر پایه اصول پایه یادگیری ماشینی احتمالاتی ساخته شده‌اند و رویکردهای مشابهی در طراحی معماری، روش‌های آموزش، فرایندهای تولید داده و ارزیابی مدل دنبال می‌کنند. با این حال، چند فاکتور کلیدی این دو نوع مدل را از هم متمایز می‌کند.

کاربردهای مدل‌های زبان کوچک

SLMها کاربردهای گسترده‌ای دارند، از جمله:

- چت‌بات‌ها: ارائه پشتیبانی مشتری و تعامل با کاربران به صورت مکالمه‌ای.
- تولید محتوا: کمک به نویسندگان در تولید ایده‌ها یا حتی پیش‌نویس مقالات کامل.
- آموزش: یاری دانش‌آموزان در انجام تکالیف نوشتاری یا یادگیری زبان‌های جدید.
- دسترسی: ایجاد ابزارهایی برای افراد دارای معلولیت، مانند سیستم‌های تبدیل متن به صدا.

اندازه

تمایز اصلی بین LLMها و SLMها در مقیاس مدل‌ها نهفته است. LLMها، مانند ChatGPT (GPT-4)، ممکن است حدود ۱.۷۶ تریلیون پارامتر داشته باشند، در حالی که مدل‌های زبان کوچک متن‌باز مانند Mistral 7B با حدود ۷ میلیارد پارامتر طراحی شده‌اند که تعداد پارامترهای بسیار کمتری است. این اختلاف عمدتاً ناشی از تفاوت‌ها در معماری مدل و فرایندهای آموزش است. برای مثال، ChatGPT از سازوکار توجه خودکار در چارچوب کدکننده-رمزگذار استفاده می‌کند، در حالی که Mistral 7B از مکانیزم توجه مرور شونده بهره می‌برد که اجازه آموزش کارآمدتر در مدل فقط رمزگذار را می‌دهد. این تفاوت معماری تأثیرات عمیقی بر پیچیدگی و عملکرد این مدل‌ها دارد.

درک

SLMها معمولاً برای کارایی بهتر در حوزه‌های خاص بهینه‌سازی شده‌اند، که آن‌ها را بسیار تخصصی و در عین حال محدود در توانایی ارائه درک گسترده زمینه‌ای در حوزه‌های مختلف می‌کند. در مقابل، LLMها هدف‌شان شبیه‌سازی هوش انسانی در سطح جامع‌تری است. این مدل‌ها با داده‌های وسیع و متنوع آموزش می‌بینند و برای عملکرد خوب در بسیاری از حوزه‌ها طراحی شده‌اند و انعطاف‌پذیری و سازگاری بیشتری دارند. بنابراین، LLMها برای طیف وسیع‌تری از وظایف پایین‌دستی مانند پردازش زبان طبیعی و برنامه‌نویسی مناسب‌ترند.

محاسبات

آموزش و استقرار LLMها فرآیندهای پرهزینه‌ای هستند که معمولاً نیازمند زیرساخت‌های محاسباتی قوی شامل خوشه‌های گسترده GPU هستند. برای مثال، آموزش مدلی مانند ChatGPT از ابتدا ممکن است به هزاران GPU برای مدت طولانی نیاز داشته باشد. در مقابل، SLMها به دلیل پارامترهای کمتر، از نظر منابع محاسباتی دسترسی‌پذیرترند. مدل‌هایی مثل Mistral 7B

را می‌توان روی ماشین‌های محلی با توانایی GPU متوسط آموزش داد و اجرا کرد، اگرچه آموزش هنوز چند ساعتی به چند GPU نیاز دارد.

تورش

تورش یک مسئله شناخته‌شده در LLMها است که عمدتاً ناشی از طبیعت داده‌های آموزش است. این مدل‌ها معمولاً متکی به داده‌های خام و در دسترس عمومی از اینترنت هستند که ممکن است برخی گروه‌ها را کم‌نمایش یا نادرست نشان دهند، برچسب‌گذاری نادرست داشته باشند یا تورش‌های زبانی ناشی از گویش، تفاوت‌های جغرافیایی و قواعد دستوری را منعکس کنند. علاوه بر این، پیچیدگی معماری LLM می‌تواند به‌طور ناخواسته تورش را افزایش دهد که بدون تنظیم دقیق دقیق ممکن است شناسایی نشود. از سوی دیگر، SLMها که روی داده‌های محدودتر و تخصصی‌تر آموزش دیده‌اند، به طور ذاتی کمتر در معرض این تورش‌ها قرار دارند، اگرچه به آن‌ها مصون نیستند.

استنتاج

اندازه کاهش یافته SLMها به آن‌ها مزیت قابل توجهی در سرعت استنتاج می‌دهد، که امکان تولید خروجی‌ها را به‌طور مؤثر بر روی سخت‌افزار محلی بدون نیاز به پردازش موازی گسترده فراهم می‌کند. در مقابل، LLMها به دلیل اندازه و پیچیدگی خود، اغلب به منابع محاسباتی موازی قابل توجهی برای دستیابی به زمان‌های استنتاج قابل قبول نیازمندند. حضور کاربران همزمان متعدد نیز زمان پاسخگویی LLMها را به‌ویژه در مقیاس بزرگ کاهش می‌دهد.

خلاصه اینکه، هرچند LLMها و SLMها بنیانی مشترک در یادگیری ماشینی دارند، اما در اندازه مدل، نیازهای منابع، درک زمینه‌ای، آسیب‌پذیری به تورش و سرعت استنتاج تفاوت‌های قابل توجهی دارند. این تفاوت‌ها نمایانگر تناسب هر کدام با موارد استفاده متفاوت است، به طوری که LLMها چندمنظوره‌تر ولی پرمصرف‌تر و SLMها کارآمدتر در حوزه‌های خاص با نیازهای محاسباتی کمتر هستند.

توجه: در این درس، ما SLM را با استفاده از Microsoft Phi-3 / 3.5 به عنوان مثال معرفی خواهیم کرد.

معرفی خانواده Phi-3 / Phi-3.5

خانواده Phi-3 / 3.5 عمدتاً هدفمند برای سناریوهای متنی، بینایی و عامل (MoE) است:

Phi-3 / 3.5 Instruct

عمدتاً برای تولید متن، تکمیل چت و استخراج اطلاعات محتوا و غیره.

Phi-3-mini

مدل زبان ۳.۸ میلیارد پارامتری در Microsoft Foundry، Hugging Face و Ollama در دسترس است. مدل‌های Phi-3 در معیارهای کلیدی به طور قابل توجهی از مدل‌های هم‌اندازه و بزرگ‌تر بهتر عمل می‌کنند (اعداد معیار در پایین، هر چه بیشتر بهتر). Phi-3-mini مدل‌هایی با دو برابر اندازه خود را پشت سر می‌گذارد، در حالی که Phi-3-small و Phi-3-medium از مدل‌های بزرگ‌تر از جمله GPT-3.5 بهتر عمل می‌کنند.

Phi-3-small و medium

با تنها ۷ میلیارد پارامتر، Phi-3-small در معیارهای مختلف زبان، استدلال، کدنویسی و ریاضیات از GPT-3.5T پیشی می‌گیرد.

Phi-3-medium با ۱۴ میلیارد پارامتر این روند را ادامه می‌دهد و از Gemini 1.0 Pro بهتر عمل می‌کند.

Phi-3.5-mini

می‌توان آن را ارتقاء Phi-3-mini دانست. در حالی که تعداد پارامترها تغییر نکرده‌اند، توانایی پشتیبانی از چندین زبان را بهبود داده (پشتیبانی از ۲۰+ زبان: عربی، چینی، چک، دانمارکی، هلندی، انگلیسی، فنلاندی، فرانسوی، آلمانی، عبری، مجاری، ایتالیایی، ژاپنی، کره‌ای، نروژی، لهستانی، پرتغالی، روسی، اسپانیایی، سوئدی، تایلندی، ترکی، اوکراینی) و پشتیبانی قوی‌تری برای زمینه‌های طولانی اضافه کرده است.

Phi-3.5-mini با ۳.۸ میلیارد پارامتر از مدل‌های هم‌اندازه بهتر عمل می‌کند و در حد مدل‌هایی با دو برابر اندازه‌اش است.

Phi-3 / 3.5 Vision

می‌توان مدل Instruct از Phi-3/3.5 را توانایی Phi در فهمیدن دانست، و Vision همان چیزی است که به Phi چشم برای درک جهان می‌دهد.

Phi-3-Vision

Phi-3-vision، با تنها ۴.۲ میلیارد پارامتر، این روند را ادامه می‌دهد و در وظایف استدلال بینایی عمومی، OCR و درک جدول و نمودار از مدل‌های بزرگ‌تری مانند Claude-3 Haiku و Gemini 1.0 Pro V پیشی می‌گیرد.

Phi-3.5-Vision

Phi-3.5-Vision نیز یک ارتقاء برای Phi-3-Vision است که پشتیبانی از چندین تصویر را اضافه کرده است. می‌توان آن را به عنوان بهبود بینایی در نظر گرفت، به طوری که نه تنها تصاویر بلکه ویدیوها را نیز می‌بینند.

Phi-3.5-vision در وظایف OCR، درک جدول و نمودار از مدل‌های بزرگ‌تر مانند Claude-3.5 Sonnet و Gemini 1.5 Flash بهتر است و در وظایف استدلال دانش بینایی عمومی در حد آن‌ها است. همچنین پشتیبانی از ورودی چندفریمی دارد، یعنی استدلال روی چندین تصویر ورودی انجام می‌دهد.

Phi-3.5-MoE

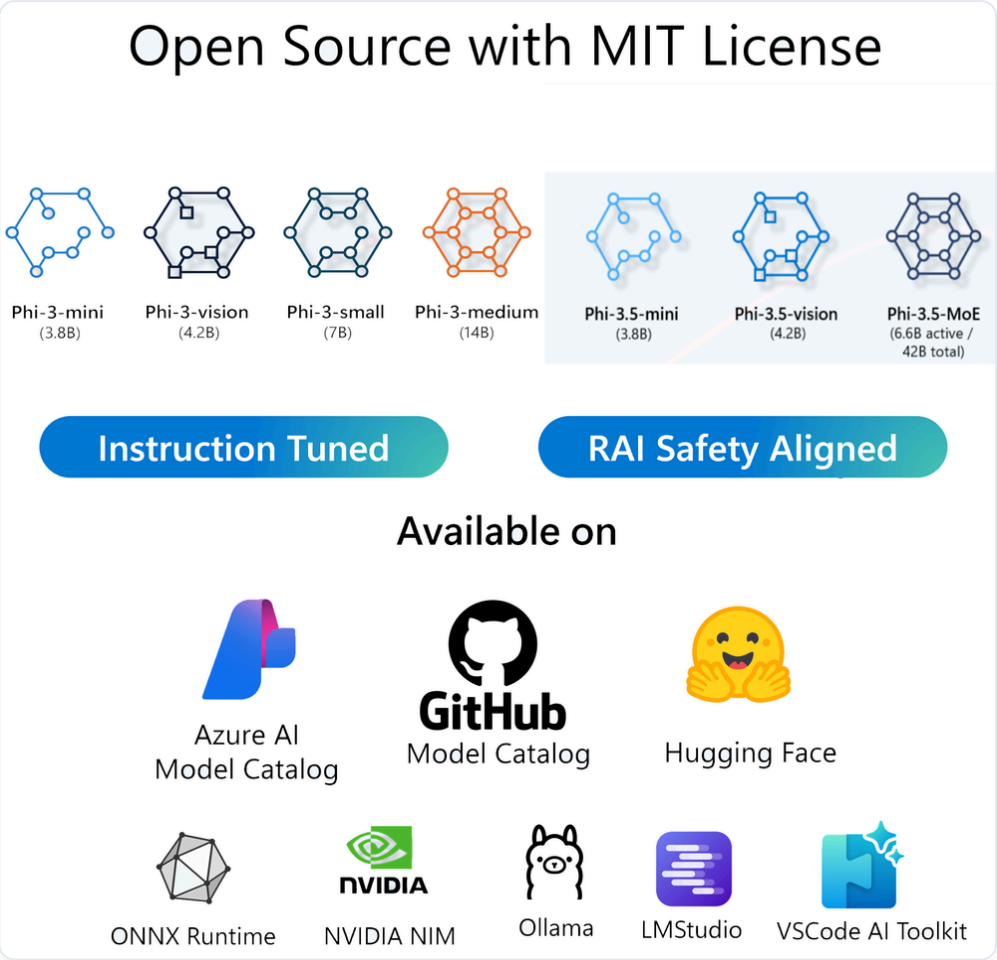
Mixture of Experts (MoE) امکان پیش‌آموزش مدل‌ها را با محاسبات بسیار کمتر فراهم می‌کند، که یعنی می‌توان اندازه مدل یا مجموعه داده را به طور چشمگیری با همان بودجه محاسباتی مدل چگال افزایش داد. به خصوص، یک مدل MoE باید کیفیت مشابه نمونه چگال خود را در مراحل پیش‌آموزش بسیار سریع‌تر به دست آورد.

Phi-3.5-MoE شامل ۱۶ ماژول خبره ۳.۸ میلیارد پارامتری است. Phi-3.5-MoE با تنها ۶.۶ میلیارد پارامتر فعال، به سطحی مشابه در استدلال، درک زبان و ریاضیات با مدل‌های بسیار بزرگ‌تر می‌رسد.

می‌توان مدل خانواده Phi-3/3.5 را بر اساس سناریوهای مختلف استفاده کرد. برخلاف LLM، می‌توان Phi-3/3.5-mini یا Phi-3/3.5-Vision را روی دستگاه‌های لبه‌ای مستقر کرد.

چگونه از مدل‌های خانواده Phi-3/3.5 استفاده کنیم

قصد داریم Phi-3/3.5 را در سناریوهای مختلف به کار ببریم. در ادامه، استفاده Phi-3/3.5 را بر اساس سناریوهای متفاوت بررسی می‌کنیم.



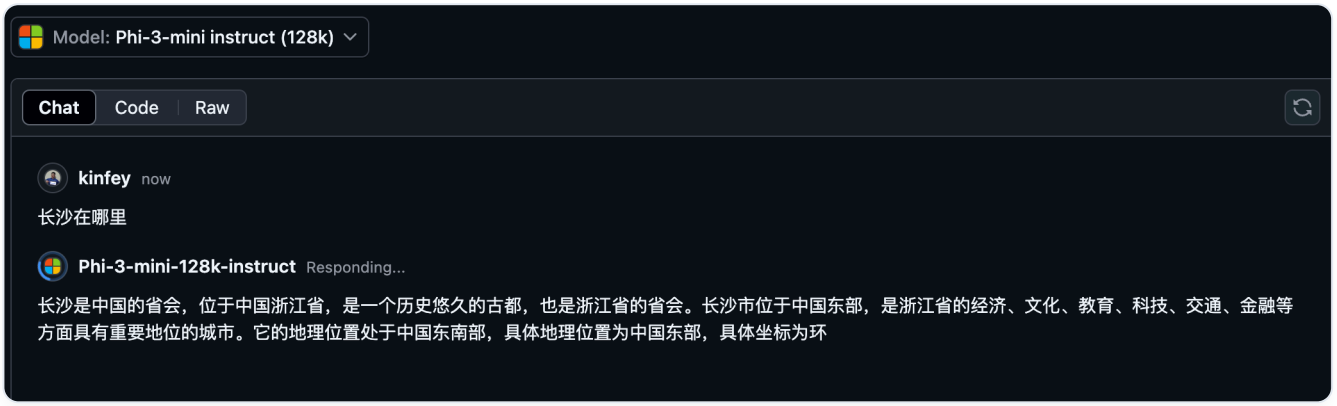
استنتاج از طریق APIهای ابری

مدل‌های Microsoft Foundry

> **یادداشت تکمیلی**
>
> **توجه:** گیت‌هاب مدل‌ها در پایان ژوئیه ۲۰۲۶ بازنشسته می‌شود. مدل‌های Microsoft Foundry جایگزین مستقیم آن هستند.

مدل‌های Microsoft Foundry ساده‌ترین راه است. شما می‌توانید به سرعت به مدل Phi-3/3.5-Instruct از طریق کاتالوگ مدل Foundry دسترسی پیدا کنید. با ترکیب SDK استنتاج Azure AI / OpenAI SDK، می‌توانید از طریق کد به API دسترسی داشته و فراخوانی Phi-3/3.5-Instruct را انجام دهید. همچنین می‌توانید اثرات مختلف را از طریق محیط Playground امتحان کنید.

- نمونه: مقایسه اثرات Phi-3-mini و Phi-3.5-mini در سناریوهای چینی



Microsoft Foundry

یا اگر بخواهیم از مدل‌های بینایی و MoE استفاده کنیم، می‌توان از Microsoft Foundry برای تکمیل فراخوانی‌ها بهره برد. اگر علاقه‌مندید، می‌توانید «کتابچه راهنمای Phi-3» را برای یادگیری نحوه فراخوانی MoE، Vision، Phi-3/3.5 Instruct از طریق Microsoft Foundry مطالعه کنید برای خواندن این لینک کلیک کنید

NVIDIA NIM

علاوه بر کاتالوگ مدل‌های ابری Microsoft Foundry، می‌توانید از NVIDIA NIM برای تکمیل فراخوانی‌های مرتبط استفاده کنید. شما می‌توانید به NVIDIA NIM مراجعه کنید تا فراخوانی‌های API خانواده Phi-3/3.5 را انجام دهید. NVIDIA NIM (خدمات میکرو استنتاج NVIDIA) مجموعه‌ای از خدمات میکرو استنتاج شتاب‌داده‌شده است که به توسعه‌دهندگان کمک می‌کند مدل‌های هوش مصنوعی را به طور مؤثر در محیط‌های مختلف از جمله ابرها، مراکز داده و ایستگاه‌های کاری مستقر کنند.

در اینجا چند ویژگی کلیدی NVIDIA NIM آورده شده است:

- **سهولت استقرار:** NIM امکان استقرار مدل‌های هوش مصنوعی با یک دستور را فراهم می‌کند و ادغام در جریان‌های کاری موجود را ساده می‌سازد.
- **عملکرد بهینه‌شده:** از موتورهای استنتاج پیش‌بهینه‌شده NVIDIA مانند TensorRT و TensorRT-LLM استفاده می‌کند تا تاخیر پایین و توان عملیاتی بالا را تضمین کند.
- **مقیاس‌پذیری:** NIM از مقیاس‌بندی خودکار روی Kubernetes پشتیبانی می‌کند و به این ترتیب می‌تواند به‌طور مؤثر بارهای کاری متغیر را مدیریت کند.
- **امنیت و کنترل:** سازمان‌ها می‌توانند با میزبانی خودکار خدمات ریزمعماری NIM روی زیرساخت مدیریت شده خود، کنترل داده‌ها و برنامه‌های خود را حفظ کنند.
- **رابط‌های استاندارد API:** NIM رابط‌های استاندارد صنعتی را ارائه می‌دهد که ساخت و ادغام برنامه‌های هوش مصنوعی مانند چت‌بات‌ها، دستیارهای هوش مصنوعی و غیره را آسان می‌سازد.

NIM بخشی از NVIDIA AI Enterprise است که هدفش ساده‌سازی استقرار و عملیاتی‌سازی مدل‌های هوش مصنوعی است، به‌گونه‌ای که آنها به‌طور کارآمد روی GPUهای NVIDIA اجرا شوند.

- نمایش: استفاده از NVIDIA NIM برای فراخوانی Phi-3.5-Vision-API [برای بازدید کلیک کنید]

اجرای Phi-3/3.5 به‌صورت محلی

استنتاج در رابطه با Phi-3، یا هر مدل زبانی مانند GPT-3، به فرآیند تولید پاسخ‌ها یا پیش‌بینی‌ها بر اساس ورودی دریافتی اشاره دارد. هنگامی که یک پرسش یا دستور به Phi-3 می‌دهید، شبکه عصبی آموزش‌دیده‌ی آن به تجزیه و تحلیل الگوها و روابط موجود در داده‌های آموزشی می‌پردازد تا محتمل‌ترین و مرتبط‌ترین پاسخ را استنتاج کند.

Hugging Face Transformers **Hugging Face Transformer** یک کتابخانه قدرتمند طراحی شده برای پردازش زبان طبیعی (NLP) و سایر کارهای یادگیری ماشین است. در اینجا چند نکته کلیدی درباره آن آمده است:

1. **مدل‌های پیش‌آموزش‌دیده:** هزاران مدل آموزش دیده ارائه می‌دهد که می‌توان برای وظایفی مانند طبقه‌بندی متن، شناسایی موجودیت‌های نام‌برده‌شده، پاسخ به سوال، خلاصه‌سازی، ترجمه و تولید متن از آنها استفاده کرد.
2. **قابلیت همکاری فریمورک‌ها:** این کتابخانه از چندین فریمورک یادگیری عمیق شامل PyTorch، TensorFlow و JAX پشتیبانی می‌کند که امکان آموزش مدل در یک فریمورک و استفاده در فریمورک دیگر را می‌دهد.
3. **قابلیت‌های چندرسانه‌ای:** علاوه بر NLP، Hugging Face Transformers از وظایف پردازش تصویر (مثلاً طبقه‌بندی تصویر، تشخیص شیء) و پردازش صوت (مثلاً تشخیص گفتار، طبقه‌بندی صوت) نیز پشتیبانی می‌کند.
4. **سهولت استفاده:** این کتابخانه رابط‌ها و ابزارهایی را برای دانلود و تنظیم دقیق آسان مدل‌ها ارائه می‌دهد که آن را برای مبتدیان و متخصصان قابل دسترسی می‌کند.

5. **جامعه و منابع:** Hugging Face دارای جامعه زنده و مستندات، آموزش‌ها و راهنماهای گسترده‌ای است که به کاربران کمک می‌کند شروع کنند و بیشترین بهره را از کتابخانه ببرند. مستندات رسمی یا مخزن گیت‌هاب آنها GitHub reposito ry.

این متد رایج‌ترین روش است، اما به شتاب‌دهی GPU نیاز دارد. به هر حال، سناریوهایی مانند Vision و MoE محاسبات زیادی نیاز دارند که روی CPU اگر کم‌دقت یا کوانتیزه نباشند بسیار کند خواهند بود.

- نمایش: استفاده از Transformer برای فراخوانی Phi-3.5-Instruct برای بازدید کلیک کنید
- نمایش: استفاده از Transformer برای فراخوانی Phi-3.5-Vision برای بازدید کلیک کنید
- نمایش: استفاده از Transformer برای فراخوانی Phi-3.5-MoE برای بازدید کلیک کنید

Ollama **Ollama** پلتفرمی است طراحی شده تا اجرای مدل‌های زبان بزرگ (LLMs) به‌صورت محلی روی دستگاه شما آسان‌تر شود. این پلتفرم از مدل‌های مختلفی مانند Mistral، Phi 3، Llama 3.1 و Gemma 2 پشتیبانی می‌کند. Ollama روند کار را با بسته‌بندی وزن مدل، پیکربندی و داده‌ها در یک بسته واحد ساده می‌کند و استفاده کاربران برای سفارشی‌سازی و ایجاد مدل‌های خود را آسان‌تر می‌نماید. Ollama برای macOS، لینوکس و ویندوز موجود است. این ابزار عالی است اگر می‌خواهید مدل‌های LLM را بدون وابستگی به خدمات ابری آزمایش یا مستقر کنید. Ollama آسان‌ترین روش است، فقط کافی است دستور زیر را اجرا کنید.

Terminal / Bash

```
ollama run phi3.5
```

Foundry Local

Foundry Local زمان اجرا آفلاین و در دستگاه مایکروسافت برای اجرای مدل‌هایی مانند Phi به‌طور کامل روی سخت‌افزار خودتان است - بدون نیاز به اشتراک Azure، کلید API یا اتصال شبکه. این به طور خودکار بهترین ارائه‌دهنده اجرا (NPU، GPU یا CPU) را انتخاب می‌کند و یک نقطه پایانی سازگار با OpenAI ارائه می‌دهد، بنابراین کدهای موجود `openai` Azure AI/ Inference SDK می‌توانند با حداقل تغییرات به آن اشاره کنند. برای شروع به مستندات Foundry Local مراجعه کنید.

Terminal / Bash

```
winget install Microsoft.FoundryLocal
foundry model run phi-3.5-mini
```

یا به‌طور مستقیم از SDK در پایتون استفاده کنید:

Terminal / Bash

```
pip install foundry-local-sdk
```

Python

```
from foundry_local import FoundryLocalManager

manager = FoundryLocalManager("phi-3.5-mini")
print(manager.endpoint, manager.api_key)
```

ONNX Runtime برای GenAI

ONNX Runtime یک شتاب‌دهنده یادگیری ماشین مستقل از پلتفرم برای استنتاج و آموزش است. ONNX Runtime برای هوش مصنوعی مولد (GENAI) ابزاری قدرتمند است که به شما کمک می‌کند مدل‌های هوش مصنوعی مولد را به طور مؤثر روی پلتفرم‌های مختلف اجرا کنید.

ONNX Runtime چیست؟

ONNX Runtime یک پروژه متن‌باز است که استنتاج با عملکرد بالا از مدل‌های یادگیری ماشین را فراهم می‌کند. این پروژه از مدل‌ها با فرمت Open Neural Network Exchange (ONNX) پشتیبانی می‌کند، که استانداردی برای نمایش مدل‌های یادگیری ماشین است. استنتاج ONNX Runtime می‌تواند تجربیات سریع‌تر کاربر و هزینه‌های کمتر را فراهم کند و از مدل‌های فریمورک‌های یادگیری عمیق مانند PyTorch و TensorFlow/Keras و همچنین کتابخانه‌های یادگیری ماشین کلاسیک مانند scikit-learn، LightGBM، XGBoost و غیره پشتیبانی می‌کند. ONNX Runtime با سخت‌افزار، درایورها و سیستم‌های عامل مختلف سازگار است و با بهره‌گیری از شتاب‌دهنده‌های سخت‌افزاری، به همراه بهینه‌سازی‌ها و تبدیل‌های گراف، عملکرد بهینه را ارائه می‌دهد.

هوش مصنوعی مولد چیست؟

هوش مصنوعی مولد به سیستم‌های هوش مصنوعی اشاره دارد که می‌توانند بر اساس داده‌های آموزش‌دیده شده، محتوای جدیدی مانند متن، تصویر یا موسیقی تولید کنند. نمونه‌هایی از آن شامل مدل‌های زبانی مانند GPT-3 و مدل‌های تولید تصویر مانند Stable Diffusion است. کتابخانه ONNX Runtime برای GenAI حلقه هوش مصنوعی مولد را برای مدل‌های ONNX فراهم می‌کند، شامل استنتاج با ONNX Runtime، پردازش لگیت‌ها، جستجو و نمونه‌گیری و مدیریت کش KV.

ONNX Runtime برای GENAI

ONNX Runtime برای GENAI قابلیت‌های ONNX Runtime را برای پشتیبانی از مدل‌های هوش مصنوعی مولد گسترش می‌دهد. چند ویژگی کلیدی عبارتند از:

- **پشتیبانی گسترده از پلتفرم:** روی پلتفرم‌های مختلف از جمله ویندوز، لینوکس، macOS، اندروید و iOS کار می‌کند.
- **پشتیبانی مدل:** از بسیاری از مدل‌های معروف هوش مصنوعی مولد مانند BLOOM، GPT-Neo، LLaMA و غیره پشتیبانی می‌کند.
- **بهینه‌سازی عملکرد:** شامل بهینه‌سازی برای شتاب‌دهنده‌های سخت‌افزاری مختلف مانند GPUهای NVIDIA، GPU های AMD و غیره است.
- **سهولت استفاده:** رابط‌های برنامه‌نویسی (API) برای ادغام آسان در برنامه‌ها فراهم می‌کند، به‌طوری که بتوانید با کد حداقلی متن، تصویر و سایر محتوا را تولید کنید.
- کاربران می‌توانند متد سطح بالا generate() را فراخوانی کنند یا هر تکرار مدل را در یک حلقه اجرا کنند، به‌طور هم‌زمان یک توکن تولید کنند و در صورت تمایل پارامترهای تولید را درون حلقه به‌روزرسانی کنند.
- ONNX Runtime همچنین از جستجوی greedy/beam و نمونه‌گیری TopK، TopP برای تولید توالی توکن‌ها و پردازش لگیت‌های داخلی مانند جریمه‌های تکرار پشتیبانی می‌کند. شما همچنین می‌توانید به آسانی امتیازدهی سفارشی اضافه کنید.

شروع کار

برای شروع با ONNX Runtime برای GENAI می‌توانید این مراحل را دنبال کنید:

نصب ONNX Runtime:

Python

```
pip install onnxruntime
```

نصب افزونه‌های هوش مصنوعی مولد:

Python

```
pip install onnxruntime-genai
```

اجرای مدل: در اینجا یک مثال ساده با پایتون آورده شده است:

Python

```
import onnxruntime_genai as og

model = og.Model('path_to_your_model.onnx')

tokenizer = og.Tokenizer(model)

input_text = "Hello, how are you?"

input_tokens = tokenizer.encode(input_text)

output_tokens = model.generate(input_tokens)

output_text = tokenizer.decode(output_tokens)

print(output_text)
```

نمایش: استفاده از ONNX Runtime GenAI برای فراخوانی Phi-3.5-Vision

Python

```
import onnxruntime_genai as og
model_path = './Your Phi-3.5-vision-instruct ONNX Path'
img_path = './Your Image Path'
model = og.Model(model_path)
processor = model.create_multimodal_processor()
tokenizer_stream = processor.create_stream()
text = "Your Prompt"
prompt = "<|user|>\n"
prompt += "<|image_1|>\n"
prompt += f"{text}<|end|>\n"
prompt += "<|assistant|>\n"
image = og.Images.open(img_path)
inputs = processor(prompt, images=image)
params = og.GeneratorParams(model)
params.set_inputs(inputs)
params.set_search_options(max_length=3072)
generator = og.Generator(model, params)
while not generator.is_done():
    generator.compute_logits()
    generator.generate_next_token()
    new_token = generator.get_next_tokens()[0]
    output = tokenizer_stream.decode(new_token)
    print(tokenizer_stream.decode(new_token), end='', flush=True)
```

سایر روش‌ها

علاوه بر روش‌های مرجع Ollama، ONNX Runtime و Foundry Local، ما می‌توانیم مرجع مدل‌های کوانتیزه شده را براساس روش‌های مرجع مدل ارائه شده توسط تولیدکنندگان مختلف کامل کنیم. مانند فریمورک Apple MLX همراه با Apple Metal، Qualcomm QNN با NPU، Intel OpenVINO با CPU/GPU، و غیره. می‌توانید محتوای بیشتری را از کتاب آشپزی Phi-3 دریافت کنید.

بیشتر

ما اصول خانواده Phi-3/3.5 را یاد گرفته‌ایم، اما برای یادگیری بیشتر درباره SLM به دانش بیشتری نیاز داریم. پاسخ‌ها را می‌توانید در کتاب آشپزی Phi-3 بیابید. اگر می‌خواهید بیشتر بیاموزید، لطفاً به **کتاب آشپزی Phi-3** مراجعه کنید.

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی **Co-op Translator** ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۱۹ — هوش مصنوعی مولد مایکروسافت

درس ۲۰ از ۲۱

ساخت و توسعه با مدل‌های خانواده Mistral

بررسی مدل‌های پیشرفته فرانسوی Mistral 7B و Mixtral 8x7B، معماری مخلوط متخصصین (MoE) و استفاده کاربردی از آن‌ها

مقدمه

این درس شامل موارد زیر است:

- بررسی مدل‌های مختلف میسترال
- درک موارد استفاده و سناریوهای هر مدل
- بررسی نمونه‌های کدی که ویژگی‌های منحصر به فرد هر مدل را نشان می‌دهد.

مدل‌های میسترال

در این درس، سه مدل مختلف میسترال را بررسی خواهیم کرد: **Mistral Small**، **Mistral Large** و **Mistral Nemo**.

هر یک از این مدل‌ها به صورت رایگان در Microsoft Foundry Models در دسترس است. کد در این دفترچه از این مدل‌ها برای اجرای کد استفاده خواهد کرد.

> **یادداشت تکمیلی**
>
> **توجه:** GitHub Models تا پایان ژوئیه ۲۰۲۶ بازنشسته می‌شود. در اینجا جزئیات بیشتری درباره استفاده از Microsoft Foundry Models برای نمونه‌سازی اولیه با مدل‌های هوش مصنوعی آورده شده است.

Mistral Large 2 (2407)

میسترال بزرگ ۲ در حال حاضر مدل شاخص میسترال است و برای استفاده سازمانی طراحی شده است.

این مدل ارتقاء یافته مدل اصلی Mistral Large است و ویژگی‌های زیر را ارائه می‌دهد:

- پنجره متنی بزرگ‌تر - ۱۲۸k در مقابل ۳۲k
- عملکرد بهتر در وظایف ریاضی و برنامه‌نویسی - دقت میانگین ۷۶.۹٪ در مقابل ۶۰.۴٪

- عملکرد چندزبانه افزایش یافته - زبان‌ها شامل: انگلیسی، فرانسوی، آلمانی، اسپانیایی، ایتالیایی، پرتغالی، هلندی، روسی، چینی، ژاپنی، کره‌ای، عربی و هندی.

با این ویژگی‌ها، میسترال بزرگ در موارد زیر برجسته است:

- *تولید تقویت‌شده با بازیابی (RAG)* - به دلیل پنجره متن بزرگ‌تر
- *فراخوانی توابع* - این مدل دارای فراخوانی توابع بومی است که امکان ادغام با ابزارها و APIهای خارجی را فراهم می‌کند. این فراخوانی‌ها می‌توانند به صورت همزمان یا یکی پس از دیگری به ترتیب انجام شوند.
- *تولید کد* - این مدل در تولید پایتون، جاوا، تایپ‌اسکریپت و C++ برجسته است.

نمونه RAG با استفاده از Mistral Large 2

در این مثال، از Mistral Large 2 برای اجرای الگوی RAG روی یک سند متنی استفاده می‌شود. سؤال به زبان کره‌ای نوشته شده و درباره فعالیت‌های نویسنده قبل از دانشگاه پرسیده است.

از مدل Embeddings Cohere برای ایجاد بردارهایی از سند متنی و همچنین سؤال استفاده می‌کند. برای این نمونه، از بسته Python faiss به عنوان مخزن بردار استفاده شده است.

پرامپتی که به مدل میسترال ارسال می‌شود شامل هم سؤال‌ها و هم بخش‌های بازیابی‌شده شبیه به سؤال است. سپس مدل پاسخ به زبان طبیعی ارائه می‌دهد.

Python

```
pip install faiss-cpu
```

Python

```
import requests
import numpy as np
import faiss
import os

from azure.ai.inference import ChatCompletionsClient
from azure.ai.inference.models import SystemMessage, UserMessage
from azure.core.credentials import AzureKeyCredential
from azure.ai.inference import EmbeddingsClient

# پروژه «مرور کلی» این موارد را از صفحه Microsoft Foundry خود بگیرید
endpoint = os.environ["AZURE_INFERENCE_ENDPOINT"]
model_name = "Mistral-large"
token = os.environ["AZURE_INFERENCE_CREDENTIAL"]

client = ChatCompletionsClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(token),
)

response = requests.get('https://raw.githubusercontent.com/run-
llama/llama_index/main/docs/docs/examples/data/paul_graham/paul_graham_essay.txt')
text = response.text

chunk_size = 2048
chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]
len(chunks)

embed_model_name = "cohere-embed-v3-multilingual"

embed_client = EmbeddingsClient(
        endpoint=endpoint,
        credential=AzureKeyCredential(token)
)

embed_response = embed_client.embed(
    input=chunks,
    model=embed_model_name
)



text_embeddings = []
for item in embed_response.data:
    length = len(item.embedding)
    text_embeddings.append(item.embedding)
text_embeddings = np.array(text_embeddings)


d = text_embeddings.shape[1]
index = faiss.IndexFlatL2(d)
index.add(text_embeddings)

question = "저자가 대학에 오기 전에 주로 했던 두 가지 일은 무엇이었나요?"

question_embedding = embed_client.embed(
    input=[question],
    model=embed_model_name
)

question_embeddings = np.array(question_embedding.data[0].embedding)


D, I = index.search(question_embeddings.reshape(1, -1), k=2) # شاخص ،فاصله
retrieved_chunks = [chunks[i] for i in I.tolist()[0]]
```

```
prompt = f"""
Context information is below.
---------------------
{retrieved_chunks}
---------------------
Given the context information and not prior knowledge, answer the query.
Query: {question}
Answer:
"""


chat_response = client.complete(
    messages=[
        SystemMessage(content="You are a helpful assistant."),
        UserMessage(content=prompt),
    ],
    temperature=1.0,
    top_p=1.0,
    max_tokens=1000,
    model=model_name
)

print(chat_response.choices[0].message.content)
```

Mistral Small

Mistral Small مدل دیگری از خانواده مدل‌های میسترال در دسته بندی پرمیوم/سازمانی است. همانطور که نام آن نشان می‌دهد، این مدل یک مدل زبان کوچک (SLM) است. مزایای استفاده از Mistral Small عبارتند از:

- صرفه‌جویی در هزینه نسبت به مدل‌های LLM میسترال مانند Mistral Large و NeMo - کاهش قیمت ۸۰٪
- تأخیر کم - پاسخ سریع‌تر نسبت به مدل‌های LLM میسترال
- انعطاف‌پذیر - امکان استقرار در محیط‌های مختلف با محدودیت‌های کمتر در منابع مورد نیاز.

Mistral Small برای موارد زیر عالی است:

- وظایف مبتنی بر متن مانند خلاصه‌سازی، تحلیل احساسات و ترجمه.
- کاربردهایی که درخواست‌های مکرر دارند به دلیل به صرفه بودن هزینه
- وظایف کد با تأخیر کم مانند بررسی و پیشنهاد کد

مقایسه Mistral Small و Mistral Large

برای نشان دادن تفاوت‌های تأخیر بین Mistral Small و Large، سلول‌های زیر را اجرا کنید.

شما باید تفاوتی در زمان پاسخ‌دهی بین ۳ تا ۵ ثانیه مشاهده کنید. همچنین به طول و سبک پاسخ‌ها برای همان پرامپت توجه کنید.

Python

```
import os
endpoint = os.environ["AZURE_INFERENCE_ENDPOINT"]
model_name = "Mistral-small"
token = os.environ["AZURE_INFERENCE_CREDENTIAL"]

client = ChatCompletionsClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(token),
)

response = client.complete(
    messages=[
        SystemMessage(content="You are a helpful coding assistant."),
        UserMessage(content="Can you write a Python function to the fizz buzz test?"),
    ],
    temperature=1.0,
    top_p=1.0,
    max_tokens=1000,
    model=model_name
)

print(response.choices[0].message.content)
```

Python

```
import os
from azure.ai.inference import ChatCompletionsClient
from azure.ai.inference.models import SystemMessage, UserMessage
from azure.core.credentials import AzureKeyCredential

endpoint = os.environ["AZURE_INFERENCE_ENDPOINT"]
model_name = "Mistral-large"
token = os.environ["AZURE_INFERENCE_CREDENTIAL"]

client = ChatCompletionsClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(token),
)

response = client.complete(
    messages=[
        SystemMessage(content="You are a helpful coding assistant."),
        UserMessage(content="Can you write a Python function to the fizz buzz test?"),
    ],
    temperature=1.0,
    top_p=1.0,
    max_tokens=1000,
    model=model_name
)

print(response.choices[0].message.content)
```

Mistral NeMo

در مقایسه با دو مدل دیگر که در این درس بررسی شدند، Mistral NeMo تنها مدل رایگانی است که دارای مجوز Apache2 است.

این مدل به عنوان ارتقاء مدل منبع باز قبلی میسترال، Mistral 7B، دیده می‌شود.

برخی ویژگی‌های دیگر مدل NeMo عبارتند از:

- *توکن‌سازی کارآمدتر:* این مدل از توکنایزر Tekken به جای tiktoken رایج‌تر استفاده می‌کند. این باعث عملکرد بهتر در زبان‌ها و کدهای بیشتر می‌شود.
- *تنظیم دقیق:* مدل پایه برای تنظیم دقیق در دسترس است. این امکان انعطاف بیشتر برای موارد استفاده‌ای که نیاز به تنظیم دقیق دارند را فراهم می‌کند.
- *فراخوانی توابع بومی* - مانند Mistral Large، این مدل برای فراخوانی توابع آموزش دیده است. این ویژگی آن را به یکی از اولین مدل‌های منبع باز با این قابلیت تبدیل می‌کند.

مقایسه توکنایزرها

در این نمونه، نحوه توکن‌سازی Mistral NeMo را در مقایسه با Mistral Large بررسی خواهیم کرد.

هر دو نمونه همان پرامپت را می‌گیرند اما باید ببینید که NeMo تعداد توکن‌های کمتری نسبت به Mistral Large برمی‌گرداند.

Terminal / Bash

```
pip install mistral-common
```

Python

```
# وارد کردن بسته‌های مورد نیاز:
from mistral_common.protocol.instruct.messages import (
    UserMessage,
)
from mistral_common.protocol.instruct.request import ChatCompletionRequest
from mistral_common.protocol.instruct.tool_calls import (
    Function,
    Tool,
)
from mistral_common.tokens.tokenizers.mistral import MistralTokenizer

# بارگذاری توکنایزر میسترال

model_name = "open-mistral-nemo"

tokenizer = MistralTokenizer.from_model(model_name)

# توکنیزه کردن یک لیست از پیام‌ها
tokenized = tokenizer.encode_chat_completion(
    ChatCompletionRequest(
        tools=[
            Tool(
                function=Function(
                    name="get_current_weather",
                    description="Get the current weather",
                    parameters={
                        "type": "object",
                        "properties": {
                            "location": {
                                "type": "string",
                                "description": "The city and state, e.g. San Francisco, CA",
                            },
                            "format": {
                                "type": "string",
                                "enum": ["celsius", "fahrenheit"],
                                "description": "The temperature unit to use. Infer this from
the user's location.",
                            },
                        },
                        "required": ["location", "format"],
                    },
                )
            )
        ],
        messages=[
            UserMessage(content="What's the weather like today in Paris"),
        ],
        model=model_name,
    )
)
tokens, text = tokenized.tokens, tokenized.text

# شمارش تعداد توکن‌ها
print(len(tokens))
```

Python

```
# وارد کردن بسته‌های مورد نیاز:
from mistral_common.protocol.instruct.messages import (
    UserMessage,
)
from mistral_common.protocol.instruct.request import ChatCompletionRequest
from mistral_common.protocol.instruct.tool_calls import (
    Function,
    Tool,
)
from mistral_common.tokens.tokenizers.mistral import MistralTokenizer

# بارگذاری توکنایزر میسترال

model_name = "mistral-large-latest"

tokenizer = MistralTokenizer.from_model(model_name)

# توکنیزه کردن یک لیست پیام‌ها
tokenized = tokenizer.encode_chat_completion(
    ChatCompletionRequest(
        tools=[
            Tool(
                function=Function(
                    name="get_current_weather",
                    description="Get the current weather",
                    parameters={
                        "type": "object",
                        "properties": {
                            "location": {
                                "type": "string",
                                "description": "The city and state, e.g. San Francisco, CA",
                            },
                            "format": {
                                "type": "string",
                                "enum": ["celsius", "fahrenheit"],
                                "description": "The temperature unit to use. Infer this from
the user's location.",
                            },
                        },
                        "required": ["location", "format"],
                    },
                )
            )
        ],
        messages=[
            UserMessage(content="What's the weather like today in Paris"),
        ],
        model=model_name,
    )
)
tokens, text = tokenized.tokens, tokenized.text

# شمارش تعداد توکن‌ها
print(len(tokens))
```

یادگیری اینجا متوقف نمی‌شود، سفر را ادامه دهید

پس از اتمام این درس، مجموعه Generative AI Learning collection ما را برای ادامه ارتقای دانش خود در زمینه هوش مصنوعی مولد بررسی کنید!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۲۰ – هوش مصنوعی مولد مایکروسافت

درس ۲۱ از ۲۱

ساخت و توسعه با مدل‌های خانواده Meta و Llama

نقش بنیادین متا در هوش مصنوعی متن‌باز، معماری Llama 3، قابلیت‌های چندزبانه، پروانه‌های انتشار و کاربرد در پروژه‌های مقیاس‌پذیر

مقدمه

این درس موارد زیر را پوشش می‌دهد:

- بررسی دو مدل اصلی خانواده Meta - Llama 3.1 و Llama 3.2
- درک کاربردها و سناریوهای هر مدل
- نمونه کد برای نشان دادن ویژگی‌های منحصربه‌فرد هر مدل

خانواده مدل‌های Meta

در این درس، ما ۲ مدل از خانواده Meta یا "گله Llama" را بررسی خواهیم کرد - Llama 3.1 و Llama 3.2.

این مدل‌ها در نسخه‌های مختلف عرضه شده و در کاتالوگ مدل‌های Microsoft Foundry در دسترس هستند.

> **یادداشت تکمیلی**
>
> **توجه:** مدل‌های GitHub تا پایان ژوئیه ۲۰۲۶ بازنشسته خواهند شد. در اینجا جزئیات بیشتری در مورد استفاده از مدل‌های Microsoft Foundry برای نمونه‌سازی با مدل‌های هوش مصنوعی آمده است.

نسخه‌های مدل‌ها:

- Llama 3.1 - 70B Instruct
- Llama 3.1 - 405B Instruct
- Llama 3.2 - 11B Vision Instruct
- Llama 3.2 - 90B Vision Instruct

توجه: Llama 3 نیز در مدل‌های Microsoft Foundry موجود است اما در این درس پوشش داده نمی‌شود

Llama 3.1

با ۴۰۵ میلیارد پارامتر، Llama 3.1 در دسته مدل‌های منبع‌باز زبان قرار می‌گیرد.

این مدل به‌روزرسانی مدل قبلی Llama 3 است و امکانات زیر را ارائه می‌دهد:

- پنجره متنی بزرگ‌تر - ۱۲۸ هزار توکن در مقابل ۸ هزار توکن
- حداکثر توکن‌های خروجی بزرگ‌تر - ۴۰۹۶ در مقابل ۲۰۴۸
- پشتیبانی چندزبانه بهتر - به دلیل افزایش تعداد توکن‌های آموزش

این‌ها به Llama 3.1 اجازه می‌دهند در ساخت برنامه‌های GenAI کاربردهای پیچیده‌تری را مدیریت کند، شامل:

- فراخوانی تابع بومی - توانایی فراخوانی ابزارها و توابع خارجی خارج از روند LLM
- عملکرد بهتر در RAG - به علت پنجره متنی بزرگ‌تر
- تولید داده‌های مصنوعی - توانایی ایجاد داده‌های مؤثر برای کارهایی مانند تنظیم دقیق مدل

فراخوانی تابع بومی

Llama 3.1 برای فراخوانی توابع یا ابزارها تنظیم دقیق شده تا مؤثرتر باشد. همچنین دو ابزار داخلی دارد که مدل بر اساس درخواست کاربر تشخیص می‌دهد باید استفاده شوند. این ابزارها عبارتند از:

- **Brave Search** - می‌توان از آن برای دریافت اطلاعات به‌روز مانند وضعیت هوا با انجام جستجوی وب استفاده کرد
- **Wolfram Alpha** - می‌توان برای محاسبات ریاضی پیچیده‌تر از آن استفاده کرد بنابراین نیاز به نوشتن توابع خودتان نیست

همچنین شما می‌توانید ابزارهای سفارشی خود را ایجاد کنید که LLM بتواند آنها را فراخوانی کند.

در نمونه کد زیر:

- ابزارهای در دسترس (brave_search, wolfram_alpha) را در درخواست سیستم تعریف می‌کنیم.
- درخواست کاربر را می‌فرستیم که درباره وضعیت هوا در یک شهر خاص سوال می‌کند.
- LLM با فراخوانی ابزار Brave Search پاسخ خواهد داد که شبیه این است

```
<|python_tag|>brave_search.call(query="Stockholm weather")
```

*توجه: این مثال فقط فراخوانی ابزار را نشان می‌دهد، اگر می‌خواهید نتایج را دریافت کنید باید حساب رایگان در صفحه API Brave ایجاد کرده و خود تابع را تعریف کنید.

Python

```
import os
from azure.ai.inference import ChatCompletionsClient
from azure.ai.inference.models import AssistantMessage, SystemMessage, UserMessage
from azure.core.credentials import AzureKeyCredential

# خود دریافت کنید Microsoft Foundry این موارد را از صفحه «بررسی کلی» پروژه
token = os.environ["AZURE_INFERENCE_CREDENTIAL"]
endpoint = os.environ["AZURE_INFERENCE_ENDPOINT"]
model_name = "Meta-Llama-3.1-405B-Instruct"

client = ChatCompletionsClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(token),
)


tool_prompt=f"""
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Environment: ipython
Tools: brave_search, wolfram_alpha
Cutting Knowledge Date: December 2023
Today Date: 23 July 2024

You are a helpful assistant<|eot_id|>
"""

messages = [
    SystemMessage(content=tool_prompt),
    UserMessage(content="What is the weather in Stockholm?"),

]

response = client.complete(messages=messages, model=model_name)

print(response.choices[0].message.content)
```

Llama 3.2

علیرغم اینکه یک مدل LLM است، یکی از محدودیت‌های Llama 3.1 عدم چندرسانه‌ای بودن آن است. به عبارت دیگر، ناتوانی در استفاده از انواع ورودی مانند تصاویر به‌عنوان درخواست و ارائه پاسخ است. این قابلیت یکی از ویژگی‌های اصلی Llama 3.2 است. این ویژگی‌ها همچنین شامل موارد زیر است:

- چندرسانه‌ای بودن - توانایی ارزیابی همزمان درخواست‌های متنی و تصویری
- نسخه‌های اندازه کوچک تا متوسط (11B و 90B) - این ارائه گزینه‌های استقرار منعطف را فراهم می‌کند،
- نسخه‌های فقط متنی (1B و 3B) - این امکان مدل را برای استقرار روی دستگاه‌های لبه‌ای / موبایل با تأخیر کم فراهم می‌کند

پشتیبانی چندرسانه‌ای گامی بزرگ در دنیای مدل‌های منبع‌باز محسوب می‌شود. نمونه کد زیر هم یک تصویر و هم متن را به عنوان درخواست می‌گیرد تا تحلیلی از تصویر را توسط Llama 3.2 90B ارائه دهد.

پشتیبانی چندرسانه‌ای با Llama 3.2

Python

```
import os
from azure.ai.inference import ChatCompletionsClient
from azure.ai.inference.models import (
    SystemMessage,
    UserMessage,
    TextContentItem,
    ImageContentItem,
    ImageUrl,
    ImageDetailLevel,
)
from azure.core.credentials import AzureKeyCredential

# پروژه «نمای کلی» صفحه از موارد را این Microsoft Foundry خود بگیرید
token = os.environ["AZURE_INFERENCE_CREDENTIAL"]
endpoint = os.environ["AZURE_INFERENCE_ENDPOINT"]
model_name = "Llama-3.2-90B-Vision-Instruct"

client = ChatCompletionsClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(token),
)

response = client.complete(
    messages=[
        SystemMessage(
            content="You are a helpful assistant that describes images in details."
        ),
        UserMessage(
            content=[
                TextContentItem(text="What's in this image?"),
                ImageContentItem(
                    image_url=ImageUrl.load(
                        image_file="sample.jpg",
                        image_format="jpg",
                        detail=ImageDetailLevel.LOW)
                ),
            ],
        ),
    ],
    model=model_name,
)

print(response.choices[0].message.content)
```

یادگیری اینجا پایان نمی‌یابد، مسیر را ادامه دهید

پس از اتمام این درس، مجموعه یادگیری هوش مصنوعی مولد ما را بررسی کنید تا دانش خود را در حوزه هوش مصنوعی مولد ارتقا دهید!

سلب مسئولیت: این سند با استفاده از سرویس ترجمه هوش مصنوعی Co-op Translator ترجمه شده است. در حالی که ما در تلاش برای دقت هستیم، لطفاً توجه داشته باشید که ترجمه‌های خودکار ممکن است شامل خطاها یا نادرستی‌هایی باشند. سند اصلی به زبان مادری خود باید به عنوان منبع معتبر در نظر گرفته شود. برای اطلاعات حیاتی، ترجمه حرفه‌ای انسانی توصیه می‌شود. ما در قبال هرگونه سوء تفاهم یا برداشت نادرست ناشی از استفاده از این ترجمه مسئولیتی نداریم.

بازگشت به فهرست مطالب

درس ۲۱ — هوش مصنوعی مولد مایکروسافت